

```
function 할일카드({ 제목, 끝났나, 눌렀을때 }) {  
  const [열림, 열기] = useState(false);  
  useEffect(() => { fetch("/api/할일").then((r) => r.json()); }, []);  
  return (  
    <li className={끝났나 ? "끝" : ""} onClick={눌렀을때}>  
      {제목}  
    </li>  
  );  
}
```

2 권 · 설 계 와 도 구

React

화면을 컴포넌트로 쪼개고, state 로 바꾸고,
서버에서 받아 옵니다.

브라우저 화면이 어떻게 바뀌는지를 매번 적어 두었습니다.

5

단원

29

개념

138

직접 해보기

70

문제

차례

06	폼과 입력	11
01	제어 컴포넌트	12
02	여러 입력 한번에	22
03	체크박스과 선택	32
04	form submit	42
05	입력을 목록에 담기	46
07	state 설계와 불변성	51
01	불변하게 바꾸기: 배열	52
02	불변하게 바꾸기: 객체	67
03	state 끌어올리기	70
04	자식이 부모에게 알리기	82
05	state를 어디에 둘까	94
08	파일 나누기와 스타일링	105
01	도구가 해 주는 일	106
02	import 와 export	112
03	컴포넌트 파일로 나누기	126
04	CSS 넣기.css	140
04	CSS 넣기	142
04	CSS 넣기.module.css	148
05	다른팀.css	150
05	styled-components	151
09	useEffect와 API	173
01	useEffect 기본	174
02	정리 함수	185
03	fetch 로 받아오기	198
04	로딩과 에러	211

05	값이 바뀌면 다시 받기	213
06	useEffect 실수	214
10	useRef와 커스텀 훅	237
01	useRef 로 요소 다루기	238
02	useRef 로 값 기억하기	244
03	커스텀 훅	258
04	훅의 규칙	261
05	memo 와 useMemo	266
부록 가	연습문제·종합연습 정답	281
부록 나	심화문제 정답	331

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

준비물 — Node.js 하나

이 자료는 처음부터 끝까지 **실제 개발에서 쓰는 방식(Vite 프로젝트)** 으로 진행합니다. 그래서 첫 시간에 Node.js 를 확인하고 프로젝트를 한 번 켜는 것으로 시작합니다.

JS자료에서 이미 설치했다면 그대로 쓰면 됩니다. 없다면 <https://nodejs.org> 에서 **왼쪽 LTS** 버튼으로 받아 설치하세요. 설치 후 **터미널을 전부 닫고 새로 여세요.**

확인하는 법 — 터미널에 이렇게 칩니다.

```
node -v
npm -v
```

v22.14.0 / 11.16.0 처럼 숫자가 나오면 됩니다.

★ **Node 는 22 이상이어야 합니다.** 앞의 숫자가 22보다 작으면(예: v18.x, v20.x) 이 자료의 도구가 안 됩니다. <https://nodejs.org> 에서 LTS 를 새로 받아 설치하세요.

시작하는 다섯 줄

1. 터미널을 연다
2. React자료/실습프로젝트 폴더로 간다
3. npm install 라고 친다 ← 맨 처음 한 번만. 10초쯤 걸립니다
4. npm run dev 라고 친다
5. 브라우저에서 터미널에 뜬 주소(<http://localhost:5173/>)를 연다

3번은 맨 처음 한 번만 하면 됩니다. 이 자료가 쓰는 도구들을 인터넷에서 받아 오는 명령입니다. 받고 나면 node_modules 라는 폴더가 생기는데, 그게 도구 창고입니다. 그 뒤로는 4번(npm run dev)만 하면 됩니다.

인터넷이 필요합니다. 실습실에서 안 되면 강사에게 말하세요.

막히면 [실습프로젝트/src/00_개발환경_만들기/개념01_설치하고_켜기.md](#) 를 보세요. 터미널을 한 번도 안 써 본 사람 기준으로 썼습니다.

VS Code

코드를 쓰는 편집기입니다. <https://code.visualstudio.com> 에서 받으세요. **파일 → 폴더 열기**로 실습프로젝트 폴더를 여는 편이 훨씬 편합니다. **Ctrl + `** 로 터미널을 열면 이미 그 폴더에 서 있습니다.

3분만 만져 보고 오세요

화면이 떴으면 왼쪽 목록에서 [01_React_시작하기 / 개념01 왜 React를 쓰나](#) 를 고르세요.

같은 화면이 두 개 있습니다.

- ① 은 여러분이 JS자료에서 하던 방식으로 만든 것
- ② 는 React 로 만든 것

두 '답기' 버튼을 번갈아 눌러 보세요. **화면은 똑같이 움직입니다.** 그런데 F12 → Console 을 보면 이렇게 찍힙니다.

[JS 방식] 내가 직접 고친 곳: 3군데
[React 방식] 내가 직접 고친 곳: 0군데 (데이터만 바꿈)

이 차이가 이 자료의 전부입니다. 지금 코드가 하나도 안 읽혀도 괜찮습니다.

그리고 마지막(13단원)에는 **할 일 목록·장바구니·사용자 검색·메모장** 네 개를 직접 만듭니다. JS자료 13단원에서 만들었던 할 일 목록을, 이번에는 React 로 다시 만듭니다.

폴더 순서

단원	제목	어디에
00	개발환경 만들기	실습프로젝트 (읽는 문서 .md)
01	React 시작하기	실습프로젝트
02	JSX	실습프로젝트
03	컴포넌트와 props	실습프로젝트
04	state와 이벤트	실습프로젝트
05	조건부 렌더링과 리스트	실습프로젝트
06	폼과 입력	실습프로젝트
07	state 설계와 불변성	실습프로젝트
08	파일 나누기와 스타일링	실습프로젝트 (개념01은 읽는 문서)

09	useEffect와 API	실습프로젝트
10	useRef와 커스텀 훅	실습프로젝트
11	React Router	실습프로젝트
12	Context와 useReducer	실습프로젝트
13	종합 프로젝트	실습프로젝트
14	서버와 연동	실습프로젝트 (터미널 두 개)
15	부록 — 타입스크립트	실습프로젝트 (안 해도 됩니다)

앞 단원을 건너뛰면 뒤 단원이 안 읽힙니다. 순서대로 가세요.

14단원은 서버를 같이 켭니다

useForm 으로 폼을 만들고, FormData 로 파일을 올리고, 내가 만든 서버에 붙습니다. **PART 3(백엔드)**로 넘어가는 다리입니다.

```
터미널 ①  cd 실습프로젝트          → npm run dev
터미널 ②  cd 실습프로젝트/14단원_서버 → node 서버.js
```

서버는 설치할 것이 없습니다. Node 만 있으면 node 서버.js 로 바로 돛니다. 서버 코드는 읽지 않아도 됩니다. 만드는 법은 PART 3 에서 배웁니다.

자료는 전부 실습프로젝트/src 안에 있습니다

```
React자료/
├ _검증도구/           ← 강사용. 학생은 볼 필요 없습니다
└ 실습프로젝트/       ← ★ 자료는 전부 여기
  ├── 14단원_서버/     ← 14단원에서 같이 켜는 연습용 서버
  └── src/
      ├── _ui/         ← 자료가 쓰는 부품 (읽지 않아도 됩니다)
      ├── 00_개발환경_만들기/ ← 첫 시간에 읽는 문서 한 장 (.md)
      ├── 01_React_시작하기/
      ├── 02_JSX/
      ├── ...
      └── 15_부록_타입스크립트/
```

왼쪽 목록에 저절로 나타납니다. 단원 폴더에 .jsx 를 만들면 등록 없이 잡힙니다.

이렇게 진행합니다

[터미널] npm run dev 를 켜 둔 채로 그대로 둡니다 ← 건드리지 않습니다
[VS Code] src 안의 예제 파일을 열어 읽고 고칩니다
[브라우저] 왼쪽에서 예제를 고르고, 저장하면 저절로 바뀌는 화면을 봅니다

- **새로고침(F5)은 누르지 마세요.** 왼쪽에서 고른 예제가 풀립니다. 저장이면 충분합니다.
- **화면이 안 바뀌면 터미널부터 보세요.** 문법이 틀리면 브라우저까지 가지도 못합니다.

분량

단원	16개 (00 준비 + 01~14 + 부록 1개)
수업 파일	107개
개념 섹션	439개
 직접 해보기	367개
연습문제	220문항
표준 진행 시간	71.5시간

자세한 시간표와 강사용 안내는 [수업_진행_가이드.md](#) 에 있습니다.

폼과 입력

OPEN 06_폼과_입력

```
function NameEcho() {  
  function handleChange(e) {  
    setText(e.target.value);  
    return (  

```

06

- 01 제어 컴포넌트
- 02 여러 입력 한번에
- 03 체크박스과 선택
- 04 form submit
- 05 입력을 목록에 담기
- 연습문제

제어 컴포넌트

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

JS자료 11단원 개념03 에서 입력칸을 이렇게 다뤘습니다.

```
nameInput.addEventListener("input", (e) => {
  out1.textContent = e.target.value; // ← 화면을 '내가' 고쳤습니다
});
```

값을 읽는 부분(`e.target.value`)은 React 에서도 똑같습니다. 달라지는 것은 그 다음입니다. React 에서는 화면을 직접 고치지 않습니다. 값을 state 에 넣기만 하면 화면은 알아서 따라옵니다. (04단원)

이 파일에서 하는 일은 세 가지입니다. ① 입력값을 state 에 담는다 → `onChange` ② 입력칸에 보이는 글자를 state 로 정한다 → `value` ③ ② 만 하고 ① 을 빠뜨리면 어떻게 되는지 직접 본다

`value` 와 `onChange` 를 짝으로 붙인 입력칸을 '제어 컴포넌트' 라고 부릅니다. 이름은 지금 외우지 않아도 됩니다. 이 파일을 다 보면 저절로 이해됩니다.

onChange 로 값 받기

섹션 1

입력칸에 글자를 칠 때마다 실행되는 일을 붙여 봅니다. React 에서 쓰는 이름은 `onChange` 입니다.

이름 때문에 헷갈리기 쉬운 곳이 하나 있습니다. JS자료 11단원에서 배운 순수 자바스크립트의 `change` 이벤트는 "다 치고 칸을 벗어났을 때" 한 번만 실행됐습니다. React 의 `onChange` 는 그렇지 않습니다. **글자 하나마다** 실행됩니다. 즉 JS자료의 `input` 이벤트와 같은 시점입니다. 이름만 `change` 입니다.

`e` 가 무엇인지도 그대로입니다. `e.target` 은 이벤트가 일어난 요소, `e.target.value` 는 그 입력칸에 들어 있는 글자입니다. (JS자료 11단원 개념02)

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function NameEcho() {
  const [text, setText] = useState("");

  function handleChange(e) {
```

`e.target.value` 는 언제나 문자열입니다. 숫자 칸이어도 문자열입니다.

```

    setText(e.target.value);
  }

  return (
    <div className="demo"> <h3>① onChange 로 값 받기</h3> <input onChange=
    {handleChange} placeholder="이름을 입력하세요" /> <div className="output">입력한 값:
    {text}</div> </div>
  );
}

```

처음 화면입니다. state 가 빈 문자열이라 뒤가 비어 있습니다.

화면	입력한 값:

여기에 "김민준" 을 한 글자씩 쳐 보세요. 아래 줄이 글자마다 바뀝니다.

화면	입력한 값: 김
	입력한 값: 김민
	입력한 값: 김민준

무슨 일이 일어난 걸까요? 한 글자를 칠 때마다 이렇게 됩니다. 키를 누름 → onChange 실행 → setText(새 값) → state 가 바뀜 → NameEcho 함수가 처음부터 다시 실행됨 → 화면이 새로 그려짐 (04단원 개념03)

out.textContent = ... 같은 줄이 한 줄도 없다는 점을 보세요. 화면을 고치는 일은 React 가 합니다.

➤ 직접 해보기

1

입력한 글자 수도 함께 보여 주세요. (힌트: {text.length} — JS자료 01단원의 .length 입니다)

value 를 붙이면 무엇이 달라지나

섹션 2

섹션 1의 입력칸에는 이상한 점이 하나 있습니다. 화면에 보이는 글자를 '두 곳' 이 따로 갖고 있다는 것입니다.

(1) 입력칸 자신이 갖고 있는 글자 ← 브라우저가 관리합니다 (2) state 에 담아 둔 text ← 우리가 관리합니다

지금은 둘이 같습니다. 그런데 코드로 입력칸을 비우고 싶으면 어떻게 할까요? setText("") 를 불러도 (2)만 바뀝니다. (1)은 그대로라 칸의 글자가 안 지워집니다.

그래서 입력칸에 value 를 붙입니다.

```
<input value={text} onChange={...} />
```

value={text} 는 이런 뜻입니다.

“이 칸에 보이는 글자는 언제나 text 다.”

이러면 (1)이 사라집니다. 값이 state 한 곳에만 있게 됩니다. 아래 데모에서 ‘지우기’ 와 ‘이서연 넣기’ 버튼을 눌러 보세요. 버튼이 state 만 바꾸는데 입력칸의 글자까지 따라 바뀝니다.

```
function ControlledName() {  
  const [text, setText] = useState("김민준");
```

함수 안에 그냥 둔 `console.log` 는 이 컴포넌트가 그려질 때마다 실행됩니다. (04단원 개념03 에서 렌더링 횟수를 셀 때 쓴 방법입니다)

```
console.log("[제어] 다시 그려집니다. text =", text);
```

F12 콘솔 [제어] 다시 그려집니다. text = 김민준

```
return (  
  <div className="demo"> <h3>② value + onChange – 제어 컴포넌트</h3> <input value=  
    {text} onChange={(e) => setText(e.target.value)} />  
    <div> <button onClick={() => setText("")}>지우기</button> <button onClick={()  
=> setText("이서연")}>이서연 넣기</button> </div> <div className="output">지금 state:  
    {text}</div> </div>  
  );  
}
```

처음 화면입니다. 입력칸 안에 이미 “김민준” 이 들어 있습니다.

화면 입력칸 = 김민준 / 지금 state: 김민준

화면(누르면): ‘지우기’ 를 누르면 입력칸이 텅 비고, 지금 state: 도 비어 버립니다. 화면(누르면): ‘이서연 넣기’ 를 누르면 입력칸에 이서연 이 들어갑니다.

콘솔도 함께 보세요. 글자를 한 자 칠 때마다 [제어] 줄이 한 줄씩 늘어납니다. state 가 바뀌면 컴포넌트 함수가 다시 실행되기 때문입니다.

정리하면 이렇습니다. value = state 를 화면으로 내보내는 길 (state → 입력칸) onChange = 사용자가 친 글자를 state 로 들이는 길 (입력칸 → state) 두 길이 다 있어야 값이 돕니다. 한 쪽만 있으면 다음 섹션처럼 됩니다.

▣ 직접 해보기 2

‘박지훈 넣기’ 버튼을 하나 더 만들어 보세요. (위 두 버튼 중 하나를 복사해서 글자만 바꾸면 됩니다)

onChange 를 빼면 — 직접 고장을 내 봅니다

섹션 3

아래 BrokenInput 은 value 만 붙이고 onChange 를 일부러 뺐습니다.

```
<input value={text} /> ← onChange 가 없습니다
```

데모 ③의 '고장난 입력칸 켜기' 버튼을 누른 뒤, 나타난 칸에 아무 글자나 쳐 보세요.

★ 한 글자도 안 써집니다. 키보드는 멀쩡한데 칸이 꼼짝도 안 합니다.

```
function BrokenInput() {
  const [text, setText] = useState("김민준");

  console.log("[고장] 다시 그려집니다. text =", text);
```

```
F12 콘솔 [고장] 다시 그려집니다. text = 김민준
```

```
return (
  <div> <input value={text} placeholder="여기에 글자를 쳐 보세요"
/> <div className="output">지금 state: {text}</div> <button onClick={() =>
  setText("이서연")}>state 를 바꿔 보기</button> </div>
);
}

function BrokenDemo() {
```

페이지를 열자마자 켜지지 않게 해 두었습니다. 켜는 순간 콘솔에 경고가 하나 나오는데, 그것이 이 섹션의 볼거리입니다.

```
const [show, setShow] = useState(false);

return (
  <div className="demo"> <h3>③ onChange 없는 입력칸 (고장)</h3> <button onClick=
{() => setShow(true)}>고장난 입력칸 켜기</button>
    {show && <BrokenInput />}
  </div>
);
}
```

화면(누르면): 버튼을 누르면 아래에 "김민준" 이 든 입력칸이 나타납니다.

그 칸에 글자를 쳐도 화면이 하나도 안 바뀝니다.
(고고 싶으면 왼쪽에서 다른 예제를 골랐다 돌아오세요)

1 키를 누른다

2 브라우저가 일단 입력칸의 글자를 바꾼다

3 onChange 가 없으니 setText 를 부르는 코드가 없다 → state 는 그대로 "김민준" 이다

4 이 칸은 value="김민준" 으로 고정된 제어 컴포넌트이므로, React 가 그 즉시

입력칸의 DOM 값을 다시 "김민준" 으로 되돌려 놓는다 (컴포넌트를 다시 그리는 것과는 다른 일입니다)

5. 그래서 칸은 계속 "김민준" 으로 보인다

한 줄로 줄이면 이렇습니다. ★ 화면은 state 를 그대로 비춘다. state 가 안 바뀌면 화면도 안 바뀐다.

콘솔을 보세요. [고장] 줄이 켜질 때 딱 한 번만 찍히고, 아무리 타이핑해도 늘어나지 않습니다. 컴포넌트가 다시 그려진 것이 아니라, React 가 입력칸의 값만 그때그때 원래대로 되돌려 놓은 것입니다. 반대로 'state 를 바꿔 보기' 버튼을 누르면 [고장] 줄이 하나 늘고 (이번엔 진짜로 다시 그려집니다) 입력칸 글자도 이서연으로 바뀝니다.

React 도 이걸 실수로 보고 콘솔에 빨간 경고를 냅니다. 전문은 이렇습니다.

You provided a `value` prop to a form field without an `onChange` handler. This will render a read-only field. If the field should be mutable use `defaultValue`. Otherwise, set either `onChange` or `readOnly`.

우리말로 옮기면 이렇습니다. "value 는 줬는데 onChange 를 안 줬습니다. 읽기 전용 칸이 됩니다. 고칠 수 있어야 하면 defaultValue 를 쓰고, 아니면 onChange 나 readOnly 중 하나를 붙이세요."

즉 고치는 길이 세 개입니다. (a) onChange 를 붙인다 → 제어 컴포넌트. 이 자료는 이 방법을 씁니다. (b) value 대신 defaultValue 를 쓴다 → 섹션 5에서 봅니다. (c) readOnly 를 붙인다 → 일부러 못 고치게 할 때만 씁니다.

이 경고는 "고장" 을 알리는 것이지 에러가 아닙니다. 화면은 계속 돌아갑니다. 그래서 더 위험합니다. 못 보고 지나치기 쉽습니다.

▹ 직접 해보기 3

BrokenInput 의 input 에 readOnly 를 붙여 보세요. `<input value={text} readOnly />` 저장하면 → 다시 켜 보면 경고가 사라집니다.

state 를 거치면 값에 손을 댈 수 있다

섹션 4

섹션 3 이 알려 준 것을 뒤집으면 쓸모가 생깁니다. "setState 를 안 부르면 화면이 안 바뀐다" → 조건에 안 맞을 때 일부러 setState 를 안 부르면, 그 글자는 안 들어갑니다.

아래는 10자까지만 받는 메모 칸입니다. 11번째 글자부터는 아예 안 써집니다. 사용자가 친 글자가 곧바로 화면에 가지 않고 우리 손을 한 번 거치기 때문에 이런 일이 가능합니다. value 를 붙인 덕분입니다.

```
const MAX_LENGTH = 10;

function LimitedMemo() {
  const [memo, setMemo] = useState("아메리카노");

  console.log("[메모] 지금 길이:", memo.length);
}
```

F12 콘솔 [메모] 지금 길이: 5

```
function handleChange(e) {
  const value = e.target.value;

  if (value.length > MAX_LENGTH) {
    return; // state 를 안 바꿉니다 → 화면도 안 바뀝니다 (섹션 3 그대로)
  }
  setMemo(value);
}

return (
  <div className="demo"> <h3>④ 10자까지만 받는 칸</h3> <input value=
{memo} onChange={handleChange} /> <div className="output">
  {memo.length} / {MAX_LENGTH} 자
</div> </div>
);
}
```

화면 입력칸 = 아메리카노 / 5 / 10 자

뒤에 "한잔주세요" 를 이어서 쳐 보세요. 10자까지는 들어갑니다.

화면 아메리카노한잔주세요 / 10 / 10 자

그 뒤로는 아무리 쳐도 글자가 안 늘어납니다. 11번째에서 return 하기 때문입니다.

섹션 1처럼 value 를 안 붙였다면 이렇게 못 막습니다. 입력칸이 자기 글자를 스스로 갖고 있어서, state 를 안 바꿔도 글자는 들어갑니다.

MAX_LENGTH 를 5 로 바꾸고 저장하세요. 이미 들어 있는 "아메리카노" 는 어떻게 되나요?

value 대신 defaultValue 를 쓰면

섹션 5

섹션 3 의 경고가 알려 준 두 번째 길입니다.

```
<input defaultValue="아메리카노" />
```

defaultValue 는 "처음 한 번만 이 글자를 넣어 뒤라" 는 뜻입니다. 그 다음부터는 입력칸이 자기 글자를 스스로 갖습니다. 그래서 잘 써집니다. 경고도 안 납니다. React 가 값을 관리하지 않겠다고 선언한 셈이니깐요.

편해 보이지만 잃는 것이 있습니다. - React 가 그 값을 모릅니다. 화면 다른 곳에 같이 보여 줄 수 없습니다. - 버튼으로 비우거나 채울 수 없습니다. - 섹션 4처럼 글자 수를 막을 수도 없습니다.

값을 읽으려면 입력칸을 직접 붙잡아야 하는데, 그 방법(useRef)은 10단원입니다. 이 자료는 06단원부터 끝까지 value + onChange 만 씁니다.

```
function DefaultValueDemo() {
  const [copied, setCopied] = useState("(아직 모릅니다)");

  return (
    <div className="demo"> <h3>⑤ defaultValue - React 가 값을 모르는 칸
    </h3> <input defaultValue="아메리카노" /> <div> <button onClick={() => setCopied("
    (그래도 여전히 모릅니다))}>
      옆 칸의 값을 여기로 가져오기
    </button> </div> <div className="output">가져온 값: {copied}</div> </div>
  );
}
```

화면 입력칸 = 아메리카노 / 가져온 값: (아직 모릅니다)

이 칸은 글자가 잘 써집니다. 경고도 안 납니다. 그런데 버튼을 눌러도 옆 칸의 글자를 가져올 수가 없습니다. 화면(누르면): 가져온 값: (그래도 여전히 모릅니다)

가져올 코드를 쓸 수가 없어서 이렇게 적어 두었습니다. 이것이 defaultValue 의 한계입니다. 값이 React 밖에 있습니다.

⑤ 의 입력칸 글자를 지우고 "라떼" 로 바꿔 보세요. 그 다음 왼쪽에서 다른 예제를 골랐다 돌아오면 무엇이 보일까요? 먼저 예상해 보세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

onChange 에 괄호를 붙임

```
<input value={text} onChange={handleChange(e)} />
```

04단원 개념01 과 같은 실수입니다. 괄호를 붙이면 '함수' 가 아니라 '함수를 지금 실행한 결과' 를 넘기는 것이 됩니다. 게다가 여기서는 e 라는 이름이 그 자리에 없어서 ReferenceError: e is not defined 로 화면이 통째로 안 그려집니다. onChange={handleChange} 처럼 이름만 넘기세요.

이런 실수를 합니다

e.target.value 가 아니라 e.target 을 넣음

```
setText(e.target);
```

state 에 입력칸 요소 자체가 들어갑니다. 그걸 화면에 그리려다가 Objects are not valid as a React child 에러가 나고 화면이 멈춥니다. .value 를 빠뜨리지 마세요.

이런 실수를 합니다

onChange 를 소문자로 씀

```
<input value={text} onchange={handleChange} />
```

아무 일도 안 일어납니다. 글자가 안 써집니다. React 는 onChange 처럼 중간이 대문자인 이름만 이벤트로 봅니다. 콘솔에는 Unknown event handler property **onchange** 경고가 나옵니다.

이런 실수를 합니다

value 자리에 setter 를 넣음

```
<input value={setText} onChange={(e) => setText(e.target.value)} />
```

함수를 화면에 넣은 셈이라 칸이 비어 보이고 경고가 납니다. value 에는 '지금 보여 줄 값' 을 넣습니다.

이런 실수를 합니다

화살표 함수의 화살표를 빠뜨림 [SyntaxError]

```
<input value={text} onChange={(e) setText(e.target.value)} />
```

⇒ 가 없어서 Babel 이 파일 전체를 못 읽습니다. 화면이 통째로 비고 콘솔에 SyntaxError 가 납니다. 눈으로만 보세요.

화면 조립

정리

- 1 **onChange** 는 글자 하나마다 실행됩니다. 값은 `e.target.value` 로 읽고, 언제나 문자열입니다.
- 2 `value={state}` 는 “이 칸에 보이는 글자는 언제나 이 state 다” 라는 뜻입니다.
- 3 `value` 와 `onChange` 를 짝으로 붙인 입력칸을 **제어 컴포넌트**라고 합니다. 값이 `state` 한 곳에만 있습니다.
- 4 `onChange` 를 빼면 글자가 한 자도 안 써집니다. **화면은 state 를 그대로 비추기 때문**입니다. React 도 콘솔에 경고를 냅니다.
- 5 `state` 를 거치므로 값에 손을 댈 수 있습니다. 조건에 안 맞으면 `setState` 를 안 부르면 됩니다.
- 6 `defaultValue` 는 첫 값만 정합니다. React 가 그 값을 모르므로 이 자료에서는 쓰지 않습니다.

```
export default function Concept01() {
  return (
    <div> <h1>개념 01 - 제어 컴포넌트</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br /> <strong>직접 타이핑해 보는 파일입니다.</strong> 읽기만 하면
      절반도 안 남습니다. 입력칸이 나오면 손을 올리고 글자를 쳐 보세요.
      <p> <div> <NameEcho /> <ControlledName /> <BrokenDemo /> <LimitedMemo
      /> <DefaultValueDemo /> </div> </div>
    );
  }
}
```

직접 해보기 정답

```
1) <div className="output">입력한 값: {text} ({text.length}자)</div>
// 화면: 입력한 값: 김민준 (3자)
```

→ 아무것도 안 쳤을 때는 (0자) 입니다.

```
2) <button onClick={() => setText("박지훈")}>박지훈 넣기</button>
// 화면(누르면): 입력칸과 '지금 state:' 가 함께 박지훈 으로 바뀝니다.
```

→ 버튼은 `state` 만 바꿉니다. 입력칸을 건드리는 코드는 한 줄도 없습니다.

```
3) <input value={text} readOnly />
// 화면: 여전히 글자가 안 써집니다. 대신 콘솔의 빨간 경고가 사라집니다.
```

→ readOnly 는 “일부러 못 고치게 한 것” 이라고 React 에게 알리는 표시입니다.

동작은 그대로고, 경고만 없어집니다.

4) 화면: 아메리카노 / 5 / 5 자 → 이미 들어 있던 다섯 글자는 그대로 남습니다.

handleChange 는 '글자를 칠 때' 만 실행되기 때문입니다.

첫 값은 검사를 거치지 않고 그냥 들어갑니다.

한 글자만 더 쳐 보면 그때부터 막힙니다.

5) 다시 “아메리카노” 로 돌아옵니다. → 왼쪽에서 다른 예제를 골랐다 돌아오면 컴포넌트가 처음부터 다시 만들어집니다.

defaultValue 는 '처음 한 번' 의 값이라 다시 아메리카노 가 들어갑니다.

쳐 넣은 “라떼” 는 어디에도 저장되지 않았습니다.

여러 입력 한번에

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념01 에서 입력칸 하나를 state 로 다뤘습니다.

```
const [text, setText] = useState("");


```

그런데 실제 폼은 칸이 하나가 아닙니다. 이름·전화번호·메모 세 칸이면 위 두 줄을 세 번 써야 합니다. 그래도 되는지, 더 나은 길이 있는지 봅시다.

이 파일에서 하는 일 ① 칸마다 state 를 따로 두기 — 그대로 늘리는 방법

(그 전에 '대괄호로 키 이름 정하기' 문법을 섹션 2에서 먼저 봅시다)

② 객체 state 하나에 모으기 — 스프레드를 빼면 무슨 일이 생기나 ③ name 속성으로 핸들러 하나가 모든 칸을 처리하기 ④ 숫자 칸을 다룰 때 조심할 것

입력칸마다 state 하나씩

섹션 1

가장 단순한 방법입니다. 개념01 을 그대로 두 번 쓴 것뿐입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function TwoStates() {
  const [name, setName] = useState("김민준");
  const [phone, setPhone] = useState("");

  return (
    <div className="demo"> <h3>① 칸마다 state 하나씩</h3> <div>
      이름 <input value={name} onChange={(e) => setName(e.target.value)} />
    </div> <div>
      전화 <input value={phone} onChange={(e) => setPhone(e.target.value)} />
    </div> <div className="output">
      {name} / {phone}
    </div> </div>
  );
}
```

화면 김민준 /

전화 칸에 "010" 을 치면 아래 줄이 이렇게 바뀝니다.

화면 김민준 / 010

이 방법이 나쁜 것이 아닙니다. 칸이 두세 개면 이게 제일 읽기 쉽습니다. 어느 칸이 어느 state 인지 눈으로 바로 보이니까요.

다만 칸이 여섯 개가 되면 이런 줄이 여섯 벌 생깁니다.

```
const [a, setA] = useState("");
const [b, setB] = useState("");
```

... 여섯 번 그리고 "폼을 전부 비우기" 를 만들려면 setA("") 부터 setF("") 까지 여섯 줄을 또 써야 합니다. 그때가 아래 방법으로 옮길 때입니다.

👉 직접 해보기 1

메모 칸을 하나 더 만들어 보세요. state 한 줄, input 한 줄이면 됩니다.

대괄호로 키 이름 정하기

섹션 2

다음 섹션으로 가기 전에 처음 보는 문법을 하나 배웁니다. 객체를 만들 때 키 이름 자리를 대괄호로 감싸는 문법입니다.

{ key: 1 } ← key 라는 '글자 그대로' 가 이름이 됩니다 { [key]: 1 } ← key 라는 '변수 안에 든 글자' 가 이름이 됩니다

즉 대괄호는 "이 자리 이름을 지금 계산해서 넣어라" 는 표시입니다. 배열에서 arr[i] 로 자리를 계산해 꺼내던 것과 같은 느낌입니다. 직접 찍어 보면 바로 보입니다.

```
const field = "phone";

console.log({ field: "010-1234-5678" });
```

F12 콘솔 { field: '010-1234-5678' }

```
console.log({ [field]: "010-1234-5678" });
```

F12 콘솔 { phone: '010-1234-5678' }

위는 field 라는 이름의 키가 만들어졌고, 아래는 field 안에 든 글자인 phone 이 키가 됐습니다.

여기에 09단원 스프레드를 같이 쓰면 이런 모양이 됩니다.

```
const before = { name: "김민준", phone: "", memo: "" };

console.log({ ...before, [field]: "010-1234-5678" });
```

```
F12 콘솔 { name: '김민준', phone: '010-1234-5678', memo: '' }
```

- 1 ...before → before 의 키 세 개를 그대로 펼쳐 담는다
- 2 [field] → 그 다음 phone 이라는 키를 하나 더 담는다
- 3 이름이 겹치므로 뒤에 담은 값이 앞의 것을 덮는다

원래 before 는 그대로 있습니다. 새 객체가 하나 만들어진 것입니다.

```
console.log(before);
```

```
F12 콘솔 { name: '김민준', phone: '', memo: '' }
```

▹ 직접 해보기 2

field 를 "memo" 로 바꾸고 저장해 보세요. 세 번째 줄이 어떻게 바뀔지 먼저 예상해 보세요.

객체 state 하나에 모으기

섹션 3

이제 세 칸의 값을 객체 하나에 담아 봅니다.

```
const [form, setForm] = useState({ name: "김민준", phone: "", memo: "" });
```

읽을 때는 form.name 처럼 씁니다. 바꿀 때가 중요합니다. 이렇게 씁니다.

```
setForm({ ...form, name: "이서연" });
```

form.name = "이서연" 처럼 직접 고치면 안 됩니다. (04단원 개념06) 언제나 새 객체를 만들어서 넣습니다. 왜 그래야 하는지는 07단원에서 제대로 다룹니다. 지금은 모양만 익히세요. 대신 ...form 을 빠뜨리면 어떻게 되는지는 지금 봅니다.

아래 데모의 버튼 세 개를 차례로 눌러 보세요.

```
function SpreadDemo() {
  const [info, setInfo] = useState({ name: "김민준", age: 20, memo: "아메리카노" });

  return (
    <div className="demo"> <h3>② 스프레드를 빼면 무슨 일이 생기나
  </h3> <div className="output">
      이름: {info.name} / 나이: {info.age} / 메모: {info.memo}
    </div>
  </div>
  )
}
```


스프레드로 이름 바꾸기

```

    </button> <button onClick={() => setInfo({ name: "박지훈" })}>
      스프레드 없이 이름 바꾸기
    </button> <button onClick={() => setInfo({ name: "김민준", age: 20, memo:
"아메리카노" })}>
      >
        되돌리기
      </button> </div> </div>
  );
}

```

화면 이름: 김민준 / 나이: 20 / 메모: 아메리카노

화면(누르면): '스프레드로 이름 바꾸기' → 이름: 이서연 / 나이: 20 / 메모: 아메리카노

이름만 바뀌고 나머지는 그대로입니다.

화면(누르면): '스프레드 없이 이름 바꾸기' → 이름: 박지훈 / 나이: / 메모:

★ 나이와 메모가 통째로 사라졌습니다.

에러는 하나도 안 납니다. 콘솔도 조용합니다. 그래서 위험합니다.

`setInfo({ name: "박지훈" })` 은 "이름만 있는 새 객체" 를 넣은 것이어서,

원래 있던 `age` 와 `memo` 는 그냥 없어진 것입니다. `...info` 는 "나머지는 하던 대로 가져가라" 는 뜻입니다.

▹ 직접 해보기

3

'스프레드 없이 이름 바꾸기' 버튼의 `setInfo` 를 `setInfo({ ...info, name: "박지훈" })` 로 고쳐 보세요.
나이와 메모가 남는지 확인하세요.

name 으로 핸들러 하나 만들기

섹션 4

칸이 세 개면 `onChange` 도 세 개 써야 할까요? 안 그래도 됩니다.

입력칸에 `name` 속성을 붙여 두면, 이벤트가 왔을 때 `e.target.name` 으로 "어느 칸에서 온 이벤트인지" 를 알 수 있습니다. (`e.target` 이 그 입력칸이니, 그 칸에 적어 둔 `name` 을 읽는 것뿐입니다)

```

<input name="phone" ... /> → e.target.name 은 "phone"

```

여기에 섹션 2의 대괄호 키를 붙이면 핸들러 하나로 모든 칸을 처리할 수 있습니다.

```

setForm({ ...form, [name]: value });

```

이름 칸에서 오면 [name] 이 "name" 이 되고, 전화 칸에서 오면 [name] 이 "phone" 이 됩니다. 한 줄이 세 칸을 다 담당합니다.

```
const EMPTY_FORM = { name: "", phone: "", memo: "" };

function OneHandlerForm() {
  const [form, setForm] = useState({
    name: "김민준",
    phone: "010-1234-5678",
    memo: "아메리카노",
  });

  function handleChange(e) {
    const name = e.target.name; // 어느 칸에서 왔나 const value = e.target.value; //
    무엇을 쳤나 setForm({ ...form, [name]: value });
  }

  return (
    <div className="demo">
      <h3>③ 핸들러 하나로 세 칸</h3>
      <div>
        이름 <input name="name" value={form.name} onChange={handleChange} />
      </div>
      <div>
        전화 <input name="phone" value={form.phone} onChange={handleChange} />
      </div>
      <div>
        메모 <input name="memo" value={form.memo} onChange={handleChange} />
      </div>
      <div className="output">
        {form.name} / {form.phone} / {form.memo}
      </div>
      <div>
        <button onClick={() => console.log("지금 폼:", form)}>
          콘솔에 지금 폼 찍기
        </button>
        <button onClick={() => setForm(EMPTY_FORM)}>전부 지우기</button>
      </div>
    </div>
  );
}
```

화면 김민준 / 010-1234-5678 / 아메리카노

세 칸 아무 데나 글자를 쳐 보세요. 그 칸만 바뀝니다. 메모 칸의 글자를 모두 지우고 "라떼" 라고 쳐 보세요.

화면 김민준 / 010-1234-5678 / 라떼

화면(누르면): '콘솔에 지금 폼 찍기' → 폼 객체가 통째로 콘솔에 찍힙니다.

```
콘솔(메모를 라떼로 바꾼 뒤 누르면): 지금 폼: { name: '김민준', phone: '010-1234-5678', memo: '라떼' }
```

화면(누르면): '전부 지우기' → 세 칸이 한꺼번에 비고 아래 줄에는 // 만 남습니다.

이것이 객체로 모아 둔 값어치입니다. - 비우기가 setForm(EMPTY_FORM) 한 줄입니다. 칸이 열 개여도 한 줄입니다. - 서버로 보낼 때도 form 하나만 넘기면 됩니다. (09단원에서 씁니다) - 칸을 하나 더 늘려도 핸들러는 그대로입니다. input 한 줄만 추가하면 됩니다.

그럼 언제 나누고 언제 합칠까요? 기준은 간단합니다. 같이 움직이고 같이 비워지는 값이면 → 객체 하나로 합친다 (한 폼의 칸들) 서로 아무 상관 없는 값이면 → state 를 따로 둔다 칸이 두 개뿐이면 굳이 합치지 않아도 됩니다. 섹션 1 이 더 읽기 쉽습니다.

▹ 직접 해보기 4

주소 칸을 하나 더 만들어 보세요. useState 의 초기값에 address 를 넣고 input 을 한 줄 추가하면 끝입니다. handleChange 는 한 글자도 안 고쳐도 됩니다.

숫자 칸은 문자열로 들어옵니다

섹션 5

type="number" 인 칸도 e.target.value 는 문자열입니다. 화면에 숫자가 보인다고 숫자가 아닙니다. JS자료 11단원 개념03 과 같은 함정입니다.

그래서 계산하기 전에 Number() 로 바꿉니다. 안 바꾸면 * 는 우연히 맞고 + 만 이상해집니다. 아래에서 직접 보세요.

```
function CartForm() {
  const [item, setItem] = useState({ price: "4500", count: "2" });

  function handleChange(e) {
    setItem({ ...item, [e.target.name]: e.target.value });
  }

  const total = Number(item.price) * Number(item.count);

  console.log("[장바구니] 합계:", total);
}
```

F12 콘솔 [장바구니] 합계: 9000

```
return (
  <div className="demo"> <h3>④ 라떼 몇 잔? - 숫자 칸</h3> <div>
    가격 <input type="number" name="price" value={item.price} onChange=
    {handleChange} /> </div> <div>
    개수 <input type="number" name="count" value={item.count} onChange=
```

```
{handleChange} /> </div> <div className="output">합계: {total}원
</div> <div className="output">Number 없이 + 로 더하면: {item.price + item.count}
</div> </div>
);
}
```

화면 합계: 9000원
 Number 없이 + 로 더하면: 45002

아래 줄을 보세요. 4500 + 2 가 4502 가 아니라 45002 입니다. 둘 다 문자열이라 + 가 '이어 붙이기' 로 동작한 것입니다.

개수 칸을 비워 보세요. value 가 빈 문자열이 되고 Number("") 은 0 이라 합계가 0원이 됩니다.

화면 합계: 0원

참고로 Number 는 계산할 때만 씁니다. state 에는 문자열 그대로 둡니다. 숫자로 바꿔서 넣으면 칸을 다 지웠을 때 0 이 튀어나와 지워지지 않습니다.

▹ 직접 해보기 5

가격을 6000, 개수를 3 으로 바꿔 보세요. 합계가 얼마가 되나요? 아래 줄은 무엇이 되나요?

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

스프레드를 빼먹음 (섹션 3)

```
setForm({ [name]: value });
```

에러가 안 납니다. 다른 칸의 값이 조용히 사라집니다. 게다가 그 칸의 value 가 undefined 가 되면서 "controlled 였던 입력칸이 uncontrolled 가 됐다" 는 경고까지 따라옵니다. ...form 을 먼저 쓰는 것을 습관으로 만드세요.

이런 실수를 합니다

대괄호를 안 씁니다

```
setForm({ ...form, name: value });
```

어느 칸을 고쳐도 name 키만 바뀝니다. 여기서 name 은 변수가 아니라 '글자 그대로의 name' 이기 때문입니다. 전화 칸을 쳐도 이름이 바뀌는 이상한 품이 됩니다.

이런 실수를 합니다

input 에 name 속성을 안 붙임

```
<input value={form.phone} onChange={handleChange} />
```

e.target.name 이 빈 문자열이 됩니다. 그러면 { "": "010" } 처럼 이름 없는 키가 생기고, 화면에는 아무 변화가 없습니다. 역시 에러는 안 납니다.

이런 실수를 합니다

객체를 직접 고침

```
form.name = "아서연";
```

04단원 개념06 에서 본 것과 같습니다. 화면이 안 바뀝니다. set 함수를 부르지 않았으니 React 는 바뀐 줄을 모릅니다.

이런 실수를 합니다

계산할 때 Number 를 빼먹음 (섹션 5)

```
const total = item.price + item.count;
```

45002 처럼 이상한 값이 나옵니다. 곱하기만 우연히 맞습니다.

이런 실수를 합니다

중괄호를 하나 빠뜨림 [SyntaxError]

```
setForm({ ...form, [name]: value });
```

Babel 이 파일 전체를 못 읽어서 화면이 통째로 빡니다. 눈으로만 보세요.

화면 조립

정리

- 1 칸이 두세 개면 **state** 를 따로 두는 것이 제일 읽기 쉽습니다.
- 2 { [key]: 1 } 처럼 대괄호로 감싸면 **변수 안에 든 글자**가 키 이름이 됩니다.
- 3 객체 state 는 setForm({ ...form, 바꿀것 }) 처럼 새 객체를 만들어 넣습니다. 스프레드를 빼면 나머지 값이 조용히 사라집니다.
- 4 input 에 **name** 을 붙이면 e.target.name 으로 어느 칸인지 알 수 있습니다.
- 5 setForm({ ...form, [name]: value }) 한 줄이면 핸들러 하나가 모든 칸을 담당합니다.
- 6 숫자 칸도 e.target.value 는 **문자열**입니다. 계산 직전에만 Number() 를 씁니다.

```
export default function Concept02() {
  return (
    <div> <h1>개념 02 - 여러 입력 한번에</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br /> <strong>직접 타이핑해 보는 파일입니다.</strong> 입력칸이 나오면
      손을 올리고 글자를 쳐 보세요.
      </p> <div> <TwoStates /> <SpreadDemo /> <OneHandlerForm /> <CartForm
    /> </div> </div>
  );
}
```

직접 해보기 정답

```
1) const [memo, setMemo] = useState("");
<div>메모 <input value={memo} onChange={(e) => setMemo(e.target.value)} /></div>
```

그리고 output 줄을 {name} / {phone} / {memo} 로 바꿉니다.

```
// 화면: 김민준 / /
```

→ state 한 줄과 input 한 줄이 늘었습니다. 칸이 늘 때마다 이만큼씩 늘어납니다.

```
2) // 콘솔: { name: '김민준', phone: '', memo: '010-1234-5678' }
```

→ field 가 "memo" 가 되었으니 memo 키를 덮어씁니다.

```
phone 은 빈 문자열 그대로 남습니다.
```

```
3) // 화면(누르면): 이름: 박지훈 / 나이: 20 / 메모: 아메리카노
```

→ ...info 가 나이와 메모를 그대로 가져오고, 그 뒤에 이름만 덮어씁니다.

4) useState 초기값을 { name: ..., phone: ..., memo: ..., address: "서울" } 로 하고

```
<div>주소 <input name="address" value={form.address} onChange={handleChange} /></div>
```

한 줄만 추가하면 됩니다.

```
// 화면: 주소 칸에 글자를 치면 그 칸만 바뀝니다.
```

→ handleChange 는 [name] 으로 키를 계산하므로 고칠 것이 없습니다.

```
이것이 핸들러를 하나로 만든 이득입니다.
```

```
5) // 화면: 합계: 18000원  
// 화면: Number 없이 + 로 더하면: 60003
```

→ $6000 \times 3 = 18000$ 입니다.

아래 줄은 "6000" 과 "3" 을 이어 붙여 "60003" 이 됩니다.

체크박스과 선택

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념01:02 에서 다룬 것은 전부 '글자를 치는 칸' 이었습니다. 짝은 언제나 `value + onChange` 였습니다.

그런데 체크박스는 글자를 치는 칸이 아닙니다. 켜졌나 꺼졌나만 있습니다. 그래서 짝이 다릅니다.

글자 칸	<code>value + onChange</code>	값은 <code>e.target.value</code> (문자열)
체크박스	<code>checked + onChange</code>	값은 <code>e.target.checked</code> (true / false)

JS자료 11단원 개념03 에서 배운 것과 똑같습니다. 순수 자바스크립트에서도 체크박스는 `.value` 가 아니라 `.checked` 를 읽었습니다. React 라고 달라지지 않습니다. 읽는 방법은 그대로고, 담는 곳만 state 로 바뀝니다.

이 파일에서 하는 일 ① 체크박스 하나 (동의 여부) ② 라디오 — 여럿 중 하나만 ③ select — 목록에서 고르기 ④ 체크박스 여러 개 — 배열 state ⑤ 글자 칸과 체크박스를 한 핸들러로

체크박스는 checked

섹션 1

체크박스에는 `value` 대신 `checked` 를 붙입니다.

```
<input type="checkbox" checked={agree} onChange={...} />
```

“이 칸이 켜져 있는지는 언제나 `agree` 가 정한다” 는 뜻입니다. 개념01 의 `value` 와 하는 일이 똑같습니다. 이름만 `checked` 입니다.

읽을 때도 `e.target.value` 가 아니라 `e.target.checked` 를 씁니다. 아래 데모에서 체크박스를 켜 보면 그 이유가 바로 보입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function AgreeCheck() {
  const [agree, setAgree] = useState(false);
  const [lastValue, setLastValue] = useState("(아직 안 눌렀습니다)");

  console.log("[동의] agree =", agree);
}
```

F12 콘솔 [동의] agree = false


```
function handleChange(e) {
  setAgree(e.target.checked); // * .checked 를
  읽습니다 setLastValue(e.target.value); // 비교용으로 .value 도 담아 둡니다
}

return (
  <div className="demo"> <h3>① 체크박스 하나
</h3> <label> <input type="checkbox" checked={agree} onChange={handleChange} />
약관에
    동의합니다
    </label> <div className="output">동의: {agree ? "함" : "안 함"}
</div> <div className="output">e.target.value 는: {lastValue}</div> <button disabled=
{!agree}>주문하기</button> </div>
  );
}
```

화면 동의: 안 함 / e.target.value 는: (아직 안 눌렀습니다)

'주문하기' 버튼은 눌리지 않는 회색 상태입니다.

화면(누르면): 체크박스를 켜면 → 동의: 함 / e.target.value 는: on

'주문하기' 버튼이 살아납니다.

화면(누르면): 다시 끄면 → 동의: 안 함 / e.target.value 는: on

★ 켜도 꺼도 e.target.value 는 계속 "on" 입니다. 값이 안 바뀝니다. 체크박스의 value 는 "이 칸이 켜졌을 때 보낼 값" 이라서 켜졌는지 꺼졌는지와는 상관이 없습니다. 그래서 체크박스에서 .value 를 읽으면 언제나 같은 값만 나옵니다. 켜짐/꺼짐은 .checked 로 읽어야 합니다.

버튼의 disabled 는 02단원 개념03 의 불리언 속성입니다.

disabled={!agree} 는 "동의를 안 했으면 못 누르게" 라는 뜻입니다.

체크박스 state 하나가 화면 두 곳을 동시에 바꿉니다.

▹ 직접 해보기 1

동의하지 않았을 때 버튼 글자를 "동의를 필요합니다" 로 바꿔 보세요. (힌트: 삼항 연산자 — 05단원 개념01)

라디오 — 여럿 중 하나만

섹션 2

라디오 버튼은 여러 개가 한 묶음으로 움직입니다. 하나를 고르면 나머지가 저절로 꺼집니다.

여기서 헛갈리기 쉬운 점이 하나 있습니다. 라디오는 checked 와 value 를 '둘 다' 씁니다. 하는 일이 다릅니다.

```
value="S"           골랐을 때 무슨 값이 되나
checked={size === "S"} 지금 이게 골라진 것인가 (true / false)
```

checked 자리에 비교식을 그대로 넣은 것을 보세요. size === "S" 는 true 아니면 false 입니다. 그게 그대로 checked 가 됩니다. 05단원에서 조건을 화면에 쓰던 것과 같은 방식입니다.

state 는 하나뿐입니다. 라디오가 세 개여도 size 하나로 충분합니다. "지금 골라진 것" 이 하나뿐이니 값을 값도 하나면 됩니다.

```
const SIZES = [
  { value: "S", label: "작은 것" },
  { value: "M", label: "보통" },
  { value: "L", label: "큰 것" },
];

function SizePicker() {
  const [size, setSize] = useState("M");

  return (
    <div className="demo"> <h3>② 라디오 - 사이즈 고르기</h3>
      {SIZES.map((item) => (
        <label key={item.value} style={{ marginRight: "12px"
      }}> <input type="radio" name="size" value={item.value} checked={size ===
      item.value} onChange={(e) => setSize(e.target.value)}
        />
        {item.label}
      </label>
    ))}
    <div className="output">고른 사이즈: {size}</div> </div>
  );
}
```

화면 고른 사이즈: M (처음부터 '보통' 이 켜져 있습니다)

화면(누르면): '큰 것' 을 누르면 → 고른 사이즈: L

라디오는 e.target.value 를 읽습니다. 체크박스과 다릅니다. 켜짐/꺼짐이 아니라 "무엇을 골랐나" 가 알고 싶은 것이니까요.

name="size" 를 셋 다 같게 적었습니다. 브라우저가 한 묶음으로 보게 하는 표시입니다. React 가 checked 로 이미 관리하고 있어서 없어도 동작은 하지만, 붙여 두면 키보드 화살표로 옮겨 다닐 수 있어서 쓰는 사람이 편합니다.

map 과 key 는 05단원 개념02·03 그대로입니다. 라디오를 손으로 세 번 쓰지 않고 배열을 돌려 그렸습니다.

직접 해보기

2

SIZES 에 { value: "XL", label: "아주 큰 것" } 을 더해 보세요. 다른 곳은 한 줄도 안 고쳐도 됩니다.

select — 목록에서 고르기

섹션 3

select 는 React 에서 모양이 조금 달라집니다.

순수 HTML <option value="M" selected>보통</option> ← option 에 표시 React <select value={menu} onChange={...}> ← select 에 value

option 에는 아무것도 안 붙입니다. select 하나에만 value 를 줍니다. 글자 칸의 value 와 똑같이 생각하면 됩니다.

고른 값은 option 의 value 입니다. 화면에 보이는 글자가 아닙니다. JS자료 11단원 개념03 에서 "큰 것" 을 골라도 "L" 이 나왔던 것과 같습니다.

```
const MENU_PRICE = {
  아메리카노: 4000,
  라떼: 4500,
  케이크: 6000,
};

function MenuSelect() {
  const [menu, setMenu] = useState("아메리카노");
```

객체에서 대괄호로 꺼내는 방법입니다. (JS자료 07단원)

```
const price = MENU_PRICE[menu];

console.log("[메뉴] 고른 것:", menu, "/ 가격:", price);
```

F12 콘솔 [메뉴] 고른 것: 아메리카노 / 가격: 4000

```
return (
  <div className="demo"> <h3>③ select - 메뉴 고르기</h3> <select value=
{menu} onChange={(e) => setMenu(e.target.value)}>
    <option value="아메리카노">아메리카노</option> <option value="라떼">라떼
</option> <option value="케이크">케이크</option> </select> <div className="output">
    {menu} - {price}원
  </div> </div>
);
}
```

화면(누르면): 목록에서 '케이크' 를 고르면 → 케이크 — 6000원

가격은 state 로 두지 않았습니다. menu 만 있으면 계산되기 때문입니다. 이렇게 "계산할 수 있는 값은 state 로 두지 않는다" 는 07단원 개념05 에서 다룹니다.

참고로 여러 개를 고를 수 있는 select(multiple)도 있습니다. 그때는 value 에 배열을 넣습니다. 잘 안 쓰니 여기서는 넘어갑니다. 여러 개를 고르게 하려면 다음 섹션의 체크박스가 훨씬 편합니다.

직접 해보기 3

MENU_PRICE 에 삼각김밥: 1200 을 넣고 option 도 한 줄 추가해 보세요.

여러 개 체크 — 배열 state

섹션 4

토픽처럼 "여러 개를 동시에 고를 수 있는" 화면입니다. 체크박스를 세 개 두고 state 를 세 개 만들어도 되지만, 개수가 늘면 힘들고 "몇 개 골랐나" 를 세기도 번거롭습니다.

고른 것만 배열에 담으면 편합니다.

```
const [picked, setPicked] = useState(["시럽"]);
```

그러면 체크박스마다 이렇게 물어보면 됩니다.

```
checked={picked.includes(t)} ← 배열 안에 있으면 켜진 것 (JS자료 06단원)
```

켜고 끌 때는 배열을 새로 만들어 넣습니다. 꺼짐 → [...picked, 값] 값을 더한 새 배열 꺼짐 → picked.filter((t) => t !== 값) 그것만 뺀 새 배열 (JS자료 08단원)

둘 다 원래 배열을 건드리지 않고 새 배열을 만듭니다. ★ push 를 쓰면 안 되는데, 그 이유는 07단원에서 제대로 설명합니다. 지금은 이 모양만 눈에 익히세요.

```
const TOPPINGS = ["시럽", "휘핑크림", "샷 추가"];

function ToppingPicker() {
  const [picked, setPicked] = useState(["시럽"]);

  console.log("[토픽]", picked);
}
```

F12 콘솔 [토픽] ['시럽']

```
function handleChange(e) {
  const value = e.target.value;

  if (e.target.checked) {
```

```

} else {
  setPicked(picked.filter((item) => item !== value));
}
}

return (
  <div className="demo"> <h3>④ 토핑 여러 개 고르기</h3>
    {TOPPINGS.map((topping) => (
      <label key={topping} style={{ marginRight: "12px"
    }}> <input type="checkbox" value={topping} checked=
    {picked.includes(topping)} onChange={handleChange}
      />
      {topping}
    </label>
    ))}
    <div className="output">고른 것: {picked.join(", ")}
  </div> <div className="output">모두 {picked.length}개</div> </div>
);
}

```

화면 고른 것: 시럽 / 모두 1개

06

화면(누르면): '휘핑크림' 을 켜면 → 고른 것: 시럽, 휘핑크림 / 모두 2개 화면(누르면): 이어서 '시럽' 을 끄면
→ 고른 것: 휘핑크림 / 모두 1개 화면(누르면): 셋 다 끄면 → 고른 것: / 모두 0개

여기서는 체크박스에 value 를 직접 적어 줬습니다("시럽" 등). 섹션 1 처럼 value 를 안 적으면 전부 "on" 이라 누가 눌렀는지 알 수 없습니다. 체크박스가 여러 개일 때는 value 를 꼭 적어 주세요.

join(", ") 은 배열을 글자로 이어 붙입니다. (JS자료 06단원) 배열을 그대로 {picked} 라고 쓰면
"시럽휘핑크림" 처럼 붙어 나옵니다.

▹ 직접 해보기

4

TOPPINGS 에 "얼음 많이" 를 더해 보세요. 체크박스가 네 개로 늘어나는지 확인하세요.

글자 칸과 체크박스를 한 핸들러로

섹션 5

개념02 에서 name 과 대괄호 키로 핸들러를 하나로 묶었습니다.

```
setForm({ ...form, [name]: value });
```

체크박스가 켜이면 한 군데만 손보면 됩니다. 값을 e.target.value 에서 읽을지 e.target.checked 에서 읽을지 갈라 주는 것입니다.

어느 쪽인지는 e.target.type 으로 압니다. 그 칸의 type 속성 글자가 들어 있습니다. 글자 칸 → "text"
체크박스 → "checkbox" 라디오 → "radio" 숫자 칸 → "number" select → "select-one" (하나만
고르는 select 라는 뜻입니다)

```
function OrderForm() {
  const [order, setOrder] = useState({
    name: "김민준",
    takeout: true,
    size: "M",
  });

  console.log("[주문]", order);
}
```

F12 콘솔 [주문] { name: '김민준', takeout: true, size: 'M' }

```
function handleChange(e) {
  const name = e.target.name;
```

체크박스면 checked, 아니면 value — 이 한 줄이 전부입니다

```
const value = e.target.type === "checkbox" ? e.target.checked : e.target.value;

setOrder({ ...order, [name]: value });
}

return (
  <div className="demo"> <h3>⑤ 한 핸들러로 섞어 쓰기</h3> <div>
    이름 <input name="name" value={order.name} onChange={handleChange}
  /> </div> <div> <label> <input type="checkbox" name="takeout" checked=
{order.takeout} onChange={handleChange}
  />
    포장하기
  </label> </div> <div>
    사이즈
    <select name="size" value={order.size} onChange=
{handleChange}> <option value="S">작은 것</option> <option value="M">보통
</option> <option value="L">큰 것</option> </select> </div> <div className="output">
  {order.name} / {order.takeout ? "포장" : "매장"} / {order.size}
</div> </div>
);
}
```

화면 김민준 / 포장 / M

화면(누르면): '포장하기' 를 끄면 → 김민준 / 매장 / M 화면(누르면): 사이즈를 '큰 것' 으로 바꾸면 → 김민준 / 매장 / L

칸이 세 개고 종류도 세 가지인데 핸들러는 하나입니다. select 에도 name 을 붙였습니다. select 도 e.target.name 을 그대로 줍니다.

`true / false` 를 화면에 그대로 쓰면 아무것도 안 보입니다. 그래서 `{order.takeout ? "포장" : "매장"}` 처럼 글자로 바꿔서 보여 줍니다.

▹ 직접 해보기 5

"일회용 컵 안 받기" 체크박스를 하나 더 만들어 보세요. 초기값은 `false` 로 두세요. `handleChange` 는 안 고쳐도 됩니다.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 `//` 를 붙이면 돌아옵니다.

이런 실수를 합니다

체크박스에서 `e.target.value` 를 읽음

```
setAgree(e.target.value);
```

언제나 "on" 이 들어갑니다. 꺾는데도 "on" 입니다. 게다가 "on" 은 truthy 라서 조건문이 늘 참이 됩니다. 에러는 안 납니다. 섹션 1의 데모에서 직접 확인한 그대로입니다.

이런 실수를 합니다

체크박스에 `checked` 만 주고 `onChange` 를 안 붙임

```
<input type="checkbox" checked={agree} />
```

아무리 눌러도 체크가 안 됩니다. 개념01 섹션 3과 완전히 같은 일입니다. 콘솔에 이런 경고가 나옵니다. You provided a `checked` prop to a form field without an `onChange` handler. ... value 자리에 `checked` 만 들어갔을 뿐 나머지는 똑같은 문장입니다.

이런 실수를 합니다

option 에 `selected` 를 씌

```
<option value="M" selected>보통</option>
```

React 가 콘솔에 이렇게 알려 줍니다. Use the `defaultValue` or `value` props on `<select>` instead of setting `selected` on `<option>`. "option 에 `selected` 를 붙이지 말고 select 에 `value` 를 주세요" 라는 뜻입니다.

이런 실수를 합니다

체크를 풀 때 배열에서 안 뺌

```
setPicked([...picked, value]); ← if 없이 이 줄만 씌
```

꺾다 꺾다 할 때마다 같은 토핑이 계속 쌓입니다. "고른 것: 시럽, 시럽, 시럽" 이 되고 개수도 계속 늘어납니다. 에러는 안 납니다. 커짐/꺼짐을 `if` 로 갈라 줘야 합니다.

이런 실수를 합니다

체크박스 여러 개에 value 를 안 적음

```
<input type="checkbox" checked={...} onChange={handleChange} />
```

e.target.value 가 전부 "on" 이라 어느 것을 눌렀는지 구분이 안 됩니다. 배열에 "on" 만 쌓입니다.

이런 실수를 합니다

JSX 속성 사이에 쉼표를 찍음 [SyntaxError]

```
<input type="checkbox", checked={agree} />
```

JSX 속성은 쉼표 없이 띄어쓰기로만 나열합니다. 쉼표를 찍으면 Babel 이 파일 전체를 못 읽어 화면이 통째로 빕니다.

화면 조립

정리

- 1 체크박스는 value 가 아니라 **checked** 를 씁니다. 값도 e.target.checked 로 읽습니다 (true / false).
- 2 체크박스의 e.target.value 는 켜도 꺼도 늘 같습니다. 안 적으면 "on" 입니다.
- 3 라디오는 value 와 checked={조건} 를 함께 씁니다. state 는 고른 값 하나면 됩니다.
- 4 select 는 option 이 아니라 **select 에 value** 를 줍니다. 고른 값은 option 의 value 입니다.
- 5 여러 개 체크는 **배열 state** 로 다룹니다. 켜면 스프레드로 더하고, 끄면 filter 로 뺍니다.
- 6 e.target.type 으로 갈라 주면 글자 칸과 체크박스를 한 핸들러로 처리할 수 있습니다.

```
export default function Concept03() {
  return (
    <div> <h1>개념 03 - 체크박스와 선택</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br /> <strong>직접 눌러 보는 파일입니다.</strong> 체크박스를 켜다
      켜다 하고, 목록을 이것저것 골라 보세요.
    </p> <div> <AgreeCheck /> <SizePicker /> <MenuSelect /> <ToppingPicker
  /> <OrderForm /> </div> </div>
  );
}
```


직접 해보기 정답

```
1) <button disabled={!agree}>{agree ? "주문하기" : "동의를 필요합니다"}</button>
// 화면: 동의가 필요합니다 (체크 전)
// 화면(누르면): 체크하면 → 주문하기
```

→ state 하나가 버튼의 글자와 눌림 여부를 함께 정합니다.

2) SIZES 배열에 한 줄만 더합니다. { value: "XL", label: "아주 큰 것" },

```
// 화면: 라디오가 네 개가 됩니다. '아주 큰 것' 을 고르면 → 고른 사이즈: XL
```

→ map 이 배열을 돌면서 그리므로 화면 코드는 손덜 것이 없습니다.

3) MENU_PRICE 에 삼각김밥: 1200, 을 넣고 <option value="삼각김밥">삼각김밥</option> 을 추가합니다.

```
// 화면(누르면): '삼각김밥' 을 고르면 → 삼각김밥 - 1200원
```

→ option 의 value 와 MENU_PRICE 의 키가 같아야 합니다.

다르면 가격 자리가 비어 버립니다.

```
4) const TOPPINGS = ["시럽", "휘핑크림", "샷 추가", "얼음 많이"];
// 화면: 체크박스가 네 개가 됩니다.
// 화면(누르면): '얼음 많이' 를 켜면 → 고른 것: 시럽, 얼음 많이 / 모두 2개
```

→ handleChange 도 배열 state 도 그대로입니다.

5) useState 초기값에 noCup: false 를 넣고 아래 한 줄을 추가합니다.

```
<label><input type="checkbox" name="noCup" checked={order.noCup}>
```

```
onChange={handleChange} /> 일회용 컵 안 받기</label>
```

```
// 화면(누르면): 켜면 order 의 noCup 이 true 가 됩니다.
```

→ e.target.type 이 "checkbox" 라서 handleChange 가 알아서 checked 를 읽습니다.

콘솔의 [주문] 줄에 noCup 이 함께 찍히는지 보세요.

form submit

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

이 파일은 JS자료 11단원 개념04 와 하는 이야기가 같습니다.

```
form.addEventListener("submit", (e) => {
  e.preventDefault();
  ...
});
```

React 에서도 그대로입니다. 붙이는 문법만 바뀝니다.

```
<form onSubmit={handleSubmit}>
```

그러니 이 파일에서 진짜로 새로 배우는 것은 많지 않습니다. 대신 JS자료에서 짚었던 함정들이 React 에서도 그대로 있는지 직접 확인합니다.

이 파일에서 하는 일 ① `onSubmit` 과 `e.preventDefault()` ② form 안 버튼의 type — 안 적으면 제출 버튼이 됩니다 (직접 재현) ③ 검사하고 예러 문구 보여 주기 ④ 제출한 뒤 정리하기

onSubmit 과 preventDefault

섹션 1

1 입력칸에서 Enter 만 쳐도 제출됩니다

2 버튼 클릭이든 Enter 든 받는 곳이 `onSubmit` 한 군데입니다

버튼에 `onClick` 을 붙이는 대신 form 에 `onSubmit` 을 붙이세요. 그래야 Enter 를 친 사람도 같은 대접을 받습니다.

`e` 는 지금까지와 같은 이벤트 객체입니다. `e.preventDefault()` 도 그대로입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function GreetForm() {
  const [name, setName] = useState("김민준");
  const [message, setMessage] = useState("아직 제출 전입니다");

  function handleSubmit(e) {
    e.preventDefault(); // * 이 줄이 없으면 페이지가
    새로고침됩니다 console.log("submit 실행됨:", name);
  }
}
```

F12 콘솔 submit 실행됨: 김민준

```
setMessage(name + "님 환영합니다");
}

return (
  <div className="demo"> <h3>① onSubmit - 버튼과 Enter 가 같은 곳으로
</h3> <form onSubmit={handleSubmit}> <input value={name} onChange={(e) =>
setName(e.target.value)} />
    <button type="submit">확인</button> </form> <div className="output">{message}
</div> </div>
);
}
```

화면 아직 제출 전입니다

화면(누르면): '확인' 을 누르면 → 김민준님 환영합니다

이번에는 버튼을 누르지 말고, 입력칸을 클릭한 뒤 Enter 만 쳐 보세요.

화면 김민준님 환영합니다 ← 버튼을 누른 것과 똑같습니다

브라우저가 Enter 를 받으면 제출 버튼을 대신 눌러 주기 때문입니다. 그래서 onSubmit 한 곳만 만들어 두면 두 가지 길이 모두 여기로 모입니다.

e.preventDefault() 를 빼면 어떻게 될까요? form 의 기본 동작은 "입력값을 서버로 보내고 페이지를 새로고침" 입니다. React 화면에서 새로고침이 일어나면 state 가 전부 처음 값으로 돌아갑니다. 지금까지 쳐 놓은 것이 통째로 날아갑니다. "버튼을 눌렀는데 화면이 초기화돼요" 의 원인은 거의 항상 이것입니다.

form 을 쓰면 preventDefault 를 반사적으로 같이 쓴다고 생각하세요.

▹ 직접 해보기 1

글로 읽으면 안 와닿습니다. 10초면 됩니다.

① 위 `e.preventDefault()`; 줄 맨 앞에 `//` 를 붙여 주석 처리합니다. ② 저장하고 왼쪽에서 다른 예제를 골랐다 돌아온 뒤, 입력칸을 "이서연" 으로 바꾸고 '확인' 을 누릅니다. ③ 무슨 일이 일어나는지, 주소창이 어떻게 되는지 보세요. ④ 확인했으면 `//` 를 다시 지워 놓으세요.

form 안 버튼의 type

섹션 2

JS자료 11단원 개념04 섹션4 에서 본 함정입니다. React 에서도 똑같습니다.

```
<button>지우기</button>           → type="submit" 과 같습니다
<button type="submit">제출</button> → 제출합니다
<button type="button">지우기</button> → 제출하지 않습니다
```

form 안의 button 은 type 을 안 적으면 기본이 submit 입니다. 그래서 "지우기" 나 "취소" 버튼에 `type="button"` 을 안 붙이면 그 버튼도 폼을 제출해 버립니다.

말로만 하면 안 믿깁니다. 아래에서 직접 세어 보세요. 버튼을 누를 때마다 무슨 일이 몇 번 일어났는지 숫자로 보여 줍니다.

```
function ButtonTypeDemo() {
  const [submitCount, setSubmitCount] = useState(0);
  const [clearCount, setClearCount] = useState(0);

  function handleSubmit(e) {
    e.preventDefault();

    console.log("[type] 폼이 제출됐습니다");
  }
}
```

F12 콘솔 [type] 폼이 제출됐습니다

04단원 개념05 의 함수형 갱신입니다. 한 번 누를 때 두 가지 일이 겹쳐 일어나므로 이 방식이 안전합니다.

```
setSubmitCount((prev) => prev + 1);
}

function handleClear() {
  setClearCount((prev) => prev + 1);
}

return (
  <div className="demo"> <h3>② 버튼의 type - 세 개를 차례로 눌러 보세요
  </h3> <form onSubmit={handleSubmit}> <button type="submit">제출
  </button> <button type="button" onClick={handleClear}>
    지우기 (type="button")
    </button> <button onClick={handleClear}>지우기 (type 없음)
  </button> </form> <div className="output">
```

```

        제출 {submitCount}번 / 지우기 {clearCount}번
      </div> </div>
    );
  }

```

화면 제출 0번 / 지우기 0번

화면(누르면): '제출' → 제출 1번 / 지우기 0번 화면(누르면): '지우기 (type="button")' → 제출 1번 / 지우기 1번 화면(누르면): '지우기 (type 없음)' → 제출 2번 / 지우기 2번

★ 마지막을 보세요. 지우기만 눌렀는데 제출 숫자까지 하나 올라갑니다. type 을 안 적어서 이 버튼도 제출 버튼이 된 것입니다. onClick 이 먼저 실행되고, 이어서 폼이 제출됩니다.

실제 앱에서는 이렇게 나타납니다. "지우기를 눌렀는데 저장까지 돼요" "취소를 눌렀는데 화면이 새로고침돼요" (preventDefault 도 없을 때)

form 안에 버튼을 넣을 때는 이렇게 생각하세요. 제출하는 버튼은 딱 하나. 나머지는 전부 type="button".

◀ 직접 해보기

2

세 번째 버튼에 type="button" 을 붙여 보세요. 누를 때 제출 숫자가 안 올라가는지 확인하세요.

06

검사하고 에러 문구 보여 주기

섹션 3

제출을 받으면 값이 쓸 만한지 먼저 봅니다. 안 맞으면 에러 문구를 state 에 넣고 그 자리에서 return 합니다. JS자료 11단원 개념04 의 초기 반환 패턴 그대로입니다.

다른 점은 하나뿐입니다.

```

JS자료  errorMsg.textContent = "...";   ← 화면을 직접 고쳤습니다
React    setError("...")                ← state 만 바꿉니다

```

화면에 문구를 그리는 일은 return 안의 {error} 가 알아서 합니다.

trim() 을 쓰는 이유도 같습니다. 스페이스만 세 칸 친 " " 는 빈 문자열이 아니라서 if (!value) 로는 안 걸립니다.

```

function JoinForm() {
  const [form, setForm] = useState({ name: "", phone: "" });
  const [error, setError] = useState("");
  const [result, setResult] = useState("아직 제출 전입니다");

  function handleChange(e) {
    setForm({ ...form, [e.target.name]: e.target.value });
  }
}

```

입력을 목록에 담기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

05단원에서는 이미 만들어져 있는 배열을 `map` 으로 그렸습니다. 개념04 에서는 제출한 값 하나를 화면에 보여줬습니다. 이 파일에서 둘을 붙입니다. 사용자가 친 글을 배열에 쌓고, 그 배열을 그립니다.

이게 되면 할 일 목록·장바구니·메모장 같은 화면의 절반이 완성됩니다. JS자료 13단원 종합03 에서 만든 할 일 목록을 떠올리면서 보세요.

이 파일에서 하는 일 ① 입력한 글을 배열에 더하기 — 스프레드로 새 배열 만들기 ② 더한 뒤 입력칸 비우기 · 개수 세기 · 비었을 때 안내하기 ③ 공백만 친 것과 이미 있는 것 걸러내기 ④ 항목마다 id 를 붙여 담기

★ 미리 한 가지만 말해 둡니다. 배열에 값을 더할 때 `push` 를 쓰지 않습니다. 스프레드로 새 배열을 만듭니다. 왜 그래야 하는지는 07단원 개념01 에서 실제로 재현해 보며 설명합니다. 여기서는 이 모양만 눈에 익히세요.

입력한 글을 배열에 더하기

섹션 1

필요한 state 는 두 개입니다.

```
const [text, setText] = useState("");    입력칸에 지금 쳐 놓은 글자
const [todos, setTodos] = useState([]);  지금까지 쌓인 목록
```

둘은 하는 일이 다릅니다. `text` 는 아직 확정되지 않은 값입니다. 칠 때마다 바뀝니다. `todos` 는 제출해서 확정된 값들입니다. 제출할 때만 바뀝니다.

더하는 줄은 이렇습니다.

```
setTodos([...todos, text]);
```

`...todos, text` · 는 09단원 스프레드입니다.

"`todos` 안의 것들을 그대로 펼쳐 담고, 그 뒤에 `text` 를 하나 더 담은 새 배열" 이라는 뜻입니다. 원래 `todos` 는 손대지 않습니다.

목록을 그리는 것은 05단원 개념02 의 `map` 그대로입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function AddOnly() {
  const [text, setText] = useState("우유 사기");
  const [todos, setTodos] = useState(["장보기"]);

  function handleSubmit(e) {
    e.preventDefault();

    console.log("[할일] 추가:", text);
  }
}
```

F12 콘솔 [할일] 추가: 우유 사기

```
setTodos([...todos, text]);
}

return (
  <div className="demo"> <h3>① 목록에 더하기 (입력칸은 그대로)
</h3> <form onSubmit={handleSubmit}> <input value={text} onChange={(e) =>
setText(e.target.value)} />
  <button type="submit">추가</button> </form> <ul>
    {todos.map((todo, index) => (
      <li key={index}>{todo}</li>
    ))}
  </ul> </div>
);
}
```

화면 목록에 '장보기' 한 줄이 있고, 입력칸에는 '우유 사기' 가 들어 있습니다.

화면(누르면): '추가' 를 누르면 목록이 두 줄이 됩니다.

장보기
우유 사기

그런데 입력칸에는 '우유 사기' 가 그대로 남아 있습니다. 한 번 더 누르면 '우유 사기' 가 또 들어갑니다. 불편합니다. 그 부분은 섹션 2에서 고칩니다.

key 자리에 index 를 썼습니다. 05단원 개념03 에서 "index 를 key 로 쓰면 문제가 생긴다" 고 배웠습니다. 그 문제는 항목이 지워지거나 순서가 바뀔 때 생깁니다. 지금 이 목록은 뒤에 붙이기만 하므로 아직 괜찮습니다. 삭제가 생기는 순간 문제가 됩니다. 그래서 섹션 4에서 id 로 바꿉니다.

▹ 직접 해보기 1

초기 목록을 ["장보기", "빨래하기"] 로 바꿔 보세요. 왼쪽에서 다른 예제를 골랐다 돌아오면 처음부터 두 줄이 보입니다.

더한 뒤 정리하기

섹션 2

추가가 끝나면 입력칸을 비웁니다. 개념04 섹션4 에서 한 것과 같습니다.

```
setText("");
```

한 화면에서 set 함수를 두 번 부르고 있는데 괜찮습니다. React 는 둘을 모아서 한 번에 처리하고, 화면은 한 번만 다시 그립니다.

목록이 비었을 때 안내 문구도 넣어 줍니다. 05단원 개념05 그대로입니다. 여기서 조심할 것이 하나 있습니다.

```
{memos.length && <p>...</p>}      ← 0 이 화면에 찍히는 함정  
{memos.length === 0 && <p>...</p>} ← 안전합니다
```

아래에서는 안전한 쪽을 썼습니다. === 0 은 true 아니면 false 니까요.

```
function MemoBoard() {  
  const [text, setText] = useState("장보기 메모");  
  const [memos, setMemos] = useState([]);  
  
  function handleSubmit(e) {  
    e.preventDefault();  
  
    setMemos([...memos, text]);  
    setText(""); // * 입력칸 비우기 console.log(`[메모] 모두 ${memos.length + 1}개`);  
  }  
  
  return (  
    <div className="demo"> <h3>② 더하고 입력칸 비우기</h3> <form onSubmit=  
      {handleSubmit}> <input value={text} onChange={(e) => setText(e.target.value)}  
        placeholder="메모를 쓰고 Enter"  
      />  
      <button type="submit">추가</button> </form> <div className="output">모두  
      {memos.length}개</div>  
      {memos.length === 0 && <div className="output">아직 메모가 없습니다</div>}
```

F12 콘솔 [메모] 모두 1개

```
  }  
  
  return (  
    <div className="demo"> <h3>② 더하고 입력칸 비우기</h3> <form onSubmit=  
      {handleSubmit}> <input value={text} onChange={(e) => setText(e.target.value)}  
        placeholder="메모를 쓰고 Enter"  
      />  
      <button type="submit">추가</button> </form> <div className="output">모두  
      {memos.length}개</div>  
      {memos.length === 0 && <div className="output">아직 메모가 없습니다</div>}
```


폼과 입력

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

직접 쳐 보세요. F12 → Console 도 함께.

문제 1 (개념01)

입력칸에 친 글자가 아래 줄에 바로 나오게 하세요. input 에 value 와 onChange 를 붙이면 됩니다.

기대 화면 "김민준" 을 치면 → 입력한 값: 김민준
글자는 써지는데 아래 줄이 안 바뀌면 onChange 만 붙이고
setText 를 안 부른 것입니다.
한 글자도 안 써지면 value 만 붙인 것입니다.

```
import { useState } from "react";

function Q1() {
  const [text, setText] = useState("");

  return (
    <div className="demo"> <h3>문제 1 – 제어 컴포넌트</h3>
      { /* TODO: 아래 input 에 value 와 onChange 를 붙이세요 */ }
      <input placeholder="이름을 입력하세요" /> <div className="output">입력한 값:
      {text}</div> </div>
    );
  }
}
```

문제 2 (개념01)

아래 빈 상자에 지금 친 글자 수를 "3자" 형태로 보여 주세요. input 은 이미 다 되어 있습니다. 상자 안만 채우면 됩니다.

기대 화면 아무것도 안 치면 → 0자
"김민준" 을 치면 → 3자
숫자만 나오면 뒤에 "자" 를 안 붙인 것입니다.

```
function Q2() {
  const [memo, setMemo] = useState("");
```


state 설계와 불변성

OPEN 07_state_설계와_불변성

```
function WrongPushDemo() {  
  function handleWrongAdd() {  
    return (  
      <div className="demo">
```

01 불변하게 바꾸기: 배열

02 불변하게 바꾸기: 객체

03 state 끌어올리기

04 자식이 부모에게 알리기

05 state를 어디에 둘까

- 연습문제

불변하게 바꾸기: 배열

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

06단원 개념05에서 입력한 값을 목록에 담을 때 이렇게 썼습니다.

```
setItems([...items, 새것]);
```

그때 “왜 이렇게 써야 하는지는 07단원에서 설명합니다” 라고 넘어갔습니다. 이제 그 약속을 지킬 차례입니다.

JS자료 06단원에서는 배열에 값을 넣을 때 `push` 를 썼습니다.

```
cart.push("아메리카노");
```

React 에서 이렇게 하면 어떻게 될까요? 에러가 날까요? 에러는 안 납니다. 그런데 화면도 안 바뀝니다. 이 파일은 그것부터 직접 보는 데서 시작합니다.

★ 이 파일에서 배우는 ‘추가 · 삭제 · 수정’ 세 가지 형태는 13단원 종합 프로젝트까지 계속 씁니다. 여기서 손에 익혀 두세요.

push 로 담으면 화면이 안 바뀝니다

섹션 1

아래 컴포넌트는 JS자료에서 배운 대로 `push` 를 씁니다. 그리고 바뀐 배열을 `set` 함수에 그대로 넣습니다. 자연스러워 보입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function WrongPushDemo() {
  const [items, setItems] = useState(["아메리카노"]);
```

이 state 는 “화면을 억지로 다시 그리게 하는” 용도로만 씁니다. 섹션 1의 마지막에서 왜 필요한지 설명합니다.

```
const [redrawCount, setRedrawCount] = useState(0);
```

컴포넌트 몸통에 적은 `console.log` 는 ‘화면을 그릴 때마다’ 찍힙니다. 04단원 개념03에서 렌더링 횟수를 세어 볼 때 쓴 방법입니다.

```
console.log("[데모①] 화면을 그렸습니다. 지금 items.length:", items.length);
```

```
F12 콘솔    [데모①] 화면을 그렸습니다. 지금 items.length: 1
            [데모①] 화면을 그렸습니다. 지금 items.length: 1
```

↑ 개발 중에는 같은 줄이 두 번 찍힙니다(09단원 개념02에서 설명).

```
function handleWrongAdd() {
  items.push("케이크"); // ← JS자료 06단원에서 배운 방법 setItems(items); // ← 바뀐
  배열을 그대로 넣습니다 console.log("[잘못된 방법] push 뒤 items.length:",
  items.length);
}
```

```
F12 콘솔    [잘못된 방법] push 뒤 items.length: 2
```

```
console.log("[잘못된 방법] 배열 안의 값:", items.join(", "));
```

```
F12 콘솔    [잘못된 방법] 배열 안의 값: 아메리카노, 케이크
```

```
}

return (
  <div className="demo">
    <h3>① push 로 담기 - 일부러 고장 난 예제</h3>
    <button onClick={handleWrongAdd}>케이크 담기 (push)</button>
    <button onClick={() => setRedrawCount(redrawCount + 1)}>
      화면 강제로 다시 그리기
    </button>
    <div className="output">담긴 개수: {items.length}</div>
    <ul>
      {items.map((item, index) => (
```

이 목록은 뒤에만 붙습니다. 순서가 안 바뀌므로 index 를 key 로 써도 05단원에서 본 문제가 생기지 않습니다.

```
      <li key={index}>{item}</li>
    )}
  </ul>
</div>
);
}
```

데모 ① 의 '케이크 담기 (push)' 를 세 번 눌러 보세요.

화면(누르면): 아무것도 안 바뀝니다. "담긴 개수: 1" 그대로,

목록도 "아메리카노" 한 줄 그대로입니다.

그런데 콘솔은 다릅니다.

잘못된 방법 · push 뒤 items.length: 2

잘못된 방법 · push 뒤 items.length: 3

잘못된 방법 · push 뒤 items.length: 4

배열은 분명히 늘어났습니다. 화면만 그대로입니다.

콘솔을 더 자세히 보세요. “[데모①] 화면을 그렸습니다” 는 페이지를 열 때 찍히고, 버튼을 아무리 눌러도 더는 안 찍힙니다. (처음에 같은 줄이 두 번 나오는 것은 개발 중 설정 때문입니다 — 09단원 개념02. 여기서 볼 것은 ‘그 뒤로 안 늘어난다’ 는 쪽입니다) React 가 화면을 다시 그리는 일 자체를 하지 않은 것입니다.

이제 ‘화면 강제로 다시 그리기’ 를 눌러 보세요. 이 버튼은 items 와 아무 상관 없는 다른 state 를 바꿉니다. 그것 때문에 컴포넌트가 다시 실행되고, 그제서야

화면(누르면): 담긴 개수: 4 / 아메리카노 · 케이크 · 케이크 · 케이크

가 한꺼번에 나타납니다.

이것이 이 예제의 핵심입니다. · 데이터는 진짜로 바뀌어 있었습니다. · 에러도 경고도 안 났습니다. · 그런데 화면만 안 바뀌었습니다.

에러가 나면 오히려 고치기 쉽습니다. 이걸 조용히 틀립니다. 초보자가 “React 가 이상하다” 고 느끼는 순간의 대부분이 이것입니다.

▫ 직접 해보기 1

데모 ① 의 ‘케이크 담기 (push)’ 를 두 번 누른 뒤, ‘화면 강제로 다시 그리기’ 를 한 번 누르세요. 화면의 ‘담긴 개수’ 가 몇으로 나오는지 확인하세요.

React 는 “같은 것인가” 로 판단합니다

섹션 2

왜 화면을 안 그렸을까요? React 의 판단 기준이 이렇게 때문입니다.

“새로 받은 값이 전에 갖고 있던 값과 같은 것인가?”

같다 → 바뀐 게 없다. 다시 그릴 필요 없다. 아무 일도 안 한다.

다르다 → 바뀌었다. 컴포넌트를 다시 실행해서 화면을 새로 그린다.

여기서 ‘같은 것’ 의 뜻이 중요합니다. 배열과 객체는 ‘내용이 같은가’ 가 아니라 ‘같은 물건인가’ 로 비교합니다. JS자료 09단원에서 본 이야기입니다. 콘솔로 다시 확인해 봅시다.

```
const listA = ["아메리카노", "케이크"];
const listB = listA; // 이름표만 하나 더 붙인 것
const listC = [...listA]; // 내용은 같지만 새로 만든 배열 console.log(listA === listB);
```

F12 콘솔 true

```
console.log(listA === listC);
```

F12 콘솔 false

listC 는 내용이 똑같은데도 false 입니다. 같은 물건이 아니라 '똑같이 생긴 다른 물건' 이기 때문입니다.

비유하면 이렇습니다. listA 와 listB 는 하나의 상자에 이름표 두 개를 붙인 것입니다. listC 는 안에 든 것이 똑같은 새 상자입니다.

비유는 여기까지입니다. 실제로 일어나는 일은 이렇습니다. 배열을 변수에 넣으면 변수에는 '그 배열이 있는 자리' 가 저장됩니다. listB = listA 는 그 자리를 그대로 베낀 것이라 결국 한 배열입니다.

...listA · 는 값을 하나씩 꺼내 새 배열을 만든 것이라 자리가 다릅니다.

push 는 상자 안에 물건을 하나 더 넣는 일입니다. 상자 자체는 그대로입니다.

```
listB.push("라떼");

console.log(listA.length);
```

F12 콘솔 3

```
console.log(listA.join(", "));
```

F12 콘솔 아메리카노, 케이크, 라떼

```
console.log(listA === listB);
```

F12 콘솔 true

listB 에 넣었는데 listA 도 늘었습니다. 애초에 한 배열이니 당연합니다. 그리고 push 한 뒤에도 listA === listB 는 여전히 true 입니다.

섹션 1에서 벌어진 일이 정확히 이것입니다.

```
items.push("케이크"); ← 안에 든 것만 바뀔. 배열 자체는 같은 것
setItems(items);      ← React: "전이랑 같은 배열이네. 할 일 없음"
```

React 는 배열을 하나하나 열어 보지 않습니다. 그건 값이 많을 때 너무 느립니다. 대신 '같은 것인지' 만 한번 비교합니다. 그래서 아주 빠릅니다. 그 대신 우리가 지켜야 할 약속이 하나 생깁니다.

★ state 를 바꿀 때는 원래 것을 고치지 말고 '새것' 을 만들어서 넣는다.

이것을 '불변하게 다룬다' 고 합니다. 이 단원의 제목이기도 합니다.

▫ 직접 해보기

2

아래 두 줄의 결과를 먼저 예상한 뒤 콘솔로 확인하세요. `const x = [1, 2]; console.log(x === [1, 2]);`

추가 — [...items, 새것]

섹션 3

이제 제대로 된 방법입니다. 새 배열을 만들어서 넣습니다.

```
setCart([...cart, 새것]);
```

JS자료 09단원 개념03 섹션6에서 본 그 문법입니다. 대괄호를 새로 열었으니 결과는 반드시 '새 배열' 입니다.

항목마다 불일 번호입니다. 화면에 보이는 값이 아니므로 state 가 아니어도 됩니다. (04단원에서 본 것처럼 그냥 변수는 바뀌어도 화면을 다시 그리지 않습니다. 여기서는 그게 오히려 알맞습니다. 번호는 화면에 안 나오니까요.)

```
let nextCartId = 3;

function AddDemo() {
  const [cart, setCart] = useState([
    { id: 1, name: "아메리카노", price: 4000 },
    { id: 2, name: "케이크", price: 6000 },
  ]);

  function handleAdd(menuName, menuPrice) {
    const nextCart = [...cart, { id: nextCartId, name: menuName, price: menuPrice }];
    nextCartId = nextCartId + 1;

    console.log("[담기] 새 배열인가?", nextCart !== cart);
  }
}
```

F12 콘솔 [담기] 새 배열인가? true

```
setCart(nextCart);
}

return (
  <div className="demo"> <h3>② 담기 - 스프레드로 새 배열 만들기
  </h3> <button onClick={() => handleAdd("라떼", 4500)}>라떼 담기
```



```

</button> <button onClick={() => handleAdd("삼각김밥", 1200)}>삼각김밥 담기
</button> <div className="output">담긴 개수: {cart.length}</div> <ul>
  {cart.map((item) => (
    <li key={item.id}>
      {item.name} - {item.price}원
    </li>
  ))}
</ul> </div>
);
}

```

화면(누르면): '라떼 담기' 를 누르면 목록에 "라떼 — 4500원" 이 바로 붙고

'담긴 개수' 도 2 에서 3 으로 올라갑니다.

데모 ① 과 딱 한 줄이 다릅니다.

```

items.push("케이크"); setItems(items);    ← 안 됨
setCart([...cart, 새것]);                  ← 됨

```

앞은 '같은 배열의 안을 고친 것' 이고, 뒤는 '새 배열을 만든 것' 입니다.

여기서는 항목을 객체로 만들었습니다. { id, name, price } 세 가지를 담습니다. id 를 따로 두는 이유는 두 가지입니다. · 05단원에서 배운 key 로 쓰기 위해서 · 섹션 4·5에서 "어느 항목인지" 를 가리키기 위해서 같은 메뉴를 두 번 담아도 id 는 다르므로 서로 구별됩니다.

앞에 붙이고 싶으면 순서만 바꾸면 됩니다.

```
setCart([새것, ...cart]);
```

▹ 직접 해보기 3

AddDemo 의 `return` 안에 버튼을 하나 더 만들어 "아메리카노" 4000원을 담을 수 있게 하세요.

▹ 직접 해보기 4

`handleAdd` 의 첫 줄을 아래처럼 바꾸고 눌러 보세요. `const nextCart = cart; nextCart.push({ id: nextCartId, name: menuName, price: menuPrice });` 화면이 어떻게 되는지, 콘솔의 '새 배열인가?' 가 무엇으로 바뀌는지 확인한 뒤 원래대로 되돌리세요.

삭제 — filter

섹션 4

JS자료 06단원에서 배열에서 값을 뺄 때 `splice` 를 썼습니다. `splice` 도 `push` 와 마찬가지로 원본을 바꿉니다. 그래서 React 에서는 안 씁니다.

대신 filter 를 씁니다. filter 는 조건에 맞는 것만 골라 '새 배열' 을 돌려줍니다(JS자료 08단원). 원본은 건드리지 않습니다.

```
setCart(cart.filter((item) => item.id !== 지울id));
```

읽는 법: "id 가 지울 id 와 다른 것만 남긴 새 배열" 지울 것을 고르는 게 아니라 '남길 것' 을 고른다는 점이 헛갈리기 쉽습니다.

```
function RemoveDemo() {
  const [cart, setCart] = useState([
    { id: 1, name: "아메리카노", price: 4000 },
    { id: 2, name: "케이크", price: 6000 },
    { id: 3, name: "라떼", price: 4500 },
  ]);

  function handleRemove(targetId) {
    const nextCart = cart.filter((item) => item.id !== targetId);

    console.log("[빼기] 지운 뒤 개수:", nextCart.length, "/ 원본 개수:",
      cart.length);
  }
}
```

F12 콘솔 [빼기] 지운 뒤 개수: 2 / 원본 개수: 3

```
console.log("[빼기] filter 가 돌려준 것은 새 배열인가?", nextCart !== cart);
```

F12 콘솔 [빼기] filter 가 돌려준 것은 새 배열인가? true

```
setCart(nextCart);
}

return (
  <div className="demo"> <h3>③ 빼기 - filter 로 새 배열 만들기</h3>
    {cart.length === 0 ? (
      <div className="output">장바구니가 비었습니다</div>
    ) : (
      <ul>
        {cart.map((item) => (
          <li key={item.id}>
            {item.name}
            <button onClick={() => handleRemove(item.id)}>빼기</button> </li>
        ))}
      </ul>
    )}
  </div>
);
}
```

화면(누르면): '케이크' 옆 빼기를 누르면 그 줄만 사라집니다.

전부 빼면 "장바구니가 비었습니다" 가 나옵니다(05단원 개념05).

콘솔의 '지운 뒤 개수: 2 / 원본 개수: 3' 을 눈여겨보세요. filter 를 부른 뒤에도 원본 cart 는 여전히 3개입니다. 원본을 안 건드립니다. splice 였다면 원본 개수도 함께 2 로 줄어듭니다.

버튼에 () ⇒ handleRemove(item.id) 로 감싼 이유가 기억나시나요?

onClick={handleRemove(item.id)} 라고 쓰면 화면을 그리는 순간 실행됩니다.

04단원 개념01에서 본 "괄호를 붙이면 안 되는 이유" 입니다. 인자를 넘겨야 할 때는 이렇게 화살표 함수로 감쌉니다.

▫ 직접 해보기

5

handleRemove 의 !== 를 === 로 바꿔서 눌러 보세요. 화면이 어떻게 되는지 보고 원래대로 되돌리세요.

수정 — map

섹션 5

목록 안의 항목 하나만 고치고 싶을 때가 있습니다. 장바구니라면 수량 늘리기, 할일목록이라면 완료 표시입니다.

이때는 map 을 씁니다. map 도 새 배열을 돌려줍니다(JS자료 08단원).

```
setCart(cart.map((item) => (item.id === 고칠id ? 고친것 : item)));
```

읽는 법: "고칠 id 면 고친 것으로, 아니면 원래 것 그대로" 고칠 것 하나만 새로 만들고 나머지는 그대로 둡니다.

'고친 것' 은 이렇게 만듭니다.

```
{ ...item, qty: item.qty + 1 }
```

JS자료 09단원 개념04 섹션5에서 배운 조합입니다. "item 을 복사해서 qty 만 바꾼 새 객체" 라는 뜻입니다. 객체 state 자체를 다루는 이야기는 다음 파일(개념02)에서 자세히 합니다.

```
function UpdateDemo() {
  const [cart, setCart] = useState([
    { id: 1, name: "아메리카노", price: 4000, qty: 1 },
    { id: 2, name: "케이크", price: 6000, qty: 1 },
  ]);

  function handlePlus(targetId) {
    const nextCart = cart.map((item) =>
      item.id === targetId ? { ...item, qty: item.qty + 1 } : item
    );
```

누른 줄과 안 누른 줄이 각각 어떻게 되었는지 확인합니다. find 는 조건에 맞는 첫 항목을 돌려줍니다 (JS자료 08단원 개념04).

```
const beforeItem = cart.find((item) => item.id === targetId);
const afterItem = nextCart.find((item) => item.id === targetId);

console.log("[수량] 누른 줄은 새 객체인가?", afterItem !== beforeItem);
```

F12 콘솔 [수량] 누른 줄은 새 객체인가? true

```
const otherBefore = cart.filter((item) => item.id !== targetId);
const otherAfter = nextCart.filter((item) => item.id !== targetId);

console.log("[수량] 안 누른 줄은 그대로인가?", otherAfter[0] === otherBefore[0]);
```

F12 콘솔 [수량] 안 누른 줄은 그대로인가? true

```
setCart(nextCart);
}

return (
  <div className="demo"> <h3>④ 수량 늘리기 - map 으로 한 줄만 바꾸기</h3> <ul>
    {cart.map((item) => (
      <li key={item.id}>
        {item.name} {item.qty}개
        <button onClick={() => handlePlus(item.id)}>+1</button> </li>
      )))}
    </ul> </div>
);
}
```

화면(누르면): 아메리카노 옆 +1 을 누르면 "아메리카노 2개" 가 됩니다.

케이크 줄은 그대로입니다.

콘솔 두 줄이 map 이 하는 일을 그대로 보여 줍니다. 누른 줄은 새 객체가 되었고, 안 누른 줄은 원래 객체를 그대로 다시 썼습니다. 목록 전체를 새로 만드는 것이 아니라, 필요한 줄만 새로 만드는 것입니다. 물론 목록 (배열) 자체는 map 이 새로 만들어 주므로 화면은 다시 그려집니다.

★ 여기까지가 이 파일의 핵심 세 줄입니다. 외우려 하지 말고 형태를 익히세요.

```
추가  setCart([...cart, 새것])
삭제  setCart(cart.filter((item) => item.id !== 지울id))
수정  setCart(cart.map((item) => (item.id === 고칠id ? { ...item, 바꿀것 } : item)))
```

13단원 종합 프로젝트의 할일목록과 장바구니가 이 세 줄로 굴러갑니다.

▹ 직접 해보기

6

UpdateDemo 에 '-1' 버튼을 만들어 수량을 1 줄이세요. (수량이 0 아래로 내려가는 것은 신경 쓰지 않아도 됩니다)

원본을 바꾸는 메소드와 안 바꾸는 메소드

섹션 6

지금까지 배운 배열 메소드는 두 종류로 나뉩니다. React 에서는 이 구분이 아주 중요합니다.

— 원본을 바꾼다 — state 에 그냥 쓰면 안 됨

```
push · pop · unshift · shift · splice · sort · reverse
```

— 새 배열을 돌려준다 — 그대로 써도 됨

```
map · filter · slice · concat · [...배열]
```

외우기 어렵다면 이렇게 확인하세요. 그 메소드의 결과를 변수에 담아 쓰는지, 아니면 부르는 것만으로 끝나는지 봅니다.

```
const 결과 = 배열.filter(...) → 결과를 받아 쓴다. 새 배열이다.
배열.push(...) → 결과를 안 받는다. 원본을 고친 것이다.
```

sort 가 특히 함정입니다. 정렬은 '보기 좋게 바꾸는 일' 같아서 원본을 바꾼다는 생각이 잘 안 듭니다. 하지만 바꿉니다. 그래서 항상 복사한 뒤에 정렬합니다.

```
const priceList = [6000, 4000, 4500];

const sortedCopy = [...priceList].sort((a, b) => a - b);
console.log(sortedCopy.join(", "));
```

F12 콘솔 4000, 4500, 6000

```
console.log(priceList.join(", "));
```

F12 콘솔 6000, 4000, 4500

...**priceList** · 로 새 배열을 만든 다음 그것을 정렬했습니다.

원본 priceList 는 그대로입니다.

```
function SortDemo() {
  const [cart, setCart] = useState([
    { id: 1, name: "케이크", price: 6000 },
    { id: 2, name: "아메리카노", price: 4000 },
    { id: 3, name: "라떼", price: 4500 },
  ]);

  function handleSort() {
```

...**cart** · 를 빼먹으면 원본 배열을 정렬한 뒤 그것을 다시 넣게 됩니다.

그러면 섹션 1과 똑같은 일이 벌어집니다. 화면이 안 바뀝니다.

```
const sorted = [...cart].sort((a, b) => a.price - b.price);
setCart(sorted);
}

return (
  <div className="demo"> <h3>⑤ 정렬 - 복사한 뒤에 sort</h3> <button onClick=
{handleSort}>싼 것부터 정렬</button> <ul>
    {cart.map((item) => (
      <li key={item.id}>
        {item.name} - {item.price}원
      </li>
    ))}
  </ul> </div>
);
}
```

화면(누르면): 아메리카노 4000 → 라떼 4500 → 케이크 6000 순서로 바뀝니다.

⇒ 직접 해보기

7

handleSort 의 [...cart] 를 cart 로 바꿔서 눌러 보세요. 화면이 어떻게 되는지 확인한 뒤 되돌리세요.

위에서 만든 데모 다섯 개를 화면에 붙입니다. 01단원에서 본 createRoot / render 두 줄입니다.

정리

- 1 React 는 **이전 값과 같은 것**으로 다시 그릴지 정합니다. 내용을 하나하나 비교하지 않습니다.
- 2 그래서 `push` 로 배열 안을 고치고 같은 배열을 넣으면 **에러 없이 화면만 안 바뀝니다.**
- 3 추가는 `setCart([...cart, 새것])`
- 4 삭제는 `setCart(cart.filter((item) => item.id !== 지울id))`
- 5 수정은 `setCart(cart.map((item) => (item.id === 고칠id ? { ...item, 바꿀것 } : item)))`
- 6 `sort-reverse-splice` 는 원본을 바꿉니다. `[...cart]` 로 복사한 뒤에 쓰세요.
- 7 이 세 가지 형태는 13단원 종합 프로젝트까지 계속 씁니다.

```
export default function Concept01() {
  return (
    <div> <h1>개념 01 - 불변하게 바꾸기: 배열</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br />
      이 파일의 데모 ① 은 <strong>일부러 고장 난 예제</strong>입니다. 버튼을
      눌러도 화면이 안 바뀌는 것이 정상입니다. 왜 그런지가 이 단원의 시작입니다.
      </p> <div> <WrongPushDemo /> <AddDemo /> <RemoveDemo /> <UpdateDemo
      /> <SortDemo /> </div> </div>
    );
  }
}
```

자주 하는 실수

섹션 7

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

push 한 뒤 같은 배열을 넣기 — 섹션 1의 데모 ① 입니다

에러도 경고도 안 납니다. 화면만 조용히 안 바뀝니다. “버튼이 안 먹는다” 싶으면 이것부터 의심하세요.

이런 실수를 합니다

스프레드를 빼먹기

```
const cartExample = ["아메리카노", "케이크"];
console.log([cartExample, "라떼"].length);
```

F12 콘솔 2

```
console.log([...cartExample, "라떼"].length);
```

F12 콘솔 3

...을 빼면 배열 안에 배열이 통째로 들어갑니다. 길이가 2 입니다. 화면에는 목록이 이상하게 한 줄만 나오거나 아무것도 안 나옵니다.

이런 실수를 합니다

sort · reverse 를 state 에 바로 쓰기

```
setCart(cart.sort(...)); ← cart 를 정렬한 뒤 같은 배열을 넣음  
setCart(cart.reverse()); ← 마찬가지로
```

화면이 안 바뀝니다. sort 와 reverse 는 원본을 바꾸고 '자기 자신' 을 돌려주기 때문입니다. [...cart] 를 앞에 붙이세요.

이런 실수를 합니다

filter 의 조건을 반대로 쓰기

```
const idList = [1, 2, 3];  
console.log(idList.filter((id) => id !== 2).join(", "));
```

F12 콘솔 1, 3

```
console.log(idList.filter((id) => id === 2).join(", "));
```

F12 콘솔 2

filter 는 '남길 것' 을 고릅니다. === 로 쓰면 지우려던 것만 남습니다. 화면에서 하나를 지웠는데 그것만 남으면 이 실수입니다.

이런 실수를 합니다

map 에서 안 고칠 항목을 안 돌려주기

```
const qtyList = [{ id: 1, qty: 1 }, { id: 2, qty: 1 }];  
const brokenList = qtyList.map((item) => {  
  if (item.id === 1) {  
    return { ...item, qty: 2 };  
  }  
});  
console.log(brokenList[1]);
```

F12 콘솔 undefined

else 쪽에서 아무것도 안 돌려주면 그 자리가 `undefined` 가 됩니다. 화면에는 "Cannot read properties of `undefined`" 에러가 납니다. 삼항 연산자로 쓰면 이 실수를 하기 어렵습니다.

이런 실수를 합니다

대괄호 안에서 쉼표를 빼먹기 → **[SyntaxError]**

```
setCart([...cart "라떼"]);
```

... 뒤에 쉼표가 필요합니다. 파일 전체가 안 돌아갑니다.

직접 해보기 정답

1) 담긴 개수: 3 이 됩니다.

```
// 화면: 담긴 개수: 3 / 아메리카노 · 케이크 · 케이크
```

→ push 두 번은 배열을 진짜로 늘렸습니다. 화면만 늦게 따라온 것입니다.

'늦게 따라왔다' 는 말이 무섭게 들린다면 맞습니다. 그래서 안 씁니다.

```
2) const x = [1, 2];
console.log(x === [1, 2]);
// 콘솔: false
```

→ 내용은 같지만 오른쪽은 그 자리에서 새로 만든 다른 배열입니다.

배열끼리의 `===` 는 내용을 안 봅니다.

```
3) <button onClick={() => handleAdd("아메리카노", 4000)}>아메리카노 담기</button>
// 화면(누르면): 목록에 "아메리카노 - 4000원" 이 한 줄 더 생깁니다.
```

→ 이미 있는 아메리카노와 이름이 같아도 id 가 다르므로 따로 그려집니다.

4) 화면이 안 바뀝니다. '담긴 개수: 2' 와 두 줄짜리 목록 그대로입니다.

```
// 콘솔: [담기] 새 배열인가? false
```

→ `nextCart` 와 `cart` 가 같은 배열이 되어 버렸습니다. 섹션 1과 같은 상황입니다.

배열에는 진짜로 들어갔지만 `React` 가 그것을 알 방법이 없습니다.

5) 누른 것 하나만 남고 나머지가 전부 사라집니다.

```
// 화면(누르면): '케이크' 의 빼기를 누르면 케이크만 남습니다.
```

→ `filter` 는 '남길 것' 을 고르는 것이기 때문입니다.

```
6) function handleMinus(targetId) {
```

```
  setCart(cart.map((item) =>  
    item.id === targetId ? { ...item, qty: item.qty - 1 } : item  
  ));
```

```
}
```

그리고 `return` 안에 `<button onClick={() => handleMinus(item.id)}>-1</button>`

```
// 화면(누르면): "아메리카노 1개" 에서 -1 을 누르면 "아메리카노 0개"
```

→ +1 과 부호만 다릅니다. `map` 의 형태는 똑같습니다.

7) 화면이 안 바뀝니다.

```
// 화면(누르면): 케이크 - 6000원 / 아메리카노 - 4000원 / 라떼 - 4500원
```

→ `sort` 는 `cart` 를 그 자리에서 정렬하고 `cart` 자신을 돌려줍니다.

그래서 `setCart` 에 들어가는 것은 '전과 같은 배열' 입니다.

정렬은 이미 끝났는데 화면만 안 따라온 상태입니다.

데모 ① 처럼 다른 `state` 를 하나 바꿔 강제로 다시 그리게 하면
그때 정렬된 모습이 한꺼번에 나타납니다.

불변하게 바꾸기: 객체

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념01에서 배열 state 를 배웠습니다. 정리하면 이랬습니다.

· React 는 '전과 같은 것인지' 로 다시 그릴지 정한다 · 그래서 배열 안을 고치지 말고 새 배열을 만들어 넣는다

객체도 똑같습니다. 문법만 대괄호에서 중괄호로 바꿉니다.

```
배열  setCart([...cart, 새것])
객체  setUser({ ...user, name: "이서연" })
```

그런데 객체에는 배열에 없던 함정이 하나 더 있습니다. 객체 안에 객체가 들어 있을 때입니다. JS자료 09단원 개념04 섹션6에서 본 '얕은 복사' 이야기가 여기서 그대로 나옵니다. 그 함정을 화면으로 직접 보는 것이 이 파일의 목표입니다.

객체 state 를 직접 고치면 화면이 안 바뀝니다

섹션 1

JS자료 07단원에서 객체의 값을 바꿀 때 이렇게 했습니다.

```
user.name = "이서연";
```

React 에서 이렇게 하면 어떻게 될까요? 개념01의 push 와 같은 일이 벌어집니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function WrongObjectDemo() {
  const [user, setUser] = useState({ name: "김민준", age: 20 });
  const [redrawCount, setRedrawCount] = useState(0);

  console.log("[데모①] 화면을 그렸습니다. 지금 이름:", user.name);
```

F12 콘솔 [데모①] 화면을 그렸습니다. 지금 이름: 김민준

```
function handleWrongChange() {
  const beforeObject = user; // 고치기 전의 객체를 기억해 둡니다

  user.name = "이서연"; // ← JS자료 07단원에서 배운 방법 setUser(user); // ← 고친
  객체를 그대로 넣습니다 console.log("[잘못된 방법] 고친 뒤 user.name:", user.name);
}
```

F12 콘솔 [잘못된 방법] 고친 뒤 user.name: 이서연

```
console.log("[잘못된 방법] 전과 같은 객체인가?", beforeObject === user);
```

F12 콘솔 [잘못된 방법] 전과 같은 객체인가? true

```
}

return (
  <div className="demo"> <h3>① 객체를 직접 고치기 - 일부러 고장 난 예제
</h3> <button onClick={handleWrongChange}>이름을 이서연으로</button> <button onClick=
{() => setRedrawCount(redrawCount + 1)}>
  화면 강제로 다시 그리기
</button> <div className="output">
  이름: {user.name} / 나이: {user.age}
</div> </div>
);
}
```

데모 ①의 '이름을 이서연으로'를 눌러 보세요.

화면(누르면): 안 바뀝니다. "이름: 김민준 / 나이: 20" 그대로입니다. 콘솔에는 "고친 뒤 user.name: 이서연"이 찍힙니다.

그리고 '화면 강제로 다시 그리기'를 누르면

화면(누르면): 이름: 이서연 / 나이: 20

이 됩니다. 개념01의 데모 ①과 완전히 같은 상황입니다. 값은 진짜 바뀌어 있었고, 화면만 안 따라온 것입니다.

콘솔의 "전과 같은 객체인가? true"가 이유를 말해 줍니다. 속성을 고쳐도 객체 자체는 그대로입니다. 상자 안의 물건만 바꾼 것입니다. React는 상자만 보고 "전과 같네"하고 넘어갑니다.

▹ 직접 해보기 1

데모 ①에서 '이름을 이서연으로'를 세 번 누른 뒤 '화면 강제로 다시 그리기'를 눌러 보세요. 이름이 무엇으로 나오는지 확인하세요.

{ ...user, 바꿀것 } 으로 새 객체 만들기

섹션 2

제대로 된 방법은 이것입니다. JS자료 09단원 개념04에서 배운 문법 그대로입니다.

```
setUser({ ...user, name: "이서연" });
```

읽는 법: "user 를 통째로 펼쳐 놓고, name 만 새 값으로 덮어쓴 새 객체"

중괄호를 새로 열었으니 결과는 반드시 새 객체입니다. 콘솔로 먼저 확인해 봅시다.

```
const baseUser = { name: "김민준", age: 20 };
const changedUser = { ...baseUser, name: "이서연" };

console.log(changedUser);
```

```
F12 콘솔    { name: '이서연', age: 20 }
```

```
console.log(baseUser);
```

```
F12 콘솔    { name: '김민준', age: 20 }
```

```
console.log(baseUser === changedUser);
```

```
F12 콘솔    false
```

name 만 바뀌었고 age 는 따라왔습니다. 그리고 원본은 그대로입니다. 같은 이름을 뒤에 다시 쓰면 덮어쓴다는 규칙(JS자료 09단원 개념04 섹션3)입니다.

```
function ProfileDemo() {
  const [user, setUser] = useState({ name: "김민준", age: 20, city: "서울" });

  function handleRename() {
    setUser({ ...user, name: "이서연" });
  }

  function handleBirthday() {
    setUser({ ...user, age: user.age + 1 });
  }

  function handleMove() {
```

두 가지를 한 번에 바꿀 수도 있습니다. 뒤에 나란히 적으면 됩니다.

```
setUser({ ...user, city: "대구", age: user.age + 1 });
}

return (
```

state 끌어올리기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

03단원에서 컴포넌트를 여러 개 만들었습니다. 04단원에서 컴포넌트가 자기 state 를 갖는 법을 배웠습니다.

그런데 이런 상황이 생깁니다.

장바구니 목록과 '담긴 개수' 를 각각 다른 컴포넌트로 만들었다.
둘 다 '지금 담긴 것' 을 알아야 한다.

컴포넌트가 각자 state 를 가지면 어떻게 될까요? 이 파일은 그것부터 직접 보고 시작합니다.

각자 state 를 가지면 어긋납니다

섹션 1

아래 두 컴포넌트는 똑같은 목록으로 시작합니다. 그런데 각자 자기 state 를 갖고 있습니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function BrokenCartList() {
  const [items, setItems] = useState(["아메리카노"]);

  function handleAdd() {
    setItems([...items, "라떼"]); // 개념01에서 배운 제대로 된
    방법입니다 console.log("[목록 컴포넌트] 내 items 의 개수:", items.length + 1);
  }
}
```

F12 콘솔 [목록 컴포넌트] 내 items 의 개수: 2

```
  }

  return (
    <div> <button onClick={handleAdd}>라떼 담기</button> <ul>
      {items.map((name, index) => (
        <li key={index}>{name}</li>
      ))}
    </ul> </div>
  );
```

```
}

function BrokenCartCount() {
```

이 컴포넌트도 '자기' items 를 갖고 있습니다. 배열 구조분해에서 뒤쪽을 안 쓸 때는 이렇게 앞만 꺼낼 수 있습니다 (JS자료 09단원 개념01). 여기서는 값을 바꿀 일이 없어서 set 함수를 안 꺼냈습니다.

```
const [items] = useState(["아메리카노"]);

return <div className="output">담긴 개수: {items.length}개</div>;
}

function BrokenCartDemo() {
  return (
    <div className="demo"> <h3>① 각자 state 를 가진 두 컴포넌트 - 일부러 고장 난
예제</h3> <BrokenCartList /> <BrokenCartCount /> </div>
  );
}
```

데모 ①의 '라떼 담기'를 눌러 보세요.

화면(누르면): 목록에는 "라떼"가 붙습니다.

그런데 아래 '담긴 개수'는 1개 그대로입니다.

두 컴포넌트가 서로를 모르기 때문입니다. BrokenCartList의 items와 BrokenCartCount의 items는 이름만 같을 뿐 완전히 다른 두 개의 state입니다.

useState는 '컴포넌트마다' 따로 만들어집니다. 컴포넌트를 두 번 쓰면 state도 두 개 생깁니다. 서로 영향을 안 줍니다. 04단원에서 배운 그대로인데, 그게 여기서는 문제가 됩니다.

이 버그는 화면이 커질수록 무섭습니다. 어떤 곳은 3개라고 하고 어떤 곳은 1개라고 합니다. 무엇이 맞는지 아무도 모릅니다.

▸ 직접 해보기 1

데모 ①의 '라떼 담기'를 세 번 누르고 목록의 줄 수와 '담긴 개수'를 각각 세어 보세요.

공통 부모로 올립니다

섹션 2

답은 간단합니다. 값을 한 군데에만 둡니다. 두 컴포넌트를 함께 담고 있는 '부모'로 state를 옮깁니다. 이것을 state 끌어올리기(lifting state up)라고 부릅니다.

옮기는 순서는 세 단계입니다.

① 두 컴포넌트가 함께 봐야 하는 값을 고른다 → items ② 그 값을 가장 가까운 공통 부모로 옮긴다 → CartApp ③ 자식에게는 props로 내려보낸다 → <CartList items={items} />

자식은 이제 state 를 안 가집니다. 받은 것을 그리기만 합니다.

```
function CartList({ items }) {
```

props 를 매개변수 자리에서 바로 구조분해했습니다(03단원 개념03).

```
  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} - {item.price}원
        </li>
      ))}
    </ul>
  );
}

function CartCount({ items }) {
  return <div className="output">담긴 개수: {items.length}개</div>;
}

let nextCartId = 3;

function CartApp() {
```

★ state 는 여기 한 곳에만 있습니다.

```
const [items, setItems] = useState([
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "케이크", price: 6000 },
]);

function handleAdd() {
  setItems([...items, { id: nextCartId, name: "라떼", price: 4500 }]);
  nextCartId = nextCartId + 1;
}

return (
  <div className="demo"> <h3>② state 를 부모로 올리기</h3> <button onClick=
{handleAdd}>라떼 담기</button> <CartList items={items} /> <CartCount items={items}
/> </div>
);
}
```

화면(누르면): 목록에 "라떼 — 4500원" 이 붙고

같은 순간 '담긴 개수' 도 2개 → 3개가 됩니다.

두 자식이 같은 items 를 받아 그리므로 어긋날 방법이 없습니다. '맞춰 주는 코드' 를 한 줄도 안 썼다는 점이 중요합니다. 값이 하나뿐이니 맞출 것도 없습니다.

자식이 state 를 잃으면서 오히려 좋아진 것이 있습니다. CartCount 는 이제 '어떤 목록이든' 받아서 개수를 보여 줍니다. 장바구니에도 쓰고, 할일목록에도 그대로 쓸 수 있습니다. 자기 데이터를 갖고 있었다면 그 화면에서만 쓸 수 있었습니다.

컴포넌트를 이렇게 두 종류로 보면 도움이 됩니다. 받은 것만 그리는 컴포넌트 — 재사용하기 쉽습니다. 개수를 늘려도 됩니다. state 를 갖고 있는 컴포넌트 — 그 화면의 주인 노릇을 합니다. 한 화면에 주인은 적을수록 좋습니다. 바뀌는 곳이 적다는 뜻이니까요.

담기 버튼은 지금 부모에 있습니다. 자식 안에 있는 버튼으로 부모의 값을 바꾸려면 방법이 하나 더 필요합니다. 그건 개념04에서 배웁니다.

▫ 직접 해보기

2

CartApp 에 <CartCount items={items} /> 를 한 줄 더 쓰세요. 개수 줄이 두 개가 되고, 담을 때 둘 다 같이 바뀝니다.

어디까지 올릴까 — 가장 가까운 공통 부모

섹션 3

"부모로 올린다" 고 했는데, 부모가 여럿이면 어디까지 올려야 할까요? 답은 '그 값을 쓰는 컴포넌트를 전부 담고 있는, 가장 가까운 부모' 입니다.

아래 데모의 구조는 이렇습니다.

```

CartPage      ← items 를 여기에 둡니다
├─ CartHeader (개수를 보여 줌)
└─ CartBody
    └─ CartList (목록을 보여 줌)
  
```

CartHeader 와 CartList 를 둘 다 담고 있는 가장 가까운 부모가 CartPage 입니다. 그래서 items 는 CartPage 에 둡니다.

CartBody 는 items 를 화면에 쓰지 않습니다. 그냥 아래로 넘기기만 합니다. 이렇게 '지나가기만 하는' 컴포넌트가 많아지면 코드가 지저분해집니다. 그 문제를 푸는 방법은 12단원(Context)에서 배웁니다. 두세 단계까지는 props 로 내려보내는 것이 오히려 읽기 쉽습니다.

```
function CartHeader({ items }) {
  return <div className="output">🛒 담긴 물건 {items.length}개</div>;
}

function CartBody({ items }) {
```

자기는 안 쓰고 아래로 넘기기만 합니다.

```
return (
  <div> <CartList items={items} /> </div>
);
}

function CartPage() {
  const [items, setItems] = useState([
    { id: 1, name: "아메리카노", price: 4000 },
  ]);

  function handleAdd() {
    setItems([...items, { id: nextCartId, name: "케이크", price: 6000 }]);
    nextCartId = nextCartId + 1;
  }

  return (
    <div className="demo"> <h3>③ 두 단계 아래까지 내려보내기</h3> <button onClick=
{handleAdd}>케이크 담기</button> <CartHeader items={items} /> <CartBody items={items}
/> </div>
  );
}
```

화면(누르면): 위의 “🛒 담긴 물건 1개” 가 2개가 되고

아래 목록에도 “케이크 - 6000원” 이 함께 붙습니다.

두 단계 떨어져 있어도 값은 하나입니다. 그래서 항상 같이 움직입니다.

반대로 너무 위로 올리면 어떻게 될까요? 예를 들어 이 items 를 페이지 맨 위 App 까지 올리면, items 와 상관없는 컴포넌트들까지 props 를 받아 넘겨야 합니다. ‘가장 가까운’ 공통 부모를 찾는 이유가 이것입니다.

◦ 직접 해보기 3

CartBody 안에 <CartCount items={items} /> 를 추가하세요. (섹션 2에서 만든 컴포넌트를 그대로 쓸 수 있습니다)

안 올려도 되는 state 도 있습니다

섹션 4

모든 state 를 올리면 안 됩니다. 올리는 건 '함께 봐야 하는 값' 만입니다.

아래 데모의 각 줄에는 '설명 보기' 버튼이 있습니다. 어느 줄이 펼쳐져 있는지는 그 줄만 알면 됩니다. 옆줄은 알 필요가 없습니다. 그래서 이 state 는 각 줄 컴포넌트 안에 그대로 둡니다.

```
function CartRow({ item }) {
```

이 state 는 이 줄에만 필요합니다. 올리지 않습니다.

```
const [isOpen, setIsOpen] = useState(false);

return (
  <li>
    {item.name}
    <button onClick={() => setIsOpen(!isOpen)}>
      {isOpen ? "설명 닫기" : "설명 보기"}
    </button>
    {isOpen ? <div className="output">{item.desc}</div> : null}
  </li>
);
}

function RowStateDemo() {
  const [items] = useState([
    { id: 1, name: "아메리카노", desc: "쓴맛이 강한 기본 커피입니다" },
    { id: 2, name: "케이크", desc: "달콤한 디저트입니다" },
  ]);

  return (
    <div className="demo"> <h3>④ 안 올려도 되는 state</h3> <ul>
      {items.map((item) => (
        <CartRow key={item.id} item={item} />
      ))}
    </ul> </div>
  );
}
```

화면(누르면): 아메리카노의 '설명 보기' 를 눌러도 케이크는 닫힌 채입니다.

둘은 서로 모릅니다. 그리고 그게 맞습니다.

판단 기준은 하나입니다.

이 값을 두 곳 이상에서 봐야 하나?
그렇다 → 공통 부모로 올린다
아니다 → 쓰는 컴포넌트 안에 그대로 둔다

미리 올려 두면 나중에 편할 것 같지만 그렇지 않습니다. 위로 올릴수록 props 가 길어지고, 어디서 바뀌는지 찾기 어려워집니다. 필요해질 때 올리면 됩니다. 옮기는 건 어렵지 않습니다.

▹ 직접 해보기 4

CartRow 의 `useState(false)` 를 `useState(true)` 로 바꿔 보세요. 왼쪽에서 다른 예제를 골랐다면 돌아오면 두 줄이 어떤 상태로 시작하는지 보세요.

props 는 자식이 고칠 수 없습니다

섹션 5

“자식도 items 를 갖고 있는데 거기서 바로 고치면 안 되나요?” 안 됩니다. 03단원 개념05에서 배운 대로 props 는 읽기 전용입니다.

그런데 React 는 이것을 막아 주지 않습니다. 자식에서 `props.items.push(...)` 를 하면 에러가 안 납니다. 대신 아주 이상한 일이 벌어집니다. 직접 봅시다.

몰래 담는 것들에 붙일 번호입니다. 05단원에서 배운 key 가 겹치지 않게 하려고 둡니다.

```
let sneakyId = 101;

function SneakyChild({ items }) {
  function handleSneak() {
```

✗ 이렇게 하면 안 됩니다. 부모의 배열을 몰래 고치는 것입니다.

```
items.push({ id: sneakyId, name: "몰래 담긴 것", price: 0 });
sneakyId = sneakyId + 1;

console.log("[자식] 몰래 push 했습니다. 배열 길이:", items.length);
```

F12 콘솔 [자식] 몰래 push 했습니다. 배열 길이: 2

```
}

return <button onClick={handleSneak}>자식이 몰래 담기</button>;
}

function SneakyDemo() {
  const [items, setItems] = useState([
    { id: 1, name: "아메리카노", price: 4000 },
```

```

  });
  const [redrawCount, setRedrawCount] = useState(0);

  return (
    <div className="demo"> <h3>⑤ 자식이 props 를 고치면 - 일부러 고장 난 예제
    </h3> <SneakyChild items={items} /> <button onClick={() => setRedrawCount(redrawCount
    + 1)}>
      화면 강제로 다시 그리기
    </button> <CartList items={items} /> </div>
  );
}

```

데모 ⑤의 '자식이 몰래 담기'를 두 번 누른 뒤 '화면 강제로 다시 그리기'를 눌러 보세요.

화면(누르면): 몰래 담기 두 번 — 화면은 그대로 "아메리카노" 한 줄 화면(누르면): 강제로 다시 그리기 — 갑자기 "몰래 담긴 것" 두 줄이 나타납니다

개념01의 push 문제가 그대로 재현됐습니다. props로 받은 배열은 사실 '부모의 state 배열 그 자체'입니다. 자식이 그것을 고치면 부모 데이터가 오염되는데, 부모는 set 함수를 안 불렀으니 화면을 다시 그릴 생각이 없습니다.

게다가 이 코드는 나중에 읽을 때 최악입니다. "items가 언제 바뀌었지?"를 찾으려면 자식들을 전부 뒤져야 합니다.

"그럼 React가 막아 주면 되잖아요?"라고 생각할 수 있습니다. 자바스크립트에는 "이 객체는 못 고친다"고 표시하는 방법이 있긴 합니다. 그런데 화면을 그릴 때마다 모든 props에 그 처리를 하면 그만큼 느려집니다. 그래서 React는 막지 않습니다. 경고도 안 띄웁니다. 지키는 것은 온전히 우리 몫입니다. 그래서 규칙을 알아 두는 것이 중요합니다.

★ 규칙: state를 바꾸는 것은 그 state를 가진 컴포넌트뿐입니다.

그럼 자식 안의 버튼으로 목록에 담으려면 어떻게 할까요? 부모가 '바꾸는 함수'를 만들어서 props로 내려보내면 됩니다. 다음 파일(개념04)에서 그 방법을 배웁니다.

▸ 직접 해보기 5

데모 ⑤의 SneakyChild에서 items.push(...) 줄을 지우고 console.log만 남겨 보세요. '화면 강제로 다시 그리기'를 눌러도 목록이 그대로인지 확인하세요.

위에서 만든 데모 다섯 개를 화면에 붙입니다.

정리

- 1 `useState` 는 **컴포넌트마다 따로** 만들어집니다. 같은 값을 각자 가지면 어긋납니다.
- 2 두 컴포넌트가 같은 값을 봐야 하면 그 값을 **가장 가까운 공통 부모**로 옮깁니다. 이것이 state 끌어올리기입니다.
- 3 부모는 `props` 로 내려보내고, 자식은 받은 것을 그리기만 합니다.
- 4 값이 한 곳뿐이므로 **어긋날 방법이 없습니다**. 맞추는 코드가 필요 없습니다.
- 5 한 컴포넌트만 쓰는 값(펼침 여부 같은 것)은 **올리지 않습니다**. 필요해질 때 올리면 됩니다.
- 6 `props` 는 읽기 전용입니다. 자식이 고치면 에러 없이 부모 데이터만 오염됩니다.
- 7 자식 안의 버튼으로 부모 값을 바꾸는 방법은 개념04에서 배웁니다.

```
export default function Concept03() {
  return (
    <div> <h1>개념 03 – state 끌어올리기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br />
      데모 ① 과 데모 ⑤ 는 <strong>일부러 고장 난 예제</strong>입니다.
    </p> <div> <BrokenCartDemo /> <CartApp /> <CartPage /> <RowStateDemo
  /> <SneakyDemo /> </div> </div>
  );
}
```

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

같은 값을 두 컴포넌트가 각자 `state` 로 갖기 — 섹션 1의 데모 ① 입니다

에러가 안 납니다. 두 화면이 서로 다른 값을 보여 줄 뿐입니다. “여기는 3개인데 저기는 1개” 라면 이 실수입니다.

이런 실수를 합니다

props 로 받은 값을 자식의 state 초기값으로 복사하기

```
const [myItems, setMyItems] = useState(items);
```

useState 의 초기값은 '처음 한 번' 만 쓰입니다. 부모의 items 가 바뀌어도 자식의 myItems 는 그대로입니다. 이 실수는 개념05에서 화면으로 재현해 봅니다.

이런 실수를 합니다

props 이름을 잘못 적기

```
<CartList item={items} /> ← items 가 아니라 item 이라고 씀
```

자식에서 items 는 undefined 가 됩니다. undefined.map(...) 이 되어 "Cannot read properties of undefined" 에러가 납니다. 에러 문구에 나온 이름이 props 이름과 같은지부터 확인하세요.

이런 실수를 합니다

값을 넘길 때 중괄호를 빠뜨리기

```
const countExample = 3;
console.log(typeof countExample);
```

F12 콘솔 number

```
console.log(typeof "countExample");
```

F12 콘솔 string

<CartCount count="count" /> 라고 쓰면 숫자가 아니라 "count" 라는 글자가 갑니다. 에러는 안 나고 화면에 이상한 글자가 나옵니다. 값을 넘길 때는 중괄호가 필요합니다(02단원 개념03).

이런 실수를 합니다

자식에서 props 를 고치기 — 섹션 5의 데모 ㉔ 입니다

에러가 안 납니다. 부모 데이터만 조용히 오염됩니다.

이런 실수를 합니다

컴포넌트를 다른 컴포넌트 '안에서' 만들기 → 잘 돌아가는 것처럼 보임

```
function CartApp() {
  function CartList() { ... } ← 부모 함수 안에 만들면
```

... } 부모가 다시 그려질 때마다 CartList 가 '새 컴포넌트' 로 취급됩니다. 그 안의 state 가 매번 처음 값으로 돌아갑니다. 컴포넌트는 항상 맨 바깥에 만드세요.

이런 실수를 합니다

JSX 에서 태그를 안 닫기 → [SyntaxError]

```
<CartList items={items}
```

스스로 닫는 태그는 </> 로 닫아야 합니다(02단원 개념04). 파일 전체가 안 돌아갑니다.

직접 해보기 정답

1) 목록은 4줄(아메리카노 + 라떼 3줄)이고 '담긴 개수' 는 1개입니다.

```
// 화면: 아메리카노 · 라떼 · 라떼 · 라떼 / 담긴 개수: 1개
```

→ 두 컴포넌트가 서로 다른 state 를 보고 있어서 그렇습니다.

```
2) <CartList items={items} />
<CartCount items={items} />
<CartCount items={items} />    ← 한 줄 더
// 화면(누르면): 개수 줄 두 개가 동시에 2개 → 3개가 됩니다.
```

→ 값이 하나이므로 몇 곳에서 보든 항상 같습니다.

```
3) function CartBody({ items }) {
```

```
  return (
    <div> <CartList items={items} /> <CartCount items={items} /> </div>
  );
```

```
}
// 화면(누르면): 위쪽 헤더와 아래쪽 개수가 같이 2개가 됩니다.
```

→ CartBody 는 items 를 화면에 안 쓰지만 아래로 넘겨야 하므로 props 로 받습니다.

4) 두 줄 다 설명이 펼쳐진 채로 시작합니다.

```
// 화면: 두 줄 모두 '설명 닫기' 버튼과 설명이 보입니다.
```

→ 컴포넌트를 두 번 썼으니 state 도 두 개 생기고, 둘 다 초기값 true 입니다.

각자의 값이라 하나를 닫아도 다른 하나는 그대로입니다.

5) 목록이 "아메리카노" 한 줄 그대로입니다.

```
// 화면: 아메리카노 - 4000원
```

→ push 를 지웠으니 부모 배열이 오염되지 않았습니다.

화면이 안 바뀌는 건 같지만, 데이터가 깨끗하다는 점이 다릅니다.

자식이 부모에게 알리기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념03에서 state 를 부모로 올렸습니다. 그리고 이렇게 끝냈습니다.

· state 를 바꾸는 것은 그 state 를 가진 컴포넌트뿐이다 · 그럼 자식 안의 버튼으로는 어떻게 바꾸나? → 이 파일에서 배웁니다

답은 생각보다 단순합니다. 값을 내려보내듯이, '바꾸는 함수' 도 내려보내면 됩니다.

부모 → 자식	값(items)	: 화면에 그려라
부모 → 자식	함수(onAdd)	: 필요할 때 이걸 불러라
자식 → 부모	함수를 부른다	: "저 눌렀어요" 라고 알리는 것

자식은 무슨 일이 일어나는지 몰라도 됩니다. 그냥 부모가 준 함수를 부르기만 합니다. state 는 부모가 알아서 바꿉니다.

함수를 값처럼 넘기는 것은 JS자료 08단원의 콜백과 같은 이야기입니다. 그때도 함수를 다른 함수에 '넘겼습니다'. 여기서는 컴포넌트에 넘깁니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

let nextCartId = 100;
```

목록을 그리는 컴포넌트는 이 파일에서 여러 번 씁니다.

```
function CartList({ items }) {
  if (items.length === 0) {
    return <div className="output">장바구니가 비었습니다</div>;
  }
  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
}
```

부모가 '바꾸는 함수' 를 내려보냅니다

섹션 1

자식은 버튼만 갖고 있습니다. state 는 하나도 없습니다. 부모에게서 onAdd 라는 함수를 받아, 눌리면 그 함수를 부릅니다.

```
function AddButton({ onAdd }) {
  function handleClick() {
    console.log("[자식] 부모가 준 함수를 부릅니다");
  }
}
```

F12 콘솔 [자식] 부모가 준 함수를 부릅니다

```
onAdd(); // ← 부모에게 알립니다
}

return <button onClick={handleClick}>라떼 담기</button>;
}

function ParentCart() {
  const [items, setItems] = useState([
    { id: 1, name: "아메리카노" }
  ]);
}
```

이 함수가 자식에게 내려갑니다.

```
function handleAdd() {
  console.log("[부모] handleAdd 가 실행됐습니다");
}
```

F12 콘솔 [부모] handleAdd 가 실행됐습니다

```
setItems([...items, { id: nextCartId, name: "라떼" }]);
nextCartId = nextCartId + 1;
}

return (
  <div className="demo">
    <h3>① 함수를 props 로 내려보내기</h3>
    <AddButton onAdd={handleAdd} />
    <CartList items={items} />
  </div>
);
}
```

화면(누르면): 목록에 "라떼" 가 한 줄 붙습니다.

콘솔에 찍히는 순서를 보세요.

자식 · 부모가 준 함수를 부릅니다

부모 · handleAdd 가 실행됐습니다

자식이 먼저, 부모가 나중입니다. 자식이 부모를 '부른' 것입니다.

여기서 넘긴 것은 함수를 부른 결과가 아니라 함수 자체입니다.

```
<AddButton onAdd={handleAdd} />      ← 함수 자체를 넘김 (괄호 없음)
<AddButton onAdd={handleAdd()} />     ← 부른 결과를 넘김 (틀림)
```

04단원 개념01의 "onClick 에 괄호를 붙이면 안 되는 이유" 와 같은 이야기입니다. 괄호를 붙이면 화면을 그리는 순간 실행돼 버립니다.

그리고 setItems 는 부모 안에만 있습니다. 자식은 setItems 를 모릅니다. 자식이 아는 것은 "onAdd 라는 걸 부르면 뭔가 된다" 뿐입니다.

▫ 직접 해보기

1

AddButton 의 글자를 "케이크 담기" 로 바꾸고 부모의 handleAdd 도 케이크를 담도록 고쳐 보세요.

값을 같이 넘기기

섹션 2

항목마다 '빼기' 버튼이 있다면, 자식은 어느 항목인지 알려 줘야 합니다. 함수를 부를 때 값을 같이 넘기면 됩니다.

```
onRemove(item.id)
```

다만 onClick 에는 함수를 넘겨야 하므로 화살표 함수로 한 겹 감쌉니다.

```
onClick={() => onRemove(item.id)}
```

읽는 법: "눌리면, onRemove 를 item.id 와 함께 불러라" 04단원 개념01에서 배운 '인자를 넘길 때 화살표로 감싸기' 그대로입니다.

```
function CartItemRow({ item, onRemove }) {
  return (
    <li>
      {item.name}
      <button onClick={() => onRemove(item.id)}>빼기</button> </li>
    );
}

function RemovableCart() {
  const [items, setItems] = useState([
```

```

{ id: 1, name: "아메리카노" },
  { id: 2, name: "케이크" },
  { id: 3, name: "라떼" },
]);

function handleRemove(targetId) {
  console.log("[부모] 빼기 요청을 받았습니다. id:", targetId);

```

F12 콘솔 [부모] 빼기 요청을 받았습니다. id: 1

```

setItems(items.filter((item) => item.id !== targetId));
}

return (
  <div className="demo"> <h3>② 값을 같이 넘기기</h3> <ul>
    {items.map((item) => (
      <CartItemRow key={item.id} item={item} onRemove={handleRemove} />
    ))}
  </ul> </div>
);
}

```

화면(누르면): '아메리카노' 옆 빼기를 누르면 그 줄만 사라집니다.

콘솔에는 "id: 1" 이 찍힙니다.

자식 셋이 전부 '같은 함수' 를 받았습니다. handleRemove 하나뿐입니다. 그런데 각자 자기 id 를 넣어 부르므로 부모는 어느 것인지 알 수 있습니다.

key 는 CartItemRow 에 붙였습니다. map 이 만드는 바로 그 자리에 붙입니다. key 를 자식 안의 li 에 붙이면 05단원에서 본 경고가 그대로 납니다.

▹ 직접 해보기 2

CartItemRow 에 '맨 뒤로' 버튼을 하나 더 만들고, 부모에서 그 항목을 맨 뒤로 옮기는 함수를 내려보내세요. (힌트: filter 로 맨 배열 뒤에 그 항목을 붙입니다)

이름 짓는 법

섹션 3

이름에는 관습이 있습니다. 지키면 코드가 훨씬 잘 읽힙니다.

부모가 만드는 함수 handle 로 시작 handleAdd, handleRemove 자식에게 넘길 때 props on 으로 시작 onAdd, onRemove

```
<AddButton onAdd={handleAdd} />
      ↑      ↑
    자식이 쓸 이름 부모가 만든 함수
```

왜 이렇게 나을까요? on~ 은 “이런 일이 생기면” 이라는 뜻입니다. 자식 입장의 이름입니다. handle~ 은 “그 일을 처리한다” 는 뜻입니다. 부모 입장의 이름입니다.

onClick · onChange · onSubmit 이 전부 on 으로 시작하는 것과 같은 규칙입니다. 자식 컴포넌트를 쓰는 사람은 <AddButton onAdd={...} /> 만 보고 “답을 때 부를 함수를 주면 되는구나” 를 바로 압니다.

이름을 다르게 지어도 동작은 똑같습니다. 규칙이 아니라 약속입니다. 다만 다른 사람의 코드를 읽을 때 이 약속이 거의 항상 지켜져 있습니다.

▣ 직접 해보기 3

데모 ② 의 props 이름 onRemove 를 onDelete 로 바꿔 보세요. 부모와 자식 두 곳을 같이 고쳐야 합니다. 한 곳만 고치면 어떤 에러가 나는지도 확인해 보세요.

자식은 무슨 일이 일어나는지 몰라도 됩니다

섹션 4

같은 자식 컴포넌트를 여러 번 쓰고, 부모만 다르게 처리하는 예제입니다. 메뉴 버튼 자식은 “눌리면 onPick 을 메뉴와 함께 부른다” 만 압니다. 그것을 담기로 쓸지 지우기로 쓸지는 부모가 정합니다.

```
function MenuButton({ menu, onPick }) {
  return <button onClick={() => onPick(menu)}>{menu.name}</button>;
}

const menuList = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
];

function MenuPickerCart() {
  const [items, setItems] = useState([]);

  function handlePick(menu) {
```

어느 버튼을 눌러도 그 메뉴 이름이 찍힙니다. 아래는 ‘라떼’ 를 눌렀을 때입니다.

```
console.log("[부모] 고른 메뉴:", menu.name);
```

```
F12 콘솔    [부모] 고른 메뉴: 라떼
```

```

setItems([...items, { id: nextCartId, name: menu.name }]);
  nextCartId = nextCartId + 1;
}

return (
  <div className="demo"> <h3>③ 자식 셋이 같은 함수를 다른 값으로 부르기</h3>
    {menuList.map((menu) => (
      <MenuButton key={menu.id} menu={menu} onPick={handlePick} />
    ))}
    <CartList items={items} /> </div>
);
}

```

화면(누르면): 어느 버튼을 눌러도 그 메뉴가 목록에 담깁니다.

처음에는 "장바구니가 비었습니다" 가 보입니다.

MenuButton 안에는 setItems 도, items 도 없습니다. 그래서 이 컴포넌트는 다른 화면에서도 그대로 쓸 수 있습니다. 부모만 다른 handlePick 을 주면 "메뉴 삭제 버튼" 으로도 씁니다.

이것이 함수를 내려보내는 방식의 이득입니다. 자식은 '무엇이 일어났는지' 만 알리고, '어떻게 처리할지' 는 부모가 정합니다.

▣ 직접 해보기 4

handlePick 을 고쳐서, 이미 담긴 메뉴를 또 누르면 담기지 않게 하세요. (힌트: items 안에 같은 name 이 있는지 find 로 확인)

입력한 값을 부모에게 알리기

섹션 5

06단원에서 만든 입력 폼을 자식 컴포넌트로 떼어 봅시다.

여기서 중요한 판단이 하나 있습니다. · 입력칸에 지금 뭐가 쳐져 있는지 → 자식만 알면 됩니다. 자식 state 로 둡니다. · 담긴 목록 → 부모가 갖고 있습니다.

개념03 섹션4에서 본 판단 기준 그대로입니다. 그래서 자식은 자기 state 를 갖되, 담을 때만 부모에게 알립니다.

```

function CartForm({ onAdd }) {
  const [text, setText] = useState(""); // 이 컴포넌트만 쓰는
  값입니다 function handleSubmit(e) {
    e.preventDefault(); // 새로그침 막기 (06단원 개념04) if (text.trim() === "") {
      console.log("[자식] 빈 값이라 부모에게 알리지 않았습니다");
    }
  }
}

```

F12 콘솔 [자식] 빈 값이라 부모에게 알리지 않았습니다

```

return;
}

    onAdd(text.trim()); // 다듬은 값만 부모에게 넘깁니다 setText(""); // 입력칸
    비우기도 자식의 일입니다
}

return (
    <form onSubmit={handleSubmit}> <input id="addNameInput" type="text" value=
    {text} onChange={(e) => setText(e.target.value)}
        placeholder="답을 것을 입력하세요"
    />
    <button type="submit">답기</button> </form>
);
}

function FormCart() {
    const [items, setItems] = useState([]);

    function handleAddByName(name) {
        setItems([...items, { id: nextCartId, name: name }]);
        nextCartId = nextCartId + 1;
    }

    return (
        <div className="demo"> <h3>④ 입력 컴포넌트가 부모에게 알리기
        </h3> <CartForm onAdd={handleAddByName} /> <CartList items={items} /> </div>
    );
}

```

화면(누르면): "삼각김밥" 을 치고 '답기' 를 누르면

목록에 "삼각김밥" 이 붙고 입력칸이 비워집니다.

화면(누르면): 아무것도 안 치고 '답기' 를 누르면 목록은 그대로이고

콘솔에 "빈 값이라 부모에게 알리지 않았습니다" 가 찍힙니다.

부모의 handleAddByName 은 '이름 하나' 만 받습니다. 입력칸이 있었는지, 어떻게 다듬었는지는 모릅니다. 알 필요도 없습니다. 나중에 입력칸을 select 로 바꿔도 부모는 한 줄도 안 고쳐도 됩니다. 자식이 부모에게 넘길 값을 잘 고르는 것이 핵심입니다. 여기서는 다듬은 문자열 하나면 충분했습니다.

▹ 직접 해보기 5

CartForm 이 부모에게 이름 대신 { name: 다듬은값, price: 0 } 객체를 넘기도록 고치세요. 부모 handleAddByName 도 함께 고쳐야 합니다.

위에서 만든 데모 네 개를 화면에 붙입니다.

정리

- 1 자식이 부모의 state 를 바꾸려면, 부모가 **바꾸는 함수를 props** 로 내려보냅니다.
- 2 자식은 그 함수를 부르지만 합니다. `setItems` 는 부모 안에만 있습니다.
- 3 값을 같이 넘길 때는 `onClick={() => onRemove(item.id)}` 처럼 화살표로 감쌉니다.
- 4 이름은 부모가 `handleXxx`, props 는 `onXxx` 로 짓는 것이 관습입니다.
- 5 자식은 **무슨 일이 생겼는지만** 알립니다. 어떻게 처리할지는 부모가 정합니다.
- 6 입력칸의 글자처럼 자식만 쓰는 값은 자식 state 로 그대로 둡니다.
- 7 `onClick` 에 괄호를 붙이면 그리는 순간 실행되어 화면이 통째로 빕니다.

```
export default function Concept04() {
  return (
    <div> <h1>개념 04 - 자식이 부모에게 알리기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br />
      이 파일의 데모는 전부 <strong>제대로 동작하는</strong> 예제입니다. 눌러
      보면서 콘솔에 찍히는 순서를 함께 보세요.
      </p> <div> <ParentCart /> <RemovableCart /> <MenuPickerCart /> <FormCart
    /> </div> </div>
  );
}
```

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

onClick에 괄호를 붙이기 → 화면이 통째로 빡니다

```
<button onClick={onRemove(item.id)}>빼기</button>
```

화면을 그리는 순간 onRemove가 실행됩니다. onRemove 안에서 setItems를 부르니 또 그려지고, 또 실행되고... 콘솔에 "Too many re-renders"가 뜨고 화면이 비어 버립니다. 값을 넘길 때는 반드시 () ⇒ onRemove(item.id)로 감싸세요. (이 항목은 주석을 풀어 확인해 봐도 됩니다. 확인했으면 꼭 다시 //를 붙이세요. 다시 붙이고 저장하면 원래대로 돌아옵니다.)

이런 실수를 합니다

감쌀 필요가 없는데 감싸기

```
<AddButton onAdd={() => handleAdd} />
```

놀러도 아무 일이 없습니다. 이 화살표 함수는 handleAdd를 '돌려주기만' 하고 부르지 않습니다. 넘길 값이 없으면 onAdd={handleAdd}로 그냥 넘기세요.

이런 실수를 합니다

props 이름을 한쪽만 고치기

```
<CartItemRow onDelete={handleRemove} /> 안데 자식에서는 onRemove를 부름
```

```
TypeError: onRemove is not a function
```

이 납니다. "is not a function"이 나오면 props 이름 철자부터 맞춰 보세요.

이런 실수를 합니다

함수를 따옴표로 넘기기

```
const handleExample = () => 1;
console.log(typeof handleExample);
```

```
F12 콘솔    function
```

```
console.log(typeof "handleExample");
```

```
F12 콘솔    string
```

onAdd="handleAdd"라고 쓰면 함수가 아니라 글자가 넘어갑니다. 자식에서 부르는 순간 "is not a function"이 납니다.

이런 실수를 합니다

자식에서 부모 배열을 직접 고치기

items.push(...) 로 해결하려는 것 — 개념03 섹션5에서 본 그 문제입니다. 에러 없이 화면만 안 바뀝니다. 부모가 준 함수를 부르세요.

이런 실수를 합니다

화살표 함수의 괄호를 안 닫기 → [SyntaxError]

```
<button onClick={() => onRemove(item.id)}>빼기</button>
```

소괄호 짝이 안 맞습니다. 파일 전체가 안 돌아갑니다.

직접 해보기 정답

```
1) function AddButton({ onAdd }) {
```

```
...
return <button onClick={handleClick}>케이크 담기</button>;
```

```
}
function handleAdd() {
```

```
setItems([...items, { id: nextCartId, name: "케이크" }]);
nextCartId = nextCartId + 1;
```

```
}
// 화면(누르면): 목록에 "케이크" 가 붙습니다.
```

→ 자식은 글자만, 부모는 담는 내용만 고쳤습니다. 서로 상관하지 않습니다.

2) 자식:

```
<button onClick={() => onMoveLast(item.id)}>맨 뒤로</button>
```

부모:

```
function handleMoveLast(targetId) {
```

```
const target = items.find((item) => item.id === targetId);
setItems([...items.filter((item) => item.id !== targetId), target]);
```

```

}
<CartItemRow ... onMoveLast={handleMoveLast} />
// 화면(누르면): 아메리카노의 '맨 뒤로' 를 누르면 순서가
//    //    케이크 · 라떼 · 아메리카노 가 됩니다.

```

→ filter 로 뺀 새 배열 뒤에 그 항목을 붙였습니다. 원본은 안 건드립니다.

3) 부모의 `<CartItemRow ... onDelete={handleRemove} />` 와 자식의 `function CartItemRow({ item, onDelete })` 및

```
onClick={() => onDelete(item.id)} 를 함께 고칩니다.
```

한 곳만 고치면:

```
// 콘솔: TypeError: onRemove is not a function
```

→ 자식이 받지 못한 이름을 부르면 `undefined` 를 부르는 것이 됩니다.

4) `function handlePick(menu) {`

```

const already = items.find((item) => item.name === menu.name);
if (already) {
  return;
}
setItems([...items, { id: nextCartId, name: menu.name }]);
nextCartId = nextCartId + 1;

```

```

}
// 화면(누르면): 아메리카노를 두 번 눌러도 목록에는 한 줄만 있습니다.

```

→ find 는 못 찾으면 `undefined` 를 돌려줍니다(JS자료 08단원 개념04).

`undefined` 는 거짓 취급이라 `if` 가 그냥 넘어갑니다.

5) 자식:

```
onAdd({ name: text.trim(), price: 0 });
```

부모:

```
function handleAddByName(newItem) {
```

```

  setItems([...items, { id: nextCartId, name: newItem.name }]);
  nextCartId = nextCartId + 1;

```

```
}  
// 화면(누르면): 동작은 전과 똑같습니다.
```

→ 넘기는 모양만 바꿨습니다. 값이 여러 개가 되면 객체로 묶어 넘깁니다.

state를 어디에 둘까

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

여기까지 오면서 state 를 ‘어떻게 바꾸는지’ 를 배웠습니다. 이 파일은 그 앞의 질문을 다룹니다.

이 값을 애초에 state 로 뒤야 하나?

초보자가 가장 많이 만드는 버그가 여기서 나옵니다. 화면에 보이는 값이 많으니 그만큼 state 를 만드는 것입니다.

```
const [items, setItems] = useState([]);
const [count, setCount] = useState(0);    ← 담긴 개수
const [total, setTotal] = useState(0);    ← 총액
```

그렇듯해 보입니다. 그런데 이렇게 하면 반드시 어긋납니다. ‘반드시’ 라고 쓴 이유를 직접 보여 드리겠습니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

let nextCartId = 100;
```

개수를 state 로 두면 어긋납니다

섹션 1

아래 컴포넌트는 items 와 count 를 둘 다 state 로 갖고 있습니다. 담을 때는 두 줄을 나란히 적었으니 잘 맞습니다. 문제는 나중에 ‘빼기’ 기능을 붙일 때 생깁니다.

```
function BrokenCountCart() {
  const [items, setItems] = useState([
    { id: 1, name: "아메리카노", price: 4000 },
    { id: 2, name: "케이크", price: 6000 },
  ]);
  const [count, setCount] = useState(2); // ← items 로 셀 수 있는
  값입니다 function handleAdd() {
    setItems([...items, { id: nextCartId, name: "라떼", price: 4500 }]);
    setCount(count + 1); // 개수도 잊지 않고 같이 올립니다
    nextCartId = nextCartId + 1;
  }
}
```

state 설계와 불변성

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

직접 눌러 보세요. F12 → Console 도 함께.

새 항목에 붙일 번호입니다. 필요하다면 쓰세요. (화면에 안 보이는 값이라 state 가 아니어도 됩니다)

```
import { useState } from "react";

let nextId = 100;
```

문제 1 (개념01 추가)

‘케이크 담기’를 누르면 목록 맨 뒤에 “케이크”가 붙게 하세요. push 를 쓰면 안 됩니다.

기대 화면 누를 때마다 목록에 “케이크”가 한 줄씩 늘어납니다.
 눌러도 아무 변화가 없으면 push 를 쓰고 `setItems(items)` 를 한 것입니다.

```
function Q1() {
  const [items, setItems] = useState(["아메리카노"]);

  function handleAdd() {
```

여기에 코드를 쓰세요

```
    }

    return (
      <div className="demo"> <h3>문제 1 – 담기</h3> <button onClick={handleAdd}>케이크
담기</button> <ul>
        {items.map((name, index) => (
          <li key={index}>{name}</li>
        ))}
      </ul> </div>
```

```
);  
}
```

문제 2 (개념01 삭제)

각 줄의 '빼기'를 누르면 그 줄만 사라지게 하세요.

기대 화면 '케이크'의 빼기를 누르면 아메리카노와 라떼만 남습니다.
누른 것만 남고 나머지가 사라지면 filter 조건을 반대로 쓴 것입니다.

```
function Q2() {  
  const [items, setItems] = useState([  
    { id: 1, name: "아메리카노" },  
    { id: 2, name: "케이크" },  
    { id: 3, name: "라떼" },  
  ]);  
  
  function handleRemove(targetId) {
```

여기에 코드를 쓰세요

```
  }  
  
  return (  
    <div className="demo"> <h3>문제 2 - 빼기</h3> <ul>  
      {items.map((item) => (  
        <li key={item.id}>  
          {item.name}  
          <button onClick={() => handleRemove(item.id)}>빼기</button> </li>  
        )</li>  
      )</ul>  
    );  
  )  
}
```

문제 3 (개념01 수정)

각 줄의 '+1'을 누르면 그 줄의 수량만 1 올라가게 하세요.

기대 화면 아메리카노의 +1 을 누르면 "아메리카노 2개",
 케이크는 "케이크 1개" 그대로입니다.
 두 줄이 같이 올라가면 조건 없이 전부 바꾼 것입니다.

```
function Q3() {
  const [items, setItems] = useState([
    { id: 1, name: "아메리카노", qty: 1 },
    { id: 2, name: "케이크", qty: 1 },
  ]);

  function handlePlus(targetId) {
```

여기에 코드를 쓰세요

```

  }

  return (
    <div className="demo"> <h3>문제 3 - 수량 늘리기</h3> <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} {item.qty}개
          <button onClick={() => handlePlus(item.id)}>+1</button> </li>
        )))}
    </ul> </div>
  );
}
```

문제 4 (개념01 정렬)

'싼 것부터' 를 누르면 목록이 가격 오름차순으로 바뀌게 하세요. sort 는 원본을 바꾼다는 점에 주의하세요.

기대 화면 아메리카노 4000 → 라떼 4500 → 케이크 6000 순서가 됩니다.
 눌러도 화면이 그대로면 복사하지 않고 바로 sort 한 것입니다.

```
function Q4() {
  const [items, setItems] = useState([
    { id: 1, name: "케이크", price: 6000 },
    { id: 2, name: "아메리카노", price: 4000 },
    { id: 3, name: "라떼", price: 4500 },
  ]);

  function handleSort() {
```

여기에 코드를 쓰세요

```

  }

  return (
    <div className="demo"> <h3>문제 4 - 정렬</h3> <button onClick={handleSort}>싼
    것부터</button> <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} - {item.price}원
        </li>
      ))}
    </ul> </div>
  );
}
```

문제 5 (개념02 객체 한 속성)

‘대구로 이사’를 누르면 city 만 “대구”로 바뀌게 하세요. 이름과 나이는 그대로 남아야 합니다.

기대 화면 김민준 / 20살 / 대구
나이 자리가 빈칸이 되면 ...user 를 빠뜨린 것입니다.

```
function Q5() {
  const [user, setUser] = useState({ name: "김민준", age: 20, city: "서울" });

  function handleMove() {
```

여기에 코드를 쓰세요

```

    }

    return (
      <div className="demo"> <h3>문제 5 – 객체의 한 속성만 바꾸기</h3> <button onClick=
{handleMove}>대구로 이사</button> <div className="output">
      {user.name} / {user.age}살 / {user.city}
    </div> </div>
    );
  }

```

문제 6 (개념02 토글)

‘얼음 켜기/끄기’를 누를 때마다 ice 가 true ↔ false 로 뒤집히게 하세요. menu 는 그대로 남아야 합니다.

기대 화면 아메리카노 / 얼음 있음 → 누르면 얼음 없음 → 또 누르면 얼음 있음
한 번 바뀐 뒤 안 움직이면 !order.ice 대신 고정값을 넣은 것입니다.

```

function Q6() {
  const [order, setOrder] = useState({ menu: "아메리카노", ice: true });

  function handleToggle() {

```

여기에 코드를 쓰세요

```

    }

    return (
      <div className="demo"> <h3>문제 6 – 불리언 뒤집기</h3> <button onClick=
{handleToggle}>얼음 켜기/끄기</button> <div className="output">
      {order.menu} / 얼음 {order.ice ? "있음" : "없음"}
    </div> </div>
    );
  }

```

문제 7 (개념02 중첩 객체)

'L 사이즈로'를 누르면 option.size 만 "L" 이 되게 하세요. option.ice 와 menu 는 그대로 남아야 합니다.

기대 화면 아메리카노 / L 사이즈 / 얼음 있음
'얼음' 자리가 빈칸이 되면 option 안쪽을 펼치지 않은 것입니다.

```

function Q7() {
  const [order, setOrder] = useState({
    menu: "아메리카노",
    option: { size: "M", ice: true },
  });

  function handleSizeUp() {

```

여기에 코드를 쓰세요

```

  }

  return (
    <div className="demo"> <h3>문제 7 – 중첩 객체 바꾸기</h3> <button onClick=
{handleSizeUp}>L 사이즈로</button> <div className="output">
      {order.menu} / {order.option.size} 사이즈 / 얼음 {order.option.ice ? "있음" :
"없음"}
    </div> </div>
  );
}

```

문제 8 (개념03 끌어올리기)

지금은 Q8List와 Q8Count가 각자 items state를 갖고 있어 어긋납니다. items를 Q8로 옮기고, 두 자식에게 props로 내려보내세요. 담기 버튼은 Q8에 그대로 두세요.

기대 화면 '라떼 담기'를 누르면 목록이 2줄이 되고
아래 '담긴 개수'도 함께 2개가 됩니다.
목록만 늘고 개수가 1개 그대로면 아직 안 올린 것입니다.

```
function Q8List() {
```

state를 지우고 props로 items를 받으세요

```
const [items] = useState(["아메리카노"]);

return (
  <ul>
    {items.map((name, index) => (
      <li key={index}>{name}</li>
    ))}
  </ul>
);
}
```

```
function Q8Count() {
```

state를 지우고 props로 items를 받으세요

```
const [items] = useState(["아메리카노"]);

return <div className="output">담긴 개수: {items.length}개</div>;
}

function Q8() {
```

여기에 `items state` 를 만들고 `handleAdd` 를 완성하세요

```
function handleAdd() {
```

여기에 코드를 쓰세요

```
}

return (
  <div className="demo"> <h3>문제 8 – state 끌어올리기</h3> <button onClick=
{handleAdd}>라떼 담기</button> <Q8List /> <Q8Count /> </div>
);
}
```

문제 9 (개념04 함수를 props 로)

Q9Button 은 버튼만 갖고 있습니다. 부모 Q9 가 `handleAdd` 를 만들어 `onAdd` 라는 이름으로 내려보내고, 자식이 눌리면 그 함수를 부르게 하세요.

기대 화면 버튼을 누를 때마다 목록에 "삼각김밥" 이 한 줄씩 늘어납니다.
"onAdd is not a function" 이 나오면 부모가 안 내려보낸 것입니다.

```
function Q9Button(props) {
```

`props` 에서 `onAdd` 를 꺼내 버튼이 눌리면 부르세요

```
return <button>삼각김밥 담기</button>;
}
```

```
function Q9() {
  const [items, setItems] = useState([]);

  function handleAdd() {
    setItems([...items, { id: nextId, name: "삼각김밥" }]);
    nextId = nextId + 1;
  }

  return (
    <div className="demo"> <h3>문제 9 - 함수를 자식에게 내려보내기</h3>
      { /* TODO: Q9Button 에 onAdd 를 넘기세요 */ }
      <Q9Button /> <ul>
        {items.map((item) => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul> </div>
    );
}
```

문제 10 (개념04 값 같이 넘기기)

Q10Row 의 '빼기' 를 누르면 그 항목이 사라지게 하세요. 부모의 handleRemove 는 이미 완성돼 있습니다. 자식이 자기 id 를 같이 넘겨서 부르도록 고치면 됩니다.

기대 화면 '케이크' 의 빼기를 누르면 케이크만 사라집니다.
누르기도 전에 목록이 사라지거나 화면이 통째로 비면
onClick 에 괄호를 붙인 것입니다.

```
function Q10Row({ item, onRemove }) {
  return (
    <li>
      {item.name}
      { /* TODO: 아래 버튼이 눌리면 onRemove 를 item.id 와 함께 부르세요 */ }
      <button>빼기</button> </li>
    );
}

function Q10() {
  const [items, setItems] = useState([
    { id: 1, name: "아메리카노" },
    { id: 2, name: "케이크" },
  ]);

  function handleRemove(targetId) {
    setItems(items.filter((item) => item.id !== targetId));
  }
}
```


파일 나누기와 스타일링

OPEN 08_파일_나누기와_스타일링

```
return <h1>안녕하세요</h1>;
```

```
npm install react-hook-form
```

```
"vite": "^8.2.0"
```

```
function Section1Demo() {  
01   도구가 해 주는 일
```

```
02   import 와 export
```

```
03   컴포넌트 파일로 나누기
```

```
04   CSS 넣기.css
```

```
04   CSS 넣기
```

```
04   CSS 넣기.module.css
```

```
05   다른팀.css
```

```
05   styled-components
```

```
-   연습문제
```

```
-   연습문제
```

```
-   연습문제
```

도구가 해 주는 일

지금까지 일급 단원을 오는 동안 여러분은 계속 이렇게 했습니다.

파일을 고친다 → Ctrl + S → 브라우저가 알아서 바뀐다

너무 당연해서 아무 생각도 안 했을 것입니다. 그런데 저 한 줄 뒤에서 도구가 꽤 많은 일을 대신하고 있습니다.

이 문서는 **그 일이 무엇인지** 밝히는 문서입니다. 그리고 그중 하나(import 와 export)는 이 단원에서 직접 배웁니다.

이 문서는 읽기만 하는 문서입니다

섹션 1

여기서는 아무것도 안 고칩니다. 10분쯤 읽고 넘어가면 됩니다. 손을 움직이는 것은 개념02부터입니다.

먼저 결론을 말해 두겠습니다.

> 도구가 해 주는 일은 크게 네 가지입니다. > 파일 나누기 · JSX 번역 · 남의 도구 가져오기 · 저장하면 바로 반영. > 네 가지 다 “없으면 못 하는 것” 이 아니라 “**없으면 아주 번거로운 것**” 입니다.

아래에서 하나씩 봅니다.

하나 — 파일을 나눌 수 있게 해 줍니다

섹션 2

도구가 없으면

브라우저는 원래 자바스크립트 파일 사이에 담을 넘을 방법이 없습니다.

<script> 두 개를 나란히 실행하면 두 파일의 이름이 **한 통에 섞입니다**. 한쪽에서 `function Menu()` 를 만들고 다른 쪽에서도 `function Menu()` 를 만들면, 나중에 실린 쪽이 앞의 것을 조용히 덮어씹습니다. 예러도 안 납니다.

그래서 옛날에는 이렇게 했습니다.

- 한 파일에 전부 몰아넣는다 → 파일이 700줄, 1000줄이 된다 - 아니면 이름 앞에 접두어를 붙여 겹치지 않게 한다 → `카페_Menu`, `상점_Menu`

두 방법 다 좋지 않습니다.

지금은

여러분이 만든 파일 하나하나가 **자기 방을 하나씩 가집니다**. 그 방에서 밖으로 내보낸 것만 다른 파일이 볼 수 있습니다.

내보내는 쪽

```
export default function Summary({ items }) { ... }
```

가져오는 쪽

```
import Summary from "../_ui/Summary.jsx";
```

여러분은 이미 이 두 줄을 01단원부터 계속 써 왔습니다. `Summary` 를 예제마다 복사해 넣지 않았습니까. 한 곳에 두고 가져다 썼습니다.

고칠 곳이 한 곳으로 모인다 — 03단원에서 컴포넌트를 배우며 들었던 그 말이, 파일 단위에서도 그대로 성립합니다.

`import` 와 `export` 의 규칙은 **개념02**에서 정면으로 다룹니다.

지금 이 프로젝트의 `src` 폴더에는 `.jsx`·`.tsx` 파일이 **118개**, 합쳐서 **48,000줄이 넘게** 들어 있습니다. 한 파일에 몰아넣었다면 어땠을지 상상해 보세요.

둘 — JSX 를 미리 번역해 둡니다

섹션 3

도구가 없으면

브라우저는 JSX 를 모릅니다. 이런 코드를 만나면 문법 오류로 봅니다.

```
return <h1>안녕하세요</h1>;
```

02단원에서 확인했듯이, 이 줄은 결국 함수 호출 하나로 바뀝니다. 그 번역을 누군가는 해야 합니다.

번역기를 **브라우저에 실어서** 페이지가 열릴 때마다 번역하게 할 수도 있습니다. 실제로 그렇게 하는 방법이 있고, 02단원에서 그 번역기를 직접 불러 봤습니다.

다만 그 번역기 자체가 작지 않습니다.

| 파일 | 크기 | |---|---:| | [@babel/standalone](#) 의 `babel.min.js` | **약 2,400 KB** |

페이지를 열 때마다 2.4 MB 를 받아서, 그때부터 번역을 시작하는 것입니다. 화면이 늦게 뜹니다. 그리고 브라우저 콘솔이 이렇게 경고합니다.

```
You are using the in-browser Babel transformer.
Be sure to precompile your scripts for production
```

“연습이면 몰라도 실제 서비스에서는 미리 번역해 두라” 는 말입니다.

지금은

번역은 **여러분이 저장하는 순간** 끝납니다. 브라우저에는 이미 번역된 자바스크립트가 갑니다.

그래서 번역기를 실을 필요가 없고, 콘솔에 그 경고도 안 나옵니다. 02단원에서 `@babel/standalone` 을 불러온 것은 예외입니다. 화면을 만들려고가 아니라, **바뀐 모습을 보여 주려고** 가져온 것이었습니다.

셋 — 남이 만든 도구를 가져다 쓸 수 있게 해 줍니다

섹션 4

도구가 없으면

React 말고 다른 라이브러리를 쓰고 싶다면 이런 순서를 밟아야 합니다.

- 1 그 라이브러리 홈페이지에서 브라우저용 파일을 찾는다
- 2 내려받아 폴더에 넣는다
- 3 `<script>` 로 싣는다
- 4 그 라이브러리가 다른 라이브러리를 필요로 하면, 그것도 같은 순서로 반복한다

4번이 문제입니다. 필요한 것이 필요한 것을 부르고, 그게 또 부릅니다. 그리고 **싣는 순서가 틀리면 조용히 안 됩니다.**

게다가 브라우저에 바로 실을 수 있는 파일(UMD 빌드)을 **아예 안 내놓는** 라이브러리가 점점 늘고 있습니다. React 도 19부터는 그 빌드를 없앴습니다.

지금은

한 줄이면 됩니다.

```
npm install react-hook-form
```

필요한 것이 무엇을 또 필요로 하는지는 `npm` 이 알아서 따라갑니다. 싣는 순서도 도구가 정합니다.

이 자료가 실제로 그렇게 가져다 쓰는 것들입니다.

| 도구 | 어디서 쓰나 | |---|---| | `react-router-dom` | 11단원 — 화면 여러 개 오가기 | | `styled-components` | 이 단원 개념05 | | `react-hook-form` | 14단원 — 폼 | | `@babel/standalone` | 02단원 — JSX 가 무엇으로 바뀌는지 보여 주려고 |

`node_modules` 폴더가 **146 MB** 쯤 되는 이유가 이것입니다. 직접 열어 볼 일은 없습니다. 지워도 `npm install` 로 다시 만들 수 있습니다.

넷 — 저장하면 바뀐 곳만 알아 끼웁니다

섹션 5

도구가 없으면

고칠 때마다 저장하고, 브라우저로 가서 F5 를 눌러야 합니다. 그러면 페이지가 처음부터 다시 시작합니다.

여기서 진짜 불편한 것은 F5 를 누르는 손가락이 아닙니다. **화면 상태가 전부 사라진다는** 점입니다.

- 폼에 열 칸을 채워 놓고 → 마지막 줄을 고치려고 F5 → 열 칸이 다 비워집니다 - 목록을 세 번 걸러 놓고 → 색을 고치려고 F5 → 처음 목록으로 돌아갑니다

06단원 폼 예제를 만들 때 이걸 계속 겪었을 것입니다.

지금은

저장하면 **바뀐 파일만** 알아 끼웁니다. 페이지는 그대로입니다. 숫자를 다섯 번 눌러 올려 놓고 코드를 고쳐도 숫자는 5 그대로 남습니다.

이것을 HMR(Hot Module Replacement) 이라고 부릅니다. 이름은 몰라도 됩니다.

> **오히려 F5 는 누르지 마세요.** > 왼쪽에서 고른 예제가 풀려서 맨 앞 예제로 돌아갑니다.

그래서 그 도구가 뭔가요

섹션 6

이름은 **Vite** 입니다. 실습프로젝트/package.json 에 적혀 있습니다.

```
"vite": "^8.2.0"
```

npm run dev 를 칠 때 켜지는 것이 이것입니다. 하는 일을 한 줄로 줄이면 이렇습니다.

> 여러분이 저장한 파일을 **브라우저가 이해할 수 있는 모양으로 바꿔서 건네주는 사람.**

React 와는 다른 물건입니다. 헛갈리기 쉬운 곳이라 한 번 정리합니다.

|| 하는 일 || ---|---| **React** | 화면을 어떻게 그릴지 정하는 라이브러리 || **Vite** | 파일을 모아 번역해서 브라우저에 건네는 도구 |

React 는 Vite 없이도 돌아갑니다. Vite 는 React 말고 다른 것에도 씁니다. 둘은 짝이 아니라, 자주 같이 쓰이는 두 물건입니다.

같은 자리에서 쓰는 다른 도구도 있습니다(webpack, Parcel, Next.js 등). 지금 그 차이를 알 필요는 없습니다. **하는 일이 같다** 는 것만 기억하세요.

① 터미널을 켜 줘야 합니다

`npm run dev` 를 끄면 화면이 안 뜹니다. 컴퓨터를 껐다 켜면 다시 켜야 합니다. 터미널 창을 실수로 닫는 것이 가장 흔한 사고입니다.

② 파일을 더블클릭해서는 못 엽니다

`.jsx` 파일을 더블클릭하면 편집기가 열리거나 아무 일도 안 일어납니다. 브라우저는 `.jsx` 를 모릅니다. 반드시 `npm run dev` 를 거쳐야 합니다.

③ `node_modules` 가 생깁니다

146 MB 쯤 됩니다. 파일 수가 수만 개라 복사가 아주 느립니다. USB 로 옮길 때는 이 폴더를 빼고 옮긴 뒤, 받은 쪽에서 `npm install` 하는 편이 빠릅니다.

④ 에러가 두 군데에서 납니다

문법이 틀리면 터미널, 실행 중에 터지면 브라우저입니다. 01단원 개념04에서 그 구별을 배웠습니다. 화면이 안 바뀌면 터미널부터 보세요.

안 바뀌는 것 — 이게 더 중요합니다

01~07단원에서 배운 React 문법은 전부 그대로입니다.

컴포넌트, props, `useState`, `map`, `key`, `onChange`, 불변 갱신 — 하나도 안 바뀝니다. 이 단원에서 새로 배우는 것은 **React 문법이 아니라 파일을 다루는 방법**입니다.

|| 지금까지 | 이 단원부터 | |---|---|---| | 컴포넌트 만들기 | `function Menu() {}` | 그대로 | | state | `useState` | 그대로 | | 파일 나누기 | 한 파일에 다 넣었다 | `import / export` 로 나눈다 | | 스타일 | `className="demo"` (공용 CSS) | 예제마다 CSS 를 따로 붙인다 |

다음 문서에서 할 일

개념02부터 손을 움직입니다.

- **개념02** — `import` 와 `export`. 지금까지 써 온 그 두 줄의 정체 - **개념03** — 컴포넌트를 파일로 나누기 - **개념04** — CSS 를 예제마다 붙이기 (일반 CSS · CSS Modules) - **개념05** — `styled-components` (안 해도 됩니다)

정리

섹션 10

- 1 도구가 해 주는 일은 네 가지입니다. **파일 나누기 · JSX 번역 · 남의 도구 가져오기 · 저장하면 바로 반영.**
- 2 **파일 나누기** — 브라우저는 원래 파일 사이에 담이 없습니다. 이름이 한 통에 섞여 조용히 덮어씹습니다. `import/export` 가 파일마다 방을 만들어 줍니다. (src 에 118파일 48,000여 줄)
- 3 **JSX 번역** — 브라우저는 JSX 를 모릅니다. 번역기를 브라우저에 실행하면 2.4 MB 를 매번 받고 콘솔에 경고가 납니다. 지금은 저장하는 순간 번역이 끝나 있습니다.
- 4 **남의 도구** — `npm install` 한 줄이면 필요한 것까지 알아서 따라옵니다. 직접 받아 `<script>` 로 실행 시절에는 순서가 틀리면 조용히 안 됐습니다.
- 5 **저장하면 바로** — F5 를 안 눌러도 됩니다. 더 중요한 것은 화면 상태가 안 사라진다는 점입니다.
- 6 그 도구 이름이 **Vite** 입니다. **React 와 다른 물건**입니다. React 는 화면을 그리는 라이브러리, Vite 는 파일을 번역해 건네는 도구입니다.
- 7 대신 터미널을 켜 줘야 하고, 더블클릭으로는 못 열고, `node_modules`(146 MB)가 생기고, 에러가 터미널과 브라우저 두 군데에서 납니다.
- 8 **React 문법은 하나도 안 바뀝니다.** 이 단원에서 배우는 것은 파일을 다루는 방법입니다.

import와 export

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념01은 읽는 문서였습니다. 여기서부터 코드입니다.

이 파일에서 배우는 것은 딱 두 단어입니다. `export` 와 `import`. 그런데 이 두 단어가 08단원 전체의 이유입니다. 개념01에서 “파일을 못 나눈다” 를 첫 번째 한계로 꼽았는데, 그것을 푸는 문법이 이 둘입니다.

★ `import` 는 사실 01단원부터 계속 봐 왔습니다. 파일 맨 위에 늘 이런 줄이 있었습니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

지금까지는 “그냥 있는 줄” 이었습니다. 이 파일에서 그 줄의 정체를 봅니다. 왜 필요한지, 중괄호는 언제 붙이는지, 경로는 어떻게 읽는지 — 섹션 5까지 가면 다 나옵니다.

★ 이 파일 맨 위(바로 아래)에 `import` 줄이 여러 개 있습니다. 그게 오늘의 주제입니다. 읽으면서 하나씩 돌아와 보세요.

★ 콘솔에 같은 줄이 두 번씩 찍힙니다. 정상입니다. `main.jsx` 의 `StrictMode` 때문입니다(09단원 개념02 섹션1). 여러분이 잘못된 게 아닙니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

import Greeting from "../_부품/Greeting.jsx";
import Hello from "../_부품/Greeting.jsx";
import { americanoPrice, lattePrice, formatPrice, shopLabel } from "../_부품/menuPrices.js";
import { cakePrice as dessertPrice } from "../_부품/menuPrices.js";
import PriceTag, { defaultNote } from "../_부품/PriceTag.jsx";
```

파일 안에서 만든 것은 그 파일 안에서만 삽니다

섹션 1

01~07단원에서 이런 코드를 썼습니다.

```
function MenuRow({ name, price }) { ... }
```


그런데 이렇게 만든 MenuRow 는 그 파일 안에서만 씁니다. 03단원에서 만든 것을 05단원에서 또 쓰고 싶어도 부를 방법이 없어서, 복사해서 붙여 넣어야 했습니다. 그리고 붙여 넣는 순간 두 벌이 됩니다.

이제는 다릅니다. 파일마다 이런 문이 하나씩 달려 있다고 생각하세요.

```
export - 이 파일에서 밖으로 내보낼 것을 정한다
import - 다른 파일이 내보낸 것을 가져온다
```

비유하면 export 는 '가게 진열대', import 는 '장 보기' 입니다. 진열대에 안 올린 물건은 손님이 살 수 없습니다.

비유는 여기까지입니다. 실제로 일어나는 일은 이렇습니다. Vite 가 import 줄을 보고 "아, 저 파일도 필요하구나" 하고 그 파일을 읽어 옵니다. 그리고 그 파일이 export 한 것만 이쪽으로 건네줍니다.

```
export 를 안 붙인 것은 애초에 건네줄 목록에 없습니다.
```

이 파일 맨 위에 이런 줄이 있습니다.

```
import Greeting from "../_부품/Greeting.jsx";
```

그래서 아래에서 Greeting 을 마치 이 파일에서 만든 것처럼 쓸 수 있습니다. _부품/Greeting.jsx 를 열어 보세요. 03단원에서 만들던 컴포넌트와 똑같이 생겼습니다. 앞에 export default 가 붙은 것만 다릅니다.

```
function Section1Demo() {
```

가져온 Greeting 이 무엇인지 콘솔로 확인해 봅시다.

```
console.log(typeof Greeting);
```

```
F12 콘솔    function
```

평범한 함수입니다. 특별한 물건이 아닙니다.

```
return (
  <div className="demo"> <h3>① 다른 파일에서 가져온 컴포넌트
</h3> <Greeting name="김민준" />
  { /* 화면: 김민준님, 안녕하세요. */ }
  <Greeting name="이서연" />
  { /* 화면: 이서연님, 안녕하세요. */ }
</div>
);
}
```

➤ 직접 해보기

1

위 Section1Demo 에 <Greeting name="박지훈" /> 을 한 줄 더 넣으세요. 저장하면 F5 없이 화면이 바뀝니다.

_부품/Greeting.jsx 는 이렇게 생겼습니다.

```
export default function Greeting({ name }) { ... }
```

export default 는 “이 파일의 대표는 이것이다” 라는 뜻입니다. 한 파일에 딱 하나만 둘 수 있습니다. 대표가 둘일 수는 없으니까요.

가져올 때는 중괄호 없이 이름만 씁니다.

```
import Greeting from "../_부품/Greeting.jsx";
```

여기서 중요한 것이 하나 있습니다.

★ export default 로 나온 것은 가져오는 쪽이 이름을 마음대로 지어도 됩니다.

“대표를 하나 주세요” 라고만 말했으니, 그걸 뭐라고 부를지는 받는 쪽 마음입니다. 실제로 이 파일 맨 위에는 같은 파일을 두 번 가져온 줄이 있습니다.

```
import Greeting from "../_부품/Greeting.jsx";
import Hello from "../_부품/Greeting.jsx";
```

이름만 다르지 알맹이는 같은 물건입니다. 아래에서 확인합니다.

```
function Section2Demo() {
  const same = Greeting === Hello;

  console.log(same);
}
```

F12 콘솔 true

같다고 나왔습니다. 두 번 가져왔지만 파일은 한 번만 읽힙니다. 이름표만 두 개 붙인 것입니다.

```
return (
  <div className="demo"> <h3>② 같은 파일, 이름만 다르게 가져오기
</h3> <Hello name="박지훈" />
  { /* 화면: 박지훈님, 안녕하세요. */ }
  <p className="output">Greeting 과 Hello 가 같은 물건인가: {String(same)}</p>
  { /* 화면: Greeting 과 Hello 가 같은 물건인가: true */ }
</div>
);
}
```

String(...) 은 값을 글자로 바꾸는 함수입니다. true / false 를 그냥 {} 안에 넣으면 화면에 아무것도 안 나오기 때문입니다(05단원 개념05).

그럼 이름을 아무렇게나 지어도 되나요? 됩니다. 다만 두 가지를 지켜세요.

[1] 컴포넌트라면 반드시 대문자로 시작하세요.

소문자로 지으면 **React** 가 HTML 태그로 오해합니다(01단원 개념04).

[2] 되도록 파일 이름과 같게 지으세요.

Greeting.jsx 에서 가져온 것을 **Zzz** 라고 부르면 나중에 찾을 수가 없습니다.

이 자료는 앞으로 [2] 를 지킵니다. Hello 는 이 섹션을 보여 주려고 한 번 쓴 것입니다.

▣ 직접 해보기 2

파일 맨 위의 `import Hello ...` 줄에서 `Hello` 를 `Annyeong` 으로 바꾸고, `Section2Demo` 안의 `<Hello ... />` 도 `<Annyeong ... />` 로 바꾸세요. 화면이 그대로인지 확인하세요.

이름 있는 export — 여러 개 내보내기

섹션 3

대표가 하나뿐이면 값 여러 개를 어떻게 내보낼까요? 그때는 `export` 를 만드는 자리마다 하나씩 붙입니다. 이것을 ‘이름 있는 export’ 라고 합니다.

`_부품/menuPrices.js` 는 이렇게 생겼습니다.

```
export const americanoPrice = 4000;
export const lattePrice = 4500;
export const cakePrice = 6000;
export function formatPrice(price) { ... }
```

가져올 때는 중괄호 안에 이름을 적습니다. 필요한 것만 골라 담으면 됩니다.

```
import { americanoPrice, lattePrice, formatPrice } from "../_부품/menuPrices.js";
```

★ `export default` 와 결정적으로 다른 점이 있습니다.

이름 있는 export 는 이름을 정확히 맞춰야 합니다.

```
import { americanoPrice } 라고 적었는데 그 파일에 그런 이름이 없으면 못 가져옵니다.
```

“대표 하나 주세요” 가 아니라 “americanoPrice 라는 걸 주세요” 라고 말한 것이기 때문입니다.

그래도 이름을 바꾸고 싶을 때가 있습니다. 그때는 `as` 를 씁니다.

```
import { cakePrice as dessertPrice } from "../_부품/menuPrices.js";
```

“cakePrice 를 주는데, 여기서는 dessertPrice 라고 부르겠다” 는 뜻입니다. `as` 는 이름이 겹칠 때 씁니다. 두 파일에서 똑같이 `formatPrice` 를 가져와야 한다면 한쪽을 `as` 로 바꿔야 합니다. 겹치지 않는데 굳이

바꾸지는 마세요. 찾기만 어려워집니다.

```
function Section3Demo() {  
  console.log(americanoPrice);  
}
```

F12 콘솔 4000

| 코드 | ▶ F12 콘솔 |
|---|----------|
| console.log(lattePrice); | ▶ 4500 |
| console.log(formatPrice(americanoPrice)); | ▶ 4000원 |
| console.log(dessertPrice); | ▶ 6000 |

★ export 를 안 붙인 것은 밖으로 안 나옵니다.

menuPrices.js 에는 이런 줄도 있습니다.

```
const shopName = "동네 카페"; // ← export 가 없습니다  
export function shopLabel() { return `${shopName} 메뉴판`; }
```

shopName 은 export 가 없으니 이 파일에서는 볼 수 없습니다.

```
console.log(typeof shopName);
```

F12 콘솔 undefined

여기서 shopName 은 “없는 이름” 입니다. (typeof 는 없는 이름에 써도 에러가 안 나는 몇 안 되는 문법입니다. 그냥 console.log(shopName) 이라고 쓰면 ReferenceError 가 납니다)

그런데 shopLabel 은 잘 돌아갑니다.

```
console.log(shopLabel());
```

F12 콘솔 동네 카페 메뉴판

shopLabel 은 menuPrices.js 안에 있으니 shopName 이 잘 보입니다. 이것이 파일을 나누는 또 하나의 이득입니다. 밖에 안 보여도 되는 것은 안 보이게 둘 수 있습니다. 그러면 다른 파일에서 실수로 건드릴 일이 없습니다.

```
return (  
  <div className="demo"> <h3>③ 값과 함수를 가져다 쓰기</h3> <p className="output">  
    {shopLabel()}</p>  
    { /* 화면: 동네 카페 메뉴판 */  
    <ul> <li>아메리카노 {formatPrice(americanoPrice)}</li>  
      { /* 화면: 아메리카노 4000원 */  
      <li>라떼 {formatPrice(lattePrice)}</li>  
    }  
  )
```

```

    { /* 화면: 라떼 4500원 */ }
    <li>케이크 {formatPrice(dessertPrice)}</li>
    { /* 화면: 케이크 6000원 */ }
  </ul> </div>
);
}

```

▹ 직접 해보기 3

_부품/menuPrices.js 를 열어 export `const riceBallPrice = 1200;` 을 추가하고, 이 파일의 import 줄에도 `riceBallPrice` 를 넣으세요. (아직 화면에 넣지는 마세요. 다음 문제에서 합니다)

▹ 직접 해보기 4

위 Section3Demo 의 목록에 `<li>삼각김밥 {formatPrice(riceBallPrice)}</li>` 를 한 줄 넣으세요.

경로 읽는 법

섹션 4

import 줄의 따옴표 안에 들어가는 것이 '경로' 입니다. 세 가지 모양이 있습니다.

[1] ./ 로 시작 — 나와 같은 폴더에서 찾아라 [2] ../ 로 시작 — 한 단계 위 폴더로 나가서 찾아라 [3] 아무것도 없음 — node_modules 에서 찾아라 (남이 만든 도구)

이 파일 맨 위 줄들을 다시 보면 셋이 다 들어 있습니다.

```

import { useState } from "react";           ← [3]
import Summary from "../_ui/Summary.jsx";    ← [2]
import Greeting from "../_부품/Greeting.jsx"; ← [1]

```

폴더 모양으로 보면 이렇습니다.

```

src/
  _ui/
    Summary.jsx      ← ../_ui/Summary.jsx
08_파일_나누기와_스타일링/
  개념02_import와_export.jsx ← 지금 이 파일
  _부품/
    Greeting.jsx     ← ../_부품/Greeting.jsx

```

./ 는 '개념02 파일이 들어 있는 폴더', 즉 08_파일_나누기와_스타일링 폴더입니다. 그 안의 _부품 폴더로 들어가니 ../_부품/Greeting.jsx 가 됩니다.

../ 는 거기서 한 단계 나갑니다. src 폴더입니다. 그 안의 _ui 폴더로 들어가니 ../_ui/Summary.jsx 가 됩니다.

★ 경로는 늘 '그 줄이 적힌 파일' 을 기준으로 읽습니다. `_부품/MenuBoard.jsx` 를 열어 보면 `"/MenuRow.jsx"` 라고 적혀 있습니다. 같은 `_부품` 폴더 안에 있으니 `./` 만으로 충분한 것입니다. 같은 파일을 가리키는데도 경로가 다릅니다. 어디서 부르는가에 따라 달라집니다.

헛갈리면 `./` 를 빠뜨리는 실수가 가장 많습니다.

```
import Greeting from "_부품/Greeting.jsx"; ← ./ 가 없습니다
```

이렇게 쓰면 [3] 으로 읽혀서 `node_modules` 에서 `_부품` 이라는 도구를 찾습니다. 그런 게 없으니 실패합니다. 실수 섹션에서 실제 메시지를 봅니다.

★ 확장자를 쓰나요? Vite 는 `.jsx` 를 생략해도 찾아 줍니다. 하지만 이 자료는 항상 붙입니다. 이유는 두 가지입니다.

- 붙여 두면 그 줄만 보고도 어떤 파일인지 압니다.
- 도구를 바꿨을 때 생략이 안 되는 경우가 있습니다.

★ 다만 [3] 번, 즉 `node_modules` 의 도구에는 확장자를 붙이지 않습니다.

"react" 이지 "react.js" 가 아닙니다. 그건 파일 이름이 아니라 도구 이름입니다.

```
function Section4Demo() {
  const [open, setOpen] = useState(false);

  console.log("경로 세 가지를 화면에 그렸습니다");
}
```

F12 콘솔 경로 세 가지를 화면에 그렸습니다

```
return (
  <div className="demo"> <h3>④ 경로 세 가지 다시 보기</h3> <button onClick={() =>
setOpen(!open)}>{open ? "접기" : "펼치기"}</button>
  { /* 화면: [펼치기] 버튼만 보입니다 */ }
  { open && (
    <div className="output"> <div> <code>"react"</code> - node_modules
에서 찾습니다 (남이 만든 도구)
    </div> <div> <code>"../_ui/Summary.jsx"</code> - 한 단계 위로
나가서 찾습니다
    </div> <div> <code>"../_부품/Greeting.jsx"</code> - 나와 같은
폴더에서 찾습니다
    </div> </div>
  ) }
  { /* 화면(누르면): 경로 세 줄이 나타나고 버튼 글자가 [접기] 로 바뀝니다 */ }
</div>
);
}
```

▹ 직접 해보기 5

파일 맨 위의 Summary import 줄을 "../_ui/Summary.jsx" 에서 "../_ui/Summary.jsx" 로 바꿔 보세요. 화면이 어떻게 되는지 확인하고 다시 되돌리세요.

한 파일에서 둘 다 쓰기

섹션 5

export default 와 이름 있는 export 는 한 파일에 같이 있어도 됩니다. _부품/PriceTag.jsx 가 그렇게 생겼습니다.

```
export const defaultNote = "가장 많이 팔립니다";      ← 이름 있는 export
export default function PriceTag({ ... }) { ... }      ← 대표
```

가져올 때는 이렇게 한 줄에 씁니다.

```
import PriceTag, { defaultNote } from "../_부품/PriceTag.jsx";
```

순서가 정해져 있습니다. 대표가 앞, 중괄호가 뒤입니다. 중괄호가 없는 쪽이 대표라고 기억하면 됩니다.

★ 중괄호가 눈에 익지 않나요? JS자료 09단원의 객체 구조분해와 똑같이 생겼습니다.

```
const { useState } = 어떤객체;      ← 구조분해 (JS자료 09단원)
import { useState } from "react";   ← 이름 있는 import (지금 배우는 것)
```

둘 다 “여러 개 중에서 이 이름을 골라 꺼낸다” 는 뜻이라 모양이 닮았습니다. 하지만 완전히 다른 문법입니다. 헷갈리지 않게 차이를 적어 둡니다.

```
const { useState } = 어떤객체;
```

이미 만들어져 있는 '객체' 에서 속성을 꺼내는 것입니다.
함수 안에서도 쓸 수 있고, 그 객체가 없으면 에러가 납니다.

```
import { useState } from "react";
```

"react" 라는 도구에서 useState 라는 export 를 가져오는 것입니다.
Vite 가 node_modules 에서 react 를 찾아 읽어 옵니다.
이 줄은 반드시 파일 맨 위에 있어야 합니다. 함수 안에는 못 씁니다.

아래에서 확인해 봅시다. 04단원에서 만든 카운터와 한 글자도 안 다릅니다. 그때도 지금도 맨 위의 import 줄이 useState 를 가져다 준 것입니다.

```
function Section5Demo() {
  const [count, setCount] = useState(0);

  console.log(defaultNote);
}
```

F12 콘솔 가장 많이 팔립니다

```
function handleAdd() {
  setCount(count + 1);

  console.log("담은 잔 수:", count + 1);
}
```

F12 콘솔 담은 잔 수: 1

```

}

return (
  <div className="demo"> <h3>⑤ default 와 이름 있는 export 를 함께 / import 한
  useState</h3> <PriceTag name="아메리카노" price={americanoPrice} />
  { /* 화면: 아메리카노 - 4000원 / 가장 많이 팔립니다 */}

  <PriceTag name="케이크" price={dessertPrice} note="달아요" />
  { /* 화면: 케이크 - 6000원 / 달아요 */}

  <p className="output">기본 문구는 &quot;{defaultNote}&quot; 입니다.</p>
  { /* 화면: 기본 문구는 "가장 많이 팔립니다" 입니다. */}

  <p className="output">지금 {count}잔 담았습니다.</p>
  { /* 화면: 지금 0잔 담았습니다. */}

  <button onClick={handleAdd}>아메리카노 담기</button>
  { /* 화면(누르면): 지금 1잔 담았습니다. */}
</div>
);
}
```

▹ 직접 해보기 6

Section5Demo 에 <PriceTag name="라떼" price={lattePrice} /> 를 한 줄 넣으세요. note 를 안 넘기면 무엇이 나올까요?

▹ 직접 해보기 7

_부품/PriceTag.jsx 의 defaultNote 를 "오늘의 추천" 으로 바꾸세요. 화면에서 몇 군데가 바뀌는지 세어 보세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

★ 08단원부터는 에러가 두 곳에 나타납니다. 둘 다 보세요.

- 브라우저 화면 (빨간 글씨가 화면을 덮기도 합니다)
- npm run dev 를 켜 둔 터미널

터미널 쪽이 더 자세할 때가 많습니다.

이런 실수를 합니다

./ 를 빠뜨림

```
import Greeting from "_부품/Greeting.jsx";
```

경로가 아니라 도구 이름으로 읽힙니다. node_modules 에서 찾다가 실패합니다. 터미널과 화면에 이런 줄이 뜹니다. Failed to resolve import "_부품/Greeting.jsx" from ... resolve 는 "어디 있는지 알아내다" 라는 뜻입니다. 이 메시지가 보이면 90%는 ./ 나 ../ 를 빠뜨린 것입니다.

이런 실수를 합니다

이름 있는 export 인데 중괄호를 안 씀 ★ 이걸 에러가 안 납니다

```
import americanoPrice from "./_부품/menuPrices.js";
```

에러가 안 납니다. 화면도 안 멈춥니다. 그런데 값이 undefined 입니다. "그 파일의 대표를 주세요" 라고 말했는데, menuPrices.js 에는 대표가 없기 때문입니다. 화면에는 4000 이 나와야 할 자리가 비어 버립니다. ★ 이 실수는 조용히 틀리는 쪽이라 제일 위험합니다. 값이 undefined 로 나오면 중괄호부터 확인하세요.

이런 실수를 합니다

export default 인데 중괄호를 씀

```
import { Greeting } from "./_부품/Greeting.jsx";
```

이건 에러가 납니다. 화면이 빨간 상자로 바뀝니다. The requested module ... does not provide an export named 'Greeting' "Greeting 이라는 이름으로 내보낸 게 없다" 는 뜻입니다. 맞습니다. Greeting.jsx 는 default 로 내보냈지 이름을 붙여 내보내지 않았습니니다.

이런 실수를 합니다

export 를 아예 안 붙임

부품 파일에서 `function` 앞의 `export default` 를 지우면 이렇게 됩니다. The requested module ... does not provide an export named 'default' 새 부품 파일을 만들었는데 이 메시지가 나오면 `export` 를 빠뜨린 것입니다.

이런 실수를 합니다

import 를 함수 안에 씀

```
function Bad(){  
  import Greeting from "../_부품/Greeting.jsx";  
  return <Greeting name="김민준" />;  
}
```

[SyntaxError] 입니다. 화면이 통째로 빡니다. `import` 는 파일 맨 위에만 쓸 수 있습니다. Vite 가 파일을 읽기도 전에 `import` 줄부터 먼저 훑어서 "어떤 파일들이 더 필요한지" 를 알아내기 때문입니다. 그래서 조건문 안이나 함수 안에는 못 넣습니다.

이런 실수를 합니다

컴포넌트 이름을 소문자로 지음 ★ 이것도 에러가 안 납니다

```
import greeting from "../_부품/Greeting.jsx";
```

... `<greeting name="김민준" />` 에러가 안 나고 화면만 이상해집니다. React 가 소문자를 HTML 태그로 봐서 `<greeting>` 이라는 없는 태그를 만듭니다. 01단원 개념04에서 본 것과 같은 실수입니다. `export default` 는 이름을 마음대로 지어도 되지만, 대문자 규칙은 남아 있습니다.

화면

아래 `restartKey` 는 데모들을 '처음부터 다시' 그리기 위한 것입니다. 05단원에서 배운 `key` 를 그대로 씁니다. `key` 가 달라지면 React 는 그것을 고쳐 쓸 물건이 아니라 '다른 물건' 으로 보고 통째로 새로 만듭니다.

정리

- 1 export 는 이 파일에서 밖으로 내보낼 것을 정하고, import 는 다른 파일이 내보낸 것을 가져옵니다. export 를 안 붙인 것은 밖에서 볼 수 없습니다.
- 2 export default 는 파일의 대표 하나입니다. 가져올 때 중괄호를 안 쓰고, 이름을 마음대로 지어도 됩니다(컴포넌트면 대문자로).
- 3 이름 있는 export 는 개수 제한이 없습니다. 가져올 때 중괄호 안에 이름을 정확히 적어야 하고, 바꾸고 싶으면 as 를 씁니다.
- 4 경로는 ./ 면 같은 폴더, ../ 면 한 단계 위, 아무것도 없으면 node_modules 의 도구입니다. ./ 를 빠뜨리면 Failed to resolve import 가 납니다.
- 5 경로는 늘 그 줄이 적힌 파일을 기준으로 읽습니다. 같은 파일이라도 어디서 부르느냐에 따라 경로가 달라집니다.
- 6 **import** { useState } 의 중괄호는 JS자료 09단원의 객체 구조분해와 닮았지만 다른 문법입니다. 구조분해는 함수 안에서도 되지만 **import** 는 파일 맨 위에만 됩니다.
- 7 import 는 파일 맨 위에만 쓸 수 있습니다. 함수 안이나 조건문 안에는 못 씁니다.

```
export default function Concept02ImportExport() {
  const [restartKey, setRestartKey] = useState(0);

  return (
    <div> <h1>개념 02 – import 와 export</h1> <p className="guide">
      이 파일은 <strong>왼쪽 목록에서 골라</strong> 봅니다. 더블클릭이 아닙니다.
      <br /> <br /> <strong>F12 → Console</strong> 도 함께 열어 두세요. 이 파일은
      콘솔에 나오는 값이
      많습니다. 같은 줄이 <strong>두 번씩</strong> 찍히는 것은 정상입니다(09단원
      개념02의
      StrictMode).
      <br /> <br />
      이 파일이 쓰는 부품은 <code>src/08_파일_나누기와_스타일링/_부품</code> 안에
      있습니다. 그
      폴더의 파일들도 같이 열어 놓고 보세요.{" "}
      <strong>하위 폴더라서 왼쪽 목록에는 안 나타납니다.
    </strong> </p> <button onClick={() => setRestartKey(restartKey + 1)}>
      이 예제를 처음부터 다시 그리기
    </button> <div key={restartKey}> <Section1Demo /> <Section2Demo
    /> <Section3Demo /> <Section4Demo /> <Section5Demo /> </div> </div>
  );
}
```

직접 해보기 정답

1) <Greeting name="박지훈" />

```
// 화면: ① 상자에 "박지훈님, 안녕하세요." 가 한 줄 늘어납니다.
```

→ 저장만 하면 바뀝니다. F5 는 누를 필요가 없습니다.

```
2) import Annyeong from "../부품/Greeting.jsx";
```

... <Annyeong name="박지훈" />

```
// 화면: ② 상자가 그대로입니다. "박지훈님, 안녕하세요."
```

→ export default 는 받는 쪽이 이름을 정합니다. 그래서 무슨 이름이든 됩니다.

```
단 <annyeong ... /> 처럼 소문자로 지으면 화면이 빕니다(실수 6).  
import 줄만 바꾸고 <Hello ... /> 를 그대로 두면  
"Hello is not defined" 에러가 나면서 예제가 빨간 상자가 됩니다.
```

3) menuPrices.js 에 한 줄 추가:

```
export const riceBallPrice = 1200;
```

이 파일의 import 줄을 이렇게 고칩니다:

```
import { americanoPrice, lattePrice, formatPrice, shopLabel, riceBallPrice }  
from "../부품/menuPrices.js";
```

→ 화면은 아직 그대로입니다. 가져오기만 하고 안 썼기 때문입니다.

가져와 놓고 안 쓰면 아무 일도 안 일어납니다. 에러도 안 납니다.

```
4) <li>삼각김밥 {formatPrice(riceBallPrice)}</li>  
// 화면: ③ 목록에 "삼각김밥 1200원" 이 한 줄 늘어납니다.
```

→ 3번을 안 하고 4번만 하면 "riceBallPrice is not defined" 로 빨간 상자가 됩니다.

```
5) import Summary from "../ui/Summary.jsx";  
// 화면: 예제가 통째로 안 나오고, 터미널과 화면에 이런 줄이 뜹니다.  
// Failed to resolve import "../ui/Summary.jsx" from "src/08_파일_나누기와_스타일링/개념02_import와_export.jsx"
```

→ ./ 는 08_파일_나누기와_스타일링 폴더입니다. 그 안에는 _ui 가 없습니다.

_ui 는 한 단계 위인 src 안에 있으므로 ../ 로 나가야 합니다.
되돌리면 화면이 저절로 돌아옵니다.

```
6) <PriceTag name="라떼" price={lattePrice} />
// 화면: 라떼 - 4500원 / 가장 많이 팔립니다
```

→ note 를 안 넘겼으니 기본값 defaultNote 가 쓰였습니다.

기본값이 그냥 글자가 아니라 '가져온 값' 이라는 것이 이 문제의 핵심입니다.

7) _부품/PriceTag.jsx 에서:

```
export const defaultNote = "오늘의 추천";
```

```
// 화면: ⑤ 상자에서 두 군데가 바뀝니다.
//      아메리카노 카드의 작은 글씨, 그리고 "기본 문구는 ..." 줄.
```

→ 케이크 카드는 note="달아요" 를 직접 넘겼으므로 안 바뀝니다.

한 파일만 고쳤는데 여러 곳이 함께 바뀌었습니다.
개념01에서 말한 "고칠 곳이 한 곳" 이 바로 이것입니다.
확인했으면 "가장 많이 팔립니다" 로 되돌려 두세요. 뒤 설명과 어긋납니다.

컴포넌트 파일로 나누기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념02에서 `import` 와 `export` 를 배웠습니다. 이 파일에서는 그것을 실제 코드에 써 봅니다.

웁길 코드는 여러분이 이미 아는 것입니다. 03단원 개념05에서 만든 카페 메뉴판입니다.

| | |
|---------------------|------------------------------|
| BigMenuBoard | - 메뉴 세 줄을 통째로 적은 것 |
| MenuRow | - 메뉴 한 줄 |
| MenuBoard | - MenuRow 를 세 번 쓴 메뉴판 |
| MenuCard | - note 기본값이 있는 카드 |

그때는 이 넷이 전부 한 .html 파일 안에 있었습니다. 이제 파일로 나눕니다. 그리고 나누고 나면 무엇이 달라지는지 봅니다.

★ 이 파일에서 가장 중요한 것은 섹션 4입니다. JSX 를 쓰는데도 이 파일에 `React` 라는 이름이 없습니다. 섹션 4에서 이유를 봅니다. 그런데도 JSX 가 돌아갑니다. 왜 그런지를 섹션 4에서 다룹니다.

★ 콘솔에 같은 줄이 두 번씩 찍힙니다. 정상입니다(09단원 개념02에서 배우는 `StrictMode` 때문입니다).

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

import MenuBoard from "../_부품/MenuBoard.jsx";
import MenuCard from "../_부품/MenuCard.jsx";
import MenuRow from "../_부품/MenuRow.jsx";
```

웁기기 전 — 한 파일 안에 다 있는 코드

섹션 1

아래는 03단원 개념05의 `BigMenuBoard` 를 그대로 가져온 것입니다. 메뉴 세 줄을 통째로 적었습니다. 쪼개지 않은 코드입니다.

이렇게 이 파일 안에 직접 쓴 컴포넌트를 앞으로 '이 파일 안의 컴포넌트' 라고 부르겠습니다. 08단원 전에는 전부 이 모양이었습니다.

```
function BigMenuBoard() {
  return (
    <div className="output"> <h4>동네 카페 메뉴판</h4> <div> <strong>아메리카노</strong> - 4000원
    <br /> <small>가장 많이 팔립니다</small> </div> <div> <strong>라떼</strong> -
```

```

4500원
    <br /> <small>가장 많이 팔립니다</small> </div> <div> <strong>케이크</strong>
- 6000원
    <br /> <small>가장 많이 팔립니다</small> </div> </div>
);
}

```

03단원에서 이 코드의 문제를 이미 짚었습니다. - 같은 모양이 세 번 반복됩니다 - <small> 문구를 바꾸려면 세 군데를 고쳐야 합니다 - 함수 하나가 길어서 구조가 안 보입니다

그래서 03단원에서 MenuRow 로 쪼갠습니다. 그건 '한 파일 안에서' 쪼갠 것입니다. 이제 '파일로' 쪼갭니다. 두 가지는 얻는 것이 다릅니다.

— 한 파일 안에서 쪼개기 — 03단원

그 파일 안에서 고칠 곳이 한 곳이 됩니다.
하지만 다른 파일에서 쓰려면 여전히 복사해야 합니다.

— 파일로 쪼개기 — 08단원

프로젝트 전체에서 고칠 곳이 한 곳이 됩니다.
어느 파일에서든 `import` 한 줄로 가져다 씁니다.

```

function Section1Demo() {
  console.log("① 이 파일 안에 직접 쓴 BigMenuBoard 를 그렸습니다");
}

```

F12 콘솔 ① 이 파일 안에 직접 쓴 BigMenuBoard 를 그렸습니다

```

return (
  <div className="demo"> <h3>① 옮기기 전 - 이 파일 안에 통째로 적은 메뉴판
</h3> <BigMenuBoard />
  { /* 화면: 동네 카페 메뉴판 / 아메리카노 - 4000원 / 라떼 - 4500원 / 케이크 -
6000원 */}
  </div>
);
}

```

▹ 직접 해보기

1

BigMenuBoard 의 <small> 세 개를 전부 "오늘의 추천" 으로 바꾸세요. 몇 군데를 고쳤나요?

파일로 옮기기 — 세 단계

섹션 2

03단원의 MenuRow 를 파일로 옮겨 봅니다. 순서는 늘 같습니다.

1단계 · 새 파일을 만든다

```
src/08_파일_나누기와_스타일링/_부품/MenuRow.jsx
```

2단계 · 컴포넌트 함수를 통째로 옮겨 붙이고, 앞에 export default 를 붙인다

```
export default function MenuRow({ name, price }) {  
  ... 03단원 코드 그대로 ...  
}
```

3단계 · 쓰는 쪽 파일 맨 위에 import 줄을 넣는다

```
import MenuRow from "../_부품/MenuRow.jsx";
```

이게 전부입니다. **함수 안은 한 글자도 안 고칩니다.** _부품/MenuRow.jsx 를 열어서 03단원 개념05의 MenuRow 와 비교해 보세요. 앞에 export default 가 붙은 것 말고는 완전히 같습니다.

★ 왜 _부품 이라는 하위 폴더에 넣었나요?

왼쪽 메뉴는 단원 폴더 '바로 아래' 의 .jsx 만 훑습니다. MenuRow.jsx 를 08_파일_나누기와_스타일링 폴더 바로 아래에 두면 왼쪽 목록에 "MenuRow" 라는 항목이 하나 생겨 버립니다. 그건 예제가 아니라 부품이니 목록에 나오면 안 됩니다. 그래서 하위 폴더에 넣습니다. 하위 폴더는 메뉴가 안 훑습니다.

```
function Section2Demo() {  
  console.log("② 다른 파일에서 가져온 MenuRow 를 그렸습니다");  
}
```

F12 콘솔 ② 다른 파일에서 가져온 MenuRow 를 그렸습니다

```
return (  
  <div className="demo"> <h3>② 옮긴 뒤 - MenuRow 를 import 해서 세 번  
</h3> <div className="output"> <h4>동네 카페 메뉴판  
</h4> <MenuRow name="아메리카노" price={4000} /> <MenuRow name="라떼" price={4500}  
/> <MenuRow name="케이크" price={6000} /> </div>  
  { /* 화면: ① 과 똑같습니다 */ }  
</div>  
}
```

화면 ① 과 ② 를 비교해 보세요. 똑같습니다. 달라진 것은 코드입니다. - <small> 문구가 세상에 하나만 있습니다. _부품/MenuRow.jsx 안입니다. - 이 파일에서는 "제목 하나에 메뉴 세 줄" 이라는 구조만 보입니다. - 다른 단원 파일에서도 import 한 줄로 같은 MenuRow 를 씁니다.

▣ 직접 해보기 2

_부품/MenuRow.jsx 의 <small> 문구를 “오늘의 추천” 으로 바꾸세요. 화면 ㉔ 에서 몇 줄이 바뀌나요?
화면 ㉓ 은 어떤가요?

부품이 부품을 부릅니다

섹션 3

파일로 나누면 자연스럽게 층이 생깁니다. _부품/MenuBoard.jsx 를 열어 보세요. 이렇게 생겼습니다.

```
import MenuRow from "../MenuRow.jsx";

export default function MenuBoard() {
  return (
    <div className="output"> <h4>동네 카페 메뉴판
    </h4> <MenuRow name="아메리카노" price={4000} />
    ...
    </div>
  );
}
```

부품도 다른 부품을 import 합니다. 특별한 문법이 아닙니다. 그냥 같은 import 입니다.

지금 파일들이 이렇게 이어져 있습니다.

```
개념03 (지금 이 파일)
└─ MenuBoard.jsx
   └─ MenuRow.jsx
```

여기서 얻는 것이 하나 있습니다.

★ 이 파일은 MenuRow 를 몰라도 됩니다.

<MenuBoard /> 한 줄만 쓰면 메뉴판이 통째로 나옵니다. 안에 몇 줄이 들어 있는지, 그 줄이 어떻게 생겼는지 이 파일은 알 필요가 없습니다. 메뉴가 열 개로 늘어나도 이 파일은 한 글자도 안 고칩니다.

03단원에서 props 를 배울 때 “부모는 자식 속을 안 봐도 된다” 고 했던 것과 같습니다. 그것이 이제 파일 단위로 커진 것입니다.

★ 경로를 한 번 더 보세요. MenuBoard.jsx 안에서는 “../MenuRow.jsx” 입니다. (둘 다 _부품 안) 이 파일에서는 “../_부품/MenuRow.jsx” 입니다. 같은 파일을 가리키는데 경로가 다릅니다. 경로는 늘 ‘그 줄이 적힌 파일’ 이 기준입니다(개념02 섹션 4).

```
function Section3Demo() {
  const [showCards, setShowCards] = useState(false);

  console.log("③ MenuBoard 한 줄로 메뉴판 전체를 그렸습니다");
}
```

F12 콘솔 ③ MenuBoard 한 줄로 메뉴판 전체를 그렸습니다

```
return (
  <div className="demo"> <h3>③ MenuBoard 한 줄 - 안에 MenuRow 가 들어 있습니다
</h3> <MenuBoard />
  /* 화면: ① · ② 와 똑같은 메뉴판이 나옵니다 */

  <button onClick={() => setShowCards(!showCards)}>
    {showCards ? "카드 접기" : "MenuCard 도 보기"}
  </button>

  {showCards && (
    <div> <MenuCard name="아메리카노" price={4000} note="가장 많이 팔립니다" />
    /* 화면(누르면): 아메리카노 - 4000원 / 가장 많이 팔립니다 */
    <MenuCard name="삼각김밥" price={1200} />
    /* 화면(누르면): 삼각김밥 - 1200원 / 설명 없음 */
    </div>
  )}
</div>
);
}
```

MenuCard 는 note 를 안 넘기면 "설명 없음" 이 나옵니다. 03단원 개념03에서 배운 props 기본값이 그대로 살아 있습니다. 파일을 옮겼다고 문법이 달라지지 않습니다.

▹ 직접 해보기 3

_부품/MenuBoard.jsx 안에 <MenuRow name="삼각김밥" price={1200} /> 를 한 줄 넣으세요. 이 파일(개념03)은 몇 군데 고쳐야 하나요?

React 를 더 이상 안 꺼내도 되는 이유

섹션 4

★ 이 섹션이 이 파일에서 가장 중요합니다.

혹을 쓸 때는 늘 이렇게 꺼내 왔습니다.

```
import { useState } from "react";
```

그런데 useState 를 안 쓰는 파일을 보면 react 에서 꺼내 오는 줄이 아예 없습니다.

```
import React from "react"; 같은 줄이 한 줄도 없습니다.
```

_부품/MenuRow.jsx 에도 없습니다. 그런데 JSX 가 잘 돌아갑니다.

정말 없는지 확인해 봅시다.

```
function Section4Demo() {
  console.log(typeof React);
```

```
F12 콘솔    undefined
```

React 라는 이름이 아예 없습니다. 그런데도 아래 JSX 가 잘 그려집니다.

왜 그럴까요? JSX 가 무엇으로 바뀌는지부터 봐야 합니다.

옛 방식 — classic · 02단원 개념01에서 Babel 로 직접 돌려 본 그 결과입니다

```
<p>안녕</p>
↓
React.createElement("p", null, "안녕")
```

바뀐 코드 안에 React 라는 이름이 들어 있습니다. 그래서 이 방식을 쓰면 파일마다 import React from "react"; 가 꼭 있어야 했습니다. 빠뜨리면 "React is not defined" 에러가 났습니다.

지금 방식 — automatic · Vite 가 실제로 하는 일입니다

```
<p>안녕</p>
↓
import { jsx } from "react/jsx-runtime"; ← Vite 가 자동으로 넣어 줍니다
jsx("p", { children: "안녕" })
```

React 라는 이름을 안 씁니다. 대신 jsx 라는 함수를 씁니다. 그리고 그 함수를 가져오는 import 줄을 Vite 가 알아서 파일 맨 위에 넣어 줍니다. 이것을 'JSX 자동 변환' 이라고 부릅니다.

여러분이 쓴 파일에는 그 줄이 안 보입니다. 저장된 파일에도 안 들어갑니다. Vite 가 번역하는 순간에만 잠깐 끼워 넣습니다.

그래서 이 프로젝트에서는 이렇게 됩니다.

안 써도 되는 것 · import React from "react";

```
JSX 만 쓸 거라면 필요 없습니다.
```

써야 하는 것 · `import { useState } from "react";`

혹은 자동으로 안 들어옵니다. 직접 가져와야 합니다.

자동으로 들어오는 것은 'JSX 를 바꾸는 함수' 하나뿐입니다. `useState` · `useEffect` 같은 것은 여러분이 이름을 적어 가져와야 합니다. (`useEffect` 는 아직 안 배웠습니다. 09단원에서 배웁니다. 여기서는 이름만 나온 것입니다)

★ 인터넷의 옛날 예제에는 `import React from "react";` 가 맨 위에 있습니다. 자동 변환이 생기기 전에 쓴 코드입니다. 지금도 써도 에러는 안 납니다. 그냥 필요 없는 줄일 뿐입니다. 이 자료에서는 안 씁니다.

JSX 가 실제로 무엇이 되는지도 볼 수 있습니다.

```
const element = <p>아메리카노 4000원</p>;
```

```
console.log(typeof element);
```

F12 콘솔 object

```
console.log(element.type);
```

F12 콘솔 p

JSX 는 화면이 아니라 '값' 입니다. 평범한 객체입니다. 그 안에 "어떤 태그인지(type)" 가 들어 있습니다. 이 객체를 React 가 읽어서 진짜 화면을 만듭니다. 02단원 개념01에서 Babel 이 만든 결과를 찍어 봤던 것과 같은 이야기입니다.

```
return (
  <div className="demo"> <h3>④ React 없이 도는
  JSX</h3> <div className="output"> <div>typeof React 는 {String(typeof React)} 입니다
  </div>
    { /* 화면: typeof React 는 undefined 입니다 */
    <div>이 줄도 JSX 인데 잘 나옵니다</div>
    { /* 화면: 이 줄도 JSX 인데 잘 나옵니다 */
    <div>JSX 를 값으로 만들어 담아 둔 것: {element}</div>
    { /* 화면: JSX 를 값으로 만들어 담아 둔 것: 아메리카노 4000원 */
    </div> </div>
  );
}
```

⇒ 직접 해보기 4

Section4Demo 안에서 `const element2 = <strong>라떼 4500원</strong>;` 을 만들고 `console.log(element2.type)` 을 찍어 보세요. 무엇이 나올까요?

어떻게 나눌까

섹션 5

파일로 나눌 수 있게 됐다고 무조건 나누면 오히려 힘들어집니다. 03단원 개념05에서 “너무 잘게 쪼개지 마세요” 라고 한 것과 같은 이야기입니다.

— 언제 파일로 뺄까

- 뺄다 - 두 파일 이상에서 쓰인다
- 뺄다 - 그 컴포넌트만 50줄이 넘는다
- 뺄다 - 이름이 자연스럽게 붙는다 (MenuRow, PriceTag, Header)

안 뺄다 — 그 파일에서 한 번만 쓰인다 안 뺄다 — 다섯 줄짜리다

이 자료의 예제 파일들이 그 예입니다. Section1Demo · Section2Demo 같은 것은 이 파일에서만 쓰니 그냥 여기 둡니다. MenuRow 는 여러 곳에서 쓰니 파일로 뺐습니다.

— 파일 이름 규칙

컴포넌트 파일은 ‘컴포넌트 이름 그대로’ 짓습니다.

```
MenuRow.jsx → MenuRow
PriceTag.jsx → PriceTag
```

왜 이렇게 할까요? import 줄만 보고 무엇이 오는지 알기 위해서입니다.

```
import MenuRow from "../_부품/MenuRow.jsx"; ← 한눈에 보입니다
import A from "../_부품/x.jsx";             ← 열어 봐야 압니다
```

화면이 없는 파일(값·함수만 있는 파일)은 .js 로 두고 소문자로 시작합니다.

```
menuPrices.js → 값과 함수만 들어 있습니다
```

— 폴더 정리

이 자료에서는 이렇게 씁니다.

```
src/08_파일_나누기와_스타일링/
  개념02_import와_export.jsx    ← 왼쪽 메뉴에 나옵니다
  개념03_컴포넌트_파일로_나누기.jsx ← 왼쪽 메뉴에 나옵니다
  _부품/                        ← 메뉴에 안 나옵니다
    MenuRow.jsx
    MenuBoard.jsx
```

실제 프로젝트에서는 보통 이런 이름을 씁니다.

```
components/  화면 부품
pages/       화면 한 장 (11단원에서 봅니다)
utils/       값과 함수
```

이 자료가 _부품이라는 한글 이름을 쓰는 것은 “이건 예제가 아니라 부품이다”를 눈에 띄게 하려는 것뿐입니다.

— 한 파일에 컴포넌트 여러 개도 됩니다

`export default` 는 하나뿐이지만, 이름 있는 `export` 로 여러 개를 내보낼 수 있습니다.

다만 처음에는 ‘한 파일에 대표 하나’로 두는 편이 찾기 쉽습니다. 이 자료는 그 방식을 씁니다.

```
function Section5Demo() {
  const [count, setCount] = useState(0);

  function handleAdd() {
    setCount(count + 1);

    console.log("장바구니에 담았습니다. 지금", count + 1, "개");
  }
}
```

F12 콘솔 장바구니에 담았습니다. 지금 1 개

```

  }

  return (
    <div className="demo"> <h3>⑤ 파일을 나눠도 state 는 그대로입니다
  </h3> <MenuCard name="아메리카노" price={4000} note={`장바구니에 ${count}개`} />
    { /* 화면: 아메리카노 - 4000원 / 장바구니에 0개 */ }

    <button onClick={handleAdd}>담기</button>
    { /* 화면(누르면): 장바구니에 1개 */ }
  </div>
  );
}
```

state 는 이 파일에 있고, 화면은 다른 파일의 MenuCard 가 그림니다. 값은 props 로 내려갑니다. 07단원에서 배운 그대로입니다. 파일이 나뉘어도 부모-자식 관계는 하나도 안 바뀝니다.

▹ 직접 해보기 5

Section5Demo 의 MenuCard 에 note 를 넘기지 말고 지워 보세요. 담기 버튼을 눌러도 글자가 안 바뀌는 것을 확인하고 되돌리세요.

▹ 직접 해보기 6

_부품 폴더에 Footer.jsx 를 새로 만들고 export default functionFooter() { return <p>영업시간 09:00~21:00</p>; } 을 넣은 뒤, 이 파일에서 import 해서 Section5Demo 맨 아래에 넣으세요.

▹ 직접 해보기 7

방금 만든 Footer.jsx 를 _부품 폴더 밖(08_파일_나누기와_스타일링 바로 아래)으로 옮겨 보세요. 왼쪽 메뉴가 어떻게 되나요? 확인하고 다시 _부품 안으로 옮기세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

옮겨 놓고 export default 를 안 붙임

파일은 만들었는데 함수 앞에 export default 가 없는 경우입니다. 화면이 빨간 상자가 되고 이런 줄이 나옵니다. The requested module '/src/.../MenuRow.jsx' does not provide an export named 'default' 새 부품 파일을 만든 직후에 가장 많이 나는 실수입니다.

이런 실수를 합니다

옮겨 놓고 원래 자리의 함수를 안 지움

MenuRow 를 파일로 뺐는데, 이 파일에도 옛날 MenuRow 가 남아 있는 경우입니다. 그러면 이런 에러가 납니다.

SyntaxError · Identifier 'MenuRow' has already been declared

“MenuRow 라는 이름이 이미 있다” 는 뜻입니다. import 로 들어온 이름과 이 파일에서 만든 이름이 부딪힌 것입니다. 옮겼으면 원래 자리는 지우세요.

이런 실수를 합니다

부품 파일을 단위 폴더 바로 아래에 둬 ★ 에러가 안 납니다

MenuRow.jsx 를 _부품 안에 아니라 08_파일_나누기와_스타일링 바로 아래에 두면 왼쪽 메뉴에 "MenuRow" 라는 항목이 생깁니다. 에러는 안 납니다. 코드도 잘 돌아갑니다. 목록만 지저분해집니다. 게다가 그 항목을 눌러 보면 props 를 아무것도 안 받은 상태로 그려져서 "— 원" 처럼 이상하게 나옵니다 (03단원 개념05 실수 3).



7번에서 직접 확인해 보세요.

이런 실수를 합니다

부품 파일 안에서 props 이름을 바꿈

_부품/MenuRow.jsx 의 { name, price } 를 { title, price } 로 바꿔 놓고 쓰는 쪽은 name="아메리카노" 로 그대로 두는 경우입니다. 에러가 안 납니다. 이름 자리만 조용히 비어 버립니다. 파일이 나뉘어 있으면 이런 어긋남을 못 보고 지나치기 쉽습니다. 부품을 고칠 때는 그 부품을 쓰는 곳도 같이 보세요.

이런 실수를 합니다

import React from "react"; 를 안 썼다고 걱정함

실수라기보다 오해입니다. 안 써도 됩니다. 섹션 4에서 확인했습니다. 인터넷 예제를 보고 그 줄을 넣어도 에러는 안 납니다. 그냥 필요 없는 줄입니다. ★ 다만 혹은 다릅니다. useState 를 import 안 하고 쓰면 "useState is not defined" 에러가 납니다. 이건 자동으로 안 들어옵니다.

화면 —————

정리

- 1 컴포넌트를 파일로 옮기는 것은 세 단계입니다. 새 파일 만들기 → 함수를 통째로 옮기고 export default 붙이기 → 쓰는 쪽에 import 줄 넣기. 함수 안은 한 글자도 안 고칩니다.
- 2 03단원의 '한 파일 안에서 쪼개기' 는 그 파일 안에서만 이득이지만, 파일로 쪼개면 프로젝트 전체에서 고칠 곳이 한 곳이 됩니다.
- 3 부품도 다른 부품을 import 할 수 있습니다. MenuBoard 를 쓰는 쪽은 그 안의 MenuRow 를 몰라도 됩니다.
- 4 JSX 는 Vite 가 자동으로 react/jsx-runtime 의 함수로 바꿔 줍니다. 그래서 import React from 'react'; 를 안 써도 됩니다. typeof React 는 undefined 입니다.
- 5 자동으로 들어오는 것은 JSX 를 바꾸는 함수뿐입니다. useState 같은 혹은 직접 import 해야 합니다.
- 6 두 곳 이상에서 쓰이거나 길어지면 파일로 빼고, 한 번만 쓰이는 작은 것은 그대로 둡니다. 파일 이름은 컴포넌트 이름 그대로 짓습니다.
- 7 import 해서 쓸 부품은 _부품 같은 하위 폴더에 둡니다. 단원 폴더 바로 아래에 두면 왼쪽 메뉴에 예제처럼 나타납니다.

```
export default function Concept03SplitFiles() {
  const [restartKey, setRestartKey] = useState(0);

  return (
    <div> <h1>개념 03 - 컴포넌트 파일로 나누기</h1> <p className="guide">
      이 파일이 쓰는 부품은 <code>src/08_파일_나누기와_스타일링/_부품</code> 안에
      있습니다.
      <strong> MenuRow.jsx · MenuBoard.jsx · MenuCard.jsx</strong> 를 함께 열어
      놓고
      보세요.
      <br /> <br />
      세 파일 모두 <strong>03단원 개념05의 코드를 그대로 옮긴 것</strong>입니다.
      앞에{" "}
      <code>export default</code> 가 붙은 것 말고는 달라진 데가 없습니다.
      <br /> <br /> <strong>F12 → Console</strong> 도 함께 열어 두세요. 섹션 4는
      콘솔로 확인합니다.
      </p> <button onClick={() => setRestartKey(restartKey + 1)}>
        이 예제를 처음부터 다시 그리기
      </button> <div key={restartKey}> <Section1Demo /> <Section2Demo
      /> <Section3Demo /> <Section4Demo /> <Section5Demo /> </div> </div>
    );
  }
}
```

직접 해보기 정답

1) BigMenuBoard 안의 <small> 세 개를 전부 고쳐야 합니다.

```
// 화면: ① 의 세 줄이 모두 "오늘의 추천" 이 됩니다.
```

→ 두 군데만 고치면 한 줄만 옛날 문구로 남습니다. 에러가 안 나서 못 보고 지나칩니다.

```
03단원 개념05에서 했던 것과 같은 문제이고, 답도 같습니다. 쪼개면 한 곳이 됩니다.
```

2) _부품/MenuRow.jsx 안의 <small> 한 곳만 고칩니다.

```
// 화면: ② 의 세 줄이 전부 바뀝니다. ③ 의 메뉴판도 함께 바뀝니다.  
//      ① 은 안 바뀝니다.
```

→ ② 와 ③ 이 같이 바뀐 것이 핵심입니다.

```
③ 의 MenuBoard 도 안에서 같은 MenuRow 를 쓰기 때문입니다.  
① 은 이 파일 안에 따로 적은 코드라 아무 상관이 없습니다.  
한 곳을 고쳐 두 화면이 바뀌는 것 - 이것이 파일로 나누는 이유입니다.  
확인했으면 "가장 많이 팔립니다" 로 되돌려 두세요.
```

3) _부품/MenuBoard.jsx 안에 한 줄만 넣습니다.

```
<MenuRow name="삼각김밥" price={1200} />
```

```
화면      ③ 의 메뉴판에 "삼각김밥 - 1200원 / 가장 많이 팔립니다" 가 늘어납니다.
```

→ 이 파일(개념03)은 한 군데도 안 고칩니다. 그게 이 문제의 답입니다.

```
<MenuBoard /> 라고만 써 뒀으니 안이 몇 줄이든 상관이 없습니다.  
② 는 이 파일에서 MenuRow 를 직접 세 번 쓴 것이라 안 바뀝니다.
```

```
4) const element2 = <strong>라떼 4500원</strong>;  
   console.log(element2.type);  
   // 콘솔: strong
```

→ JSX 를 쓴 만큼 값이 만들어지고, type 에는 태그 이름이 들어갑니다.

```
<MenuRow /> 처럼 우리가 만든 컴포넌트를 넣으면 type 은 글자가 아니라 함수입니다.  
React 가 소문자와 대문자를 구분하는 이유가 여기 있습니다(01단원 개념04).
```

```
5) <MenuCard name="아메리카노" price={4000} />  
   // 화면: 아메리카노 - 4000원 / 설명 없음  
   //      담기를 눌러도 "설명 없음" 그대로입니다.
```

→ note 를 안 넘겼으니 MenuCard 의 기본값이 쓰입니다.

count 가 올라가도 MenuCard 에 전해 주는 값이 없으니 화면이 안 바뀝니다.
콘솔에는 숫자가 계속 올라갑니다. state 는 잘 바뀌고 있는데
화면에 안 이어져 있을 뿐입니다.

6) 새 파일 src/08_파일_나누기와_스타일링/_부품/Footer.jsx :

```
export default function Footer() {
  return <p>영업시간 09:00~21:00</p>;
}
```

이 파일 맨 위에:

```
import Footer from "../_부품/Footer.jsx";
```

Section5Demo 의 </div> 바로 앞에:

```
<Footer />
```

화면 ⑤ 상자 맨 아래에 "영업시간 09:00~21:00" 이 나옵니다.

→ import 줄을 빠뜨리면 "Footer is not defined" 로 빨간 상자가 됩니다.

export default 를 빠뜨리면 does not provide an export named 'default' 가 납니다.

7) Footer.jsx 를 08_파일_나누기와_스타일링 폴더 바로 아래로 옮기면:

```
// 화면: 왼쪽 메뉴의 08 단원에 "Footer" 라는 항목이 하나 늘어납니다.
//      그 항목을 누르면 "영업시간 09:00~21:00" 만 덩그러니 나옵니다.
```

→ 에러는 안 납니다. 다만 부품이 예제인 척 목록에 끼어든 것입니다.

import 경로도 "../Footer.jsx" 로 고쳐야 화면이 다시 돌아옵니다.
_부품 안으로 돌려놓고 경로도 "../_부품/Footer.jsx" 로 되돌리세요.

CSS 넣기.css

```

/* =====
08단원 개념05 가 쓰는 CSS - 전역(global) CSS 입니다
-----
이 파일은 개념04_CSS_넣기.jsx 의 맨 위에서 이렇게 불러 옵니다.

import "../개념04_CSS_넣기.css";

★ '전역' 이라는 말의 뜻:
한 번 불러 오면 이 파일의 규칙이 프로젝트 화면 전체에 걸립니다.
이 예제를 한 번 열었다가 다른 예제로 옮겨 가도 이 규칙은 그대로 살아 있습니다.
그래서 이름이 겹치면 남의 화면을 망가뜨릴 수 있습니다.
그것을 막으려고 이 파일의 이름은 전부 c05 로 시작하게 지었습니다.
(개념05 섹션 4에서 이 이야기를 자세히 합니다)

★ CSS 는 이 자료에서 가르치지 않습니다.
여기 있는 색·여백 같은 것은 몰라도 됩니다.
배우는 것은 "CSS 파일을 어떻게 넣고, 어떻게 이름을 붙이는가" 입니다.
===== */

.c05Card {
  background: #fff;
  border: 1px solid #ccc;
  border-left: 4px solid #2d6cdf;
  padding: 10px 14px;
  margin: 6px 0;
}

.c05Title {
  font-weight: bold;
  font-size: 16px;
}

.c05Price {
  color: #2d6cdf;
  font-weight: bold;
}

```

```
/* 두 개를 같이 붙였을 때만 달라지게 하는 것도 됩니다 */
.c05Sale {
  border-left-color: #c00;
  background: #fff8f8;
}

.c05SoldOut {
  color: #999;
  text-decoration: line-through;
}

/* 섹션 4에서 '이름 부딪힘' 을 보여 주려고 만든 상자입니다.
   같은 이름이 개념05_다른팀.css 에도 있습니다. */
.c05Box {
  border: 2px solid #2d6cdf;
  background: #eef4ff;
  padding: 10px 14px;
}
```

CSS 넣기

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

08단원의 마지막 개념 파일입니다. 개념02에서 import 를 배우고, 개념03에서 컴포넌트를 파일로 나눴습니다. 남은 것은 화면 모양, 즉 CSS 입니다.

★ 이 자료는 CSS 를 가르치지 않습니다. 색을 어떻게 고르고 여백을 어떻게 주는지는 다루지 않습니다. 여기서 배우는 것은 딱 세 가지입니다.

- 1 CSS 파일을 어떻게 프로젝트에 넣는가
- 2 className 을 어떻게 붙이는가 (여러 개 / 조건부)
- 3 이름이 겹치면 무슨 일이 나고, 어떻게 막는가

★ 콘솔에 같은 줄이 두 번씩 찍힙니다. 정상입니다(09단원 개념02에서 배우는 StrictMode 때문입니다).

```
import { useState } from "react";
import Summary from "../ui/Summary.jsx";
```

[1] 전역 CSS — 이름을 안 받습니다. 그냥 “이 파일도 같이 실어 달라” 는 뜻입니다.

```
import "../개념04_CSS_넣기.css";
import "../개념05_다른팀.css";
```

[2] CSS Modules — 이름을 받습니다. 섹션 5에서 다룹니다.

```
import styles from "../개념04_CSS_넣기.module.css";
```

스타일은 이미 들어와 있습니다

섹션 1

01단원부터 지금까지 .demo · .output 같은 클래스를 그냥 써 왔습니다. 어디에도 <style> 을 쓴 적이 없는데 모양이 나왔습니다. 그 정체가 이것입니다.

스타일은 처음부터 한 파일에 모여 있었습니다.

```
src/index.css
```

열어 보면 .demo · .output · .guide · .summary 같은 이름이 그대로 들어 있습니다. 01단원부터 쪽 쓰던 그 이름들입니다.

그런데 이 파일(개념04)은 index.css 를 import 하지 않았습니다. 그래도 화면 모양이 나옵니다. 왜일까요?

src/main.jsx 를 열어 보면 이런 줄이 있습니다.

```
import "../index.css";
```

main.jsx 는 앱이 켜질 때 딱 한 번 도는 파일입니다(01단원 개념02). 거기서 한 번 불러 두면 프로젝트 전체에 걸립니다. 그래서 각 예제 파일에서 다시 import 할 필요가 없습니다.

이것이 '전역 CSS' 입니다. 한 번 들어오면 화면 전체에 걸립니다. 편리한 만큼 위험한 점도 있습니다. 섹션 4에서 그 이야기를 합니다.

```
function Section1Demo() {
  console.log("① index.css 는 main.jsx 가 이미 불러 두었습니다");
```

F12 콘솔 ① index.css 는 main.jsx 가 이미 불러 두었습니다

```
return (
  <div className="demo"> <h3>① 이 상자들은 index.css 가 만든 모양입니다
</h3> <p className="output">className=&quot;output&quot; - 흰 상자에 회색 테두리</p>
  { /* 화면: 흰 배경에 회색 테두리가 있는 상자 */}

  <p className="error">className=&quot;error&quot; - 빨간 굵은 글씨</p>
  { /* 화면: 빨간 굵은 글씨 */}

  <p className="done">className=&quot;done&quot; - 취소선</p>
  { /* 화면: 회색 취소선이 그어진 글씨 */}
</div>
);
}
```

▹ 직접 해보기 1

src/index.css 를 열어 .output 규칙이 몇 줄인지 찾아보세요. (읽기만 하세요. 이 파일은 고치지 않습니다)

내 CSS 파일 넣기

섹션 2

예제마다 필요한 모양이 다를 수 있습니다. 그럴 때 CSS 파일을 따로 만듭니다. 이 파일은 같은 폴더에 이런 파일을 하나 두었습니다.

```
src/08_파일_나누기와_스타일링/개념04_CSS_넣기.css
```

그리고 이 파일 맨 위에서 이렇게 불러왔습니다.

```
import "../개념04_CSS_넣기.css";
```

★ 개념02에서 본 import 와 모양이 다릅니다. 이름을 안 받습니다.

```
import Greeting from "../부품/Greeting.jsx";   ← 이름을 받습니다
import "../개념04_CSS_넣기.css";               ← 이름이 없습니다
```

CSS 파일에는 가져올 값이 없기 때문입니다. 함수도 변수도 없습니다. 그래서 "가져올 건 없고, 이 파일도 같이 실어 달라" 는 뜻만 남습니다.

★ 그런데 CSS 는 자바스크립트가 아닌데 어떻게 import 가 되나요?

Vite 가 대신 처리해 줍니다. Vite 는 이 줄을 보고 CSS 파일을 읽어서, 브라우저 화면에 <style> 태그를 하나 만들어 붙입니다. HTML 에 손으로 <style> 을 넣는 것과 같은 일을 Vite 가 대신 해 주는 것입니다. 그래서 F12 → Elements 로 보면 <head> 안에 style 태그가 들어 있습니다.

★ 그리고 CSS 파일을 고치고 저장하면 화면 색만 바뀝니다.

페이지가 새로 열리지도 않고 state 도 안 사라집니다. HMR 이 CSS 에도 걸립니다.

★ .css 파일은 단원 폴더 바로 아래에 뒤도 됩니다. 왼쪽 메뉴는 .jsx 만 훑기 때문에 .css 는 목록에 안 나타납니다. (개념03에서 부품 .jsx 는 _부품 폴더에 넣어야 했던 것과 다릅니다)

파일 이름은 자유입니다. 보통은 쓰는 파일과 같은 이름으로 짓습니다. 그래야 나중에 짝을 찾기 쉽습니다.

```
function Section2Demo() {
  console.log("② 내가 만든 CSS 파일의 이름을 쓰고 있습니다");
```

F12 콘솔 ② 내가 만든 CSS 파일의 이름을 쓰고 있습니다

```
return (
  <div className="demo"> <h3>② 내 CSS 파일이 만든 모양
</h3> <div className="c05Card"> <div className="c05Title">아메리카노
</div> <div className="c05Price">4000원</div> </div>
  {/* 화면: 왼쪽에 파란 붉은 세로줄이 있는 흰 카드. 가격은 파란 붉은 글씨 */}

  <div className="c05Card"> <div className="c05Title">라떼
</div> <div className="c05Price">4500원</div> </div>
  {/* 화면: 같은 모양의 카드 하나 더 */}
</div>
);
}
```

◦ 직접 해보기 2

개념04_CSS_넣기.css 의 .c05Price 색을 #2d6cdf 에서 #2e9e4f (초록) 로 바꾸고 저장하세요.
화면이 어떻게 되나요?

Section2Demo 에 케이크 6000원 카드를 하나 더 넣으세요. (CSS 는 안 고칩니다)

className 다시 보기

섹션 3

className 은 02단원 개념03에서 이미 배웠습니다. 복습부터 합니다.

왜 class 가 아니라 className 인가?

`class` 는 자바스크립트가 이미 쓰고 있는 낱말이라 속성 이름으로 못 씁니다.
그래서 `React` 는 `className` 을 씁니다. 브라우저에 붙을 때는 `class` 가 됩니다.

여기서는 08단원에 필요한 두 가지를 더 봅니다.

[1] 여러 개 붙이기 — 공백으로 나눠 씁니다

```
<div className="c05Card c05Sale">
```

CSS 에서 .c05Card 와 .c05Sale 이 둘 다 걸립니다. .c05Sale 은 테두리 색과 배경만 바꾸므로, 카드 모양은 그대로 두고 색만 달라집니다. 이렇게 '기본 모양 + 상태' 로 나눠 두면 조합해 쓸 수 있습니다.

[2] 조건에 따라 바꾸기 — 그냥 문자열을 만들면 됩니다

```
className={isSale ? "c05Card c05Sale" : "c05Card"}
```

className 은 결국 '글자' 입니다. 그러니 05단원에서 배운 삼항 연산자로 글자를 골라 넣으면 됩니다. 새 문법이 아닙니다.

템플릿 리터럴로 써도 똑같습니다.

```
className={`c05Card ${isSale ? "c05Sale" : ""}`}
```

둘 중 읽기 편한 쪽을 쓰세요. 이 자료는 삼항 쪽을 씁니다.

[3] style={{ }} 과 어떻게 나눠 쓰나

02단원 개념03에서 `style={{ color: "red" }}` 도 배웠습니다. 둘 다 됩니다.

- className - 정해진 모양. 여러 곳에서 같이 쓰는 것. CSS 파일에 적습니다.
- style - 그때그때 계산해서 정해지는 값. 예를 들어 막대 그래프의 길이.

섞어 써도 됩니다. 보통은 `className` 을 주로 쓰고, 계산이 필요할 때만 `style` 을 씁니다.

```
function Section3Demo() {
  const [isSale, setIsSale] = useState(false);
  const [soldOut, setSoldOut] = useState(false);
```

className 은 결국 글자입니다. 무엇이 들어가는지 콘솔로 확인해 봅시다.

```
const cardClass = isSale ? "c05Card c05Sale" : "c05Card";

console.log("지금 붙은 className:", cardClass);
```

F12 콘솔 지금 붙은 className: c05Card

```
return (
  <div className="demo"> <h3>③ className 여러 개 붙이기와 조건부
</h3> <div className={cardClass}> <div className={soldOut ? "c05Title c05SoldOut" :
"c05Title"}>케이크</div> <div className="c05Price">6000원</div> </div>
  {/* 화면: 파란 세로줄 카드에 "케이크 / 6000원" */}

  <button onClick={() => setIsSale(!isSale)}>
    {isSale ? "할인 끄기" : "할인 켜기"}
  </button>
  {/* 화면(누르면): 카드 세로줄이 빨갭게 바뀌고 배경이 연분홍이 됩니다 */}

  <button onClick={() => setSoldOut(!soldOut)}>
    {soldOut ? "판매 재개" : "품절 표시"}
  </button>
  {/* 화면(누르면): "케이크" 글자에 취소선이 그어집니다 */}

  <p className="output">지금 className: {cardClass}</p>
  {/* 화면: 지금 className: c05Card */}
</div>
);
}
```

▫ 직접 해보기 4

Section3Demo 의 cardClass 를 템플릿 리터럴로 다시 쓰세요. `c05Card ${isSale ? "c05Sale" : ""}` 로 바뀌도 똑같이 도는지 보세요.

▫ 직접 해보기 5

가격 줄에 `style={{ fontSize: 20 }}` 을 붙여 보세요. className 과 style 이 같이 걸리는 것을 확인하세요.

전역 CSS 의 문제 — 이름이 부딪힙니다

섹션 4

여기가 이 파일에서 가장 중요한 부분입니다.

전역 CSS 는 한 번 불러 오면 화면 전체에 걸립니다. 그래서 두 파일이 같은 이름을 쓰면 서로 부딪힙니다.

이 폴더에는 일부러 그런 파일을 하나 더 만들어 두었습니다.

```
개념04_CSS_넣기.css → .c05Box { border: 2px solid 파랑; 배경 연파랑 }
개념05_다른팀.css   → .c05Box { border: 2px dashed 빨강; 배경 연분홍 }
```

이름이 똑같습니다. 이 파일 맨 위에서 둘 다 import 했습니다.

```
import "../개념04_CSS_넣기.css";
import "../개념05_다른팀.css";    ← 이쪽이 나중입니다
```

결과가 어떻게 될까요? 아래 화면 ④ 를 보세요.

★ 에러가 안 납니다. 경고도 없습니다. ★ 나중에 불러 온 쪽이 이깁니다. 빨간 점선이 나옵니다.

지금은 두 파일이 나란히 보이니 알아채기 쉽습니다. 그런데 실제 프로젝트에서는 CSS 파일이 스무 개쯤 됩니다. 어제 만든 화면이 오늘 갑자기 이상해졌는데, 원인은 다른 사람이 오늘 만든 CSS 파일의 이름이 겹친 것 — 이런 일이 실제로 일어납니다.

— 지금까지 쓴 해결책

이름 앞에 붙임말을 붙이는 것입니다. 이 파일이 c05 를 붙인 이유가 그것입니다.

```
.c05Card · .c05Title · .c05Price
```

겹칠 확률이 크게 줄어듭니다. 하지만 사람이 지키는 규칙이라 언젠가 깨집니다. 붙임말을 빠뜨린 것을 아무도 못 잡아 줍니다.

그리고 한 가지 더 확인할 것이 있습니다.

★ 이 예제를 한 번 열면, 다른 예제로 옮겨 가도 이 CSS 는 살아 있습니다.

전역이라는 말이 그런 뜻입니다. 예제를 바꾼다고 CSS 가 사라지지 않습니다. 만약 이 파일이 .demo 나 .output 같은 이름을 다시 정의했다면, 09단원·10단원 예제까지 전부 이상해졌을 것입니다. c05 를 붙여 둔 덕분에 아무 일도 안 일어난 것입니다.

이것을 규칙이 아니라 도구로 막는 방법이 다음 섹션의 CSS Modules 입니다.

```
function Section4Demo() {
  console.log("④ 같은 이름의 .c05Box 가 두 파일에 있습니다");
```

F12 콘솔 ④ 같은 이름의 .c05Box 가 두 파일에 있습니다

```
return (
  <div className="demo"> <h3>④ 이름이 겹치면 나중 것이 이깁니다
</h3> <div className="c05Box">
  이 상자의 className 은 c05Box 하나입니다.
```

CSS 넣기.module.css

```

/* =====
08단원 개념05 섹션 5 - CSS Modules
=====
* 파일 이름이 .module.css 로 끝납니다. 이게 전부입니다.
  Vite 는 .module.css 로 끝나는 파일을 특별하게 다룹니다.

여기 적은 card · title · price 라는 이름은
Vite 가 '아무도 안 쓸 것 같은 긴 이름' 으로 바꿔서 내보냅니다.
그래서 다른 파일에 card 라는 이름이 있어도 부딪히지 않습니다.

쓰는 쪽에서는 이렇게 씁니다.

import styles from "../개념04_CSS_넣기.module.css";
...
<div className={styles.card}>

자세한 설명은 개념05 섹션 5에 있습니다.
===== */

.card {
  background: #fff;
  border: 1px solid #ccc;
  border-left: 4px solid #2e9e4f;
  padding: 10px 14px;
  margin: 6px 0;
}

.title {
  font-weight: bold;
  font-size: 16px;
  color: #2e9e4f;
}

.price {
  font-weight: bold;
}

```

```
.sale {  
  border-left-color: #c00;  
  background: #fff8f8;  
}
```

다른팀.css

```
/* =====  
08단원 개념05 섹션 4 - '남이 만든 CSS 파일' 역할입니다  
-----  
개념04_CSS_넣기.css 에 이미 .c05Box 가 있는데,  
이 파일에도 똑같은 이름으로 .c05Box 를 만들어 두었습니다.  
  
두 파일을 다 import 하면 어떻게 될까요?  
에러는 안 납니다. 나중에 불러 온 쪽이 이깁니다.  
그 장면을 개념05 섹션 4에서 화면으로 확인합니다.  
===== */  
  
.c05Box {  
  border: 2px dashed #c00;  
  background: #fff4f4;  
  padding: 10px 14px;  
}
```

styled-components

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념04에서 CSS 를 넣는 방법 두 가지를 봤습니다.

전역 CSS `import "./파일.css"` 이름이 겹치면 조용히 덮입니다
CSS Modules `import styles from ...` Vite 가 이름을 새로 지어 줍니다

여기서는 세 번째 방법을 봅니다. **styled-components** 입니다.

★ 앞의 둘과 생각이 다릅니다. 앞의 둘은 "CSS 파일을 따로 두고 이름으로 잇는" 방식입니다. styled-components 는 **CSS 를 컴포넌트로 만듭니다**. 파일을 따로 안 둡니다.

★★ 셋 중 뭐가 맞는지는 섹션 4에서 정리합니다. 지금은 "이런 것도 있다" 가 아니라 **무엇이 달라지는가**를 보세요. 특히 섹션 2(props)는 앞의 둘로는 못 하던 것입니다.

★ 콘솔에 같은 줄이 두 번씩 찍힙니다. 정상입니다(09단원 개념02에서 배우는 StrictMode 때문입니다).

★★ 이 파일은 **일부러 경고를 냅니다**. 섹션 5에서 잘못된 코드를 돌리기 때문입니다. 콘솔에 노란 줄이 보이면 고장이 아니라 그 실습입니다.

↑ 섹션 5에서 **일부러** 잘못된 코드를 돌립니다. 그때 나는 경고를 미리 적어 둔 것입니다.

검증 도구는 여기 적힌 경고가 진짜로 나는지까지 확인합니다.

```
import { useState } from "react";
import styled from "styled-components";
import Summary from "../_ui/Summary.jsx";
```

CSS 를 컴포넌트로 만듭니다

섹션 1

아래 한 덩어리가 **컴포넌트 하나**입니다. `styled.button` 뒤에 백틱을 붙이고 그 안에 CSS 를 그대로 씁니다.

```
const 담기버튼 = styled.button`
  background: #2d6cdf;
  color: white;
  border: none;
  border-radius: 6px;
  padding: 8px 16px;
  font-size: 15px;
  font-family: inherit;
  cursor: pointer;
`;
```

★ 이제 <담기버튼>담기</담기버튼> 처럼 씁니다. className 이 없습니다.

개념04 방식 styled-components 방식

.css 파일을 만든다 만들 파일이 없습니다 className="c05Btn" <담기버튼> 이름을 서로 맞추다 맞추 이름이 없습니다

★★ 이름을 맞추 필요 없다는 게 핵심입니다. 개념04 섹션6 [실수 3]에서 "className 오타는 아무 말도 안 해 준다" 고 했습니다. 여기서는 이름을 안 쓰니 오타가 날 곳이 없습니다. <담기버튼> 을 잘못 쓰면 그냥 에러가 납니다. 조용히 지나가지 않습니다.

백틱 안은 그냥 CSS 입니다. 중첩도 됩니다.

```
const 상자 = styled.div`
  border: 1px solid #ddd;
  border-radius: 6px;
  padding: 12px;
  margin: 8px 0;

  h4 {
    margin: 0 0 6px;
    font-size: 15px;
  }
`;
```

★ h4 { ... } 는 "이 상자 **안의** h4" 라는 뜻입니다. CSS 에서 .상자 h4 { } 라고 쓰던 것과 같습니다.

★★ 밖의 h4 는 영향을 안 받습니다. 이 상자 안에서만 먹습니다.

```
function Section1Demo() {
  const [담은수, set담은수] = useState(0);

  return (
    <div className="demo"> <h3>① CSS 를 컴포넌트로</h3> <상자> <h4>삼각김밥
```



```
</h4> <div>1200원</div> <답기버튼 onClick={() => set담은수(담은수 + 1)}>답기</답기버튼> </상자> <div className="output">담은 개수: {담은수}</div> </div>
);
}
```

★ `onClick` 이 그냥 먹습니다. `<답기버튼>` 이 결국 `<button>` 이기 때문입니다. `styled-components` 가 만든 것은 **button** 을 감싼 컴포넌트입니다. 그래서 `button` 에 주던 것(`onClick`, `disabled`, `type`...)을 그대로 줄 수 있습니다.

직접 해보기 1

답기버튼의 `background` 를 `#d9534f` 로 바꿔 보세요. 저장하는 순간 화면이 바뀌니까? 담은 개수는 그대로입니까?

직접 해보기 2

`styled.button` 을 `styled.a` 로 바꾸고 `<답기버튼 href="#">` 처럼 써 보세요. 무엇이 달라집니까?

props로 모양을 바꿉니다

섹션 2

여기가 앞의 두 방법과 제일 크게 다른 곳입니다.

개념04에서 조건부 스타일을 이렇게 했습니다.

```
const cardClass = isSale ? "c05Card c05Sale" : "c05Card";
<div className={cardClass}>
```

클래스 이름을 글자로 조립했습니다. `styled-components` 는 값을 그대로 받습니다.

```
const 값표시 = styled.span`
  font-weight: bold;
  color: ${(props) => (props.$큰가 ? "#d9534f" : "#2d6cdf")};
  font-size: ${(props) => (props.$큰가 ? "20px" : "15px")};
`;
```

★★★ \$ 를 왜 붙이나 — 이게 이 섹션의 핵심입니다.

\$ 로 시작하는 props 는 스타일을 정하는 데만 쓰고 DOM 으로 안 내려갑니다. 이런 것을 transient props (지나가는 props) 라고 부릅니다.

\$ 를 빼고 `<값표시 큰가={true}>` 라고 쓰면 — `큰가` 를 진짜 HTML 속성으로 알고 `<span 큰가="true">` 를 만들려 합니다.

★★ 그러면 경고가 두 개 납니다. 진짜 크롬으로 재 본 것입니다.

① `styled-components` 가 먼저 말합니다

styled-components: it looks like an unknown prop "큰가" is being sent through to the DOM, which will likely trigger a React console error.
... or consider using transient props (`\$` prefix for automatic filtering.)

② 그 다음 React 가 실제로 화를 냅니다

Received `true` for a non-boolean attribute `큰가`.
If you want to write it to the DOM, pass a string instead:
큰가="true" or 큰가={value.toString()}.

★ ①의 마지막 줄이 답을 그대로 알려 줍니다. "transient props (\$ prefix)" 를 쓰라고요.

경고문을 끝까지 읽으면 고치는 법이 대개 그 안에 있습니다.

★★ 화면은 멀쩡하게 나옵니다. 색도 크기도 제대로 바뀝니다.

콘솔에만 쌓입니다. 그래서 안 보고 지나가기 쉽습니다. 섹션 6에서 다시 다룹니다.

```
function Section2Demo() {
  const [금액, set금액] = useState(1200);
  const 큰가 = 금액 >= 5000;

  return (
    <div className="demo"> <h3>② props 로 모양 바꾸기</h3> <div> <button onClick={() => set금액(1200)}>1200원</button>{" "}
      <button onClick={() => set금액(8000)}>8000원
    </button> </div> <div className="output">
      가격: <값표시 $큰가={큰가}>{금액}원</값표시>
      {큰가 && " ← 5000원이 넘어 빨간 큰 글씨가 됩니다"}
    </div> </div>
  );
}
```

★★ 클래스 이름을 조립하지 않았습니니다. 값을 그대로 넘겼습니니다. \$큰가={큰가} 하나로 색과 크기가 같이 바뀝니다.

CSS Modules 로 같은 것을 하려면 —

```
.큰 { color: #d9534f; font-size: 20px; }
className={` ${styles.값} ${큰가 ? styles.큰 : ""} `}
```

클래스를 하나 더 만들고 이름을 조립해야 합니다.

★ 값이 두 가지(크다/작다)면 별 차이가 없습니다.

값이 이어지는 수(0~100 같은)면 이야기가 달라집니다.
클래스로는 100개를 미리 만들 수 없습니다.

▹ 직접 해보기 3

값표시 에 \$흐림 props 를 하나 더 만들어 참이면 opacity 를 0.4 로 만들어 보세요.

▹ 직접 해보기 4

\$큰가 를 큰가 로 (달러를 빼고) 바꿔 보세요. 화면은 그대로입니까? F12 → Console 에 무엇이 뜹니까?
확인했으면 되돌려 두세요.

이름이 겹치지 않습니다

섹션 3

개념04 섹션4에서 전역 CSS 의 이름이 부딪히는 것을 봤습니다. `.card` 를 두 팀이 각자 정의하면 나중에 불러 온 쪽이 이겼습니다.

styled-components 는 클래스 이름을 **직접 짓지 않습니다**. 그래서 부딪힐 이름이 없습니다.

```
const 파랑상자 = styled.div`
  background: #eef4ff;
  padding: 8px;
  margin: 4px 0;
`;

const 초록상자 = styled.div`
  background: #eefaf0;
  padding: 8px;
  margin: 4px 0;
`;

function Section3Demo() {
  return (
    <div className="demo"> <h3>③ 이름이 겹치지 않습니다</h3> <파랑상자>
    파랑상자입니다</파랑상자> <초록상자>초록상자입니다</초록상자>
    <div className="output"> <div>파랑상자가 받은 class: {파랑상자.styledComponentId}
    </div> <div>초록상자가 받은 class: {초록상자.styledComponentId}</div> <div style=
    {{ marginTop: 6 }}>
      * F12 로 위 상자를 눌러 보세요. class 가 두 개 붙어 있습니다.
    </div> </div> </div>
  );
}
```

★ 진짜 화면에서는 이렇게 나옵니다.

```
<div class="sc-bdvwhi bjovqr">파랑상자입니다</div>
```

①

②

① 이 컴포넌트를 가리키는 이름 (styledComponentId) ② 지금 이 스타일 묶음을 가리키는 이름 (내용이 바뀌면 이것도 바뀝니다)

★★ 둘 다 사람이 안 짓습니다. 그래서 남과 겹칠 수가 없습니다. 개념04의 불임말(c05Card 같은)을 붙이던 일이 필요 없어집니다.

★ 대신 **F12** 에서 이름만 보고는 어느 컴포넌트인지 모릅니다. `sc-bdvwhi` 를 보고 "아 담기버튼이구나" 할 수가 없습니다. 이게 실제로 불편합니다. (섹션 4의 단점 칸)

▹ 직접 해보기 5

F12 → Elements 에서 파랑상자를 눌러 class 두 개를 확인하세요. 그 다음 배경색을 #ffeeee 로 바꾸고 저장해, 둘 중 어느 쪽이 바뀌는지 보세요.

셋 중 무엇을 쓰나

섹션 4

개념04의 둘과 여기의 하나를 나란히 놓습니다.

```
const 비교표 = [
  {
    이름: "전역 CSS",
    좋은점: "제일 단순합니다. 배운 CSS 그대로입니다",
    나쁜점: "★ 이름이 겹치면 조용히 덮입니다",
    쓸때: "사이트 전체에 걸리는 것 (글꼴·기본 색)",
  },
  {
    이름: "CSS Modules",
    좋은점: "겹침이 없고, 설치할 게 없습니다",
    나쁜점: "조건부 스타일에 이름을 조립해야 합니다",
    쓸때: "★ 대부분의 경우. 기본으로 삼으세요",
  },
  {
    이름: "styled-components",
    좋은점: "props 로 값을 그대로 넘깁니다",
    나쁜점: "설치가 필요하고, F12 에서 이름을 못 알아봅니다",
    쓸때: "값에 따라 스타일이 이어서 바뀔 때",
  },
];

function Section4Demo() {
  return (
    <div className="demo"> <h3>④ 셋 비교</h3>

    {비교표.map((하나) => (
      <상자 key={하나.이름}> <h4>{하나.이름}</h4> <div>○ {하나.좋은점}</div> <div>×
{하나.나쁜점}</div> <div>→ {하나.쓸때}</div> </상자>
    ))}
    </div>
  );
}
```

★★★ 이 자료의 09~13단원은 **전역 CSS(index.css)**를 기본으로 씁니다. 혼자 연습하는 지금은 그게 가장 간단하기 때문입니다(개념04 참고).

styled-components 는 **팀이 이미 쓰고 있으면** 따라 쓰세요. 혼자 시작하는 프로젝트에 굳이 넣을 이유는 없습니다.

★★ 섞어 써도 됩니다. 실제로 많이 그렇게 합니다. 전역 CSS 로 글꼴과 기본 색을 깔고, 화면은 CSS Modules 로 만들고, 값에 따라 변하는 것 몇 개만 styled-components 로 하는 식입니다.

★ “무엇이 정답인가” 보다 “**팀이 하나로 정했는가**” 가 훨씬 중요합니다. 한 프로젝트에 세 방식이 아무 규칙 없이 섞여 있으면 그게 제일 나쁩니다.

09단원에서 만들 화면을 떠올리고, 셋 중 무엇을 쓸지와 그 이유를 주석으로 적어 보세요.

★★ 컴포넌트를 함수 밖에서 만드세요

섹션 5

지금까지 만든 styled 컴포넌트는 전부 **파일 맨 위에** 있습니다. 함수 안에 넣으면 안 됩니다. 그런데 왜 안 되는지는 안 해 보면 모릅니다.

올바른 자리 · 파일 맨 위 — 한 번만 만들어집니다

```
const 바깥에서만든칸 = styled.div`
  border: 2px solid #2d6cdf;
  padding: 8px;
  margin: 4px 0;
`;

function Section5Demo() {
  const [센수, set센수] = useState(0);
```

★★★ [잘못된 자리] 함수 안 — 그럴 때마다 **새로** 만들어집니다

```
const 안에서만든칸 = styled.div`
  border: 2px solid #d9534f;
  padding: 8px;
  margin: 4px 0;
`;

return (
  <div className="demo">
    <h3>⑤ 함수 안에서 만들면</h3>

    <button onClick={() => set센수(센수 + 1)}>다시 그리기 ({센수}번)</button>

    <바깥에서만든칸>
      <div>바깥에서 만든 칸 (파랑)</div>
      <div>
        여기에 글자를 쳐 보세요: <input placeholder="아무거나" />
      </div>
      <div style={{ fontSize: 12, color: "#666" }}>
        class: {바깥에서만든칸.styledComponentId}
      </div>
    </바깥에서만든칸>

    <안에서만든칸>
      <div>안에서 만든 칸 (빨강) *</div>
      <div>
        여기에도 쳐 보세요: <input placeholder="아무거나" />
      </div>
      <div style={{ fontSize: 12, color: "#666" }}>
        class: {안에서만든칸.styledComponentId}
      </div>
    </안에서만든칸>

    <div className="output">
      양쪽 칸에 글자를 친 다음 <strong>[다시 그리기]</strong> 를 누르세요.
      <br />
      빨강 칸의 글자만 사라집니다. class 이름도 바뀝니다.
    </div>
  </div>
);
}
```

★★★ 무슨 일이 벌어지나

`styled.div` 는 **컴포넌트를 만드는 함수**입니다. 함수 안에 두면 그릴 때마다 새 컴포넌트가 만들어집니다.

React 는 “컴포넌트가 바뀌었다” 고 보고 **옛것을 버리고 새로 그립니다**. 그 안에 있던 것이 전부 없어집니다.

- input 에 친 글자가 사라집니다
- 커서(포커스)도 풀립니다
- 그 안에 useState 가 있었다면 값도 초기화됩니다

★★ 그리고 스타일이 **매번 새로 만들어져 쌓입니다**. 백 번 그러면 백 개가 쌓입니다. 화면이 점점 느려집니다.

★★ 콘솔을 열어 보세요. styled-components 가 **경고**를 냅니다.

```
The component styled.div with the id of "sc-gKckzE" has been created dynamically.
You may see this warning because you've called styled inside another component.
To resolve this only create new StyledComponents outside of any render method
and function component.
```

★ 이 자료를 만들면서 여기를 틀렸습니다.

"에러도 경고도 없다" 고 적어 놔는데, 진짜 크롬으로 돌려 보니 ****경고**가 있었습니다.**
검증 도구가 잡아 졌습니다. 감으로 적으면 이렇게 됩니다.

★★★ 그런데 안심하면 안 됩니다. 이 경고는 **개발 중에만** 나옵니다.

styled-components 는 **개발 모드일 때만** 이 경고를 내도록 만들어져 있습니다. `npm run build` 로 빌드한 결과에는 이 경고가 **아예 들어 있지 않습니다**.

· 개발 중 → 경고가 뜹니다 (다행히) · 배포 뒤 → 아무 말도 없습니다. 증상만 남습니다

★ 그래서 "배포하고 나서 가끔 입력이 지워져요" 가 됩니다.

개발할 때 콘솔을 안 봤으면 못 잡고 넘어간 것입니다.
★★ 콘솔에 노란 줄이 쌓여 있으면 배포 전에 한 번 훑으세요.

★★★ 규칙은 하나입니다. **styled 컴포넌트는 파일 맨 위에서 만든다**. 예외 없습니다. 값에 따라 바꿀 게 있으면 섹션 2처럼 props 로 넘기세요.

▫ 직접 해보기 7

두 칸에 글자를 친 다음 [다시 그리기] 를 누르세요. 어느 쪽 글자가 남습니까? class 이름은 어떻게 됩니까?

▫ 직접 해보기 8

안에서만든칸 을 함수 밖으로 옮기고(이름은 그대로) 다시 해 보세요. 이번엔 글자가 남습니까? 확인했으면 되돌려 두세요.

이런 실수를 합니다

props 에 \$ 를 안 붙임 ★ 제일 흔합니다

~~<값표시 크기={true}>~~ 라고 쓰면 경고가 두 개 뜹니다 (섹션 2).

화면은 멀쩡해서 모르고 지나갑니다. 콘솔을 열어 두세요. 스타일을 정하는 데만 쓰는 props 에는 전부 \$ 를 붙이세요. ★ 영문 prop 이름이라도 마찬가지입니다. `primary` 도 HTML 속성이 아닙니다.

이런 실수를 합니다

styled 컴포넌트를 함수 안에서 만들 ★ 섹션 5

개발 중에는 콘솔에 “has been created dynamically” 경고가 뜹니다. ★ 빌드하면 그 경고가 사라집니다. 증상만 남습니다. 콘솔의 노란 줄을 그냥 두면 이런 것을 놓칩니다.

이런 실수를 합니다

백틱 대신 괄호를 씌

`styled.button("color: red")` ← 이렇게 쓰는 게 아닙니다 `styled.buttoncolor: red;` ← 백틱을 바로 붙입니다 ★ 함수 호출이 아니라 태그드 템플릿이라는 문법입니다. 지금은 “백틱을 붙여 쓴다” 로만 알아도 됩니다.

이런 실수를 합니다

컴포넌트 이름을 소문자로 시작

`const 버튼 = ...` 처럼 한글은 괜찮습니다. 영문으로 지을 때 `const button = styled.button`` 이라고 하면 `<button>` 이 진짜 HTML 버튼과 헷갈립니다. Button 처럼 대문자로 시작하세요. (01단원 개념04에서 본 규칙 그대로입니다)

이런 실수를 합니다

CSS 에 세미콜론을 빠뜨림

백틱 안은 진짜 CSS 라서 줄마다 ; 가 필요합니다. 빠뜨리면 그 줄부터 조용히 무시됩니다. 에러가 안 납니다.

이런 실수를 합니다

설치를 안 하고 import

Failed to resolve import “styled-components” 가 뜹니다. 실습프로젝트에는 이미 넣어 뒀습니다. 새 프로젝트에서는 `npm i styled-components` 를 먼저 하세요.

화면 —————

정리 —————

- 1 styled-components 는 CSS 를 컴포넌트로 만듭니다. styled.button 뒤에 백틱을 붙이고 그 안에 CSS 를 그대로 씁니다. .css 파일도 className 도 없습니다.
- 2 만들어진 것은 결국 그 태그입니다. 은 이라서 onClick·disabled 같은 것을 그대로 줄 수 있습니다.
- 3 props 로 값을 그대로 넘겨 스타일을 바꿉니다. CSS Modules 처럼 클래스 이름을 조립하지 않아도 됩니다. 값이 이어지는 수일 때 특히 편합니다.
- 4 스타일에만 쓰는 props 에는 \$ 를 붙입니다(\$큰가). 안 붙이면 styled-components 와 React 가 각각 경고를 냅니다. 화면은 멀쩡해서 모르고 지나가기 쉽습니다.
- 5 클래스 이름을 사람이 안 짓습니다. 그래서 겹칠 일이 없습니다. 대신 F12 에서 sc-bdvwhi 같은 이름만 보여서 어느 컴포넌트인지 알아보기 어렵습니다.
- 6 ★ styled 컴포넌트는 반드시 파일 맨 위에서 만듭니다. 함수 안에서 만들면 그릴 때마다 새 컴포넌트가 되어 입력값과 포커스가 사라지고 스타일이 쌓입니다.
- 7 그 실수는 개발 중에만 콘솔 경고(has been created dynamically)로 알려 줍니다. 빌드하면 경고가 사라지고 증상만 남습니다. 그래서 콘솔의 노란 줄을 그냥 두면 안 됩니다.
- 8 이 과정의 기본은 CSS Modules 입니다. styled-components 는 팀이 이미 쓰고 있으면 따라 쓰세요. 무엇을 쓰든 팀이 하나로 정하는 게 더 중요합니다.

```
export default function Concept05Styled() {
  const [restartKey, setRestartKey] = useState(0);

  return (
    <div> <h1>개념 05 – styled-components</h1> <p className="guide">
      개념04의 CSS Modules 를 먼저 보고 오세요. <strong>비교하는 파일</strong>
      입니다.
      <br /> <br /> <strong>이 방법을 꼭 써야 하는 것은 아닙니다.</strong> 이
      단원에서 기본으로 다룬 것은
      CSS Modules 이고, 09단원부터는 다시 전역 <code>index.css</code> 만 씁니다.
      다만 styled-components 는 회사에서 아주 많이 쓰기 때문에 읽을 줄은 알아야
      합니다.
      <br /> <br /> <strong>섹션 5가 이 파일에서 제일 중요합니다.</strong> 예러가
      안 나는 함정이라
      모르면 못 찾습니다.
      </p> <button onClick={() => setRestartKey(restartKey + 1)}>
        이 예제를 처음부터 다시 그리기
      </button> <div key={restartKey}> <Section1Demo /> <Section2Demo
      /> <Section3Demo /> <Section4Demo /> <Section5Demo /> </div> </div>
```

```
);  
}
```

직접 해보기 정답

1) const 담기버튼 = styled.button`

```
background: #d9534f;  
...
```

```
`;
```

화면 버튼이 빨갛게 바뀝니다. 담은 개수는 그대로 남습니다.

→ 개념04의  2와 같습니다. Vite 가 바뀐 부분만 알아 끼우기 때문입니다.

확인했으면 #2d6cdf 로 되돌려 두세요.

2) const 담기버튼 = styled.a`

```
... (그대로)
```

```
`;  
<담기버튼 href="#" onClick={...}>담기</담기버튼>
```

화면 모양은 거의 같은데 글자에 밀줄이 생기고, 커서가 손 모양이 됩니다.

→ styled.무엇 의 '무엇' 이 진짜 만들어지는 태그입니다.

a 로 만들면 href 를 줄 수 있고, button 으로 만들면 못 줍니다.
* 모양이 같다고 아무 태그나 쓰면 안 됩니다. 누르는 것은 button 입니다.
확인했으면 button 으로 되돌려 두세요.

3) const 값표시 = styled.span`

```
font-weight: bold;  
color: ${props => (props.$큰가 ? "#d9534f" : "#2d6cdf")};  
font-size: ${props => (props.$큰가 ? "20px" : "15px")};  
opacity: ${props => (props.$흐림 ? 0.4 : 1)};
```

```
`;
```

```
<값표시 $큰가={큰가} $흐림={true}>{금액}원</값표시>
```

화면 글자가 흐려집니다.

→ props 를 몇 개든 늘릴 수 있습니다. 각각이 CSS 한 줄을 말합니다.

★ 이걸 CSS Modules 로 하려면 흐름 클래스를 따로 만들고
className 조립에 한 조각을 더 붙여야 합니다.

4) <값표시 큰가={큰가}>{금액}원</값표시>

화면 똑같습니다. 색도 크기도 그대로 바뀝니다.

```
// 콘솔: styled-components: it looks like an unknown prop "큰가" is being sent
//       through to the DOM, which will likely trigger a React console error. ...
// 콘솔: Received `true` for a non-boolean attribute `큰가`.
```

```
//       If you want to write it to the DOM, pass a string instead:
//       큰가="true" or 큰가={value.toString()}.
```

→ 두 개가 연달아 뜹니다. 앞것것이 styled-components, 뒤것것이 React 입니다.

화면이 멀쩡한 게 함정입니다. 콘솔을 안 열면 모릅니다.
8000원을 눌렀을 때만 뜹니다(그때만 true 가 넘어가니까요).
★ F12 → Elements 로 그 span 을 봐도 큰가 속성은 **안 붙어 있습니다.**
React 가 경고만 내고 붙이지는 않습니다. 그래서 더 티가 안 납니다.
확인했으면 \$큰가 로 되돌려 두세요.

5) F12 → Elements 에서 파랑상자를 누르면 이렇게 보입니다.

```
<div class="sc-bdvwhi bjovqr">파랑상자입니다</div>
```

background 를 #ffeeee 로 바꾸면 —

화면 배경이 분홍이 되고, class 두 개 중 **뒤엿것만** 바뀝니다.

→ 앞엇것(sc-...)은 "이 컴포넌트" 를 가리키므로 그대로입니다.

뒤엇것은 "지금 이 스타일 내용" 을 가리키므로 내용이 바뀌면 같이 바뀝니다.
★ 정확한 글자는 컴퓨터마다·저장할 때마다 다릅니다. 값이 아니라 모양을 보세요.
확인했으면 #eef4ff 로 되돌려 두세요.

6) 정답이 하나가 아닙니다. 이런 식이면 됩니다.

```
// 09단원 사용자 목록 화면
// → CSS Modules. 목록·카드 모양이 값에 따라 변하지 않고,
// 설치할 게 없어서 바로 시작할 수 있습니다.
// 다만 '불러오는 중' 표시의 흐름 정도를 진행률에 맞춰 바꾼다면
// 그 부분만 styled-components 가 편합니다.
```

→ ★ "무조건 이것" 이 답이 아닙니다. 이유를 댈 수 있으면 맞은 것입니다.

7) 양쪽에 글자를 치고 [다시 그리기] 를 누르면 —

화면 파랑 칸의 글자는 남고, **빨강 칸의 글자만 사라집니다.**

빨강 칸의 class 이름도 누를 때마다 바뀝니다. 파랑은 그대로입니다.

→ 빨강 칸은 그릴 때마다 **다른 컴포넌트**가 됩니다.

React 는 그것을 "다른 것" 으로 보고 옛것을 버립니다.
그 안의 input 도 같이 버려지니 글자가 사라집니다.

// 콘솔: The component styled.div with the id of "sc-..." has been created dynamically.

→ ★ 경고는 **뜸니다**. 다만 개발 중에만 뜬니다.

빌드한 결과에는 이 경고가 없습니다. 배포한 다음에는 증상만 남습니다.

8) Section5Demo 위(파일 맨 위 쪽)로 옮깁니다.

```
const 안에서만든칸 = styled.div`
  border: 2px solid #d9534f;
  padding: 8px;
  margin: 4px 0;
`;

function Section5Demo() {
  const [센수, set센수] = useState(0);
  ...
}
```

화면 이번에는 빨강 칸의 글자도 남습니다. class 이름도 안 바뀝니다.

→ 옮긴 것 말고는 아무것도 안 고쳤습니다. **자리 하나가 전부**였습니다.

★ 이름은 그대로 두는 게 좋습니다. "안에서 만들면 이렇게 된다" 를 기억하는 이름이니깐요. 확인했으면 되돌려 두세요.

.CSS

```

/* =====
08단원 연습문제가 쓰는 전역 CSS
=====
연습문제.jsx 와 연습문제_정답.jsx 가 둘 다 이 파일을 씁니다.

import "./연습문제.css";

* 이름을 전부 q08 로 시작하게 지었습니다.
전역 CSS 는 프로젝트 전체에 걸리므로, 다른 단원의 이름과 겹치면
남의 화면이 망가집니다(개념05 섹션 4).

* CSS 는 이 자료에서 가르치지 않습니다. 여기 내용은 몰라도 됩니다.
문제에서 시키는 것은 '이 이름을 className 에 붙이는 것' 뿐입니다.
===== */ /* 기본 카드 모양
*/
.q08Card {
  background: #fff;
  border: 1px solid #ccc;
  border-left: 4px solid #2d6cdf;
  padding: 10px 14px;
  margin: 6px 0;
}

/* .q08Card 와 함께 붙이면 색만 바뀝니다 */
.q08Sale {
  border-left-color: #c00;
  background: #fff8f8;
}

/* 품절 표시 */
.q08SoldOut {
  color: #999;
  text-decoration: line-through;
}

/* 가격 글씨 */
.q08Price {
  color: #2d6cdf;
  font-weight: bold;
}

```

Vite 로 옮기기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 파일을 고르세요.

고치고 저장하면 화면이 바로 바뀝니다. F12 → Console 도 함께 보세요.

문제 1~10은 기본, 11은 응용, 12는 도전, 13은 에러 확인입니다.

문제마다 '기대 결과' 가 적혀 있으니 그대로 나오는지 확인하세요.

★ 아직 안 푼 문제는 "문제 N: ..." 같은 안내 글자가 그대로 보입니다. 정상입니다.

★ 개발 중에는 StrictMode 때문에 콘솔 줄이 두 번씩 찍힙니다. 이것도 정상입니다.

★ 이 단원은 import 를 다룹니다. import 는 파일 맨 위에만 쓸 수 있으므로,

아래 [import 연습 구역] 에 줄을 추가하면서 푼다.

부품 파일들은 _부품 폴더에 이미 다 만들어 두었습니다. 열어서 읽어 보세요.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

이 파일이 쓰는 CSS 입니다. 문제 8·9가 여기 있는 이름을 씁니다.

```
import "../연습문제.css";
```

[import 연습 구역]

문제 1 · 2 · 3 · 4 · 5 · 6 · 10 · 12 는 여기에 import 줄을 씁니다. 문제를 읽고 필요한 줄을 하나씩 늘려 가세요.

여기에 import 줄을 쓰세요

여기까지

문제에서 함께 쓰는 데이터입니다. 고치지 마세요.

```
const menuItems = [
  { id: 1, name: "아메리카노", price: 4000, soldOut: false },
  { id: 2, name: "라떼", price: 4500, soldOut: false },
  { id: 3, name: "케이크", price: 6000, soldOut: true },
  { id: 4, name: "삼각김밥", price: 1200, soldOut: false },
];
```

문제 1 (개념02)

_부품/연습_Welcome.jsx 를 열어 보세요. Welcome 이라는 컴포넌트를 export default 로 내보내고 있습니다. 그것을 이 파일로 가져와서 화면에 그리세요.

할 일은 두 가지입니다. (1) [import 연습 구역] 에 import 줄을 한 줄 쓴다 (2) 아래 Q1Welcome 안의 안내 문구를 <Welcome name="김민준" /> 으로 바꾼다

기대 화면 김민준님, 환영합니다.
 화면 전체가 빨간 상자가 되고 Failed to resolve import 가 보이면
 경로 앞의 ./ 를 빠뜨린 것입니다.
 Welcome is not defined 가 나오면 import 줄을 안 쓴 것입니다.

아래 함수 안을 고치세요

```
function Q1Welcome() {
  return (
    <div className="output">문제 1: _부품/연습_Welcome.jsx 를 가져와 쓰세요</div>
  );
}
```

문제 2 (개념02)

_부품/연습_가격.js 에는 이름 있는 export 가 여러 개 있습니다. 그중 cakePrice 와 applyDiscount 두 개만 가져와서 아래 화면을 만드세요.

할 일: [import 연습 구역] 에 한 줄 쓰고, 아래 함수 안을 고치기

기대 화면 케이크 6000원 → 할인가 5400원
 값이 undefined 로 나오면 중괄호를 안 쓴 것입니다.
 does not provide an export named 이 나오면 이름을 잘못 적은 것입니다.

아래 함수 안을 고치세요

```
function Q2Discount() {
  return (
    <div className="output">문제 2: cakePrice 와 applyDiscount 를 가져오세요</div>
  );
}
```

문제 3 (개념 02)

같은 파일(_부품/연습_가격.js)의 americanoPrice 를 가져오려고 합니다. 그런데 이 파일에는 이미 americanoPrice 라는 이름이 아래에 있습니다(바로 다음 줄). 이름이 부딪히지 않게 coffeePrice 라는 이름으로 가져오세요.

기대 화면 가져온 값 4000 / 이 파일의 값 9999
Identifier 'americanoPrice' has already been declared 가 나오면
as 를 안 쓰고 그냥 가져온 것입니다.

[import 연습 구역] 에 as 를 쓴 import 줄을 넣고, 아래 함수 안을 고치세요

이 줄은 일부러 이름을 겹치게 만들어 둔 것입니다. 지우지 마세요.

```
const americanoPrice = 9999;

function Q3Rename() {
  return (
    <div className="output">
      문제 3: americanoPrice 를 coffeePrice 라는 이름으로 가져오세요 (이 파일의 값:{
    ">
    {americanoPrice})
  </div>
  );
}
```

문제 4 (개념02)

_부품/연습_배지.jsx 에는 export default(Badge)와 이름 있는 export(hotLabel)가 함께 들어 있습니다. 한 줄로 둘 다 가져와서 아래 화면을 만드세요.

기대 화면 아메리카노 옆에 파란 바탕의 "인기" 딱지가 붙습니다.
 딱지에 글자가 없으면 hotLabel 을 안 넘긴 것입니다.
 중괄호 안에 Badge 까지 넣으면 does not provide an export named
 'Badge' 가 납니다. 대표는 중괄호 밖입니다.

아래 함수 안을 고치세요

```
function Q4Badge() {  
  return (  
    <div className="output">문제 4: Badge 와 hotLabel 을 한 줄로 가져오세요</div>  
  );  
}
```

문제 5 (개념03)

아래 코드는 같은 모양이 세 번 반복됩니다. 03단원에서 배운 그 문제입니다. _부품/연습_카드.jsx 의 CoffeeCard 를 가져와서 세 줄로 줄이세요.

```
<CoffeeCard name="..." price={...} note="..." />
```

기대 화면 지금과 똑같은 카드 세 개가 나옵니다. 화면은 하나도 안 바뀝니다.
 note 를 안 넘긴 카드가 있으면 그 카드만 "설명 없음" 이 됩니다.

아래 함수 안을 고치세요

```
function Q5UseCard() {  
  return (  
    <div> <div className="output"> <strong>아메리카노</strong> - 4000원
```

.module.css

```

/* =====
08단원 연습문제 10번이 쓰는 CSS Modules 파일
=====

파일 이름이 .module.css 로 끝납니다. 이게 전부입니다.

import styles from "./연습문제.module.css";
...
<div className={styles.box}>

여기 적은 box · label 이라는 이름을
Vite 가 아무도 안 쓸 것 같은 긴 이름으로 바꿔서 내보냅니다.
그래서 불임말(q08 같은 것)을 안 붙여도 안 부딪힙니다.
===== */

.box {
  background: #fff;
  border: 1px solid #ccc;
  border-left: 4px solid #2e9e4f;
  padding: 10px 14px;
  margin: 6px 0;
}

.label {
  font-weight: bold;
  color: #2e9e4f;
}

```

08단원 마무리 파일 나누기와 스타일링

자주 나오는 질문

Q “node_modules 는 뭐예요? 지워도 돼요?”

A 첫날 npm install 로 받아 온 도구 창고입니다. 지워도 됩니다 — npm install 을 다시 치면 10초면 돌아옵니다(인터넷 필요)

Q “왜 저장만 하면 바뀌어요? 앞에서는 F5 눌렀는데”

A Vite 가 바뀐 부분만 브라우저에 밀어 넣어 줍니다. 그게 08단원으로 옮긴 이유 중 하나입니다

useEffect와 API

OPEN 09_useEffect와_API

```
function OrderDemo() {  
  useEffect(() => {  
    return (  
      <div className="demo">
```

01 useEffect 기본

02 정리 함수

03 fetch 로 받아오기

04 로딩과 에러

05 값이 바뀌면 다시 받기

06 useEffect 실수

useEffect 기본

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

08단원까지 우리가 만든 화면은 전부 이 모양이었습니다.

데이터(state) 를 바꾼다 → React 가 화면을 다시 그린다

여기까지는 '화면 안' 의 일입니다. 그런데 앱을 만들다 보면 화면 밖에도 시켜야 할 일이 생깁니다.

서버에서 글 목록을 받아온다
1초마다 시계를 움직인다
브라우저 탭 제목을 바꾼다
로그인 정보를 브라우저에 저장한다

이런 일은 "화면을 어떻게 그릴까" 와 상관이 없습니다. 화면을 그리는 김에 옆에서 같이 해야 하는 일입니다. 이 일을 맡는 것이 `useEffect` 입니다.

★ 이 단원은 인터넷에 있는 연습용 서버에서 데이터를 받아옵니다. 주소는 `https://jsonplaceholder.typicode.com` 이고, JS자료 12단원에서 쓰던 곳과 같습니다. 실습실에서 인터넷이 막혀 있으면 실습프로젝트 폴더의 `index.html` 을 열어 /오프라인_대체.js 줄을 감싼 주석(`<!--` 와 `-->`)만 지우세요. 인터넷 없이도 똑같이 동작합니다. 단, 흉내 내는 주소는 위 `jsonplaceholder` 뿐입니다. 다른 주소는 그대로 실패합니다.

★ 콘솔에 같은 줄이 두 번씩 찍히는 것을 보게 됩니다. 정상입니다. 여러분이 잘못된 것이 아닙니다. 이유는 개념02에서 정면으로 다룹니다. 이 파일에서는 "두 번 찍혀도 놀라지 않는다" 정도만 알고 넘어가세요.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

부작용이란 무엇인가

섹션 1

처음 나오는 말이라 먼저 뜻을 정합니다.

부작용(side effect)

화면을 만드는 일 말고, 그 김에 바깥세상에 일어나는 일.

이름이 무섭게 생겼지만 나쁜 뜻이 아닙니다. 약의 부작용처럼 “원래 목적 옆에서 같이 생기는 일”이라는 뜻일 뿐입니다.

컴포넌트 함수의 원래 목적은 하나입니다. 화면 설명(JSX)을 돌려주는 것. 그 목적 밖의 일이 전부 부작용입니다.

부작용이 아님 · 숫자를 더해서 화면에 넣는다

부작용이 아님 · 배열을 map 으로 돌려 목록 JSX 를 만든다

부작용 · fetch 로 서버에 요청을 보낸다

부작용 · setInterval 로 타이머를 켜다

부작용 · `document.title` 을 바꾼다

부작용 · `localStorage` 에 값을 저장한다

구분하는 방법이 하나 있습니다. “이 줄을 두 번 실행해도 아무 일 없나?” 하고 물어 보세요. 두 번 실행하면 요청이 두 번 나가거나 타이머가 두 개 생기면, 그게 부작용입니다.

▸ 직접 해보기 1

아래 다섯 가지 중 부작용인 것을 골라 보세요. (정답은 파일 맨 아래에) (가) 가격 4000 에 개수를 곱해 총액을 만든다 (나) 서버에 “이 글 조회수 +1” 요청을 보낸다 (다) 목록을 filter 로 걸러 화면에 그린다 (라) 브라우저 탭 제목을 “장바구니 3개” 로 바꾼다 (마) 1초마다 도는 타이머를 켜다

언제 실행되나

섹션 2

useEffect 의 모양은 이렇습니다.

```
useEffect(() => {
  할 일
});
```

화살표 함수를 통째로 넘깁니다. 우리가 부르는 것이 아니라 React 가 대신 불러 줍니다. 언제 부르냐면 — 화면이 실제로 다 그려진 뒤입니다.

왜 그려진 뒤일까요? 컴포넌트 함수가 도는 동안은 아직 화면이 없습니다. 화면 설명을 ‘만드는 중’ 입니다. 그 도중에 서버 요청을 보내거나 타이머를 켜면, 화면은 아직 한 글자도 안 나왔는데 바깥일부터 벌어집니다. 그래서 React 는 화면을 다 그린 다음에 몰아서 실행합니다.

아래 데모가 그 순서를 보여 줍니다.

```
function OrderDemo() {
  const [tick, setTick] = useState(0);
```

이 줄은 컴포넌트 함수 '본문' 입니다. 화면 설명을 만드는 중에 실행됩니다.

```
console.log("① 함수 본문 - 화면 설명을 만드는 중입니다");
```

F12 콘솔 ① 함수 본문 - 화면 설명을 만드는 중입니다

```
useEffect(() => {
```

이 줄은 화면이 다 그려진 뒤에 실행됩니다.

```
console.log("② useEffect - 화면이 다 그려진 뒤입니다");
```

F12 콘솔 ② useEffect - 화면이 다 그려진 뒤입니다

```
});

return (
  <div className="demo"> <h3>① 실행 순서 - 본문이 먼저, useEffect 가 나중
</h3> <p className="output">다시 그린 횟수: {tick}</p> <button onClick={() =>
setTick(tick + 1)}>다시 그리기</button> </div>
);
}
```

콘솔을 보면 ① 이 먼저, ② 가 나중입니다. 순서가 절대 뒤집히지 않습니다. 버튼을 눌러 다시 그려도 ① → ② 순서 그대로입니다.

여기서 한 가지 더 눈에 띄는 것입니다. ① 이 두 줄씩 찍힙니다. 개발 중에만 그렇습니다. 개념02에서 이유를 설명합니다. 지금은 넘어가세요.

▫ 직접 해보기

2

위 useEffect 안에 `console.log("③")` 를 한 줄 더 넣고 버튼을 눌러 보세요. ③ 이 ① 보다 먼저 찍히는 일이 있을까요?

의존성 배열 세 가지

섹션 3

위 데모의 useEffect 는 다시 그릴 때마다 매번 실행됐습니다. 그런데 대부분은 그러면 안 됩니다. 서버에서 목록을 받아오는 일이 화면을 그릴 때마다 나가면 요청이 폭주합니다.

그래서 useEffect 는 두 번째 자리에 '의존성 배열' 을 받습니다.

```
useEffect(() => { 할 일 }, [지켜볼 값]);
```


“이 배열 안의 값이 지난번과 다를 때만 실행해 줘” 라는 뜻입니다. 쓰는 방법이 딱 세 가지입니다.

A · 배열을 아예 안 씀 `useEffect(() => {...});` → 다시 그릴 때마다 매번

B · 빈 배열 `useEffect(() => {...}, []);` → 처음 한 번만

C · 값을 넣은 배열 `useEffect(() => {...}, [count]);` → count 가 바뀌었을 때만

B · 가 왜 ‘한 번만’ 인지 생각해 보면 규칙이 저절로 이해됩니다.

빈 배열은 지켜볼 값이 하나도 없다는 뜻입니다. 그러니 달라질 것도 없습니다. 그래서 처음 화면에 나타날 때 한 번 실행되고 끝입니다.

아래 데모에서 A · B · C 를 나란히 켜 두고 직접 세어 봅니다.

```
function DepsDemo() {
  const [count, setCount] = useState(0);
  const [color, setColor] = useState("파랑");
```

A · 배열을 안 씀 — 다시 그릴 때마다

```
useEffect(() => {
  console.log("A) 배열 없음 - 실행");
```

F12 콘솔 A) 배열 없음 - 실행

```
});
```

B · 빈 배열 — 처음 한 번만

```
useEffect(() => {
  console.log("B) [] - 실행");
```

F12 콘솔 B) [] - 실행

```
}, []);
```

C · [count] — count 가 바뀌었을 때만

```
useEffect(() => {
  console.log(`C) [count] - 실행 (count 는 ${count})`);
```

F12 콘솔 C) [count] - 실행 (count 는 0)
 C) [count] - 실행 (count 는 1)

```

    }, [count]));

    return (
      <div className="demo"> <h3>② 의존성 배열 세 가지를 나란히
    </h3> <p className="output">
      count: {count} / color: {color}
    </p>
    { /* 화면: count: 0 / color: 파랑 */ }
    <button onClick={() => setCount(count + 1)}>숫자 +1</button> <button onClick=
    {() => setColor(color === "파랑" ? "빨강" : "파랑")}>
      색 바꾸기
    </button> </div>
    );
  }
}

```

콘솔을 지우고(휴지통 아이콘) 버튼을 눌러 몇 줄이 늘어나는지 세어 보세요.

A(배열 없음) B([]) C([count])

숫자 +1 · 누름 늘어남 안 늘어남 늘어남

색 바꾸기 · 누름 늘어남 안 늘어남 안 늘어남

세 줄로 요약하면 이렇습니다. A는 화면이 다시 그려질 때마다 무조건 실행됩니다. B는 처음 한 번 실행되고 다시는 실행되지 않습니다. C는 count가 바뀔 때만 실행됩니다. color가 바뀌어도 가만히 있습니다.

주의 · “화면이 다시 그려진다”는 04단원 개념03에서 배운 그 렌더링입니다.

set 함수를 부르면 컴포넌트 함수가 처음부터 다시 실행되는 것 말입니다.

셋 중 무엇을 쓸지는 이렇게 고릅니다. 처음 한 번만 받아오면 되는 데이터 → [] 어떤 값이 바뀔 때마다 다시 받아야 함 → [그 값] 매번 실행해야 하는 경우 → 거의 없습니다. 대부분 실수입니다.

— B를 다시 보고 싶다면

B는 처음 한 번만 실행됩니다. 그러니 버튼을 아무리 눌러도 다시 안 찍힙니다. 다시 보려면 이 예제를 화면에서 통째로 지웠다가 새로 그려야 합니다. 화면 맨 위의 [이 예제를 처음부터 다시 그리기] 버튼이 그 일을 합니다. 그 버튼을 누르면 A · B · C가 전부 처음처럼 한 번씩 더 찍힙니다.

▫ 직접 해보기 3

위 [C]의 의존성 배열을 [count]에서 [color]로 바꿔 보세요. 이제 어떤 버튼을 눌러야 C가 찍힐까요? 바꾸기 전에 먼저 예상해 보세요.

실제로 쓰는 예 — 브라우저 탭 제목

섹션 4

지금까지는 콘솔에 글자만 찍었습니다. 진짜로 쓰는 모양을 하나 봅시다.

브라우저 탭 위에 뜨는 글자는 `document.title` 입니다. JS자료 10단원에서 쓰던 그 `document` 가 맞습니다. React 와 상관없는 브라우저 물건입니다.

그래서 이 일은 화면(JSX)으로는 절대 못 합니다. 컴포넌트가 그리는 것은 `#root` 안쪽뿐이고, 탭 제목은 그 바깥에 있으니까요. 딱 `useEffect` 가 필요한 자리입니다.

```
function TitleDemo() {
  const [cartCount, setCartCount] = useState(0);

  useEffect(() => {
```

화면 밖(브라우저 탭)을 건드립니다. 전형적인 부작용입니다.

```
document.title = `장바구니 ${cartCount}개`;

console.log("브라우저 탭 제목을 바꿨습니다:", document.title);
```

F12 콘솔 브라우저 탭 제목을 바꿨습니다: 장바구니 0개
 브라우저 탭 제목을 바꿨습니다: 장바구니 1개

```
    }, [cartCount]); // 담은 개수가 바뀔 때만 제목을 고치면 됩니다 return (
    <div className="demo"> <h3>③ 담을 때마다 탭 제목이 바뀝니다
  </h3> <p className="output">장바구니: 아메리카노 {cartCount}개</p>
    { /* 화면: 장바구니: 아메리카노 0개 */
    <button onClick={() => setCartCount(cartCount + 1)}>아메리카노 담기</button>
    { /* 화면(누르면): 장바구니: 아메리카노 1개 - 브라우저 탭 글자도 함께 바뀝니다
  */
    </div>
    );
  }
```

버튼을 누르고 브라우저 탭(창 맨 위 작은 글씨)을 보세요. 숫자가 같이 올라갑니다.

여기서 `useEffect` 를 안 쓰고 컴포넌트 본문에 `document.title = ...` 를 그냥 적으면 어떻게 될까요? 사실 지금은 잘 돌아가는 것처럼 보입니다. 하지만 화면을 그리는 도중에 바깥을 건드리는 것이라 React 가 보장해 주지 않습니다. 그리는 일과 바깥일을 섞지 않는다 — 이것이 `useEffect` 를 쓰는 이유입니다.

알아둘 것 · 이 예제는 탭 제목을 바꿔 놓고 되돌리지 않습니다.

그래서 다른 예제로 옮겨 가도 "장바구니 N개" 가 그대로 남아 있습니다.
되돌리는 방법이 개념02의 정리 함수입니다.

탭 제목을 “커피 N잔” 으로 바꿔 보세요. (템플릿 리터럴만 고치면 됩니다)

자주 하는 실수

섹션 5

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

의존성 배열을 빠뜨리고 그 안에서 state 를 바꿈

```
useEffect(() => {
  setCount(count + 1);
});
```

무한 루프입니다. state 를 바꾸면 다시 그려지고, 다시 그려지면 또 실행되고, 또 state 를 바꾸고... 끝이 없습니다. 개념06에서 안전하게 재현해 봅시다. ★ 이걸 주석을 풀지 마세요. 브라우저가 버벅입니다.

이런 실수를 합니다

혹을 조건문 안에서 부름

```
if (count > 0) {
  useEffect(() => { ... }, []);
}
```

"Rendered more hooks than during the previous render" 에러가 나면서 이 예제가 빨간 상자로 바뀝니다. 혹은 컴포넌트 맨 위에서만 부릅니다. 04단원 개념06에서 배운 그 규칙이 useEffect 에도 똑같이 적용됩니다.

이런 실수를 합니다

useEffect 를 쓸 필요가 없는데 씀

```
const [total, setTotal] = useState(0);
useEffect(() => { setTotal(price * count); }, [price, count]);
```

에러는 안 납니다. 그냥 계산하면 될 일을 state 로 만든 것입니다. `const total = price * count;` 한 줄이면 됩니다. 07단원 개념05의 파생 state 금지와 같은 이야기입니다. 개념06 섹션 3에서 다시 다룹니다.

이런 실수를 합니다

의존성 배열에 '매번 새로 만들어지는 값' 을 넣음

이건 에러가 안 나고 조용히 틀립니다. 그래서 아래에 실제로 돌려 둡니다.

```
function NewArrayDepsDemo() {
  const [tick, setTick] = useState(0);
```

이 배열은 컴포넌트가 다시 그려질 때마다 '새로' 만들어집니다. 안에 든 글자는 같지만, 새 배열이라 지난번 배열과 같은 물건이 아닙니다. (JS자료 09단원에서 배운 그것입니다. `["a"] === ["a"]` 는 `false` 입니다)

```
const menu = ["아메리카노", "라떼"];

useEffect(() => {
  console.log("[실수 4] menu 를 의존성에 넣었는데 또 실행됐습니다");
```

F12 콘솔 [실수 4] menu 를 의존성에 넣었는데 또 실행됐습니다

```
  }, [menu]); // ← 지켜보라고 넣었지만 매번 다른 배열이라 매번 실행됩니다 return (
    <div className="demo"> <h3>④ [실수 4] 의존성에 넣었는데 소용이 없는 경우
  </h3> <p className="output">다시 그린 횟수: {tick}</p> <button onClick={() =>
    setTick(tick + 1)}>다시 그리기</button> </div>
  );
}
```

버튼을 누를 때마다 위 줄이 콘솔에 또 찍힙니다. menu 의 내용은 한 번도 안 바뀌었는데 말입니다. React 는 배열 안을 들여다보지 않고 "같은 물건인가" 만 봅니다. 매번 새로 만든 배열은 매번 다른 물건이라 매번 실행됩니다.

지금 단계의 해결책은 간단합니다. ① 그 값이 진짜 필요 없으면 의존성에서 빼거나 ② 배열·객체 자체가 아니라 그 안의 값(menu.length 같은 것)을 넣습니다. 컴포넌트 밖으로 빼는 방법도 있습니다. 위 menu 는 state 가 아니니 파일 위쪽으로 옮겨도 됩니다. (매번 새로 만들지 않도록 감싸 두는 방법은 10단원에서 배웁니다)

화면

아래 `restartKey` 는 위 데모들을 '처음부터 다시' 그리기 위한 것입니다. 05단원에서 배운 `key` 를 그대로 씁니다. `key` 가 달라지면 React 는 그것을 고쳐 쓸 물건이 아니라 '다른 물건' 으로 보고 통째로 새로 만듭니다. 새로 만들어지니 `[]` 를 쓴 `useEffect` 도 처음처럼 한 번 더 실행됩니다.

정리

- 1 부작용은 화면을 만드는 일 말고 그 김에 바깥세상에 일어나는 일입니다. 서버 요청·타이머·탭 제목 바꾸기 같은 것입니다.
- 2 `useEffect(() => { 할 일 })` 는 화면이 다 그려진 뒤에 **React** 가 대신 불러 줍니다. 컴포넌트 본문이 먼저, `useEffect` 가 나중입니다.
- 3 두 번째 자리의 의존성 배열이 실행 시점을 정합니다. 안 쓰면 매번, `[]` 는 처음 한 번만, `[값]` 은 그 값이 바뀔 때만입니다.
- 4 빈 배열이 '한 번만' 인 이유는 지켜볼 값이 없어서 달라질 것도 없기 때문입니다. 외율 규칙이 아닙니다.
- 5 의존성에 매번 새로 만들어지는 배열·객체를 넣으면 매번 실행됩니다. React 는 안을 보지 않고 같은 물건인지만 봅니다.
- 6 그냥 계산하면 되는 값을 `useEffect + state` 로 만들지 마세요. 07단원의 파생 state 금지와 같은 이야기입니다.

```
export default function Concept01EffectBasics() {
  const [restartKey, setRestartKey] = useState(0);

  return (
    <div> <h1>개념 01 - useEffect 기본</h1> <p className="guide"> <strong>F12 →
    Console</strong> 을 함께 열어 두세요. 이 파일은 화면보다 콘솔에
      더 많은 것이 나옵니다.
    <br /> <br />
      콘솔에 같은 줄이 <strong>두 번씩</strong> 찍힙니다. 정상입니다. 이유는
    개념02에서
      설명합니다.
    <br /> <br /> <strong>인터넷이 막힌 실습실이라면</strong> 실습프로젝트 폴더의
    {" "}
      <code>index.html</code> 에서 <code>오프라인_대체.js</code> 줄을 감싼 주석만
      지우세요. 이 단원의 데이터 예제가 인터넷 없이도 똑같이 돌아갑니다.
    </p> <button onClick={() => setRestartKey(restartKey + 1)}>
      이 예제를 처음부터 다시 그리기
    </button> <div key={restartKey}> <OrderDemo /> <DepsDemo /> <TitleDemo
    /> <NewArrayDepsDemo /> </div> </div>
  );
}
```

직접 해보기 정답

- 1 (나) · (라) · (마) 가 부작용입니다. (가) 와 (다) 는 화면에 넣을 값을 만드는 계산이라 몇 번을 실행해도 바깥에 아무 일이 없습니다. (나) 는 실행할 때마다 조회수가 진짜로 올라갑니다. (라) 는 실행할 때마다 브라우저 탭이 바뀝니다. (마) 는 실행할 때마다 타이머가 하나씩 더 생깁니다. → “두 번 실행하면 뭔가 두 배로 일어나는가” 가 구분 기준입니다.

2 `useEffect(() => {`

```
console.log("② useEffect - 화면이 다 그려진 뒤입니다");
console.log("③");
```

```
});
// 콘솔: ① 함수 본문 - 화면 설명을 만드는 중입니다
// 콘솔: ② useEffect - 화면이 다 그려진 뒤입니다
// 콘솔: ③
```

→ ③ 이 ① 보다 먼저 찍히는 일은 없습니다.

useEffect 안의 코드는 화면이 다 그려진 뒤에만 실행되기 때문입니다.
순서가 보장된다는 것이 이 섹션의 핵심입니다.

3) `useEffect(() => {`

```
console.log(`C) [count] - 실행 (count 는 ${count})`);
```

```
}, [color]);
```

→ 이제 [색 바꾸기] 를 눌러야 C 가 찍힙니다. [숫자 +1] 은 아무리 눌러도 안 찍힙니다.

대신 찍히는 글자는 여전히 count 값입니다.
// 콘솔: C) [count] - 실행 (count 는 0)
"언제 실행할지" 를 정하는 것이 의존성 배열이고,
"무엇을 쓸지" 는 함수 안의 코드가 정합니다. 둘은 별개입니다.

4) `useEffect(() => {`

```
document.title = `커피 ${cartCount}잔`;
console.log("브라우저 탭 제목을 바꿨습니다:", document.title);
```

```
}, [cartCount]);
// 콘솔: 브라우저 탭 제목을 바꿨습니다: 커피 0잔
```

→ 버튼을 누르면 탭 글자가 “커피 1잔” 이 됩니다.

확인이 끝났으면 원래대로 되돌려 놓으세요. 위 설명의 // 콘솔: 값과 어긋납니다.

정리 함수

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념01에서 `useEffect` 로 ‘켜는’ 법을 배웠습니다. 이 파일은 ‘끄는’ 법입니다.

컨 것을 끄지 않으면 어떻게 될까요. 타이머는 화면에서 사라진 컴포넌트를 위해 계속 돛니다. 이벤트 구독은 남아서 없어진 화면을 고치려고 합니다. 앱을 오래 켜 둘수록 조금씩 무거워지고, 나중에는 이유를 못 찾는 버그가 됩니다.

그리고 이 파일에서 여러분이 계속 궁금해했을 것을 먼저 풀고 갑니다.

"왜 콘솔에 같은 줄이 두 번씩 찍히지? 내가 뭘 잘못했나?"

아무것도 잘못하지 않았습니다. 섹션 1에서 정면으로 설명합니다.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

왜 두 번 실행되나 (StrictMode)

섹션 1

이 실습프로젝트의 `src/main.jsx` 를 열어 보면 이렇게 되어 있습니다.

```
createRoot(document.querySelector("#root")).render(
  <StrictMode> <App /> </StrictMode>
);
```

StrictMode 는 “엄격 모드” 라는 뜻입니다. 개발 중에만 켜지는 검사 도구입니다. 이것이 켜져 있으면 React 가 컴포넌트를 이렇게 다룹니다.

| | |
|-----------|--------------------------------|
| 화면에 불인다 | → <code>useEffect</code> 실행 |
| 곧바로 떼어 낸다 | → 정리 함수 실행 |
| 다시 불인다 | → <code>useEffect</code> 다시 실행 |

그래서 콘솔에 effect 로그가 두 번 찍힙니다. 정상입니다.

— 왜 이런 짓을 하나

정리 함수를 제대로 썼는지 대신 검사해 주는 것입니다. 제대로 썼다면 붙였다 뗐다 다시 붙여도 결과가 똑같습니다. 안 썼다면 타이머가 두 개 생기고, 요청이 두 번 나가고, 구독이 두 개 쌓입니다. 그 차이가 콘솔에 바로 드러납니다.

즉 두 번 실행은 버그가 아니라 '건강검진' 입니다. 실제 사용자에게 배포할 때(npm run build) StrictMode 검사는 사라지고 한 번만 실행됩니다.

— 컴포넌트 본문도 두 번 실행됩니다

개념01에서 "㉠ 함수 본문" 이 두 줄씩 찍힌 이유가 이것입니다. 컴포넌트 함수가 '같은 입력이면 같은 화면' 을 돌려주는지 검사하려고 두 번 부릅니다. 그래서 컴포넌트 본문에서 부작용(요청 보내기·타이머 켜기)을 하면 두 배로 벌어집니다. 개념01에서 "본문에서 하지 말고 useEffect 에서 하라" 고 한 이유가 여기서 드러납니다.

— 두 번 도는 것은 처음 나타날 때뿐입니다

값이 바뀌어 effect 가 다시 도는 경우에는 한 번씩만 실행됩니다. 아래 데모의 버튼으로 직접 확인하세요.

— 이 자료의 표기 약속

자료의 // 콘솔: 에는 한 번만 적혀 있습니다. 실제 콘솔에 두 줄씩 보이는 것이 정상이라고 생각하세요.

— 끄고 싶다면

main.jsx 의 <StrictMode> 를 지우면 두 번 실행이 없어집니다. 하지만 지우지 마세요. 두 번 실행이 불편하다는 것은 대개 정리 함수가 빠졌다는 신호입니다. 신호를 끄는 것으로는 아무것도 안 고쳐집니다.

```
function StrictModeDemo() {  
  const [tick, setTick] = useState(0);  
  
  useEffect(() => {  
    console.log("㉠ effect 실행 - tick 은 " + tick);  
  });  
}
```

```
F12 콘솔    ㉠ effect 실행 - tick 은 0  
           ㉠ effect 실행 - tick 은 1
```

```
return () => {  
  console.log("㉡ 정리 함수 실행 - tick 은 " + tick);  
}
```

```
F12 콘솔    ㉡ 정리 함수 실행 - tick 은 0
```

```

    };
    }, [tick]);

    return (
      <div className="demo"> <h3>① 두 번 실행되는 것 직접 보기
    </h3> <p className="output">tick: {tick}</p>
      { /* 화면: tick: 0 */ }
      <button onClick={() => setTick(tick + 1)}>tick +1</button> </div>
    );
  }
}

```

콘솔에 이 순서로 찍힙니다.

화면에 처음 나타날 때 · [tick +1 을 누를 때]

- | | |
|----------------------|----------------------|
| ① effect 실행 (tick 0) | ② 정리 함수 실행 (tick 0) |
| ② 정리 함수 실행 (tick 0) | ① effect 실행 (tick 1) |
| ① effect 실행 (tick 0) | |

왼쪽은 세 줄, 오른쪽은 두 줄입니다. 처음 나타날 때만 StrictMode 검사 때문에 한 세트가 더 붙습니다.

오른쪽에서 정리 함수가 effect 보다 먼저 온다는 것도 눈여겨보세요. “치우고 나서 새로 켜다” 는 순서입니다. 섹션 4에서 다시 봅니다.

▸ 직접 해보기 1

[tick +1] 을 세 번 누르고 콘솔 줄 수를 세어 보세요. 한 번 누를 때마다 몇 줄이 늘어납니까?

정리 함수의 모양

섹션 2

useEffect 에 넘긴 함수가 '함수' 를 return 하면, React 는 그것을 정리 함수로 씁니다.

```

useEffect(() => {
  const id = setInterval(tick, 1000); // 켜기 return () => {
  // 끄기 - 이것이 정리 함수 clearInterval(id);
  };
}, []);

```

짜이 되는 것끼리 한 곳에 모여 있는 것이 이 문법의 좋은 점입니다. 켜는 코드와 끄는 코드가 파일 여기저기 흩어져 있으면 하나를 빠뜨리기 쉽습니다.

[React 가 정리 함수를 부르는 때는 딱 두 가지입니다]

① 같은 effect 를 다시 실행하기 직전

의존성 배열의 값이 바뀌어 effect 를 또 돌려야 할 때, 지난번 것부터 치웁니다.

② 컴포넌트가 화면에서 사라질 때

조건부 렌더링으로 지워지거나, 목록에서 빠지거나,
지금처럼 왼쪽 메뉴에서 다른 예제로 옮겨 갈 때입니다.
이것을 '마운트 해제' 라고 부릅니다.
반대로 화면에 처음 나타나는 것은 '마운트' 입니다.

— return 하는 것은 반드시 함수여야 합니다

숫자나 문자열을 return 하면 React 가 콘솔에 경고를 냅니다. 정리 함수가 필요 없으면 아무것도 return 하지 않으면 됩니다.

▫ 직접 해보기 2

섹션 1 데모의 정리 함수 안에 `console.log("치우는 중")` 를 한 줄 더 넣고, 왼쪽 메뉴에서 다른 예제를 골랐다가 돌아와 보세요.

안 치우면 어떻게 되나

섹션 3

말로 하면 잘 안 와닿습니다. 직접 봅시다. 똑같은 타이머를 두 개 만듭니다. 하나는 정리 함수가 있고, 하나는 없습니다.

— 정리 함수가 있는 타이머

```
function SafeTimer() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    console.log("[안전한 타이머] 켜졌습니다");  
  });  
}
```

F12 콘솔 [안전한 타이머] 켜졌습니다

```
let safety = 0; // 학생 PC 를 지키기 위한 안전장치입니다 (아래 설명) const id =  
setInterval(() => {  
  safety += 1;  
  console.log("[안전한 타이머] " + safety + "번째 신호");  
}, 1000);
```

F12 콘솔 [안전한 타이머] 1번째 신호

```

setCount(safety);

    if (safety >= 12) {
        clearInterval(id);
        console.log("[안전한 타이머] 안전장치 - 6초가 지나 스스로 멈췄습니다");
    }
}, 500);

return () => {
    clearInterval(id);
    console.log("[안전한 타이머] 정리 함수 - 끕니다");
};

```

F12 콘솔 [안전한 타이머] 정리 함수 - 끕니다

```

    };
}, []);

return (
    <p className="output">안전한 타이머: {count}번째 신호까지 받음</p>
);
}

```

정리 함수가 없는 타이머 · — 일부러 빠뜨렸습니다

```

function LeakyTimer() {
    const [count, setCount] = useState(0);

    useEffect(() => {
        console.log("[새는 타이머] 끕니다 - 정리 함수가 없습니다");
    });
}

```

F12 콘솔 [새는 타이머] 끕니다 - 정리 함수가 없습니다

```

let safety = 0;

const id = setInterval(() => {
    safety += 1;
    console.log("[새는 타이머] " + safety + "번째 신호 - 화면에서 지워도 계속 나옵니다");
}, 1000);

```

F12 콘솔 [새는 타이머] 1번째 신호 - 화면에서 지워도 계속 나옵니다

```

setCount(safety);

```

★ 안전장치 ★ 진짜 새는 코드라면 이 줄이 없어서 영원히 돕니다. 그러면 여러분 브라우저가 점점 무거워지므로, 6초 뒤 스스로 멈추게 해 두었습니다. 이 줄이 없다고 상상하면서 아래 설명을 읽으세요.

```

    if (safety >= 12) {
      clearInterval(id);
      console.log("[새는 타이머] 안전장치 - 6초가 지나 스스로 멈췄습니다");
    }
  }, 500);

```

`return` 이 없습니다. 그래서 이 타이머는 아무도 안 꺼 줍니다.

```

    }, []);

    return <p className="output">새는 타이머: {count}번째 신호까지 받음</p>;
  }

  function LeakDemo() {
    const [showLeaky, setShowLeaky] = useState(true);
    const [showSafe, setShowSafe] = useState(true);

    return (
      <div className="demo"> <h3>② 정리 함수가 있는 타이머 vs 없는 타이머</h3>

      {showSafe && <SafeTimer />}
      {showLeaky && <LeakyTimer />}
      { /* 화면: 안전한 타이머: 0번째 신호까지 받음 / 새는 타이머: 0번째 신호까지 받음
      */}

      <button onClick={() => setShowSafe(!showSafe)}>
        {showSafe ? "안전한 타이머 숨기기" : "안전한 타이머 보이기"}
      </button> <button onClick={() => setShowLeaky(!showLeaky)}>
        {showLeaky ? "새는 타이머 숨기기" : "새는 타이머 보이기"}
      </button> </div>

    );
  }

```

콘솔을 비우고 두 개를 모두 [숨기기] 해 보세요. 그리고 콘솔을 지켜보세요.

안전한 타이머 → “정리 함수 — 꺾습니다” 가 찍히고 조용해집니다. 새는 타이머 → 화면에서 사라졌는데도 0.5초마다 계속 찍힙니다.

화면에는 없는데 코드는 돌고 있습니다. 이것이 ‘메모리 누수’ 입니다. 컴포넌트는 지워졌지만 타이머가 그 컴포넌트를 붙잡고 있어서 사라지지 못합니다.

— StrictMode 가 미리 알려 줍니다

숨기기를 누르기 전에도 이미 힌트가 있었습니다. 처음 나타날 때 콘솔을 보면 이렇습니다.

안전한 타이머 : 0.5초마다 한 줄씩
 새는 타이머 : 0.5초마다 두 줄씩

섹션 1에서 본 붙였다 뗐다 다시 붙이기 때문입니다. 정리를 안 했으니 첫 번째 타이머가 안 꺼진 채로 두 번째 타이머가 또 생긴 것입니다. 두 줄씩 찍히는 것이 바로 "정리 함수를 빠뜨렸다"는 증거입니다.

— 안전장치에 대해

위 두 타이머는 6초 뒤 스스로 멈추게 해 두었습니다. 실습 중에 브라우저가 느려지지 않게 하려는 것입니다. 실제 코드에는 이런 장치가 없으니, 안 치우면 정말로 끝없이 돕니다.

▶ 직접 해보기 3

이 예제를 연 직후에 [새는 타이머 숨기기]를 누르고 콘솔을 지켜보세요. 화면에서 지웠는데도 계속 찍히니까?

값이 바뀔 때도 정리된다

섹션 4

정리 함수는 컴포넌트가 사라질 때만 불리는 것이 아닙니다. 의존성 배열의 값이 바뀌어 effect를 '다시' 실행할 때도 먼저 불립니다.

지난번 effect의 정리 함수 실행 → 새 effect 실행

순서가 이렇게 되어 있어서, 옛날 타이머가 새 타이머와 겹치는 일이 없습니다. 아래 데모에서 속도를 바꿔 보면 콘솔에 그 순서가 그대로 나옵니다.

```
function SpeedTimerDemo() {
  const [delay, setDelay] = useState(500);
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("[속도 데모] " + delay + "밀리초 타이머 시작");
  });
}
```

F12 콘솔 [속도 데모] 500밀리초 타이머 시작
 [속도 데모] 1000밀리초 타이머 시작

```
let safety = 0;

const id = setInterval(() => {
  safety += 1;
  setCount((prev) => prev + 1); // 04단원 개념05의 함수형 갱신입니다 if (safety
  >= 12) clearInterval(id);
}, delay);

return () => {
  clearInterval(id);
  console.log("[속도 데모] " + delay + "밀리초 타이머 정리");
};
```

F12 콘솔 [속도 데모] 500밀리초 타이머 정리

```
};
}, [delay]); // delay 가 바뀌면 치우고 다시 겁니다 return (
  <div className="demo"> <h3>③ 속도를 바꾸면 치우고 다시 겁니다
</h3> <p className="output">
  간격 {delay}밀리초 / 신호 {count}번
</p>
  {/* 화면: 간격 500밀리초 / 신호 0번 */}
  <button onClick={() => setDelay(delay === 500 ? 1000 : 500)}>
    간격 바꾸기 (500 ↔ 1000)
  </button> </div>
);
}
```

버튼을 누르면 콘솔에 이 두 줄이 순서대로 찍힙니다.

속도 데모 · 500밀리초 타이머 정리 ← 먼저 치우고

속도 데모 · 1000밀리초 타이머 시작 ← 새로 겁니다

만약 정리 함수가 없었다면 500 짜리 타이머가 안 꺼진 채로 1000 짜리가 하나 더 생깁니다. 버튼을 다섯 번 누르면 타이머가 여섯 개가 됩니다. 화면 숫자가 미친 듯이 올라갑니다.

◁ 직접 해보기 4

위 정리 함수의 `clearInterval(id)` 줄만 잠시 지우고 [간격 바꾸기] 를 세 번 눌러 보세요. 숫자가 어떻게 됩니까? (확인했으면 반드시 되돌려 놓으세요)

이벤트 구독도 정리 대상

섹션 5

타이머 말고도 정리해야 하는 것이 있습니다. 대표적인 것이 이벤트 구독입니다. JS자료 11단원에서 배운 `addEventListener` 입니다.

컴포넌트 안의 버튼에 붙이는 onClick 은 정리할 필요가 없습니다. 그 버튼이 사라지면 React 가 알아서 떼어 줍니다.

하지만 window 나 document 처럼 컴포넌트 바깥 물건에 붙인 것은 React 가 모릅니다. 우리가 붙였으니 우리가 떼야 합니다.

```
function SubscribeDemo() {
```

처음 값은 지금 창 너비로 시작합니다.

```
const [width, setWidth] = useState(window.innerWidth);

useEffect(() => {
  function handleResize() {
    setWidth(window.innerWidth);
  }

  window.addEventListener("resize", handleResize);
  console.log("[구독] window 의 resize 를 구독했습니다");
```

F12 콘솔 [구독] window 의 resize 를 구독했습니다

```
return () => {
```

불일 때와 똑같은 함수를 넘겨야 떼어집니다 (JS자료 11단원)

```
window.removeEventListener("resize", handleResize);
console.log("[구독] resize 구독을 해제했습니다");
```

F12 콘솔 [구독] resize 구독을 해제했습니다

```
  };
}, []);

return (
  <div className="demo"> <h3>④ 창 크기 구독하기</h3> <p className="output">지금 창
너비: {width}px</p>
  { /* 화면: 지금 창 너비: 1280px ← 숫자는 여러분 창 크기에 따라 다릅니다 */ }
</div>
);
}
```

브라우저 창의 오른쪽 끝을 잡고 좌우로 끌어 보세요. 숫자가 따라 바뀝니다.

정리 함수가 없으면 어떻게 될까요? 이 예제를 떠나도 handleResize 가 window 에 붙어 있습니다. 창 크기를 바꿀 때마다 이미 사라진 컴포넌트의 setWidth 가 불립니다. 예제를 열 때마다 하나씩 쌓이니, 열 번 오가면 열 개가 붙어 있게 됩니다.

창 너비 대신 창 높이(`window.innerHeight`)를 보여 주세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

정리 함수를 아예 안 씀 — 섹션 3에서 실제로 돌려 봤습니다.

에러가 안 나서 가장 발견하기 어렵습니다. “콘솔이 두 줄씩 찍힌다” 를 신호로 삼으세요.

이런 실수를 합니다

정리 함수 자리에 함수가 아닌 것을 return 함

```
useEffect(() => {  
  return setInterval(tick, 1000);  
}, []);
```

`setInterval` 은 타이머 번호(숫자)를 돌려줍니다. 함수가 아닙니다. React 가 "useEffect must not return anything besides a function" 경고를 냅니다. 화살표 함수를 한 줄로 쓰면 자동 `return` 이 되므로 실수하기 쉽습니다. 종괄호로 감싸서 `return` 을 없애거나, `() => clearInterval(id)` 를 돌려주세요.

이런 실수를 합니다

불일 때와 다른 함수를 떼려고 함

```
window.addEventListener("resize", () => setWidth(window.innerWidth));  
return () => window.removeEventListener("resize", () => setWidth(window.innerWidth));
```

화살표 함수를 두 번 썼으니 서로 다른 함수입니다. 하나도 안 떼어집니다. 조용히 안 떼어지고 에러도 안 납니다. 섹션 5처럼 이름 있는 함수로 만들어 두세요.

이런 실수를 합니다

두 번 실행이 싫어서 StrictMode 를 지움

증상만 사라지고 원인은 그대로입니다. 배포한 앱에서는 타이머가 하나만 생기니 당장은 괜찮아 보입니다. 그러다 사용자가 화면을 여러 번 오가면 그때 쌍입니다. 그때는 재현하기도 어렵습니다.

이런 실수를 합니다

정리 함수에서 지워야 할 것을 골라서 지움

```
return () => { if (count > 3) clearInterval(id); };
```

조건을 붙이지 마세요. 정리 함수는 “컨 것을 전부 되돌린다”가 원칙입니다. 조건이 필요하다고 느껴지면 대개 effect 를 둘로 나눠야 하는 상황입니다.

화면

정리

- 1 개발 중에는 StrictMode 가 컴포넌트를 붙였다 뗐다 다시 붙입니다. 그래서 useEffect 가 두 번 실행되고 콘솔도 두 줄씩 찍힙니다. 정상입니다.
- 2 두 번 실행은 정리 함수를 제대로 썼는지 검사하는 장치입니다. 배포한 앱에서는 한 번만 실행됩니다.
- 3 두 번 도는 것은 처음 화면에 나타날 때뿐입니다. 값이 바뀌어 다시 도는 경우는 한 번씩입니다.
- 4 useEffect 안에서 함수를 return 하면 React 가 그것을 정리 함수로 씁니다. 켜는 코드 바로 옆에 끄는 코드를 둡니다.
- 5 정리 함수는 두 때에 불립니다. 같은 effect 를 다시 실행하기 직전, 그리고 컴포넌트가 화면에서 사라질 때입니다.
- 6 타이머·window 이벤트 구독처럼 컴포넌트 바깥에 컨 것은 반드시 정리합니다. 안 치우면 화면에서 사라진 뒤에도 계속 둡니다.

```
export default function Concept02Cleanup() {
  return (
    <div> <h1>개념 02 - 정리 함수</h1> <p className="guide"> <strong>F12 →
    Console</strong> 을 열고 보세요. 이 파일은 콘솔이 주인공입니다.
    <br /> <br />
    콘솔에 같은 줄이 <strong>두 번씩</strong> 찍히는 이유를 이 파일에서
    설명합니다.
    먼저 섹션 1을 읽으세요.
    <br /> <br />이 파일의 타이머들은 <strong>6초 뒤 스스로 멈추게</strong> 해
    두었습니다.
    실습 중에 브라우저가 느려지지 않게 하려는 안전장치입니다.
    </p> <StrictModeDemo /> <LeakDemo /> <SpeedTimerDemo /> <SubscribeDemo
  /> </div>
);
}
```

직접 해보기 정답

1) 한 번 누를 때마다 두 줄씩 늘어납니다.

```
// 콘솔: ② 정리 함수 실행 - tick 은 0
// 콘솔: ① effect 실행 - tick 은 1
```

세 번 누르면 여섯 줄입니다. → 처음 나타날 때만 세 줄(① ② ①)이고, 그 뒤로는 두 줄(② ①)씩입니다.

"처음 한 번만 두 번 돈다" 를 눈으로 확인하는 실습입니다.

2) `return () => {`

```
console.log("② 정리 함수 실행 - tick 은 " + tick);
console.log("치우는 중");
```

```
};
```

→ 다른 예제를 고르는 순간 두 줄이 찍힙니다.

컴포넌트가 화면에서 사라질 때 정리 함수가 불린다는 증거입니다.
돌아오면 다시 마운트되므로 ① ② ① 이 새로 찍힙니다.

3) 계속 찍힙니다.

```
// 콘솔: [새는 타이머] 1번째 신호 - 화면에서 지워도 계속 나옵니다
```

처음 나타날 때 타이머가 두 개 생겼으므로 실제로는 두 줄씩 짝을 지어 나옵니다. → 안전장치 때문에 '타이머를 끈 지 6초' 가 되면 스스로 멈춥니다.

그러니 예제를 열자마자 숨겼다면 6초 가까이, 한참 뒤에 숨겼다면 남은 시간만큼만 나옵니다.
멈출 때는 "[새는 타이머] 안전장치 - 6초가 지나 스스로 멈췄습니다" 가 나옵니다.
이 안전장치가 없었다면 브라우저 탭을 닫을 때까지 계속 찍힙니다.
같은 시간에 안전한 타이머 쪽은 "정리 함수 - 꺾었습니다" 한 줄만 남기고 조용합니다.

4) 신호 숫자가 두 배, 세 배 빠르게 올라갑니다. 타이머를 세 번 바꿨으면 옛 타이머 세 개가 안 꺼진 채로 남아 네 개가 동시에 돕니다. → 화면만 보면 "왜 이렇게 빨라졌지?" 싶고 원인이 안 보입니다.

콘솔에도 에러가 한 줄도 없습니다. 정리 함수를 빠뜨린 버그는 이렇게 조용합니다.
확인이 끝났으면 `clearInterval(id)` 를 꼭 되돌려 놓으세요.

```
5) const [height, setHeight] = useState(window.innerHeight);
function handleResize() {
```

```
setHeight(window.innerHeight);
```

```
}
```

...

```
<p className="output">지금 창 높이: {height}px</p>
```

```
// 화면: 지금 창 높이: 720px ← 숫자는 창 크기에 따라 다릅니다
```

→ 구독하는 이벤트 이름(resize)은 그대로입니다. 꺼내 쓰는 값만 바뀝니다.

fetch 로 받아오기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요. 인터넷 연결이 필요합니다.

드디어 `useEffect` 를 가장 많이 쓰는 자리에 왔습니다. 서버에서 데이터 받아오기입니다.

좋은 소식이 있습니다. `fetch` 는 이미 다 배웠습니다. JS자료 12단원 개념03·04에서 이렇게 썼습니다.

```
const res = await fetch(주소);
const data = await res.json();
```

React 라고 해서 달라지는 것은 하나도 없습니다. 똑같은 `fetch`, 똑같은 `await` 입니다. 새로 배울 것은 딱 두 가지뿐입니다.

- ① 이 코드를 '어디에' 두는가 → `useEffect` 안
- ② 받은 데이터를 '어떻게' 화면에 넣는가 → `state` 에 담는다

인터넷이 막힌 실습실이라면 실습프로젝트 폴더의 `index.html` 에서 오프라인_대체.js 줄을 감싼 주석만 지우세요 (개념01 머리말 참고).

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

JS자료 12단원에서 쓰던 연습용 서버와 같은 곳입니다.

```
const BASE_URL = "https://jsonplaceholder.typicode.com";
```

왜 `useEffect` 안에서 부르나

섹션 1

먼저 `useEffect` 없이 해 보면 왜 필요한지 알 수 있습니다. 컴포넌트 본문에 `fetch` 를 그냥 적었다고 생각해 봅시다.

```
function PostView() {
  const [post, setPost] = useState(null);

  fetch(`${BASE_URL}/posts/1`)      ← 본문에 그냥 적으면
    .then((res) => res.json())
    .then((data) => setPost(data));  ← 여기서 state 를 바꾸고

  return <p>{post.title}</p>;
}
```

이 코드는 이렇게 됩니다.

화면을 그린다 → 요청을 보낸다 → 데이터가 온다 → state 를 바꾼다
→ 화면을 다시 그린다 → 또 요청을 보낸다 → 또 데이터가 온다 → ...

끝이 없습니다. 서버에 요청이 끝없이 나갑니다. 게다가 StrictMode 는 컴포넌트 본문을 두 번 부르니 (개념02) 요청도 두 배로 나갑니다.

그래서 규칙이 하나 생깁니다.

컴포넌트 본문에서는 요청을 보내지 않는다. useEffect 안에서 보낸다.

useEffect 는 화면을 다 그린 뒤에, 그것도 의존성 배열이 허락할 때만 실행됩니다. [] 를 붙이면 처음 한 번만 요청이 나갑니다. 이것이 우리가 원하는 동작입니다.

아래 데모로 "본문이 몇 번 실행되는지" 를 먼저 눈으로 확인하세요.

```
function RenderCountDemo() {
  const [tick, setTick] = useState(0);
```

여기가 컴포넌트 본문입니다. 이 자리에 fetch 를 두면, 이 줄이 찍힐 때마다 요청이 한 번씩 나갑니다.

```
console.log("[섹션 1] 본문 실행 - 여기에 fetch 를 두면 요청이 나갑니다");
```

```
F12 콘솔    [섹션 1] 본문 실행 - 여기에 fetch 를 두면 요청이 나갑니다
```

```
return (
  <div className="demo"> <h3>① 본문은 화면을 그릴 때마다 실행됩니다
</h3> <p className="output">다시 그린 횟수: {tick}</p>
    {/* 화면: 다시 그린 횟수: 0 */}
    <button onClick={() => setTick(tick + 1)}>다시 그리기</button>
    {/* 화면(누르면): 다시 그린 횟수: 1 */}
  </div>
);
}
```

이 데모는 요청을 보내지 않습니다. 글자만 찍습니다. 하지만 저 `console.log` 자리에 `fetch` 가 있었다면, 찍힌 줄 수만큼 요청이 나갔을 것입니다. `state` 를 바꾸는 다른 코드가 하나만 있어도 이 숫자는 끝없이 올라갑니다.

▫ 직접 해보기 1

[다시 그리기] 를 세 번 누르고 콘솔 줄 수를 세어 보세요. 그 자리에 `fetch` 가 있었다면 요청이 몇 번 나갔을까요? (개발 중에는 `StrictMode` 때문에 두 배로 찍힙니다 — 개념02)

useEffect(async () => ...) 가 안 되는 이유

섹션 2

`await` 를 쓰려면 `async` 함수 안이어야 합니다(JS자료 12단원 개념04 규칙 1). 그러니 이렇게 쓰고 싶어집니다. 누구나 처음에 이렇게 씁니다.

```
useEffect(async () => {  
  const res = await fetch(url);  
  const data = await res.json();  
  setPost(data);  
}, []);
```

← × 이렇게 쓰면 안 됩니다

문법 에러는 아닙니다. 화면도 얼추 나옵니다. 그런데 콘솔에 경고가 뜹니다.

```
useEffect must not return anything besides a function,  
which is used for clean-up.
```

왜 그럴까요? 두 가지를 이어 붙이면 답이 나옵니다.

(가) `async` 함수는 '항상 `Promise` 를 돌려준다' (JS자료 12단원 개념04 규칙 2) (나) `useEffect` 는 '돌려받은 것을 정리 함수로 쓴다' (개념02 섹션 2)

그러니 `React` 는 `Promise` 를 정리 함수라고 믿게 됩니다. 나중에 정리하려고 그것을 함수처럼 부르면 함수가 아니라 낭패입니다. 그래서 `React` 가 미리 "함수 말고 다른 걸 돌려주지 마" 라고 알려 주는 것입니다.

★ 위 코드를 진짜로 써 보고 싶다면, 확인만 하고 반드시 되돌리세요. 경고가 콘솔에 계속 남아서 다른 문제를 가립니다.

해결책 · effect 안에 async 함수를 만들고, 그 함수를 부릅니다.

```
useEffect(() => {
  async function load() {
    const res = await fetch(url);
    ...
  }
  load();
}, []);
```

← 이 함수는 async 가 아닙니다
← 안쪽에 async 함수를 만들고
← 부릅니다

useEffect 에 넘긴 바깥 함수는 여전히 평범한 함수라 아무것도 **return** 하지 않습니다. await 는 안쪽 async 함수 안에 들어 있으니 규칙도 지켜집니다. 처음 보면 번거로워 보이지만, 이유를 알고 나면 자연스러운 모양입니다.

▹ 직접 해보기

2

load 라는 이름이 마음에 안 들면 다른 이름으로 바꿔도 될까요? 아래 섹션 3의 함수 이름을 fetchPost 로 바꿔 보세요.

글 하나 받아오기

섹션 3

```
function OnePostDemo() {
```

아직 아무것도 못 받았으니 null 로 시작합니다. 이 선택은 섹션 5에서 다시 봅니다.

```
const [post, setPost] = useState(null);
```

```
useEffect(() => {
```

① effect 안에 async 함수를 만들고

```
async function loadPost() {
  const res = await fetch(`${BASE_URL}/posts/1`);
  const data = await res.json();

  console.log("받은 글의 제목:", data.title);
```

```
F12 콘솔    받은 글의 제목: sunt aut facere repellat provident occaecati excepturi
            optio reprehenderit
```

② 받은 데이터를 state 에 담습니다. 그래야 화면이 다시 그려집니다.

```
setPost(data);
}
```

③ 만든 함수를 부릅니다

```
loadPost();
}, []); // 처음 한 번만 받아옵니다 return (
  <div className="demo"> <h3>② 글 하나 받아오기</h3>
    {post === null ? (
      <p className="output">아직 못 받았습니다</p>
    ) : (
      <div className="output"> <p> <strong>{post.title}</strong> </p> <p>
{post.body}</p> </div>
    )}
    { /* 화면: 잠깐 "아직 못 받았습니다" 가 보였다가 영어 제목과 본문으로 바뀝니다
  */}
  </div>
);
}
```

여기서 일어나는 일을 시간 순서로 보면 이렇습니다.

- 1 컴포넌트가 처음 실행된다. post 는 null 이다.
- 2 화면이 그려진다. "아직 못 받았습니다" 가 보인다.
- 3 useEffect 가 실행되어 요청이 나간다.
- 4 데이터가 도착한다. setPost(data) 로 state 가 바뀐다.
- 5 컴포넌트가 다시 실행된다. 이번엔 post 에 값이 있다.
- 6 화면이 다시 그려진다. 제목과 본문이 보인다.

화면이 두 번 그려집니다. 그래서 초기값이 반드시 필요합니다. 첫 번째 화면에 넣을 것이 있어야 하니까요.

5)에서 컴포넌트가 다시 실행되면 useEffect 도 또 도는 것 아니냐고요? 의존성 배열이 [] 이니 다시 돌지 않습니다. 그래서 무한 루프가 안 생깁니다. 섹션 1의 나쁜 예와 갈리는 지점이 정확히 여기입니다.

▹ 직접 해보기 3

/posts/1 을 /posts/5 로 바꿔 보세요. 제목이 바뀹니까?

목록 받아서 그리기

섹션 4

받은 것이 배열이면 05단원에서 배운 map 을 그대로 씁니다. 서버에서 왔다고 특별한 배열이 아닙니다. 그냥 배열입니다.

```
function PostListDemo() {
```

목록이니 빈 배열로 시작합니다. 이러면 map 을 바로 써도 안전합니다.

```
const [posts, setPosts] = useState([]);

useEffect(() => {
  async function loadPosts() {
```

주소 뒤의 ?_limit=3 은 "3개만 주세요" 입니다 (JS자료 12단원 개념03)

```
const res = await fetch(`${BASE_URL}/posts?_limit=3`);
const data = await res.json();

console.log("받은 글 개수:", data.length);
```

F12 콘솔 받은 글 개수: 3

```
setPosts(data);
}

loadPosts();
}, []);

return (
  <div className="demo">
    <h3>③ 목록 받아서 그리기</h3>
    <ul>
      {posts.map((post) => (
```

key 는 05단원 개념03에서 배운 그것입니다. 서버가 준 id 를 쓰면 딱 좋습니다.

```
      <li key={post.id}>
        {post.id}. {post.title}
      </li>
    ))}
  </ul>
  { /* 화면: 1. sunt aut facere... / 2. qui est esse / 3. ea molestias quasi...
*/}
  </div>
);
}
```

초기값을 빈 배열([])로 둔 것이 중요합니다. `null` 로 두면 첫 화면에서 `null.map(...)` 이 되어 그 자리에서 터집니다. 배열을 그릴 것이면 빈 배열로, 객체 하나를 그릴 것이면 `null` 로 시작하는 것이 편합니다.

빈 배열이면 `map` 이 아무것도 안 만들고 조용히 빈 목록이 나옵니다. "받아오는 중" 이라는 표시는 개념04에서 제대로 붙입니다.

_limit=3 을 _limit=5 로 바꾸고, 콘솔의 개수도 함께 확인하세요.

첫 렌더에는 데이터가 없다

섹션 5

이 단원에서 학생들이 가장 많이 터지는 지점입니다.

```
const [user, setUser] = useState(null);
...
return <p>{user.name}</p>;    ← 첫 화면에서 여기가 터집니다
```

에러 메시지는 이렇게 나옵니다.

```
Cannot read properties of null (reading 'name')
```

데이터가 늦게 오는 것은 고칠 수 없습니다. 네트워크는 원래 느립니다. 그러니 “아직 없는 상태” 를 화면이 감당하게 만들어야 합니다. 방법은 세 가지입니다.

방법 1 · 초기값을 빈 배열·빈 문자열로 둔다 — 목록이나 글자에 잘 맞습니다

방법 2 · 조건부 렌더링으로 갈라 준다 — 05단원의 삼항 연산자나 &&

방법 3 · 일찍 return 한다 — 05단원 개념01

아래 데모는 방법 3을 씁니다.

```
function FirstRenderDemo() {
  const [user, setUser] = useState(null);
```

컴포넌트 본문입니다. 화면을 그릴 때마다 실행됩니다.

```
console.log("[화면 그리는 중] user 가 아직 null 인가?", user === null);
```

```
F12 콘솔    [화면 그리는 중] user 가 아직 null 인가? true
            [화면 그리는 중] user 가 아직 null 인가? false
```

```
useEffect(() => {
  async function loadUser() {
    const res = await fetch(`${BASE_URL}/users/1`);
    const data = await res.json();
    setUser(data);
  }

  loadUser();
}, []);
```

방법 3 · 아직 없으면 여기서 끝내 버립니다. 아래 줄은 실행되지 않습니다.

```
if (user === null) {
  return (
    <div className="demo"> <h3>④ 첫 렌더에는 데이터가 없다
  </h3> <p className="output">사용자 정보를 기다리는 중</p> </div>
  );
}
```

여기까지 왔다면 user 는 반드시 객체입니다. 마음 놓고 점을 찍어도 됩니다.

```
return (
  <div className="demo"> <h3>④ 첫 렌더에는 데이터가 없다
  </h3> <div className="output"> <p>이름: {user.name}</p> <p>이메일: {user.email}
  </p> </div>
  { /* 화면: 이름: Leanne Graham / 이메일: Sincere@april.biz */
    <button onClick={() => setUser(null)}>받은 정보 지우기</button>
    { /* 화면(누르면): 사용자 정보를 기다리는 중 - 첫 화면을 다시 볼 수 있습니다 */
  </div>
  );
}
```

콘솔을 보면 “user 가 아직 null 인가?” 가 처음엔 true, 나중엔 false 로 바뀝니다. 컴포넌트가 두 번 실행됐다는 뜻입니다. 데이터가 오기 전에 한 번, 온 뒤에 한 번.

— 일찍 return 하는 방법의 장점

아래쪽 코드에서는 user 가 null 일 경우를 다시 생각하지 않아도 됩니다. user && user.name 같은 검사를 화면 곳곳에 뿌리지 않아도 됩니다. 화면에 넣을 값이 많을수록 이 방식이 편합니다.

— 혹은 일찍 return 하기 전에 부릅니다

위 코드에서 useState 와 useEffect 가 if 문보다 위에 있습니다. 우연이 아닙니다. 혹은 컴포넌트 맨 위에서만 부를 수 있습니다(04단원 개념06). if 문 아래에 useEffect 를 두면 어떤 날은 실행되고 어떤 날은 안 되니 규칙 위반입니다.

받은 정보 지우기 · 버튼을 눌러 보세요. 첫 화면이 다시 나옵니다.

그런데 아무리 기다려도 데이터가 다시 오지 않습니다. 고장이 아닙니다. 의존성 배열이 []라서 effect 가 다시 돌지 않기 때문입니다. 다시 받아오게 하려면 화면 맨 위의 [이 예제를 처음부터 다시 그리기] 를 누르세요. 컴포넌트가 새로 만들어지면서 [] effect 도 처음처럼 한 번 실행됩니다. 값이 바뀔 때마다 다시 받아오는 방법은 개념05에서 배웁니다.

▸ 직접 해보기 5

위 일찍 `return` 을 삼항 연산자 한 개로 바꿔 보세요. (05단원 개념01에서 배운 방법입니다)

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

`useEffect(async () => { ... }, [])` — 섹션 2에서 설명했습니다.

경고가 뜨고, 정리 함수를 쓸 수 없게 됩니다.

이런 실수를 합니다

의존성 배열을 아예 안 씀

```
useEffect(() => {  
  loadPost();  
});
```

화면을 그릴 때마다 요청이 나갑니다. 받아온 데이터를 state 에 넣으면 다시 그려지고 또 요청이 나갑니다. 섹션 1의 무한 루프와 같습니다. ★ 주석을 풀지 마세요. 서버에 요청이 끝없이 나갑니다.

이런 실수를 합니다

초기값 `null` 인데 바로 점을 찍음

```
return <p>{post.title}</p>;
```

Cannot read properties of `null` 로 이 예제가 빨간 상자가 됩니다. 섹션 5의 세 가지 방법 중 하나를 쓰세요.

이런 실수를 합니다

res 를 데이터라고 생각함

```
const res = await fetch(url);
setPost(res);
```

res 는 '응답' 이지 내용이 아닙니다. res.json() 을 한 번 더 거쳐야 데이터입니다. JS자료 12단원 개념03에서 then 이 두 번이었던 것과 같은 이야기입니다.

이런 실수를 합니다

res.json() 앞에 **await** 를 빠뜨림

이건 에러가 안 납니다. 조용히 아무것도 안 나옵니다. 그래서 아래에 실제로 돌려 둡니다.

```
function MissingAwaitDemo() {
  const [title, setTitle] = useState("아직 못 받았습니다");

  useEffect(() => {
    async function loadPost() {
      const res = await fetch(`${BASE_URL}/posts/2`);

      const data = res.json(); // ← await 를 일부러 빠뜨렸습니다 console.log("[실수
5] data.title 은", data.title);
    }
  }, []);
}
```

F12 콘솔 [실수 5] data.title 은 undefined

```
setTitle(data.title);
}

loadPost();
}, []);

return (
  <div className="demo"> <h3>⑤ [실수 5] await 를 빠뜨리면 조용히 빈 화면
</h3> <p className="output">제목: {title}</p>
  { /* 화면: 제목: ← 제목 자리가 텅 빕니다. 에러는 한 줄도 안 납니다 */ }
</div>
);
}
```

await 를 빼면 data 에는 내용이 아니라 Promise 가 들어갑니다. Promise 에는 title 이라는 속성이 없으니 undefined 입니다. React 는 undefined 를 화면에 아무것도 안 그리고 넘어갑니다. 그래서 조용합니다.

이런 버그를 만나면 순서가 있습니다. ① 화면이 비었다 → ② 넣으려던 값을 console.log 로 찍어 본다 → ③ undefined 면 그 값을 만든 줄로 거슬러 올라간다 이 습관이 디버깅 시간을 가장 많이 줄여 줍니다.

화면

정리

- 1 fetch·async·await 는 JS자료 12단원에서 배운 그대로입니다. React 에서 달라지는 것은 '어디에 두는가' 와 '받은 값을 state 에 담는다' 뿐입니다.
- 2 요청은 컴포넌트 본문이 아니라 useEffect 안에서 보냅니다. 본문에서 보내면 화면을 그릴 때마다 요청이 나가고 무한 루프가 됩니다.
- 3 useEffect(async () ⇒ ...) 는 쓸 수 없습니다. async 함수는 Promise 를 돌려주는데 useEffect 는 그것을 정리 함수로 착각하기 때문입니다.
- 4 대신 effect 안에 async 함수를 만들고 그 함수를 부릅니다. 바깥 함수는 평범한 함수로 남습니다.
- 5 처음 한 번만 받아올 때는 의존성 배열을 [] 로 둡니다. 그래야 state 를 바꿔도 다시 요청하지 않습니다.
- 6 첫 화면에는 데이터가 없습니다. 초기값을 빈 배열로 두거나, 조건부 렌더링이나 일찍 return 으로 '아직 없는 상태' 를 그려 주세요.

```
export default function Concept03FetchInEffect() {
```

개념01에서 쓴 것과 같은 방법입니다. key 가 바뀌면 아래 상자들이 통째로 새로 만들어져 [] 를 쓴 useEffect 도 처음처럼 한 번 더 실행됩니다.

```
const [restartKey, setRestartKey] = useState(0);

return (
  <div> <h1>개념 03 - fetch 로 받아오기</h1> <p className="guide"> <strong>인터넷
  연결이 필요합니다.</strong> <strong>F12 → Console</strong> 과{" "}
    <strong>Network</strong> 탭을 함께 보면 요청이 나가는 것이 보입니다.
    <br /> <br />
    인터넷이 막힌 실습실이라면 실습프로젝트 폴더의 <code>index.html</code> 에서{"
  "}
    <code>오프라인_대체.js</code> 줄을 감싼 주석만 지우세요.
    <br /> <br />⑤ 번 상자는 <strong>일부러 틀리게 만든 예제</strong>입니다.
  제목 자리가
    비어 있는 것이 정상입니다.
  </p> <button onClick={() => setRestartKey(restartKey + 1)}>
    이 예제를 처음부터 다시 그리기
  </button> <div key={restartKey}> <RenderCountDemo /> <OnePostDemo
  /> <PostListDemo /> <FirstRenderDemo /> <MissingAwaitDemo /> </div> </div>
);
}
```


직접 해보기 정답

1) 세 번 누르면 여섯 줄이 늘어납니다. 한 번에 두 줄씩입니다.

```
// 콘솔: [섹션 1] 본문 실행 - 여기에 fetch 를 두면 요청이 나갑니다
```

→ 두 줄인 이유는 StrictMode 가 본문을 두 번 부르기 때문입니다(개념02).

그 자리에 fetch 가 있었다면 요청이 여섯 번 나갔을 것입니다.
버튼 세 번 누른 것만으로 말입니다.
게다가 받아온 데이터를 state 에 넣으면 다시 그려지고 또 요청이 나가서
끝이 없어집니다. 이것이 본문에서 요청을 보내면 안 되는 이유입니다.

2) 됩니다. 아무 이름이나 괜찮습니다.

```
useEffect(() => {

  async function fetchPost() {
    const res = await fetch(`${BASE_URL}/posts/1`);
    const data = await res.json();
    setPost(data);
  }
  fetchPost();

}, []);
```

→ 이름을 바꿨으면 아래 부르는 줄도 함께 바뀌어야 합니다.

"load 라고 써야 한다" 는 규칙은 없습니다. 다만 loadPost.fetchUser 처럼
무엇을 받아오는지 드러나는 이름이 나중에 읽기 좋습니다.

3) 바뀝니다.

```
const res = await fetch(`${BASE_URL}/posts/5`);
// 콘솔: 받은 글의 제목: nesciunt quas odio
```

→ 주소의 숫자 하나만 바꾸면 됩니다. 나머지 코드는 손댈 것이 없습니다.

```
4) const res = await fetch(`${BASE_URL}/posts?_limit=5`);
// 콘솔: 받은 글 개수: 5
```

→ 화면에도 다섯 줄이 나옵니다. map 은 배열 길이에 맞춰 알아서 늘어납니다.

```
5) return (
```

```

<div className="demo">
  <h3>③ 첫 렌더에는 데이터가 없다</h3>
  {user === null ? (
    <p className="output">사용자 정보를 기다리는 중</p>
  ) : (
    <div className="output"> <p>이름: {user.name}</p> <p>이메일: {user.email}</p> </div>
  )}
</div>

```

```

);
// 화면: 이름: Leanne Graham / 이메일: Sincere@april.biz

```

→ 결과는 같습니다. 다만 삼항 안쪽에서는 user 가 null 일 수 있다는 것을

계속 신경 써야 합니다. 화면에 넣을 값이 두세 개를 넘어가면
일찍 **return** 쪽이 읽기 편해집니다. 둘 중 무엇이 옳다기보다 상황에 따라 고릅니다.

로딩과 에러

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요. 인터넷 연결이 필요합니다.

개념03의 예제는 전부 “잘 받아온다” 를 전제했습니다. 실제 서비스에서는 두 가지가 더 일어납니다.

느리다 - 데이터가 1초 뒤에 올 수도 있습니다. 그동안 화면은 텅 비어 있습니다.
실패한다 - 인터넷이 끊기거나, 주소가 틀렸거나, 서버가 죽었습니다.

둘 다 우리가 막을 수 없습니다. 대신 화면이 그 상황을 감당하게 만듭니다. 이 파일에서 만드는 것이 그것입니다.

★ 이 파일은 일부러 실패하는 요청을 보냅니다. 그래서 콘솔에 빨간 줄이 몇 개 나옵니다.

Failed to load resource: the server responded with a status of 404

이 빨간 줄은 정상입니다. 우리 코드가 낸 에러가 아니라 브라우저가 “이런 요청이 실패했어요” 하고 알려 주는 것입니다. 막을 수 없고, 막을 필요도 없습니다. 실무에서도 그냥 나옵니다.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";

const BASE_URL = "https://jsonplaceholder.typicode.com";
```

09

화면은 세 가지 상태 중 하나다

섹션 1

데이터를 받아오는 화면은 언제나 셋 중 하나입니다.

로딩 · 요청을 보냈고 아직 답이 안 왔다 → “불러오는 중...”

에러 · 실패했다 → “불러오지 못했습니다”

성공 · 데이터가 있다 → 데이터를 그린다

그래서 state 도 보통 세 개를 둡니다.

```
const [data, setData] = useState(null); // 성공했을 때의 내용
const [loading, setLoading] = useState(true); // 지금 기다리는 중인가
const [error, setError] = useState(null); // 실패했다면 무슨 이유인가
```

— 왜 loading 의 초기값이 true 인가

처음 화면이 나타나는 순간 이미 요청이 나갑니다. 그러니 처음부터 로딩 중입니다. false 로 시작하면 아주 잠깐 “데이터 없음” 화면이 번쩍이고 지나갑니다.

— 세 가지가 동시에 참이면 안 됩니다

로딩이면서 에러일 수는 없습니다. 그래서 화면을 그릴 때 순서대로 갈라 줍니다. 로딩인가? → 에러인가? → 아니면 성공. 이 순서가 가장 안전합니다.

07단원 개념05에서 “state 를 최소로 두라” 고 배웠는데 세 개나 두는 것이 이상해 보일 수 있습니다. 이 셋은 서로 계산해 낼 수 없는 값이라 각자 필요합니다. data 가 null 이라는 것만으로는 아직 기다리는 중인지 실패한 것인지 알 수 없으니까요.

⇒ 직접 해보기 1

loading 의 초기값을 false 로 두면 화면이 어떻게 달라질지 말로 설명해 보세요. (정답은 파일 맨 아래에)

로딩 표시 붙이기

섹션 2

```
function LoadingDemo() {
  const [post, setPost] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function loadPost() {
      console.log("[로딩 데모] 요청을 보냈습니다");
    }
  });
}
```

F12 콘솔 [로딩 데모] 요청을 보냈습니다

```
const res = await fetch(`${BASE_URL}/posts/1`);
const data = await res.json();

console.log("[로딩 데모] 데이터가 도착했습니다");
```

F12 콘솔 [로딩 데모] 데이터가 도착했습니다

```
setPost(data);
setLoading(false); // 다 받았으니 로딩 표시를 끕니다
}

loadPost();
}, []);

return (
```

값이 바뀌면 다시 받기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요. 인터넷 연결이 필요합니다.

지금까지는 의존성 배열이 늘 [] 였습니다. 처음 한 번만 받아왔습니다. 하지만 진짜 앱은 이렇게 동작합니다.

| | |
|----------------------|--------------------|
| 사용자가 목록에서 다른 사람을 고른다 | → 그 사람 정보를 다시 받아온다 |
| 검색창에 다른 글자를 친다 | → 그 글자로 다시 검색한다 |
| 다음 페이지 버튼을 누른다 | → 다음 페이지를 다시 받아온다 |

방법은 이미 개념01에서 배웠습니다. 의존성 배열에 그 값을 넣으면 됩니다.

```
useEffect(() => { 받아오기 }, [userId]);
```

정말 이게 전부입니다. 그런데 여기서 새 문제가 하나 생깁니다. 요청을 여러 번 보내면 답이 보낸 순서대로 오지 않는다는 것입니다. 이 파일의 후반부가 그 이야기입니다.

— 연습용 서버의 사용자 이름이 영어인 이유

jsonplaceholder 는 외국에서 만든 연습용 서버라 사람 이름이 영어입니다. Leanne Graham · Ervin Howell 처럼 나오는 것이 정상입니다.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";

const BASE_URL = "https://jsonplaceholder.typicode.com";
```

의존성에 값 넣기

섹션 1

```
function UserPickerDemo() {
  const [userId, setUserId] = useState(1);
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function loadUser() {
      setLoading(true);
```

useEffect 실수

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

useEffect 는 배우기는 쉬운데 잘못 쓰기도 쉽습니다. 그 이유가 있습니다. 잘못 써도 대부분 에러가 안 납니다.

무한 루프 → 에러 없이 브라우저만 뜨거워집니다
 의존성 빠뜨리기 → 에러 없이 옛날 값을 계속 씁니다
 안 써도 되는데 씀 → 에러 없이 잘 돌아가는 것처럼 보입니다

조용히 틀리는 코드가 가장 무섭습니다. 그래서 이 파일을 따로 두었습니다. 앞의 다섯 파일에서 조금씩 예고했던 것들을 여기서 한 번에 확인합니다.

★ 이 파일의 무한 루프 예제는 안전하게 만들어 두었습니다. 버튼을 눌러야 시작되고, 다섯 번 돌면 스스로 멈춥니다. 여러분 브라우저가 얼어붙는 일은 없습니다.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

무한 루프

섹션 1

useEffect 로 낼 수 있는 사고 중 가장 큰 것입니다. 만드는 방법은 아주 간단합니다. 두 가지만 겹치면 됩니다.

① effect 안에서 state 를 바꾼다 ② 그 state 가 의존성 배열에 들어 있다 (또는 의존성 배열이 아예 없다)
 그러면 이렇게 됩니다.

effect 실행 → state 바뀜 → 화면 다시 그림 → 의존성이 달라짐
 → effect 실행 → state 바뀜 → 화면 다시 그림 → ...

멈출 이유가 없습니다. 초당 수백 번씩 돕니다. 콘솔이 순식간에 수만 줄이 되고, 노트북 팬이 돌기 시작합니다.

아래 데모는 그 상황을 그대로 만들되, 다섯 번만 돌고 멈추게 해 두었습니다.

```
const LOOP_LIMIT = 5;

function InfiniteLoopDemo() {
  const [started, setStarted] = useState(false);
  const [count, setCount] = useState(0);

  useEffect(() => {
```

안전장치 1 · 버튼을 눌러야 시작합니다

```
if (!started) return;
```

안전장치 2 · 다섯 번 돌면 멈춥니다

```
if (count >= LOOP_LIMIT) {
  console.log(`[무한 루프] 상한 ${LOOP_LIMIT}에 달아 멈췄습니다`);
```

F12 콘솔 [무한 루프] 상한 5에 달아 멈췄습니다

```
return;
}
```

여기가 문제의 두 줄입니다.

```
console.log(`[무한 루프] ${count + 1}번째 실행 - 또 state 를 바꿉니다`);
```

F12 콘솔 [무한 루프] 1번째 실행 - 또 state 를 바꿉니다
[무한 루프] 5번째 실행 - 또 state 를 바꿉니다

```
setCount(count + 1); // ← effect 안에서 state 를 바꾸고
}, [started, count]); // ← 그 state 가 의존성에 들어 있습니다 return (
  <div className="demo"> <h3>① 무한 루프 (다섯 번만 돌고 멈춥니다)
</h3> <p className="output">
  {started ? `${count}번 돌았습니다` : "아직 시작하지 않았습니다"}
</p>
  {/* 화면: 아직 시작하지 않았습니다 */}
  <button onClick={() => setStarted(true)}>무한 루프 시작</button>
  {/* 화면(누르면): 5번 돌았습니다 */}
  <button onClick={() => {
    setStarted(false);
    setCount(0);
  }}
  >
  처음으로
</button> </div>
```

```
);  
}
```

버튼을 누르면 콘솔에 여섯 줄이 순식간에 찍히고 멈춥니다. (“1번째 실행” 부터 “5번째 실행” 까지 다섯 줄 + “상한 5에 닿아 멈췄습니다” 한 줄) 안전장치 두 줄을 빼면 “N번째 실행” 로그가 멈추지 않고 끝없이 이어집니다.

— 고치는 방법 세 가지

방법 1 · 그 state 를 의존성에서 뺀다 — 함수형 갱신을 쓰면 됩니다

```
useEffect(() => {  
  setCount((prev) => prev + 1);  
}, [started]);  
04단원 개념05의 setCount((prev) => prev + 1) 입니다.  
이러면 effect 안에서 count 를 읽지 않으니 의존성에 넣을 이유가 없습니다.
```

방법 2 · 조건을 붙여 멈출 자리를 만든다 — 위 안전장치가 그것입니다

방법 3 · 애초에 effect 를 안 쓴다 — 대개 이것이 정답입니다

“숫자를 하나 올린다” 는 화면 밖의 일이 아닙니다. 부작용이 아닙니다.
버튼을 눌러서 올리고 싶으면 onClick 에서 올리면 됩니다.

— 무한 루프를 알아채는 방법

콘솔 오른쪽에 같은 줄의 개수가 미친 듯이 올라갑니다. 브라우저 탭이 무거워지고 버튼이 잘 안 눌립니다.
그럴 때는 탭을 닫고, 방금 고친 useEffect 의 의존성 배열부터 보세요.

— 의존성 배열이 아예 없어도 같은 일이 벌어집니다

개념01에서 배웠듯 배열을 안 쓰면 매번 실행됩니다. 그 안에서 state 를 바꾸면 역시 끝이 없습니다.
개념03 섹션 1의 “본문에서 fetch 하면 안 되는 이유” 와 같은 구조입니다.

▹ 직접 해보기

1

위 LOOP_LIMIT 을 5에서 3으로 바꾸고 다시 눌러 보세요. 콘솔에 몇 줄이 찍힙니까?

의존성 빠뜨리기

섹션 2

무한 루프가 무서워서 의존성 배열을 무조건 [] 로 두는 사람이 많습니다. 그러면 반대쪽 문제가 생깁니다.
effect 가 옛날 값을 계속 들고 있습니다.

effect 안의 코드는 '그 effect 가 실행된 때' 의 값을 그대로 붙잡고 있습니다. 다시 실행되지 않으면 값도 영원히 그대로입니다. 이렇게 낡아 버린 값을 오래된 값(stale value)이라고 부릅니다.

아래 데모에서 A 와 B 를 비교해 보세요.

```
function StateValueDemo() {
  const [price, setPrice] = useState(4000); // 아메리카노
```

A · price 를 쓰면서 의존성에는 안 넣었습니다

```
useEffect(() => {
  console.log(`A) 의존성 [] - 이때 본 price 는 ${price}원`);
```

F12 콘솔 A) 의존성 [] - 이때 본 price 는 4000원

```
}, []);
```

B · 제대로 넣었습니다

```
useEffect(() => {
  console.log(`B) 의존성 [price] - 지금 price 는 ${price}원`);
```

F12 콘솔 B) 의존성 [price] - 지금 price 는 4000원
 B) 의존성 [price] - 지금 price 는 4500원

```
}, [price]);
```

```
return (
  <div className="demo"> <h3>② 의존성을 빠뜨리면 옛날 값을 봅니다
</h3> <p className="output">지금 가격: {price}원</p>
  { /* 화면: 지금 가격: 4000원 */ }
  <button onClick={() => setPrice(price === 4000 ? 4500 : 4000)}>
    아메리카노 ↔ 라떼 (4000 ↔ 4500)
  </button>
  { /* 화면(누르면): 지금 가격: 4500원 */ }
</div>
);
}
```

버튼을 눌러 보세요. 콘솔에 B 만 새 값으로 찍힙니다. A 는 아무 말이 없습니다.

A 는 틀린 값을 찍은 것이 아닙니다. 아예 다시 실행되지 않았습니니다. 처음 실행될 때 본 4000원을 아직도 들고 있습니다.

— 이게 왜 위험한가

콘솔에 글자를 찍는 정도면 티가 납니다. 그런데 이런 코드라면 어떨까요.

```
useEffect(() => {
  const id = setInterval(() => {
    서버에저장(price);      ← 여기서 쓰는 price 는 영원히 4000원
  }, 5000);
  return () => clearInterval(id);
}, []);
```

사용자는 가격을 4500원으로 고쳤는데 서버에는 계속 4000원이 저장됩니다. 애러도 경고도 없습니다. 나중에 데이터를 보고 나서야 알게 됩니다.

— 규칙은 하나입니다

effect 안에서 쓰는 값은 전부 의존성 배열에 넣습니다. 넣었더니 너무 자주 도는 것 같으면, 배열을 줄이지 말고 effect 를 둘로 나누거나 함수형 갱신($prev \Rightarrow \dots$)으로 그 값을 안 읽게 만듭니다. "의존성 배열은 실행 횟수를 조절하는 손잡이가 아니라, 무엇을 쓰는지 적는 목록" 입니다.

▸ 직접 해보기

2

[A] 의 의존성 배열을 [] 에서 [price] 로 바꿔 보세요. 이제 버튼을 누르면 A 도 새 값을 찍습니까?

계산하면 될 값을 effect 로 만들기

섹션 3

07단원 개념05에서 "계산할 수 있는 값은 state 로 두지 말라" 고 배웠습니다. useEffect 를 배우고 나면 그 금지를 어기기가 훨씬 쉬워집니다. "총액 state 를 두고 effect 로 맞춰 주면 되겠네" 하는 생각이 자연스럽게 들거든요.

아래 데모에 두 방식을 나란히 두었습니다.

```
const COFFEE_PRICE = 4000; // 아메리카노 한 잔
function DerivedStateDemo() {
  const [count, setCount] = useState(2);
```

나쁜 방법 · 총액을 state 로 두고 effect 로 맞춥니다

```
const [badTotal, setBadTotal] = useState(0);

useEffect(() => {
  setBadTotal(COFFEE_PRICE * count);
  console.log(`[파생 state] effect 가 총액을 다시 계산했습니다: ${COFFEE_PRICE * count}`);
});
```

```
F12 콘솔    [파생 state] effect 가 총액을 다시 계산했습니다: 8000
            [파생 state] effect 가 총액을 다시 계산했습니다: 12000
```

```
    }, [count]);
```

좋은 방법 · 그냥 계산합니다. 한 줄입니다.

```
const goodTotal = COFFEE_PRICE * count;
```

```
console.log(`[화면 그리는 중] 나쁜 총액 ${badTotal} / 좋은 총액 ${goodTotal}`);
```

```
F12 콘솔    [화면 그리는 중] 나쁜 총액 0 / 좋은 총액 8000
            [화면 그리는 중] 나쁜 총액 8000 / 좋은 총액 8000
            [화면 그리는 중] 나쁜 총액 8000 / 좋은 총액 12000
            [화면 그리는 중] 나쁜 총액 12000 / 좋은 총액 12000
```

```
return (
  <div className="demo"> <h3>③ 총액은 계산하면 되는 값입니다
</h3> <p className="output">
    아메리카노 {count}잔 - 나쁜 총액 {badTotal}원 / 좋은 총액 {goodTotal}원
  </p>
  { /* 화면: 아메리카노 2잔 - 나쁜 총액 8000원 / 좋은 총액 8000원 */ }
  <button onClick={() => setCount(count + 1)}>한 잔 더</button>
  { /* 화면(누르면): 아메리카노 3잔 - 나쁜 총액 12000원 / 좋은 총액 12000원 */ }
</div>
);
}
```

콘솔을 보면 나쁜 방법이 왜 나쁜지 그대로 드러납니다.

화면 그리는 중 · 나쁜 총액 0 / 좋은 총액 8000 ← 첫 화면. 나쁜 쪽은 아직 0원

파생 state · effect 가 총액을 다시 계산했습니다: 8000

화면 그리는 중 · 나쁜 총액 8000 / 좋은 총액 8000 ← 이제야 맞았습니다

화면이 두 번 그려졌습니다. 그리고 그 사이에 잠깐 0원이 화면에 나갔습니다.

한 잔 더 · 를 눌러도 똑같습니다. 잠깐 12000원과 8000원이 함께 보입니다.

— 나쁜 방법의 문제 세 가지

- 1 한 박자 늦습니다. 잠깐이지만 어긋난 값이 진짜로 화면에 나옵니다.
- 2 화면을 두 번 그립니다. 한 번이면 될 일입니다.
- 3 어긋날 수 있습니다. 나중에 다른 곳에서 count 만 바꾸고

badTotal 을 안 고치면 둘이 영영 안 맞습니다.

— 판단 기준

지금 있는 값으로 계산해 낼 수 있으면 state 로 두지 않습니다. 그냥 계산합니다. 계산이 무거워서 걱정된다면 10단원의 useMemo 를 배운 뒤에 생각하세요. 대부분은 걱정할 필요가 없습니다. 곱셈 한 번은 아무것도 아닙니다.

◁ 직접 해보기 3

나쁜 총액을 화면에서 빼고, badTotal state 와 그 useEffect 도 통째로 지워 보세요. 화면이 달라집니까?

이벤트로 하면 될 일을 effect 로 하기

섹션 4

판단이 헛갈리는 마지막 경우입니다.

effect 로 할 일 · 화면에 보이는 것만으로 일어나야 하는 일

예) 이 화면이 열렸으니 글 목록을 받아온다
이 화면이 열렸으니 타이머를 켜다

이벤트로 할 일 · 사용자가 무언가를 했기 때문에 일어나는 일

예) 담기 버튼을 눌렀으니 장바구니에 넣는다
보내기 버튼을 눌렀으니 서버로 보낸다

구분하는 질문은 이것입니다.

"화면이 그냥 떠 있기만 해도 이 일이 일어나야 하나?"

그렇다면 effect, 아니라면 이벤트 핸들러입니다.

— 흔히 보는 잘못된 모양

```
const [submitted, setSubmitted] = useState(false);

useEffect(() => {
  if (submitted) {
    서버로보내기();
  }
}, [submitted]);

<button onClick={() => setSubmitted(true)}>보내기</button>
```

돌아가기는 합니다. 그런데 state 하나가 괜히 늘었고, 코드를 읽는 사람은 “보내기”를 찾으려고 두 곳을 오가야 합니다. 게다가 submitted를 false로 되돌리는 것을 잊으면 두 번째 클릭이 안 먹습니다.

아래처럼 한 곳에 두면 끝입니다.

```
function EventNotEffectDemo() {
  const [cart, setCart] = useState([]);
```

사용자가 눌렀기 때문에 일어나는 일 → 여기에 씁니다

```
function handleAdd() {
  const nextCart = [...cart, "아메리카노"];
  setCart(nextCart);

  console.log(`[이벤트에서] 담기 처리 완료 - 지금 ${nextCart.length}개`);
```

F12 콘솔 [이벤트에서] 담기 처리 완료 - 지금 1개

```
}

return (
  <div className="demo"> <h3>④ 누른 결과는 이벤트 핸들러에서
</h3> <p className="output">장바구니: {cart.length}개</p>
  { /* 화면: 장바구니: 0개 */ }
  <button onClick={handleAdd}>아메리카노 담기</button>
  { /* 화면(누르면): 장바구니: 1개 */ }
</div>
);
}
```

— 왜 nextCart를 따로 만들었나

setCart(nextCart)를 부른 다음 줄에서 cart.length를 읽으면 옛날 값이 나옵니다. 04단원 개념05에서 배운 그대로입니다. state는 즉시 바뀌지 않습니다. 그래서 새 배열을 변수에 담아 두고 그것을 씁니다.

[useEffect 를 안 써도 된다는 것이 이상하게 느껴진다면] useEffect 는 “화면과 바깥세상을 맞추는” 도구입니다. 버튼을 누른 결과를 처리하는 도구가 아닙니다. 09단원 내내 배운 것을 한 줄로 줄이면 이렇습니다. “화면이 떠 있는 동안 유지되어야 하는 것”에만 useEffect 를 씁니다.

▹ 직접 해보기 4

담기 버튼을 세 번 누르고 콘솔을 보세요. 숫자가 1, 2, 3 으로 제대로 올라갑니까?

자주 하는 실수

섹션 5

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

effect 안에서 바꾸는 state 를 의존성에 넣음 — 섹션 1의 무한 루프입니다.

★ 안전장치 없이 따라 하지 마세요.

이런 실수를 합니다

의존성 배열에 함수를 넣음

```
function loadUser() { ... }  
useEffect(() => { loadUser(); }, [loadUser]);
```

컴포넌트 안에서 만든 함수는 화면을 그릴 때마다 새로 만들어집니다. 개념01 섹션 5의 배열과 똑같은 상황입니다. 매번 다른 물건이라 매번 실행됩니다. 그 안에서 state 를 바꾸면 무한 루프가 됩니다. 지금 단계에서는 함수를 effect ‘안’ 으로 옮기면 해결됩니다(개념03에서 쓴 방식).

이런 실수를 합니다

의존성 배열을 실행 횟수 조절 손잡이로 씀

```
useEffect(() => { 서버에저장(price); }, []);
```

너무 자주 도는 게 싫어서 []로 만들면 옛날 값을 씁니다. 섹션 2입니다. 의존성 배열은 “이 effect 가 무엇을 쓰는지” 적는 목록입니다.

이런 실수를 합니다

정리 함수를 안 씀

개념02 섹션 3에서 실제로 봤습니다. 타이머와 구독은 반드시 치웁니다. 콘솔이 두 줄씩 찍히는 것이 신호입니다.

이런 실수를 합니다**안 써도 되는데 씀**

계산하면 되는 값(섹션 3), 버튼을 눌러서 하는 일(섹션 4)에는 쓰지 않습니다. useEffect 를 하나 쓸 때마다 스스로에게 물어보세요. "이건 화면 밖의 일인가? 화면이 떠 있기만 해도 일어나야 하나?"

이런 실수를 합니다**다시 받아올 때 ignore 를 안 씀**

개념05 섹션 3의 경쟁 상태입니다. 의존성에 값이 들어 있는 fetch effect 라면 `let ignore = false` → if (ignore) `return` → 정리 함수에서 `ignore = true`.

화면 ———**정리** ———

- 1 effect 안에서 바꾸는 state 를 의존성 배열에 넣으면 무한 루프가 됩니다. 에러 없이 브라우저만 무거워집니다.
- 2 무한 루프는 함수형 갱신(`prev ⇒ ...`)으로 그 값을 안 읽게 하거나, 애초에 effect 를 안 쓰는 쪽으로 고칩니다.
- 3 의존성 배열은 실행 횟수를 줄이는 손잡이가 아니라 effect 가 무엇을 쓰는지 적는 목록입니다. 빠뜨리면 옛날 값을 계속 씁니다.
- 4 지금 있는 값으로 계산할 수 있는 것은 state 로 두지 않습니다. effect 로 맞추면 한 박자 늦고 화면을 두 번 그립니다.
- 5 버튼을 눌러서 일어나는 일은 이벤트 핸들러에 씁니다. useEffect 는 화면이 떠 있는 동안 유지되어야 하는 일에만 씁니다.
- 6 useEffect 를 쓰기 전에 물어보세요. 이건 화면 밖의 일인가? 화면이 그냥 떠 있기만 해도 일어나야 하나?

```
export default function Concept06EffectMistakes() {
  return (
    <div> <h1>개념 06 – useEffect 실수</h1> <p className="guide"> <strong>F12 →
    Console</strong> 을 함께 열어 두세요.
      <br /> <br /> ① 번의 무한 루프는 <strong>안전하게</strong> 만들어 두었습니다.
    버튼을 눌러야
      시작되고 <strong>다섯 번만 돌고 멈춥니다.</strong> 브라우저가 얼어붙지
    않습니다.
      <br /> <br /> ③ 번 상자의 "나쁜 총액" 은 <strong>일부러 틀리게 만든
    예제</strong>
      입니다. 잠깐 어긋난 값이 보이는 것이 정상입니다.
```

연습문제 (14문항)

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요. 7번부터는 인터넷 연결이 필요합니다.

- 1 문제 설명을 읽습니다.
- 2 TODO 자리에 코드를 씁니다.
- 3 저장하면 화면이 저절로 새로 그려집니다. 기대 결과와 맞는지 봅니다.
- 4 막히면 개념 파일의 해당 섹션을 다시 보세요. 문제마다 어느 개념인지 적어 두었습니다.

순서는 앞에서 뒤로 갈수록 어려워집니다. 문제 1~11 기본 문제 12 [응용] 문제 13 [도전] 문제 14 에러 확인

★ 콘솔에 같은 줄이 두 번씩 찍히는 것은 정상입니다(개념02 StrictMode). 기대 결과에는 한 번만 적어 두었습니다.

★ 인터넷이 막힌 실습실이라면 실습프로젝트 폴더의 index.html 에서 오프라인_대체.js 줄을 감싼 주석만 지우세요.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";

const BASE_URL = "https://jsonplaceholder.typicode.com";
```

문제 1 (개념01)

이 컴포넌트가 화면에 처음 나타날 때 콘솔에 “09단원 시작합니다” 를 한 번 찍으세요. 처음 한 번만 실행되어야 합니다.

기대 콘솔 09단원 시작합니다
 (개발 중에는 두 줄로 보입니다. 정상입니다)
 버튼이 없으니 화면을 열자마자 찍혀야 합니다.
 한 줄도 안 나오면 useEffect 를 안 썼거나 함수를 안 넘긴 것입니다.

```
function Problem01() {
```


여기에 `useEffect` 를 쓰세요 (의존성 배열을 잊지 마세요)

```
return (
  <div className="demo"> <h3>문제 1 - 처음 나타날 때 한 번만
</h3> <p className="output">콘솔(F12)을 보세요</p> </div>
);
}
```

문제 2 (개념01)

의존성 배열을 아예 쓰지 않는 `useEffect` 를 만들어 콘솔에 “다시 그려졌습니다” 를 찍으세요. 아래 [다시 그리기] 버튼을 누를 때마다 한 줄씩 늘어나야 합니다.

기대 콘솔 다시 그려졌습니다
버튼을 누를 때마다 한 줄씩 늘어납니다.
버튼을 눌러도 안 늘어나면 의존성 배열에 [] 를 붙인 것입니다.

```
function Problem02() {
  const [tick, setTick] = useState(0);
```

여기에 `useEffect` 를 쓰세요

```
return (
  <div className="demo"> <h3>문제 2 - 매번 실행되게 하기
</h3> <p className="output">다시 그린 횟수: {tick}</p> <button onClick={() =>
setTick(tick + 1)}>다시 그리기</button> </div>
);
}
```

문제 3 (개념01)

`count` 가 바뀔 때만 콘솔에 “지금 잔 수: N” 을 찍으세요.

잔 수 +1 을 누르면 찍히고, [메뉴 바꾸기] 를 눌러도 찍히면 안 됩니다.

기대 콘솔 지금 잔 수: 0

잔 수 +1 을 누르면 → 지금 잔 수: 1

메뉴 바꾸기 를 눌렀는데도 줄이 늘어나면 의존성 배열이 비었거나 없는 것입니다.

```
function Problem03() {  
  const [count, setCount] = useState(0);  
  const [menu, setMenu] = useState("아메리카노");
```

여기에 `useEffect` 를 쓰세요

```
  return (  
    <div className="demo"> <h3>문제 3 - 특정 값이 바뀔 때만  
    </h3> <p className="output">  
      {menu} {count}잔  
      </p> <button onClick={() => setCount(count + 1)}>잔 수  
      +1</button> <button onClick={() => setMenu(menu === "아메리카노" ? "라떼" :  
      "아메리카노")}>  
        메뉴 바꾸기  
      </button> </div>  
  );  
}
```

문제 4 (개념01)

할 일 개수가 바뀔 때마다 브라우저 탭 제목을 "할일 N개" 로 바꾸세요. 탭 제목은 `document.title` 입니다.

기대 화면 창 맨 위 탭 글자가 "할일 0개" 가 됩니다.

할 일 추가 를 누르면 "할일 1개" 로 바뀝니다.

탭 글자가 안 바뀌면 `useEffect` 안에 `document.title` 을 안 넣은 것입니다.

```
function Problem04() {
  const [todoCount, setTodoCount] = useState(0);
```

여기에 useEffect 를 쓰세요

```
return (
  <div className="demo"> <h3>문제 4 - 탭 제목 바꾸기</h3> <p className="output">할
  일 {todoCount}개</p> <button onClick={() => setTodoCount(todoCount + 1)}>할 일 추가
  </button> </div>
);
}
```

문제 5 (개념02)

1초마다 seconds 를 1씩 올리는 타이머를 만드세요. 정리 함수로 타이머를 반드시 꺼 주어야 합니다.

힌트 · setInterval 안에서 state 를 올릴 때는 함수형 갱신을 쓰세요.

setSeconds((prev) => prev + 1) ← 04단원 개념05
이러면 seconds 를 읽지 않으니 의존성 배열이 [] 여도 괜찮습니다.

기대 화면 0초 → 1초 → 2초 ... 로 1초마다 올라갑니다.
숫자가 한 번에 2씩 뛰면 정리 함수를 안 쓴 것입니다.
(StrictMode 가 타이머를 두 개 만들었기 때문입니다)

```
function Problem05() {
  const [seconds, setSeconds] = useState(0);
```

여기에 useEffect 를 쓰세요. 정리 함수도 함께 쓰세요.

```
return (
  <div className="demo"> <h3>문제 5 - 타이머와 정리 함수
</h3> <p className="output">{seconds}초</p> </div>
);
}
```

문제 6 (개념02)

문제 5에서 만든 정리 함수 안에 `console.log("타이머를 껏습니다")` 를 넣으세요. 그리고 왼쪽 메뉴에서 다른 예제를 골랐다가 이 파일로 돌아오세요.

먼저 예상해 보세요. 그 줄이 언제 찍힐까요? (가) 다른 예제를 고르는 순간 (나) 이 파일로 돌아오는 순간 (다) 둘 다 (라) 안 찍힌다

기대 출력 예상한 뒤에 직접 확인하세요. 정답은 정답 파일에 있습니다.
화면에는 아무 변화가 없습니다. 콘솔만 보세요.

(문제 6은 문제 5의 코드를 고치는 문제입니다. 따로 만들 것이 없습니다)

문제 7 (개념03)

3번 글(/posts/3)을 받아와 제목을 화면에 보여 주세요. `useEffect` 안에 `async` 함수를 만들어 부르는 방식을 쓰세요.

기대 화면 ea molestias quasi exercitationem repellat qui ipsa sit aut
"아직 못 받았습니다" 그대로면 `setTitle` 을 안 불렀거나 주소가 틀린
것입니다.
화면이 빨간 상자가 되면 `useEffect(async () => ...)` 를 쓴 것입니다.

```
function Problem07() {
  const [title, setTitle] = useState("아직 못 받았습니다");
```

여기에 `useEffect` 를 쓰세요

안에 `async` 함수를 만들고, 만든 함수를 부르는 것을 잊지 마세요.

```
return (
  <div className="demo"> <h3>문제 7 - 글 하나 받아오기</h3> <p className="output">
{title}</p> </div>
);
}
```

문제 8 (개념03)

사용자 3명(/users?_limit=3)을 받아와 이름을 목록으로 그리세요. map 으로 를 만들고 key 를 붙이세요.

기대 화면 Leanne Graham / Ervin Howell / Clementine Bauch
콘솔에 key 경고가 뜨면 key 를 안 붙인 것입니다.
화면이 빨간 상자가 되면 초기값을 null 로 두고 map 을 부른 것입니다.

```
function Problem08() {
  const [users, setUsers] = useState([]);
```

여기에 useEffect 를 쓰세요

```
return (
  <div className="demo"> <h3>문제 8 - 목록 받아서 그리기</h3> <ul>{/* TODO: users
를 map 으로 그리세요 */}</ul> </div>
);
}
```

문제 9 (개념04)

문제 7과 같은 일을 하되, 받아오는 동안 "불러오는 중..." 을 보여 주세요. loading state 를 하나 더 두면 됩니다.

기대 화면 잠깐 "불러오는 중..." 이 보였다가 제목으로 바뀝니다.
"불러오는 중..." 에서 안 바뀌면 setLoading(false) 를 안 부른 것입니다.
너무 빨라서 안 보이면 F12 → Network → Slow 4G 로 바꿔 보세요.

```
function Problem09() {
  const [title, setTitle] = useState("");
  const [loading, setLoading] = useState(true);
```

여기에 `useEffect` 를 쓰세요

```
return (
  <div className="demo"> <h3>문제 9 - 로딩 표시</h3> <p className="output">{loading
? "불러오는 중..." : title}</p> </div>
);
}
```

문제 10 (개념 04)

없는 글(/posts/9999)을 요청하고, `res.ok` 를 확인해 직접 에러를 던지세요. `catch` 에서 `err.message` 를 콘솔에 찍으세요.

힌트 · `if (!res.ok) throw new Error(서버 응답 오류 (${res.status}));`

기대 콘솔 문제10 에러: 서버 응답 오류 (404)
이 줄이 안 나오면 `res.ok` 검사를 빠뜨린 것입니다. 404 는 `catch` 로 안 갑니다.
콘솔에 빨간 `Failed to load resource` 줄이 함께 나오는 것은 정상입니다.

```
function Problem10() {
  const [message, setMessage] = useState("확인 중");
```

여기에 `useEffect` 를 쓰세요

`try / catch` 로 감싸고, `catch` 안에서 `setMessage("에러가 났습니다")` 도 해 주세요.

```
return (
  <div className="demo"> <h3>문제 10 – res.ok 확인하기</h3> <p className="output">
    {message}</p> </div>
);
}
```

문제 11 (개념04)

로딩 · 에러 · 성공 세 갈래를 모두 갖춘 화면을 만드세요. 주소는 없는 글(/posts/9999)이므로 결과는 에러 화면이 되어야 합니다. 에러 문구는 "글을 불러오지 못했습니다" 로 하세요.

힌트 · 화면을 가르는 순서는 loading → error → 성공 입니다.

로딩을 끄는 일은 **finally** 에 두세요.

기대 화면 잠깐 "불러오는 중..." 이 보였다가 "글을 불러오지 못했습니다" 로 바뀝니다.
"불러오는 중..." 에서 멈춰 있으면 setLoading(false) 가 try 안에만 있는
것입니다.

```
function Problem11() {
  const [post, setPost] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
```

여기에 **useEffect** 를 쓰세요

```
return (
  <div className="demo"> <h3>문제 11 – 세 갈래 화면</h3>
    { /* TODO: loading / error / 성공 세 가지를 갈라서 그리세요 */ }
    <p className="output">여기에 세 갈래 화면을 그리세요</p> </div>
);
}
```

문제 12 [응용] (개념05)

버튼으로 사용자 번호를 고르면 그 사람 정보를 다시 받아오게 하세요. 1 · 2 · 3 번 버튼이 이미 만들어져 있습니다. **useEffect** 만 채우면 됩니다.

힌트 · 의존성 배열에 `userId` 를 넣으세요.

다시 받아올 때 로딩 표시도 다시 켜야 합니다.

기대 화면 처음에는 Leanne Graham 이 나옵니다.

2번 · 을 누르면 Ervin Howell, [3번] 을 누르면 Clementine Bauch 로 바뀝니다.

버튼을 눌러도 이름이 안 바뀌면 의존성 배열이 [] 인 것입니다.

```
function Problem12() {  
  const [userId, setUserId] = useState(1);  
  const [name, setName] = useState("");  
  const [loading, setLoading] = useState(true);
```

여기에 `useEffect` 를 쓰세요

```
  return (  
    <div className="demo"> <h3>문제 12 [응용] - 고른 사람 다시 받아오기  
</h3> <button onClick={() => setUserId(1)}>1번</button> <button onClick={() =>  
setUserId(2)}>2번</button> <button onClick={() => setUserId(3)}>3번  
</button> <p className="output">{loading ? "불러오는 중..." : name}</p> </div>  
  );  
}
```

문제 13 [도전] (개념05)

문제 12에 `ignore` 플래그를 붙여, 늦게 도착한 옛 응답이 화면을 덮어쓰지 못하게 하세요.

아래 데모는 1번 사용자만 일부러 늦게 답하도록 만들어 두었습니다(wait 함수).

경쟁 재현 · 버튼은 1번을 고른 뒤 곧바로 2번을 고릅니다.

`ignore` 없이 만들면 화면이 뒤늦게 Leanne Graham 으로 되돌아갑니다.

힌트 · effect 맨 위에 `let ignore = false;`

받은 뒤 `if (ignore) return;`
정리 함수에서 `ignore = true;`

기대 화면 [경쟁 재현] 을 눌러도 Ervin Howell 그대로여야 합니다.
Leanne Graham 으로 되돌아가면 ignore 가 동작하지 않는 것입니다.
ignore = true 를 정리 함수에 안 넣었는지 확인하세요.

```
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

function Problem13() {
  const [userId, setUserId] = useState(2);
  const [name, setName] = useState("아직 없음");
```

여기에 `useEffect` 를 쓰세요.

1번을 받을 때는 `res.json()` 뒤에 `await wait(250);` 을 넣어 일부러 늦추세요.
그래야 경쟁 상태가 재현됩니다.

```
function runRace() {
  setUserId(1);
  setTimeout(() => setUserId(2), 80);
}

return (
  <div className="demo"> <h3>문제 13 [도전] - 늦게 온 응답 버리기
</h3> <p className="output">지금 화면에 그린 이름: {name}</p> <button onClick=
{runRace}>경쟁 재현 (1번 → 곧바로 2번)</button> <button onClick={() =>
setUserId(3)}>3번 고르기 (평범하게)</button> </div>
);
}
```

문제 14 에러 확인 (개념06)

아래 코드는 무한 루프에 빠집니다. 주석을 풀지 마세요. 눈으로만 보세요.

```
const [count, setCount] = useState(0);

useEffect(() => {
  setCount(count + 1);
}, [count]);
```

물음 1. 왜 끝없이 도는지 한 문장으로 설명해 보세요. 물음 2. 이 코드가 하려던 일이 “화면에 나타날 때 숫자를 1 올린다” 라면

어떻게 고쳐야 할까요? 아래 TODO 자리에 고친 코드를 쓰세요.

힌트 · effect 안에서 count 를 읽지 않으면 의존성에 넣을 이유가 없어집니다.

04단원 개념05의 함수형 갱신을 떠올려 보세요.

기대 화면 숫자가 1 에서 멈춰 있어야 합니다.
 (개발 중에는 StrictMode 때문에 2 가 될 수도 있습니다. 그것도 정답입니다)
 숫자가 계속 올라가면 아직 루프가 돌고 있는 것입니다. 바로 되돌리세요.

```
function Problem14() {
  const [count, setCount] = useState(0);
```

여기에 고친 useEffect 를 쓰세요

```
return (
  <div className="demo"> <h3>문제 14 – 무한 루프 고치기</h3> <p className="output">
    숫자: {count}</p> </div>
  );
}
```

화면 _____

정리

- 1 문제 1~4는 개념01입니다. 의존성 배열 세 가지(없음 / [] / [값])를 구분할 수 있으면 됩니다.
- 2 문제 5~6은 개념02입니다. 켜 것은 정리 함수로 끄니다. 콘솔이 두 줄씩 찍히면 정리를 빠뜨린 신호입니다.
- 3 문제 7~8은 개념03입니다. effect 안에 async 함수를 만들어 부르고, 받은 값을 state 에 담습니다.
- 4 문제 9~11은 개념04입니다. 로딩·에러·성공 세 갈래와 res.ok 검사가 핵심입니다.
- 5 문제 12~13은 개념05입니다. 의존성에 값을 넣어 다시 받아오고, ignore 로 늦은 응답을 버립니다.
- 6 문제 14는 개념06입니다. effect 안에서 바꾸는 state 를 의존성에 넣으면 무한 루프가 됩니다.
- 7 막히면 정답 파일을 보기 전에 개념 파일의 해당 섹션을 먼저 다시 읽어 보세요.

```
export default function Unit09Exercises() {
  return (
    <div>
      <h1>09단원 연습문제 (14문항)</h1> <p className="guide">
        각 상자의 <strong>TODO</strong> 자리를 채우세요. 저장하면 화면이 저절로 다시
        그려집니다.
        <br /> <br /> <strong>F12 → Console</strong> 을 함께 열어 두세요. 콘솔로
        확인하는 문제가 많습니다.
        같은 줄이 두 번씩 찍히는 것은 정상입니다(개념02 StrictMode).
        <br /> <br />
        7번부터는 <strong>인터넷 연결이 필요합니다.</strong> 막혀 있다면 실습프로젝트
        폴더의 <code>index.html</code> 에서 <code>오프라인_대체.js</code> 줄을 감싼
        주석만
        지우세요.
        <br /> <br />
        10 · 11번은 <strong>일부러 없는 글을 요청</strong>합니다. 콘솔에 빨간{" "}
        <code>Failed to load resource ... 404</code> 줄이 나오는 것이 정상입니다.
        </p> <Problem01 /> <Problem02 /> <Problem03 /> <Problem04 /> <Problem05
        /> <Problem07 /> <Problem08 /> <Problem09 /> <Problem10 /> <Problem11 /> <Problem12
        /> <Problem13 /> <Problem14 />

      </div>
    );
  }
}
```

자주 넘어지는 자리

- 1 **StrictMode** 때문에 개발 중에는 `useEffect` 가 두 번 돕니다. 정상입니다 | 안 알려 주면 온 반이 헤맵니다
- 2 `fetch` 는 **404**를 실패로 치지 않습니다. `if (!res.ok) throw` 가 있어야 실패가 됩니다 |
- 3 의존성 배열을 빠뜨린 **무한 루프** (자료에 상한을 걸어 두어 안전합니다) |

자주 나오는 질문

Q "`async` 를 `useEffect` 에 바로 붙이면 안 돼요?"

A 안 됩니다. `useEffect` 는 정리 함수를 돌려받기를 기대하는데 `async` 는 Promise 를 돌려줍니다

useRef와 커스텀 훅

OPEN 10_useRef와_커스텀_훅

```
function DirectDom() {  
  function handleDirect() {  
    el.value = "라떼";  
  }  
  return (  

```

01 useRef 로 요소 다루기

02 useRef 로 값 기억하기

03 커스텀 훅

04 훅의 규칙

05 memo 와 useMemo

- 연습문제

useRef 로 요소 다루기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

JS자료 10단원에서 이렇게 배웠습니다.

```
const input = document.querySelector("#nameInput");
input.focus();
```

화면의 요소를 고르고, 그 요소에 무언가를 시켰습니다.

React 를 쓰는 지금도 “입력칸에 커서를 놓아 주세요” 같은 일은 필요합니다. 그런데 React 에서는 `querySelector` 를 쓰지 않습니다. 대신 `ref` 를 씁니다.

1 왜 `querySelector` 를 쓰면 안 되는지 (섹션 1)

2 그럼 무엇을 쓰는지 (섹션 2부터)

```
import { useState, useRef, useEffect } from "react";
import Summary from "../_ui/Summary.jsx";
```

화면은 React 가 관리합니다

섹션 1

01단원에서 이런 말을 했습니다. “여러분은 데이터만 바꾸고, 화면을 고치는 일은 React 가 한다”

이 말에는 뒷면이 있습니다. 화면을 고치는 일이 React 것이라면, 그 화면을 내가 직접 고치면 어떻게 될까요?

React 는 “지금 화면이 이렇게 생겼을 것이다” 를 자기 기억 속에 들고 있습니다. 그 기억과 실제 화면이 같다고 믿고, 다음에 바뀐 부분만 골라 고칩니다. 그래서 내가 몰래 화면을 고치면 React 의 기억과 실제 화면이 어긋납니다.

말로만 하면 안 믿기니 직접 해 봅니다. 아래 데모에서 [입력칸을 직접 바꾸기] 를 누르면 `querySelector` 로 입력칸을 찾아 `value` 에 “라떼” 를 직접 넣습니다. JS자료에서 하던 그대로입니다.

```
function DirectDom() {
  const [name, setName] = useState("아메리카노");
  const [tick, setTick] = useState(0);

  function handleDirect() {
```

JS자료 10단원에서 하던 방식 그대로입니다.

```
const el = document.querySelector("#nameInput");
el.value = "라떼";

console.log(`입력칸에 직접 넣은 값: ${el.value}`);
```

F12 콘솔 입력칸에 직접 넣은 값: 라떼

```
console.log(`React 가 알고 있는 값: ${name}`);
```

F12 콘솔 React 가 알고 있는 값: 아메리카노

두 줄이 다릅니다. 화면에는 “라떼” 가 보이는데 React 는 아직 “아메리카노” 인 줄 알고 있습니다. 이미 어긋났습니다.

```
}

return (
  <div className="demo"> <h3>① 화면을 직접 고치면
</h3> <input id="nameInput" value={name} onChange={(e) => setName(e.target.value)}
/>
  <button onClick={handleDirect}>입력칸을 직접 바꾸기</button> <button onClick=
{() => setTick(tick + 1)}>다른 값 바꾸기</button> <div className="output">
  React 가 아는 값: {name} / 다른 값: {tick}
</div> </div>
);
}
```

데모 ① 을 이 순서로 눌러 보세요.

1) [입력칸을 직접 바꾸기] → 입력칸 글자가 “라떼” 로 바뀝니다.

아래 “React 가 아는 값” 은 아메리카노 그대로입니다.

2) [다른 값 바꾸기] → 입력칸이 “아메리카노” 로 되돌아갑니다.

화면(누르면): 입력칸 “라떼” → 다른 값 바꾸기 → 입력칸 “아메리카노”

2)번에서 우리는 입력칸을 건드리지도 않았습니. tick 만 1 올렸습니다. 그런데 입력칸이 제멋대로 되돌아갔습니다.

이유는 이렇습니다. tick 이 바뀌었으니 React 가 이 컴포넌트를 다시 그립니다. 다시 그리면서 “입력칸의 value 는 name, 즉 아메리카노여야 한다” 를 확인합니다. 실제 화면에는 “라떼” 가 들어 있으니 다르다고 보고 아메리카노로 되돌립니다.

내가 직접 고친 것은 이렇게 조용히 지워집니다. 에러도 경고도 안 납니다. 이게 가장 나쁜 종류의 버그입니다.

정리하면 이렇습니다. React 가 그린 화면의 '내용' 은 React 가 관리합니다. 내용을 바꾸고 싶으면 state 를 바꿉니다. 화면을 직접 만지지 않습니다.

그런데 화면에는 '내용' 말고도 다룰 것이 있습니다. - 이 입력칸에 커서를 놓아라 (포커스) - 이 줄이 보이게 스크롤해라 - 이 동영상을 재생해라

이런 것은 화면에 그려지는 글자가 아닙니다. 브라우저에게 시키는 '동작'입니다. JSX 로는 적을 수가 없습니다. 그래서 요소를 직접 잡아야 합니다. 그 통로가 ref 입니다.

▹ 직접 해보기

1

데모 ① 에서 [입력칸을 직접 바꾸기] 를 누른 뒤, 입력칸에 아무 글자나 타이핑해 보세요. "라떼" 가 어떻게 되는지 보고, 왜 그런지 생각해 보세요.

useRef — React 에게 요소를 건네받기

섹션 2

쓰는 법은 세 걸음입니다.

- 1 상자를 만든다 `const inputRef = useRef(null);`
- 2 요소에 ref 로 붙인다 `<input ref={inputRef} />`
- 3 `inputRef.current` 로 쓴다 `inputRef.current.focus();`

`useRef(null)` 의 `null` 은 '처음 값' 입니다. 아직 요소가 없으니 `null` 로 시작합니다.

그러면 React 가 화면을 그린 다음, 만들어진 실제 요소를 그 상자에 넣어 줍니다. 상자 이름은 항상 `current` 입니다. 이 이름은 바꿀 수 없습니다.

`querySelector` 와 비교하면 이렇습니다.

| | |
|--|-----------------------|
| <code>querySelector("#nameInput")</code> | 내가 페이지 전체를 뒤져서 찾는다 |
| <code>ref</code> | React 가 만들면서 나에게 건네준다 |

찾을 필요가 없으니 id 를 붙이지 않아도 되고, id 가 겹칠 걱정도 없습니다. 화면에 없는 요소를 잘못 잡는 일도 없습니다.

```
function RefBasic() {  
  const boxRef = useRef(null);
```

그리는 도중에는 아직 요소가 만들어지기 전입니다.

```
console.log(`그리는 중 boxRef.current: ${String(boxRef.current)}`);
```

```
F12 콘솔    그리는 중 boxRef.current: null
```

```
useEffect(() => {
```


useEffect 는 09단원에서 배웠습니다. '화면을 다 그린 뒤' 에 실행됩니다. 그래서 여기서는 요소가 들어 있습니다.

```
console.log(`그린 뒤 boxRef.current 의 태그: ${boxRef.current.tagName}`);
```

```
F12 콘솔    그린 뒤 boxRef.current 의 태그: INPUT
```

```
console.log(`그린 뒤 boxRef.current 의 값: ${boxRef.current.value}`);
```

```
F12 콘솔    그린 뒤 boxRef.current 의 값: 라떼
```

```
    }, []);

    return (
      <div className="demo"> <h3>② ref 상자 안을 콘솔에서 확인</h3> <input ref=
        {boxRef} defaultValue="라떼" /> <p>F12 → Console 을 열어 위 두 줄을 확인하세요.
      </p> </div>
    );
  }
}
```

위 콘솔 결과가 알려 주는 것이 하나 더 있습니다.

그리는 중 → null 그린 뒤 → INPUT

ref 상자는 화면을 다 그린 뒤에 채워집니다. 그래서 컴포넌트 본문(그리는 중)에서 ref.current 를 쓰면 null 입니다. ref 를 쓰는 곳은 두 군데뿐이라고 외워도 됩니다. - 이벤트 핸들러 안 (버튼을 누른 뒤니까 이미 다 그려졌습니다) - useEffect 안 (화면을 다 그린 뒤에 실행되니까요)

참고로 defaultValue 는 "처음에만 이 값으로 두고, 그 뒤로는 브라우저가 알아서" 라는 뜻입니다. value 와 달리 React 가 계속 관리하지 않습니다. 06단원에서 배운 value 와는 다릅니다.

▫ 직접 해보기 2

RefBasic 의 useEffect 안에 아래 줄을 추가하고 콘솔에 무엇이 찍히는지 보세요.

```
console.log(boxRef.current.id);
```

입력칸에 커서 놓기 (포커스)

섹션 3

ref 를 가장 많이 쓰는 곳입니다.

왜 이걸 state 로 못 할까요? "포커스가 여기 있다" 는 화면에 그려지는 글자가 아니기 때문입니다. <input focus> 같은 JSX 는 없습니다. 브라우저에게 "이 요소에 커서를 놓아라" 라고 시켜야 합니다. 시키려면 그 요소가 손에 있어야 하고, 그래서 ref 가 필요합니다.

```
function FocusDemo() {
  const searchRef = useRef(null);
  const [keyword, setKeyword] = useState("");

  function handleFocus() {
    searchRef.current.focus();
    console.log("검색칸에 커서를 놓았습니다");
  }
}
```

F12 콘솔 검색칸에 커서를 놓았습니다

```
function handleSelect() {
```

`select()` 는 입력칸의 글자를 통째로 선택합니다. 바로 덮어쓰기 좋습니다.

```
searchRef.current.focus();
searchRef.current.select();
console.log("검색칸의 글자를 전부 선택했습니다");
```

F12 콘솔 검색칸의 글자를 전부 선택했습니다

```
}

function handleClear() {
```

지우고 나서 커서를 다시 놓아 줍니다. 실제 검색창이 이렇게 동작합니다.

```
setKeyword("");
searchRef.current.focus();
console.log("검색칸을 비우고 커서를 다시 놓았습니다");
```

F12 콘솔 검색칸을 비우고 커서를 다시 놓았습니다

```
}

return (
  <div className="demo"> <h3>③ 포커스 주기</h3> <input ref={searchRef} value=
{keyword} onChange={(e) => setKeyword(e.target.value)}
  placeholder="메뉴 검색"
  />
  <div style={{ marginTop: 8 }}> <button onClick={handleFocus}>커서 놓기
</button> <button onClick={handleSelect}>글자 전체 선택</button> <button onClick=
{handleClear}>지우고 커서 놓기</button> </div> <div className="output">검색어:
{keyword} || "(비어 있음)"</div> </div>
```

```
);
}
```

화면(누르면): [커서 놓기] 를 누르면 검색칸 테두리가 진해지고 커서가 깜빡입니다. 화면(누르면): 검색칸에 "라떼" 를 치고 [글자 전체 선택] 을 누르면 라떼가 파랗게 덮입니다. 화면(누르면): [지우고 커서 놓기] 를 누르면 검색칸이 비고 커서가 그 자리에 놓입니다.

세 번째가 실전에서 자주 쓰는 모양입니다. 값을 비우는 일(setKeyword)은 state 로, 커서를 놓는 일(focus)은 ref 로 합니다. 한 함수 안에서 둘을 같이 쓰는 것이 자연스럽습니다.

▸ 직접 해보기 3

버튼을 하나 더 만들어, 누르면 검색칸에서 커서를 빼앗도록 해 보세요. (힌트: focus 의 반대는 blur 입니다)

화면이 뜨자마자 커서 놓기

섹션 4

로그인 화면을 열면 아이디 칸에 이미 커서가 놓여 있는 것을 본 적 있을 겁니다. 09단원의 useEffect 와 ref 를 같이 쓰면 됩니다.

```
useEffect(() => { ... }, []) → 화면을 처음 그린 뒤에 한 번
```

그 안에서 ref.current.focus() → 그때는 요소가 이미 만들어져 있습니다

순서를 다시 확인하세요. 컴포넌트 함수 실행 → 화면에 그림 → ref 상자 채움 → useEffect 실행
useEffect 가 마지막이라서 ref 를 안전하게 쓸 수 있습니다.

```
function AutoFocusForm() {
  const idRef = useRef(null);
  const [id, setId] = useState("");
  const [saved, setSaved] = useState("");

  useEffect(() => {
    idRef.current.focus();
    console.log("화면이 뜨자마자 아이디 칸에 커서를 놓았습니다");
  });
}
```

F12 콘솔 화면이 뜨자마자 아이디 칸에 커서를 놓았습니다

```
}, []);

function handleSubmit(e) {
  e.preventDefault(); // 06단원에서 배웠습니다. 새로고침을 막습니다. setSaved(id);
  setId("");
  idRef.current.focus(); // 저장하고 다시 커서를 놓아 줍니다
}
```

useRef 로 값 기억하기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념01에서 `useRef` 를 '요소를 담는 통로' 로 썼습니다. 사실 `useRef` 는 요소 전용 도구가 아닙니다.

`useRef` 가 하는 일은 딱 하나입니다. "다시 그려도 안 없어지는 상자를 하나 준다"

그 상자에 요소를 넣으면 개념01이 되고, 숫자나 타이머 id 를 넣으면 이 파일이 됩니다.

그리고 아주 중요한 성질이 하나 있습니다. 상자 안의 값을 바꿔도 React 는 화면을 다시 그리지 않습니다.

state 와 정반대입니다. 이 파일은 그 차이를 눈으로 확인하는 파일입니다.

```
import { useState, useRef, useEffect } from "react";
import Summary from "../_ui/Summary.jsx";
```

useRef 가 주는 것은 '상자' 입니다

섹션 1

`useRef(0)` 을 부르면 React 는 이런 객체를 하나 만들어 돌려줍니다.

```
{ current: 0 }
```

안에 든 값을 꺼낼 때도, 넣을 때도 `current` 를 씁니다.

```
boxRef.current      꺼내기
boxRef.current = 5   넣기
```

`current` 라는 이름은 정해져 있습니다. 바꿀 수 없습니다.

이 상자에는 특별한 점이 하나 있습니다. 컴포넌트가 몇 번을 다시 그려져도 React 가 '같은 상자' 를 계속 돌려줍니다. 그래서 값이 사라지지 않습니다.

```
function RefShape() {
```

이 상자는 이 파일에서 한 번도 바꾸지 않습니다. 모양만 보는 용도입니다.

```
const boxRef = useRef(0);

console.log(boxRef);
```

```
F12 콘솔    { current: 0 }
```

```
console.log(typeof boxRef);
```

```
F12 콘솔    object
```

```
return (
  <div className="demo"> <h3>① useRef 가 돌려주는 것</h3> <p>
    useRef(0) 의 결과: <code>{"{ current: 0 }"}</code> </p> <p>F12 → Console 에서
    실제 모양을 확인하세요.</p> </div>
);
}
```

useState 와 모양부터 다릅니다.

```
const [count, setCount] = useState(0);  ← 값과 바꾸는 함수, 두 개를 받습니다
const countRef = useRef(0);             ← 상자 하나를 받습니다
```

useRef 에는 바꾸는 함수가 없습니다. 상자 안에 직접 넣습니다. 그래서 React 는 값이 바뀐 줄도 모릅니다.

▸ 직접 해보기

1

RefShape 안에서 useRef(0) 을 useRef("아메리카노") 로 바꾸고 콘솔에 무엇이 찍히는지 확인하세요.

나란히 놓고 비교하기 (이 파일의 핵심)

섹션 2

아래 데모에는 값이 두 개 있습니다. 하는 일은 똑같이 '1 올리기' 입니다.

```
refCount    useRef 로 만든 상자
stateCount  useState 로 만든 state
```

두 값을 화면에도 나란히 보여 줍니다.

ref 올리기 · 를 눌러 보고, 그 다음 [state 올리기] 를 눌러 보세요.

```
function RefVsState() {
  const refCount = useRef(0);
  const [stateCount, setStateCount] = useState(0);
```

이 줄은 컴포넌트가 다시 그려질 때마다 실행됩니다. 즉 이 줄이 콘솔에 찍혔다면 '화면을 다시 그렸다' 는 뜻입니다.

```
console.log("[다시 그림] RefVsState 를 다시 그렸습니다");
```

```
F12 콘솔    [다시 그림] RefVsState 를 다시 그렸습니다
```

```
function handleRefUp() {
```

set 함수가 없습니다. 상자에 직접 넣습니다.

```
    refCount.current = refCount.current + 1;
```

```
    console.log(`ref 를 올렸습니다. 진짜 값은 ${refCount.current} 입니다`);
```

F12 콘솔 ref 를 올렸습니다. 진짜 값은 1 입니다

(두 번째로 누르면 2, 세 번째로 누르면 3 이 찍힙니다)

```
}
```

```
function handleStateUp() {
```

```
    setStateCount(stateCount + 1);
```

```
    console.log("state 를 올렸습니다. React 가 화면을 다시 그립니다");
```

F12 콘솔 state 를 올렸습니다. React 가 화면을 다시 그립니다

```
}
```

```
return (
```

```
    <div className="demo"> <h3>② ref 와 state 를 나란히
</h3> <div className="output"> <div>화면에 보이는 ref 값: {refCount.current}
</div> <div>화면에 보이는 state 값: {stateCount}</div> </div> <div style=
{{ marginTop: 8 }}> <button onClick={handleRefUp}>ref 올리기
</button> <button onClick={handleStateUp}>state 올리기</button> </div> </div>
    );
}
```

이 순서대로 눌러 보세요. 콘솔도 같이 보셔야 합니다.

1) [ref 올리기] 를 세 번 누른다

화면(누르면): "화면에 보이는 ref 값: 0" 그대로입니다. 안 바뀝니다.
콘솔에는 진짜 값이 1, 2, 3 으로 올라간 것이 찍힙니다.

다시 그림 · 은 한 번도 안 찍힙니다.

2) [state 올리기] 를 한 번 누른다

화면(누르면): state 값이 1 이 되고,
그와 동시에 ref 값이 갑자기 3 으로 튀어나옵니다.
콘솔에 [다시 그림] 이 찍힙니다.

2)번이 이 파일에서 가장 중요한 장면입니다.

ref 값은 사실 1)번에서 이미 3 이었습니다. 상자 안에서 잘 올라가고 있었습니다. 다만 React 가 화면을 다시 그리지 않아서 화면이 옛날 값을 계속 보여 준 것입니다. state 를 올리는 순간 React 가 화면을 다시 그렸고, 그때 비로소 3 이 나왔습니다.

여기서 두 가지를 얻습니다.

① ref 를 바꾸는 것은 React 에게 아무 신호도 주지 않습니다.

그래서 화면에 보여야 하는 값을 ref 에 두면 화면이 안 바뀝니다.
에러도 안 납니다. 조용히 틀립니다.

② 반대로, 화면에 안 보여도 되는 값을 state 로 두면

바꿀 때마다 쓸데없이 화면을 다시 그립니다.

그래서 판단 기준은 하나입니다. 이 값이 바뀌면 화면도 바뀌어야 하나? → 예: state / 아니오: ref

개발 중에는 [다시 그림] 이 두 번씩 찍힙니다. 이것은 정상입니다. main.jsx 가 StrictMode 를 켜 두었기 때문입니다. 실수를 미리 찾아 주려고 개발 중에만 컴포넌트를 두 번 실행합니다. 09단원에서 useEffect 가 두 번 도는 것과 같은 이유입니다.

▸ 직접 해보기 2

[ref 올리기] 를 다섯 번 누른 뒤 [state 올리기] 를 한 번 누르면 화면의 ref 값이 얼마가 될지 먼저 예상하고, 확인해 보세요.

그냥 변수로는 왜 안 되나

섹션 3

"다시 안 그려도 되는 값이면 그냥 let 변수를 쓰면 되지 않나?" 라고 생각할 수 있습니다. 04단원 개념02에서 이미 한 번 본 이야기입니다. 다시 확인해 봅니다.

컴포넌트는 '함수' 입니다. 다시 그린다는 것은 그 함수를 처음부터 다시 실행한다는 뜻입니다. 함수 안의 let 변수는 함수가 다시 실행되면 처음 값으로 되돌아갑니다.

```
function LocalVarDemo() {
```

다시 그릴 때마다 이 줄이 다시 실행됩니다. 그래서 항상 0 부터 시작합니다.

```
let localCount = 0;

const [tick, setTick] = useState(0);

function handleLocalUp() {
  localCount = localCount + 1;
  console.log(`지역 변수는 지금 ${localCount} 입니다`);
}
```

F12 콘솔 지역 변수는 지금 1 입니다

(다시 그리기 전까지는 2, 3 으로 올라갑니다)

```
}

return (
  <div className="demo"> <h3>③ 그냥 변수는 다시 그리면 사라집니다
</h3> <div className="output"> <div>화면에 보이는 지역 변수: {localCount}</div> <div>
다시 그린 횟수 세는 state: {tick}</div> </div> <div style={{ marginTop: 8
}}> <button onClick={handleLocalUp}>지역 변수 올리기</button> <button onClick={() =>
setTick(tick + 1)}>다시 그리기</button> </div> </div>
);
}
```

1 [지역 변수 올리기] 세 번 → 콘솔에 1, 2, 3

2 [다시 그리기] 한 번

3 [지역 변수 올리기] 한 번 → 콘솔에 다시 1

3)번에서 값이 1로 돌아갔습니다. 쌓아 둔 3이 사라진 것입니다.

useRef는 바로 이 문제를 푼다. 지역 변수 다시 그리면 사라진다

| | |
|-------|---|
| state | 안 사라지지만, 바꿀 때마다 화면을 다시 그린다 |
| ref | 안 사라지고, 바뀌어도 화면을 안 그린다 ← 이 자리가 비어 있었습니다 |

▹ 직접 해보기 3

LocalVarDemo의 `let localCount = 0`을 `const localCount = useRef(0)`로 바꾸고,
`handleLocalUp`안을 `localCount.current = localCount.current + 1`로 고쳐 보세요. 그 다음 1) 2)
 3)을 다시 해 보세요.

실전 — 타이머 id 보관하기

섹션 4

여기서부터가 useRef를 실제로 쓰는 자리입니다.

JS자료 12단원에서 setTimeout 과 setInterval 을 배웠습니다.

```
const id = setInterval(함수, 1000); // 시작하면 번호(id)를 돌려줍니다
clearInterval(id); // 그 번호로 멈춥니다
```

멈추려면 시작할 때 받은 번호를 어딘가에 적어 두어야 합니다. 그 '어딘가' 가 어디여야 할까요? 셋 다 해 봅니다.

지역 변수 → 다시 그리면 사라집니다. 멈출 수가 없습니다. ← 아래에서 확인 state → 되긴 됩니다. 그런데 id 는 화면에 안 보이는 값입니다.

state 로 두면 시작할 때마다 쓸데없이 화면을 다시 그립니다.

ref → 안 사라지고, 화면도 안 건드립니다. 정답입니다.

```
function BrokenTimer() {
  const [sec, setSec] = useState(0);
  const [running, setRunning] = useState(false);
```

여기가 문제의 줄입니다. 다시 그릴 때마다 null 로 돌아갑니다.

```
let timerId = null;

function handleStart() {
  if (running) return;

  const id = setInterval(() => {
    setSec((prev) => prev + 1);
  }, 250);

  timerId = id;
  setRunning(true);

  console.log("고장난 타이머를 시작했습니다");
```

F12 콘솔 고장난 타이머를 시작했습니다

이 자료가 멈추지 않는 타이머를 남기면 곤란하니 2초 뒤에는 스스로 멈추도록 안전장치를 넣어 뒀습니다. 원래 코드에는 없는 줄입니다.

```

setTimeout(() => {
    clearInterval(id);
    setRunning(false);
}, 2000);
}

function handleStop() {
    console.log(`멈추려 했지만 기억해 둔 id 는 ${String(timerId)} 입니다`);
}

```

F12 콘솔 멈추려 했지만 기억해 둔 id 는 null 입니다

```

clearInterval(timerId); // null 을 넘기므로 아무 일도 안 일어납니다
}

return (
    <div className="demo"> <h3>④-1 고장난 타이머 (지역 변수에 id 를 뒀습니다)
</h3> <div className="output">
    {sec} 칸 {running ? "(도는 중)" : "(멈춤)"}
    </div> <div style={{ marginTop: 8 }}> <button onClick={handleStart}>시작
</button> <button onClick={handleStop}>멈춤</button> </div> </div>
);
}

```

시작 · 을 누르면 숫자가 올라갑니다. 그때 [멈춤] 을 눌러 보세요.

화면(누르면): 숫자가 계속 올라갑니다. 안 멈춥니다.

왜 그럴까요? 순서를 따라가 봅시다.

- 1 [시작] 을 누른다 → timerId 에 번호가 들어간다
- 2 setRunning(true) 때문에 화면을 다시 그린다
- 3 다시 그리면서 let timerId = null 이 또 실행된다 ← 번호가 지워졌습니다
- 4 [멈춤] 을 누르면 null 로 멈추려 한다 → 아무 일도 안 일어난다

콘솔에 “기억해 둔 id 는 null 입니다” 가 찍히는 것으로 확인할 수 있습니다.

(위 데모는 2초 뒤 스스로 멈춥니다. 자료가 안전하도록 넣어 둔 장치이고, 원래 잘못된 코드에는 그런 장치가 없어서 영영 안 멈춥니다.)

```

function RefTimer() {
    const [sec, setSec] = useState(0);
    const [running, setRunning] = useState(false);
}

```

상자에 넣어 둡니다. 다시 그려도 이 상자는 그대로입니다.

```
const timerRef = useRef(null);

function handleStart() {
  if (timerRef.current !== null) return; // 이미 돌고 있으면 무시

  timerRef.current = setInterval(() => {
    setSec((prev) => prev + 1);
  }, 250);
  setRunning(true);

  console.log("타이머를 시작했습니다");
}
```

F12 콘솔 타이머를 시작했습니다

```
}

function handleStop() {
  clearInterval(timerRef.current);
  timerRef.current = null;
  setRunning(false);

  console.log("타이머를 멈췄습니다");
}
```

F12 콘솔 타이머를 멈췄습니다

```
}

function handleReset() {
  setSec(0);
  console.log("0 으로 되돌렸습니다");
}
```

F12 콘솔 0 으로 되돌렸습니다

```
}
```

09단원 개념02의 정리 함수입니다. 이 예제를 떠나면(다른 메뉴를 고르면) 돌던 타이머를 치웁니다.

```
useEffect(() => {
  return () => {
    clearInterval(timerRef.current);
  };
}, []);

return (
  <div className="demo"> <h3>④-2 제대로 도는 타이머 (ref 에 id 를 뒀습니다)
</h3> <div className="output">
```

```

    {sec} 칸 {running ? "(도는 중)" : "(멈춤)"}
  </div> <div style={{ marginTop: 8 }}> <button onClick={handleStart}>시작
</button> <button onClick={handleStop}>멈춤</button> <button onClick={handleReset}>0
으로</button> </div> </div>
  );
}

```

화면(누르면): [시작] 을 누르면 0.25초마다 한 칸씩 올라가고,

멈춤 을 누르면 그 자리에 섭니다.

두 컴포넌트의 차이는 딱 한 줄입니다.

```

let timerId = null;           ← 고장난 쪽
const timerRef = useRef(null); ← 제대로 도는 쪽

```

여기서 sec 는 state 인 것에 주의하세요. 화면에 보여야 하는 값이니깐요. 반대로 timerRef 는 화면에 한 번도 안 나옵니다. 그래서 ref 입니다. 한 컴포넌트 안에서 state 와 ref 를 이렇게 나눠 씁니다.

▹ 직접 해보기 4

RefTimer 의 handleStop 에서 timerRef.current = null; 줄만 지워 보세요. [시작] [멈춤] [시작] 순서로 눌렀을 때 무슨 일이 생기는지 보세요.

실전 — 직전 값 기억하기

섹션 5

“방금 전에는 무엇이었지?” 를 알아야 할 때가 있습니다. 화면에 꼭 보여야 하는 값은 아니니 ref 가 알맞습니다.

방법은 이렇습니다. 화면을 다 그린 뒤(useEffect)에 지금 값을 상자에 넣어 둡니다. 그러면 다음번에 그릴 때 상자 안에는 ‘직전 값’ 이 들어 있습니다.

```

function PreviousValue() {
  const [menu, setMenu] = useState("아메리카노");
  const prevMenuRef = useRef("(없음)");

  useEffect(() => {

```

그리기가 끝난 뒤에 실행되므로, 화면에는 아직 옛날 값이 쓰였습니다.

```

    prevMenuRef.current = menu;
  }, [menu]);

  function pick(name) {
    setMenu(name);
    console.log(`고른 메뉴: ${name}`);
  }

```

F12 콘솔

```

    고른 메뉴: 라떼
    고른 메뉴: 케이크
    고른 메뉴: 삼각김밥
  
```

```

  }

  return (
    <div className="demo"> <h3>⑤ 직전 값 기억하기
    </h3> <div className="output"> <div>지금 고른 메뉴: {menu}</div> <div>직전에 고른
    메뉴: {prevMenuRef.current}</div> </div> <div style={{ marginTop: 8
    }}> <button onClick={() => pick("라떼")}>라떼</button> <button onClick={() =>
    pick("케이크")}>케이크</button> <button onClick={() => pick("삼각김밥")}>삼각김밥
    </button> </div> </div>
  );
}

```

화면(누르면): [라떼] → 지금 라떼 / 직전 아메리카노 화면(누르면): 이어서 [케이크] → 지금 케이크 / 직전 라떼

prevMenuRef 를 state 로 만들면 어떻게 될까요? setPrevMenu 를 부르면 그 순간 렌더가 한 번 더 일어납니다. 하지만 이 useEffect 의 의존성은 [menu] 뿐이고 setPrevMenu 는 menu 를 바꾸지 않으므로, 그 렌더에서 effect 가 다시 돌지는 않습니다. 09단원 개념06의 무한 루프는 effect 안에서 바꾼 state 가 그 effect 자신의 의존성에도 들어 있어서 생긴 것이라 여기와는 다릅니다. 렌더가 딱 한 번 더 늘어날 뿐, 무한 루프는 아닙니다. 그래도 ref 가 더 알맞습니다. 직전 값은 menu 가 바뀔 때 이미 일어나는 렌더에 실려서 화면에 나오므로, 굳이 렌더를 하나 더 만들 필요가 없습니다.

▣ 직접 해보기

5

같은 버튼을 두 번 연달아 누르면 “직전에 고른 메뉴” 가 어떻게 되는지 확인하고 이유를 생각해 보세요.

state 와 ref, 무엇으로 할까

섹션 6

표로 정리하면 이렇습니다.

| 묻는 것 | useState | useRef | | 만들기 | `const [x, setX] = ...` | `const xRef = useRef(...)` | | 읽기 | `x` | `xRef.current` | | 바꾸기 | `setX(새 값)` | `xRef.current = 새 값` | | 바꾸면 다시 그리나 | 예 | 아니오 | | 바로 읽으면 | 옛날 값 (04단원) | 방금 넣은 값 | | 어디에 쓰나 | 화면에 보이는 값 | 화면과 상관없는 값 |

표에서 '바로 읽으면' 줄을 눈여겨보세요. 04단원 개념05에서 배운 것입니다.

```
setCount(count + 1);
console.log(count);          // 아직 옛날 값입니다

countRef.current = countRef.current + 1;
console.log(countRef.current); // 방금 넣은 값이 그대로 나옵니다
```

ref 는 그냥 객체의 속성을 바꾸는 것이라 즉시 반영됩니다. state 는 React 에게 부탁하는 것이라 다음 렌더에 반영됩니다.

ref 를 쓸 자리는 생각보다 적습니다. 아래 정도입니다. - 요소 다루기 (개념01) - 타이머 id, 구독 해제 함수 같은 '치울 것' 보관 - 직전 값 기억 - 화면과 무관한 누적값 (예: 몇 번 눌렀는지 몰래 세기)

헛갈리면 state 로 시작하세요. 화면에 안 나오는데 자꾸 다시 그러서 곤란할 때 ref 로 옮기면 됩니다.

직접 해보기

6

아래 값들은 state 일까요 ref 일까요? 하나씩 답해 보세요. (가) 장바구니에 담긴 개수 (나) 입력칸 요소 자체 (다) 사용자가 이 화면에 들어온 시각 (라) 검색 결과 목록

자주 하는 실수

섹션 7

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

화면에 보여야 할 값을 ref 에 둬

```
const countRef = useRef(0);
<button onClick={() => (countRef.current += 1)}>담기</button>
<p>{countRef.current} 개</p>
```

에러가 안 납니다. 그런데 눌러도 화면 숫자가 안 바뀝니다. 섹션 2에서 본 그 장면입니다. 이 단원에서 가장 많이 나는 실수입니다. 화면에 나오는 값이면 state 로 바꾸세요.

이런 실수를 합니다

.current 를 빼먹음

```
countRef = countRef + 1;
```

TypeError: Assignment to constant variable. const

로 만든 상자 자체를 바꾸려 했습니다. 바꿀 것은 상자 안(current)입니다.

이런 실수를 합니다

그리는 도중에 ref 를 바꿈

```
function Bad() {
```

```
  const countRef = useRef(0);
  countRef.current = countRef.current + 1; // ← 컴포넌트 본문
  return <p>{countRef.current}</p>;
```

```
}
```

에러는 안 납니다. 그런데 값이 예상과 다릅니다. 개발 중에는 StrictMode 때문에 컴포넌트가 두 번 실행되므로 2씩 올라갑니다. 그리는 일은 '값을 읽어 화면을 만드는 일' 이어야 합니다. 값을 바꾸는 일은 이벤트 핸들러나 useEffect 안에서 하세요.

이런 실수를 합니다

ref 를 useEffect 의 의존성 배열에 넣음

```
useEffect(() => { ... }, [countRef.current]);
```

에러는 안 납니다. 그런데 값이 바뀌어도 useEffect 가 다시 안 돕니다. ref 를 바꿔도 다시 그리지 않으니, 의존성을 비교할 기회 자체가 없습니다. 다시 돌게 하고 싶으면 그 값은 state 여야 합니다.

이런 실수를 합니다

상자 안에 넣지 않고 상자를 통째로 씀

```
<p>{countRef}</p>
```

Objects are not valid as a React child (found: object with keys {current}) 상자는 객체입니다. 화면에 그릴 수 없습니다. countRef.current 로 써야 합니다.

이런 실수를 합니다

화살표 함수의 중괄호를 빼먹음 — 눈으로만

```
<button onClick={() => { countRef.current += 1 }}>답기</button>
```

[SyntaxError] 중괄호를 안 닫았습니다. 파일 전체가 멈춥니다. JSX 안에서는 괄호가 많아 자주 나는 실수입니다.

정리

- 1 useRef 는 { current: 값 } 모양의 상자를 줍니다. 다시 그려도 같은 상자가 유지됩니다.
- 2 상자 안의 값을 바꿔도 화면을 다시 그리지 않습니다. state 와 가장 큰 차이입니다.
- 3 그래서 화면에 보여야 하는 값을 ref 에 두면 화면이 안 바뀝니다. 에러 없이 조용히 틀립니다.
- 4 지역 변수는 다시 그리면 사라지고, state 는 바꿀 때마다 다시 그립니다. ref 는 그 사이를 메웁니다.
- 5 타이머 id 처럼 화면에 안 나오지만 기억해야 하는 값이 ref 의 대표 용도입니다.
- 6 판단 기준: 이 값이 바뀌면 화면도 바뀌어야 하나? 예면 state, 아니오면 ref.

```
export default function Concept02ValueRef() {
  return (
    <div> <h1>개념 02 - useRef 로 값 기억하기</h1> <p className="guide">
      이 파일은 <strong>콘솔을 함께 봐야</strong> 뜻이 통합니다. F12 → Console 을
      열어 두세요.
      <br />
      데모 ② 는 <strong>[ref 올리기] 를 세 번 누른 다음 [state 올리기]</strong>
      순서로
      눌러 보세요.
      </p> <RefShape /> <RefVsState /> <LocalVarDemo /> <BrokenTimer /> <RefTimer
    /> <PreviousValue /> </div>
  );
}
```

직접 해보기 정답

```
1) const boxRef = useRef("아메리카노");
```

콘솔에 { current: '아메리카노' } 가 찍힙니다. → 상자에는 어떤 값이든 넣을 수 있습니다. 숫자, 문자열, 객체, 요소, 함수 모두 됩니다.

2) 5 가 됩니다. → [ref 올리기] 를 다섯 번 누르는 동안 화면은 0 인 채였지만

상자 안에서는 1, 2, 3, 4, 5 로 올라가고 있었습니다.

state 올리기 · 가 화면을 다시 그리는 순간 그 5 가 화면에 나타납니다.

개발 중이라면 [다시 그림] 이 두 번 찍힙니다. StrictMode 때문이고 정상입니다.

```
3) let localCount = 0; → const localCount = useRef(0);
   localCount = localCount + 1; → localCount.current = localCount.current + 1;
```


화면에 보이는 값은 {localCount.current} 로 고칩니다.

출력 [다시 그리기] 를 눌러도 값이 사라지지 않습니다.

지역 변수 올리기 · 를 세 번 → [다시 그리기] → 화면에 3 이 나옵니다.

4) 두 번째 [시작] 이 안 먹습니다. 화면(누르면): [시작] → 숫자가 올라감 → [멈춤] → 섬 → [시작] → 그대로 서 있음 → handleStop 에서 clearInterval 로 타이머는 멈췄지만 상자는 안 비웠습니다.

상자 안에는 아직 옛날 번호가 들어 있습니다.
그래서 handleStart 첫 줄의
if (timerRef.current !== null) return;
에 걸려 "이미 돌고 있다" 고 오해하고 그냥 돌아가 버립니다.
치울 때는 clearInterval 과 함께 상자도 null 로 비워야 합니다.

5) 같은 버튼을 두 번 누르면 "직전에 고른 메뉴" 가 안 바뀝니다. 화면(누르면): [라떼] [라떼] → 지금 라떼 / 직전 아메리카노 → 두 번째 클릭에서는 menu 가 "라떼" 에서 "라떼" 로 바뀌는 셈이라

React 가 값이 같다고 보고 다시 그리지 않습니다.
다시 안 그리니 useEffect 도 안 돌고, 상자도 그대로입니다.

6) (가) state — 화면에 개수가 보여야 합니다. (나) ref — 요소는 화면에 그리는 값이 아닙니다.
개념01에서 배운 그대로입니다. (다) ref — 들어온 시각을 화면에 안 보여 준다면 ref 가 알맞습니다.

화면에 "머문 시간" 을 계속 보여 준다면 그 시간은 state 입니다.

(라) state — 목록이 화면에 그려집니다.

커스텀 훅

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

지금까지 `useState`, `useEffect`, `useRef` 를 배웠습니다. 전부 React 가 준 훅입니다. 이 파일에서는 훅을 '직접 만듭니다'.

먼저 겁먹지 않도록 결론부터 적습니다.

커스텀 훅은 새로운 문법이 아닙니다. 훅을 부르는 코드를 함수 하나로 묶은 것, 그게 전부입니다.

`import` 도, 특별한 선언도 없습니다. 그냥 함수를 만들면 됩니다. JS자료 05단원에서 "같은 코드가 반복되면 함수로 묶는다" 를 배웠습니다. 그 이야기를 훅에 그대로 적용하는 것뿐입니다.

```
import { useState, useRef, useEffect } from "react";
import Summary from "../_ui/Summary.jsx";
```

같은 코드가 자주 반복됩니다

섹션 1

아래 두 컴포넌트를 보세요. 하나는 커피 잔 수, 하나는 케이크 조각 수를 셉니다. 세는 대상만 다르고 코드는 똑같습니다.

```
function CoffeeCounterOld() {
  const [count, setCount] = useState(0);

  function increase() {
    setCount((prev) => prev + 1);
  }
  function decrease() {
    setCount((prev) => prev - 1);
  }
  function reset() {
    setCount(0);
  }

  return (
    <div className="output">
      아메리카노 {count} 잔
      <div style={{ marginTop: 6 }}> <button onClick={increase}>담기
```

```

</button> <button onClick={decrease}>빼기</button> <button onClick={reset}>비우기
</button> </div> </div>
);
}

function CakeCounterOld() {

```

위와 완전히 같은 코드입니다. 이름만 케이크로 바꿨습니다.

```

const [count, setCount] = useState(0);

function increase() {
  setCount((prev) => prev + 1);
}
function decrease() {
  setCount((prev) => prev - 1);
}
function reset() {
  setCount(0);
}

return (
  <div className="output">
    케이크 {count} 조각
    <div style={{ marginTop: 6 }}> <button onClick={increase}>담기
</button> <button onClick={decrease}>빼기</button> <button onClick={reset}>비우기
</button> </div> </div>
  );
}

function SectionOne() {
  return (
    <div className="demo"> <h3>① 코드가 겹치는 두 컴포넌트</h3> <CoffeeCounterOld
/> <CakeCounterOld /> </div>
  );
}

```

화면 담기·빼기·비우기 버튼이 두 줄로 나옵니다. 둘 다 잘 동작합니다.

잘 돌아가긴 합니다. 그런데 여기에 요구사항이 하나 붙는다고 해 봅시다. “0 아래로는 내려가지 않게 해 주세요”

그러면 decrease 를 두 군데 다 고쳐야 합니다. 컴포넌트가 다섯 개면 다섯 군데입니다. 한 군데를 빠뜨리면 거기만 조용히 다르게 동작합니다.

03단원에서 “반복되는 화면은 컴포넌트로 묶는다” 를 배웠습니다. 그런데 여기서 겹치는 것은 화면이 아닙니다. 화면은 서로 다릅니다. 겹치는 것은 state 와 그 state 를 다루는 함수들, 즉 ‘동작’ 입니다.

동작을 묶는 것이 커스텀 훅입니다.

▸ 직접 해보기 1

CoffeeCounterOld 와 CakeCounterOld 에서 서로 다른 줄이 몇 줄인지 세어 보세요.

함수로 묶으면 끝입니다 — useCounter

섹션 2

겹치는 부분(state 와 세 함수)을 그대로 잘라 함수 안에 넣습니다. 그리고 컴포넌트가 필요로 하는 것들을 `return` 으로 돌려줍니다.

```
function useCounter(initial = 0) {
```

훅 안에서 훅을 부릅니다. 이게 됩니다. 커스텀 훅은 컴포넌트와 같은 취급을 받습니다.

```
  const [count, setCount] = useState(initial);

  function increase() {
    setCount((prev) => prev + 1);
  }
  function decrease() {
    setCount((prev) => prev - 1);
  }
  function reset() {
    setCount(initial);
  }
}
```

아래는 { count: count, increase: increase, ... } 를 짧게 쓴 것입니다. 키 이름과 변수 이름이 같으면 한 번만 써도 됩니다. '속성 축약' 이라고 부릅니다.

```
  return { count, increase, decrease, reset };
}
```

쓰는 쪽은 이렇게 짧아집니다.

```
function CoffeeCounterNew() {
  const coffee = useCounter(0);

  function handleIncrease() {
    coffee.increase();
    console.log("useCounter 의 increase 를 불렀습니다");
  }
}
```

F12 콘솔 useCounter 의 increase 를 불렀습니다

```
}
```

훅의 규칙

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

04단원 개념06에서 이런 문장을 한 번 봤습니다.

"훅은 컴포넌트 맨 위에서만 부릅니다."

그때는 "그렇구나" 하고 넘어갔습니다. 이제 이유를 볼 차례입니다.

이 파일의 목표는 규칙을 외우게 하는 것이 아닙니다. React 가 값을 어떻게 기억하는지 알면 규칙은 저절로 따라옵니다. 그래서 순서가 중요합니다.

- 1 규칙이 무엇인지 (섹션 1)
- 2 React 훅내를 직접 내 보며 왜 그런지 알기 (섹션 2)
- 3 진짜 React 에서 깨뜨려 보기 (섹션 3, 4)
- 4 그럼 조건부로 하고 싶을 땐 어떻게 하나 (섹션 5)

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

규칙은 두 개뿐입니다

섹션 1

규칙 1 · 훅은 컴포넌트(또는 커스텀 훅)의 맨 위에서만 부릅니다.

조건문 안, 반복문 안, 중첩 함수 안, early `return` 뒤에서 부르지 않습니다.

규칙 2 · 훅은 React 컴포넌트나 커스텀 훅 안에서만 부릅니다.

그냥 함수, 이벤트 핸들러, 일반 파일 맨 위에서는 부르지 않습니다.

여기서 '맨 위' 는 파일의 맨 위가 아니라 '함수 본문의 바깥쪽' 이라는 뜻입니다.

```
function Menu() {
```

```

const [name, setName] = useState("");    // ← 맨 위. 좋습니다
const [price, setPrice] = useState(0);    // ← 좋습니다

if (name === "") {
  const [x, setX] = useState(0);          // ← 안 됩니다. 조건문 안입니다
}

function handleClick() {
  const [y, setY] = useState(0);          // ← 안 됩니다. 중첩 함수 안입니다
}

return <div>...</div>;

}

```

왜 이런 규칙이 있을까요? 다음 섹션에서 직접 만들어 보면 알게 됩니다.

▫ 직접 해보기

1

위 예시에서 규칙을 어긴 줄이 몇 줄인지 세어 보세요.

React 는 '부른 순서' 로 기억합니다

섹션 2

궁금한 것부터 짚고 갑시다.

```

const [name, setName] = useState("김민준");
const [age, setAge] = useState(20);

```

두 줄 다 useState 입니다. 이름도 안 알려 줬습니다. 그런데 다시 그럴 때 React 는 어느 값이 name 이고 어느 값이 age 인지 어떻게 알까요?

답은 '순서' 입니다. React 는 컴포넌트마다 상자를 여러 칸 준비해 두고, useState 가 불릴 때마다 0번 칸, 1번 칸, 2번 칸 ... 순서대로 담습니다. 이름은 안 봅니다. 몇 번째로 불렸는지만 봅니다.

말로만 하면 안 와닿으니 흉내를 내 봅니다. React 없이 순수 자바스크립트입니다.

```

let slots = []; // 값을 담는 칸들
let slotIndex = 0; // 지금 몇 번 칸을 쓸 차례인지

```

React 의 useState 를 아주 단순하게 흉내 낸 것입니다.

```
function myUseState(initial) {
  const i = slotIndex;
  slotIndex = slotIndex + 1; // 다음 혹은 다음 칸을 씁니다 if (slots[i] ===
  undefined) {
    slots[i] = initial; // 처음이면 초기값을 넣습니다
  }
  return slots[i]; // 이미 있으면 초기값은 무시하고 담아 둔 값을 돌려줍니다
}
```

컴포넌트를 한 번 그리는 것을 흉내 낸 함수입니다.

```
function myRender(useCoupon) {
  slotIndex = 0; // 그릴 때마다 0번 칸부터 다시 씁니다. React 도 이렇게 합니다. const
  name = myUseState("김민준");

  let coupon = "(안 씀)";
  if (useCoupon) {
```

규칙을 어긴 자리입니다. 조건문 안에서 혹은 부릅니다.

```
    coupon = myUseState("첫 잔 무료");
  }

  const menu = myUseState("아메리카노");

  return { name, coupon, menu };
}

function SlotSimulation() {
  function runSimulation() {
```

여러 번 눌러도 같은 결과가 나오도록 칸을 비우고 시작합니다.

```
    slots = [];
    slotIndex = 0;

    const first = myRender(false); // 쿠폰을 안 쓰는 상태로 한 번
    그립니다 console.log(
      `1번째 그리기 → 이름: ${first.name} / 쿠폰: ${first.coupon} / 메뉴:
      ${first.menu}`
    );
```

F12 콘솔 1번째 그리기 → 이름: 김민준 / 쿠폰: (안 씀) / 메뉴: 아메리카노

...slots · 는 지금 상태를 사진 찍듯 복사한 것입니다.

그냥 slots 를 찍으면 콘솔이 나중에 바뀐 값을 보여 줘서 헷갈립니다.

```
console.log(...slots);
```

F12 콘솔 ['김민준', '아메리카노']

```
const second = myRender(true); // 이번엔 쿠폰을 쓰는 상태로 다시  
그립니다 console.log(  
  `2번째 그리기 → 이름: ${second.name} / 쿠폰: ${second.coupon} / 메뉴:  
  ${second.menu}`  
);
```

F12 콘솔 2번째 그리기 → 이름: 김민준 / 쿠폰: 아메리카노 / 메뉴: 아메리카노

```
console.log(...slots);
```

F12 콘솔 ['김민준', '아메리카노', '아메리카노']

```
}  
  
return (  
  <div className="demo"> <h3>① React 흉내내기 - 칸에 순서대로 담기</h3> <p>버튼을  
  누르고 콘솔을 보세요. 두 번 그리는 것을 흉내 냅니다.</p> <button onClick=  
  {runSimulation}>순서 시뮬레이션 실행</button> </div>  
);  
}
```

콘솔 결과를 한 줄씩 따라가 봅시다.

— 1번째 그리기 — 쿠폰 안 씀

myUseState("김민준") → 0번 칸이 비었으니 "김민준" 을 넣고 돌려줍니다 (쿠폰은 건너뛴니다)

myUseState("아메리카노") → 1번 칸이 비었으니 "아메리카노" 를 넣고 돌려줍니다 칸: ['김민준', '아메리카노']

— 2번째 그리기 — 쿠폰 씀

myUseState("김민준") → 0번 칸에 이미 "김민준" 이 있으니 그것을 돌려줍니다 myUseState("첫 잔 무료") → 1번 칸에 이미 "아메리카노" 가 있습니다!

초기값은 무시되고 "아메리카노" 가 돌아옵니다

myUseState("아메리카노") → 2번 칸이 비었으니 "아메리카노" 를 넣습니다

그래서 쿠폰 자리에 메뉴 이름이 들어갔습니다.

쿠폰: 아메리카노

조건문 하나 때문에 이후의 모든 혹이 한 칸씩 밀렸습니다. React 도 정확히 이렇게 동작합니다. 그래서 규칙 1이 있는 것입니다.

다시 말하면, 규칙 1은 이렇게 바꿔 읽을 수 있습니다.

“몇 번째 그리든 혹이 같은 개수, 같은 순서로 불리게 하라”

▸ 직접 해보기

2

myRender 에서 쿠폰 부분(`let coupon` 줄부터 `if` 블록 끝까지)을 menu 줄 아래로 통째로 옮기면 2번째 그리기 결과가 어떻게 나올까요? 먼저 예상해 보고, 그 다음 실제로 옮겨서 확인하세요.

진짜 React 에서 깨뜨려 보기

섹션 3

이번에는 흥내가 아니라 진짜 React 입니다.

아래 컴포넌트는 `useState` 를 두 번 부릅니다. 개수는 늘 두 개로 같습니다. 순서만 바꿉니다. 그런데도 값이 뒤바뀝니다.

`useState` 는 배열을 돌려줍니다. `[0]` 이 값, `[1]` 이 바꾸는 함수입니다. 04단원에서는 구조분해로 받았는데, 여기서는 순서를 눈에 띄게 하려고 `[0]` 으로 씁니다.

```
function SwapDemo({ swapped }) {
  let name;
  let price;

  if (swapped) {
    price = useState(4000)[0]; // 가격이 먼저 (0번 칸)
    name = useState("아메리카노")[0]; // 이름이 나중에 (1번 칸)
  } else {
    name = useState("아메리카노")[0]; // 이름이 먼저 (0번 칸)
    price = useState(4000)[0]; // 가격이 나중에 (1번 칸)
  }

  console.log(`이름 칸에 든 값: ${name} / 가격 칸에 든 값: ${price}`);
}
```

F12 콘솔 이름 칸에 든 값: 아메리카노 / 가격 칸에 든 값: 4000
이름 칸에 든 값: 4000 / 가격 칸에 든 값: 아메리카노

```
return (
  <div className="output"> <div>이름: {name}</div> <div>가격: {price}</div> </div>
);
}

function SwapSection() {
  const [swapped, setSwapped] = useState(false);
}
```

memo 와 useMemo

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

이 파일에서 배울 도구는 셋입니다.

| | |
|-------------|---------------------------------|
| useMemo | 무거운 계산 결과를 기억해 두고 다시 안 합니다 |
| memo | props 가 그대로면 자식 컴포넌트를 다시 안 그립니다 |
| useCallback | 함수를 매번 새로 만들지 않고 그대로 씁니다 |

그런데 이 파일에서 가장 중요한 문장은 도구 설명이 아닙니다.

★ 먼저 쓰지 마세요. ★

이 문장을 먼저 읽고 아래로 내려가세요. 섹션 1이 그 이야기입니다.

```
import { useState, useMemo, useCallback, memo } from "react";
import Summary from "../_ui/Summary.jsx";
```

이 파일에서 계속 쓸 장바구니입니다.

```
const cart = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "라떼", price: 4500, count: 1 },
  { name: "케이크", price: 6000, count: 1 },
  { name: "삼각김밥", price: 1200, count: 3 },
];
```

일부러 느리게 만든 합계 계산입니다. 같은 계산을 repeat 번 반복해서 시간을 끕니다. 실제 코드에는 이런 함수가 없습니다. 진짜 느린 계산(큰 목록 정렬, 수만 줄 걸러내기)을 훔내 내려고 만든 것입니다.

```
function slowTotal(items, repeat) {
  let total = 0;
  for (let r = 0; r < repeat; r++) {
    total = 0;
    for (const item of items) {
      total = total + item.price * item.count;
    }
  }
  return total;
}

const HEAVY = 5000000; // 반복 횟수. 컴퓨터에 따라 40~120ms 쯤 걸립니다.
```

먼저 쓰지 마세요

섹션 1

이 도구들을 배우면 “그럼 전부 붙여 두면 더 빠르겠네” 라고 생각하기 쉽습니다. 그러면 대개 더 느려지고, 코드는 확실히 어려워집니다. 이유가 셋입니다.

이유 1 · 공짜가 아닙니다

useMemo 는 “의존성이 바뀌었나” 를 매번 비교합니다. memo 는 “props 가 다 같은가” 를 매번 비교합니다. 비교하는 데도 시간이 듭니다. 아낀 시간보다 비교하는 시간이 더 클 수 있습니다. 더하기 몇 번 하는 계산에 useMemo 를 붙이면 손해입니다.

이유 2 · 기억해 두려면 자리를 차지합니다

계산 결과와 의존성을 계속 들고 있어야 합니다. 메모리를 씁니다.

이유 3 · 코드가 어려워집니다

의존성 배열이 늘어납니다. 하나 빠뜨리면 옛날 값이 화면에 남습니다. 에러가 안 나는 종류라 찾기 어렵습니다. 섹션 7에서 실제로 봅니다.

그래서 순서는 언제나 이렇습니다.

- 1 먼저 그냥 만든다
- 2 느리다고 느껴지면 어디가 느린지 쟀다
- 3 쟀 결과가 가리키는 그 자리만 고친다
- 4 고친 뒤 다시 재서 정말 빨라졌는지 확인한다

2번을 건너뛰고 3번부터 하는 것을 ‘넘겨짚기’ 라고 합니다. 대부분의 화면은 이 도구들 없이도 충분히 빠릅니다. React 는 화면 전체를 다시 그리는 것처럼 보여도, 실제로 손대는 것은 바뀐 부분뿐입니다. 01단원에서 본 그대로입니다.

```
function IntroBox() {
  return (
    <div className="demo"> <h3>① 순서를 기억하세요
</h3> <div className="output"> <div>1. 그냥 만든다</div> <div>2. 느리면 켜다
</div> <div>3. 켜 자리만 고친다</div> <div>4. 다시 재서 확인한다</div> </div> <p>이
파일의 도구들은 3번에서만 씁니다.</p> </div>
  );
}
```

▹ 직접 해보기

1

위 네 단계 중 사람들이 가장 자주 건너뛰는 단계는 몇 번일까요?

재는 법

섹션 2

재는 방법은 여러 가지인데, 코드로 재는 가장 쉬운 두 가지를 봅니다.

방법 1 · performance.now()

지금 시각을 밀리초로 돌려줍니다. 앞뒤로 찍어 빼면 걸린 시간입니다.

방법 2 · console.time / console.timeEnd

같은 이름으로 찍을 맞추면 걸린 시간을 콘솔이 직접 찍어 줍니다.

화면이 얼마나 자주, 얼마나 오래 그려지는지를 제대로 보려면 React DevTools 의 Profiler 탭을 씁니다. 01단원 개념04에서 설치를 안내했습니다. 여기서는 코드로 재는 것까지만 합니다.

```
function MeasureDemo() {
  const [result, setResult] = useState("아직 안 잰습니다");

  function handleMeasure() {
    const start = performance.now();
    const total = slowTotal(cart, HEAVY);
    const ms = Math.round(performance.now() - start);

    console.log(`장바구니 합계는 ${total} 원입니다`);
  }
}
```

F12 콘솔 장바구니 합계는 22100 원입니다

```
console.log(`계산에 걸린 시간:`, ms, "ms");
```

F12 콘솔 계산에 걸린 시간:

뒤의 숫자는 컴퓨터마다 다릅니다. 40~120 사이면 정상입니다.

```

    setResult(`${total} 원 · ${ms}ms`);
  }

  function handleConsoleTime() {

```

console.time 과 console.timeEnd 는 이름을 똑같이 맞춰야 짝이 됩니다.

```

    console.time("합계 계산");
    slowTotal(cart, HEAVY);
    console.timeEnd("합계 계산");

```

콘솔에 "합계 계산: 52.3ms" 같은 줄이 나옵니다. 숫자는 컴퓨터마다 다릅니다.

```

    setResult("console.timeEnd 로 잤습니다. 콘솔을 보세요");
  }

  return (
    <div className="demo"> <h3>② 재 보기</h3> <div className="output">{result}
</div> <div style={{ marginTop: 8 }}> <button onClick={handleMeasure}>performance.now
로 재기</button> <button onClick={handleConsoleTime}>console.time 으로 재기
</button> </div> </div>
  );
}

```

40~120ms 는 사람이 느낄 수 있는 시간입니다. 화면을 그릴 때마다 이만큼 걸린다면 버튼이 굼떠 보입니다. 이 정도가 되어야 비로소 useMemo 를 꺼낼 자리입니다.

⇒ 직접 해보기 2

HEAVY 를 5000000 에서 500000 으로 (0 하나 뺍니다) 바꾸고 다시 재 보세요. 몇 ms 가 나오나요?

useMemo — 무거운 계산 건너뛰기

섹션 3

문제 상황부터 봅시다.

컴포넌트는 다시 그릴 때마다 함수 본문을 처음부터 다시 실행합니다(04단원 개념03). 본문 안에 무거운 계산이 있으면 그 계산도 매번 다시 합니다. 계산에 쓰이는 값이 하나도 안 바뀌었는데도 그렇습니다.

useMemo 는 이렇게 씁니다.

```

const 결과 = useMemo(() => 무거운계산(), [바뀌면 다시 할 값들]);

```

의존성 배열은 09단원 useEffect 에서 배운 것과 같은 규칙입니다.

```

[]          한 번만 계산합니다

```

cart · cart 가 바뀌면 다시 계산합니다

(없음) 매번 계산합니다 - 그러면 useMemo 를 쓸 이유가 없습니다

아래 데모는 같은 화면에서 두 방식을 나란히 돌립니다.

```
function MemoCompare() {  
  const [tick, setTick] = useState(0);
```

(1) useMemo 없이 — 다시 그럴 때마다 계산합니다

```
const start = performance.now();  
const totalPlain = slowTotal(cart, HEAVY);  
const plainMs = Math.round(performance.now() - start);  
console.log(`useMemo 없이 계산했습니다`, plainMs, "ms");
```

F12 콘솔 useMemo 없이 계산했습니다

(2) useMemo 로 — cart 가 안 바뀌면 계산을 건너뛴다

```
const totalMemo = useMemo(() => {  
  const s = performance.now();  
  const value = slowTotal(cart, HEAVY);  
  console.log(`useMemo 안에서 계산했습니다`, Math.round(performance.now() - s),  
    "ms");
```

F12 콘솔 useMemo 안에서 계산했습니다

```
return value;  
}, []); // cart 는 이 파일에서 바뀌지 않으므로 빈 배열입니다 return (  
  <div className="demo"> <h3>③ useMemo 있고 없고  
</h3> <div className="output"> <div>  
  useMemo 없이: {totalPlain} 원 (이번 렌더에서 {plainMs}ms 걸림)  
  </div> <div>useMemo 로: {totalMemo} 원</div> <div>버튼 누른 횟수: {tick}  
</div> </div> <div style={{ marginTop: 8 }}> <button onClick={() => setTick(tick +  
1)}>이 값만 올리기</button> </div> </div>  
);  
}
```

이 값만 올리기 · 를 여러 번 눌러 보세요. 콘솔을 함께 보셔야 합니다.

화면(누르면): 숫자가 올라갑니다. 그런데 버튼 반응이 살짝 굼뎡니다. 콘솔에는 “useMemo 없이 계산했습니다” 만 계속 찍힙니다. “useMemo 안에서 계산했습니다” 는 처음 한 번 나오고 다시 안 나옵니다.

눌러도 장바구니는 하나도 안 바뀌었습니다. 그런데 위쪽은 매번 다시 계산했습니다. 아래쪽은 "cart 가 그대로니 지난번 답을 그냥 쓰자" 하고 넘어갔습니다. 그것이 useMemo 입니다.

개발 중에는 StrictMode 때문에 "useMemo 없이 계산했습니다" 가 두 번씩 찍힙니다. 정상입니다. 09단원에서 본 그 이유와 같습니다.

여기서 놓치면 안 되는 것이 있습니다. useMemo 는 계산을 '더 빠르게' 만들지 않습니다. '덜 하게' 만들 뿐입니다. 처음 한 번은 똑같이 오래 걸립니다.

▫ 직접 해보기

3

useMemo 의 의존성 배열 [] 을 지워 보세요. (useMemo(() => {...}) 로 만듭니다) [이 값만 올리기] 를 눌렀을 때 콘솔이 어떻게 달라지나요?

memo — props 가 그대로면 자식을 다시 안 그린다

섹션 4

부모가 다시 그려지면 자식도 전부 다시 그려집니다. 기본 동작입니다. 자식이 받는 props 가 하나도 안 바뀌어도 그렇습니다.

memo 로 감싸면 React 가 이렇게 바꿉니다. "이 컴포넌트는 props 를 먼저 비교해라. 다 같으면 지난번 화면을 그대로 써라."

쓰는 법은 컴포넌트를 memo(...) 로 감싸는 것뿐입니다.

```
const MemoCard = memo(function MemoCard({ name, price }) {
  console.log(`MemoCard 를 그렸습니다: ${name}`);
```

F12 콘솔 MemoCard 를 그렸습니다: 아메리카노

```
return (
  <div className="output">
    [memo 불임] {name} - {price} 원
  </div>
);
```

```
function PlainCard({ name, price }) {
  console.log(`PlainCard 를 그렸습니다: ${name}`);
```

F12 콘솔 PlainCard 를 그렸습니다: 라떼

```

return (
  <div className="output">
    [memo 안 붙임] {name} - {price} 원
  </div>
);
}

function MemoSection() {
  const [tick, setTick] = useState(0);

  console.log("부모(MemoSection)를 그렸습니다");
}

```

F12 콘솔 부모(MemoSection)를 그렸습니다

```

return (
  <div className="demo"> <h3>④ memo 있고 없고
</h3> <MemoCard name="아메리카노" price={4000} /> <PlainCard name="라떼" price={4500}
/> <div className="output">부모의 숫자: {tick}</div> <div style={{ marginTop: 8
}}> <button onClick={() => setTick(tick + 1)}>부모만 다시 그리기
</button> </div> </div>
);
}

```

부모만 다시 그리기 · 를 눌러 보세요.

콘솔에 찍히는 것: 부모(MemoSection)를 그렸습니다 PlainCard 를 그렸습니다: 라떼

MemoCard 는 안 찍힙니다. props(name, price)가 지난번과 같기 때문입니다.

화면(누르면): 두 카드의 글자는 그대로이고 “부모의 숫자” 만 올라갑니다.

여기서 오해하기 쉬운 점 하나. PlainCard 가 다시 그려졌다고 해서 화면이 실제로 다시 칠해진 것은 아닙니다. React 는 다시 그려 본 뒤 “결과가 같네” 하고 실제 화면은 안 건드립니다. 그래서 memo 를 안 붙여도 눈에 보이는 문제는 대개 없습니다. memo 가 아끼는 것은 ‘컴포넌트 함수를 실행하는 시간’ 입니다. 그 함수가 무겁지 않으면 아낄 것도 별로 없습니다.

▫ 직접 해보기 4

MemoCard 에 price={tick} 을 넘기도록 바꾸고 [부모만 다시 그리기] 를 눌러 보세요. 콘솔이 어떻게 달라지나요?

useCallback — memo 를 무력하게 만드는 함정

섹션 5

memo 를 붙였는데도 자식이 계속 다시 그려지는 일이 있습니다. 옆에 아홉은 함수를 props 로 넘기고 있기 때문입니다.

컴포넌트를 다시 그릴 때 본문의 함수도 '새로' 만들어집니다. 함수도 객체입니다(JS자료 07단원). 객체는 모양이 같아도 새로 만들면 다른 것입니다.

```
console.log({ a: 1 } === { a: 1 });
// false - 안이 같아도 서로 다른 객체입니다
console.log(function () {} === function () {});
// false - 함수도 마찬가지로입니다
```

memo 는 props 를 === 로 비교합니다. 그러니 함수 props 는 매번 "달라졌다" 로 판정되고, memo 는 아무 일도 못 합니다.

useCallback 이 이 문제를 푼다.

```
const 함수 = useCallback(() => { ... }, [바뀌면 새로 만들 값들]);
```

의존성이 그대로면 지난번에 만든 그 함수를 다시 씁니다.

```
const ActionButton = memo(function ActionButton({ label, onPress }) {
  console.log(`ActionButton 을 그렸습니다: ${label}`);
```

```
F12 콘솔  ActionButton 을 그렸습니다: 매번 새 함수
           ActionButton 을 그렸습니다: useCallback 함수
```

```
return <button onClick={onPress}>{label} 담기</button>;
});
```

```
function CallbackSection() {
  const [tick, setTick] = useState(0);
```

(1) 다시 그릴 때마다 새로 만들어지는 함수

```
const unstablePress = () => {
  console.log("매번 새 함수 쪽 버튼을 눌렀습니다");
```

```
F12 콘솔  매번 새 함수 쪽 버튼을 눌렀습니다
```

```
};
```

(2) 언제나 같은 함수

```
const stablePress = useCallback(() => {
  console.log("useCallback 쪽 버튼을 눌렀습니다");
```

```
F12 콘솔  useCallback 쪽 버튼을 눌렀습니다
```

```
}, []);
```

useRef 와 커스텀 훅

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 파일을 고르세요.

고치고 저장하면 화면이 바로 바뀝니다. F12 → Console 도 함께 보세요.

문제 1~10은 기본, 11은 응용, 12는 도전, 13은 에러 확인입니다.

문제마다 '기대 결과'가 적혀 있으니 그대로 나오는지 확인하세요.

★ 아직 안 푼 문제는 "문제 N: ..." 같은 안내 글자가 그대로 보입니다. 정상입니다.

★ 개발 중에는 StrictMode 때문에 콘솔 줄이 두 번씩 찍힙니다. 이것도 정상입니다.

필요한 것은 미리 다 꺼내 두었습니다. 문제를 풀면서 골라 쓰세요.

```
import { useState, useRef, useEffect, memo } from "react";
import Summary from "../_ui/Summary.jsx";
```

문제에서 함께 쓰는 데이터입니다. 고치지 마세요.

```
const menuNames = [
  "아메리카노",
  "카페라떼",
  "바닐라라떼",
  "카페모카",
  "녹차라떼",
  "케이크",
  "쿠키",
  "삼각김밥",
];

const menuItems = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
  { id: 4, name: "삼각김밥", price: 1200 },
];
```

문제 1 (개념01)

커서 놓기 · 를 누르면 입력칸에 커서가 놓이게 하세요.

할 일은 세 가지입니다. useRef 로 상자 만들기 / input 에 ref 붙이기 / focus 부르기.

기대 화면 버튼을 누르면 입력칸 테두리가 진해지고 커서가 깜빡입니다.
아무 일도 안 일어나면 input 에 ref={...} 를 안 붙인 것입니다.
콘솔에 Cannot read properties of null 이 나오면
ref 는 만들었는데 input 에 안 붙인 것입니다.

아래 함수 안을 고치세요

```
function Q1Focus() {
```

TODO 1: 여기에 ref 상자를 만드세요

```
function handleFocus() {
```

TODO 1: 여기에서 입력칸에 커서를 놓으세요

```
    }

    return (
      <div className="output"> <input placeholder="여기에 커서가 와야 합니다"
/> <button onClick={handleFocus}>커서 놓기</button> </div>
    );
  }
}
```

문제 2 (개념 01)

지우고 커서 놓기 버튼을 누르면 입력칸을 비우고, 그 자리에 커서를 놓으세요.

상자(inputRef)는 이미 만들어서 input 에 붙여 두었습니다. 비우는 일과 커서를 놓는 일 중 어느 쪽이 state 이고 어느 쪽이 ref 인지 생각해 보세요.

기대 화면 입력칸이 비고, 그 자리에 커서가 깜빡입니다.
글자만 지워지고 커서가 없으면 focus() 를 안 부른 것입니다.
커서만 오고 글자가 남아 있으면 setText 를 안 부른 것입니다.

아래 함수 안을 고치세요

```
function Q2ClearAndFocus() {
  const [text, setText] = useState("아메리카노");
  const inputRef = useRef(null);

  function handleClear() {
```

TODO 2: 여기에 코드를 쓰세요

```
  }

  return (
    <div className="output"> <input ref={inputRef} value={text} onChange={(e) =>
      setText(e.target.value)}
    />
    <button onClick={handleClear}>지우고 커서 놓기</button> <div>지금 값: {text} ||
    "(비어 있음)"</div> </div>
  );
}
```

문제 3 (개념01)

케이크로 가기 · 를 누르면 목록이 '케이크' 줄까지 스크롤 내려가게 하세요.

ref 는 이미 '케이크' 줄에 붙여 두었습니다. 부르는 코드만 쓰면 됩니다. 스크롤 움직이게 하고, 케이크 줄이 상자 가운데에 오게 하세요.

기대 화면 목록 상자가 스크롤 내려가 '케이크' 가 가운데에 보입니다.
순간이동하듯 톱 움직이면 behavior 옵션을 안 준 것입니다.
케이크가 상자 맨 위에 붙으면 block 옵션을 안 준 것입니다.

아래 함수 안을 고치세요

```
function Q3Scroll() {
  const cakeRef = useRef(null);

  function handleGo() {
```

TODO 3: 여기에 코드를 쓰세요

```

    }

    return (
      <div className="output"> <button onClick={handleGo}>케이크로 가기
    </button> <div style={{ height: 110,
      overflowY: "auto",
      border: "1px solid #ccc",
      background: "#fff",
      marginTop: 6,
    }}
    > <ul>
      {menuNames.map((name) => (
        <li key={name} ref={name === "케이크" ? cakeRef : null}>
          {name}
        </li>
      ))}
    </ul> </div> </div>
    );
  }
}

```

문제 4 (개념02)

ref 올리기 · 를 누르면 상자 안의 값을 1 올리고,

콘솔에 "지금 진짜 값: N" 을 찍으세요. 화면은 손대지 마세요.

기대 화면 [ref 올리기] 를 세 번 눌러도 "화면에 보이는 값" 은 0 그대로입니다.
 그 다음 [다시 그리기] 를 누르면 화면 값이 3 으로 바뀝니다.
 기대 결과 (콘솔): 누를 때마다 지금 진짜 값: 1 / 2 / 3 이 찍힙니다.
 화면 값이 누를 때마다 바로 바뀌면 useState 로 만든 것입니다.

아래 함수 안을 고치세요

```

function Q4RefNoRender() {
  const countRef = useRef(0);
  const [tick, setTick] = useState(0);

  function handleUp() {

```

TODO 4: 여기에 코드를 쓰세요

```

    }

    return (
      <div className="output"> <div>화면에 보이는 값: {countRef.current}</div> <div>
다시 그린 횟수: {tick}</div> <div style={{ marginTop: 6 }}> <button onClick=
{handleUp}>ref 올리기</button> <button onClick={() => setTick(tick + 1)}>다시 그리기
</button> </div> </div>
    );
  }
}

```

문제 5 (개념 02)

아래 코드는 잘못됐습니다. [담기] 를 눌러도 화면 숫자가 안 바뀝니다. 화면에 보여야 하는 값이 ref 에 들어 있기 때문입니다. ref 를 state 로 바꿔서 고치세요. (화면 글자와 버튼은 그대로 두세요)

기대 화면 [담기] 를 누를 때마다 숫자가 1씩 올라갑니다.
 안 올라가면 아직 ref 인 것입니다.
 화면이 빨간 상자가 되면 .current 를 지우지 않고 남겨 둔 것입니다.

아래 함수 안을 고치세요

```

function Q5FixToState() {
  const countRef = useRef(0);

  function handleAdd() {
    countRef.current = countRef.current + 1;
  }

  return (
    <div className="output"> <div>아메리카노 {countRef.current} 잔
</div> <button onClick={handleAdd}>담기</button> </div>
  );
}

```

문제 6 (개념02)

시작·멈춤이 되는 타이머를 완성하세요. - [시작] 을 누르면 0.5초마다 1씩 올라갑니다 - [멈춤] 을 누르면 그 자리에 섭니다 - [시작] 을 두 번 눌러도 타이머가 두 개 돌면 안 됩니다 타이머 id 를 어디에 뒀야 하는지 생각해 보세요. 상자는 만들어 뒀습니다.

기대 화면 [시작] → 숫자가 올라감 → [멈춤] → 그 자리에 섭니다.

시작 · 을 두 번 눌렀을 때 두 배로 빨라지면

"이미 돌고 있나" 검사를 안 넣은 것입니다.

멈춤 · 뒤에 [시작] 이 안 먹으면 멈출 때 상자를 안 비운 것입니다.

아래 두 함수 안을 고치세요

```
function Q6Timer() {
  const [sec, setSec] = useState(0);
  const timerRef = useRef(null);

  function handleStart() {
```

TODO 6: 여기에 코드를 쓰세요

```
  }

  function handleStop() {
```

TODO 6: 여기에 코드를 쓰세요

```
  }
```

이 예제를 떠날 때 타이머를 치우는 코드입니다. 이미 적어 뒀습니다(09단원).

```
useEffect(() => {
  return () => {
    clearInterval(timerRef.current);
  };
}, []);
```


부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

06단원 연습문제 정답.jsx 정답 폼과 입력

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

직접 쳐 보세요. F12 → Console 도 함께.

문제 1

```
import { useState } from "react";

function Q1() {
  const [text, setText] = useState("");

  return (
    <div className="demo"> <h3>문제 1 – 제어 컴포넌트</h3> <input value=
    {text} onChange={(e) => setText(e.target.value)}
    placeholder="이름을 입력하세요"
    />
    <div className="output">입력한 값: {text}</div> </div>
  );
}
```

화면 입력한 값:

“김민준” 을 치면 → 화면: 입력한 값: 김민준

value 와 onChange 는 한 세트입니다. value state 를 칸에 내보내는 길 onChange 친 글자를 state 로 들이는 길 한쪽만 있으면 값이 돌지 않습니다. onChange 를 빼면 한 글자도 안 써집니다. 문제 15 에서 직접 확인합니다.

문제 2

```
function Q2() {
  const [memo, setMemo] = useState("");

  return (
    <div className="demo"> <h3>문제 2 - 글자 수 세기</h3> <input value=
{memo} onChange={(e) => setMemo(e.target.value)} />
    <div className="output">{memo.length}자</div> </div>
  );
}
```

화면 0자

“김민준” 을 치면 → 화면: 3자

글자 수를 state 로 따로 두지 않았습니다. memo 만 있으면 세어지니까요. 계산할 수 있는 값을 state 로 두면 두 값이 어긋납니다. (07단원 개념05)

06

문제 3

```
function Q3() {
  const [text, setText] = useState("아메리카노");

  return (
    <div className="demo"> <h3>문제 3 - 지우기 버튼</h3> <input value=
{text} onChange={(e) => setText(e.target.value)} />
    <button onClick={() => setText("")}>지우기</button> <div className="output">
지금 값: {text}</div> </div>
  );
}
```

화면 지금 값: 아메리카노

화면(누르면): 입력칸이 비고 → 지금 값:

입력칸을 직접 붙잡는 코드가 한 줄도 없습니다.

value={text} 로 묶어 뒀으니 state 를 비우면 칸도 따라 비워집니다.

이것이 value 를 붙이는 가장 큰 이득입니다.

문제 4

```
function Q4() {
  const [form, setForm] = useState({ name: "김민준", memo: "" });

  function handleMemo(e) {
    setForm({ ...form, memo: e.target.value });
  }

  return (
    <div className="demo"> <h3>문제 4 – 객체 state 와 스프레드</h3> <div>
      이름 <input value={form.name} readOnly /> </div> <div>
      메모 <input value={form.memo} onChange={handleMemo}
    /> </div> <div className="output">
      {form.name} / {form.memo}
    </div> </div>
  );
}
```

화면 김민준 /

메모에 “라떼” 를 치면 → 화면: 김민준 / 라떼

...form 이 없으면 setForm({ memo: ... }) 가 되어 name 이 통째로 사라집니다. 에러는 안 납니다. 이름 자리만 조용히 비어 버립니다. 객체 state 는 “나머지는 하던 대로” 를 먼저 적는 습관을 들이세요.

문제 5

```
function Q5() {
  const [form, setForm] = useState({ id: "", pw: "" });

  function handleChange(e) {
    setForm({ ...form, [e.target.name]: e.target.value });
  }

  return (
    <div className="demo"> <h3>문제 5 – 핸들러 하나로 두 칸</h3> <div>
      아이디 <input name="id" value={form.id} onChange={handleChange}
    /> </div> <div>
      비밀번호 <input name="pw" value={form.pw} onChange={handleChange}
    /> </div> <div className="output">
      {form.id} / {form.pw}
    </div> </div>
  );
}
```

```
);
}
```

화면 /

아이디에 "minjun", 비밀번호에 "1234" 를 치면 → 화면: minjun / 1234

e.target.name · 의 대괄호가 핵심입니다.

대괄호가 없으면 name 이라는 글자 그대로가 키가 되어, 어느 칸을 쳐도 form.name 만 생기고 화면은 그대로입니다. 아이디 칸에서 오면 "id", 비밀번호 칸에서 오면 "pw" 가 됩니다.

문제 6

```
function Q6() {
  const [item, setItem] = useState({ price: "4000", count: "2" });

  function handleChange(e) {
    setItem({ ...item, [e.target.name]: e.target.value });
  }

  const total = Number(item.price) * Number(item.count);

  return (
    <div className="demo"> <h3>문제 6 – 숫자 칸 합계</h3> <div>
      가격 <input type="number" name="price" value={item.price} onChange=
{handleChange} /> </div> <div>
      개수 <input type="number" name="count" value={item.count} onChange=
{handleChange} /> </div> <div className="output">합계: {total}원</div> </div>
    );
}
```

화면 합계: 8000원

개수를 3 으로 바꾸면 → 화면: 합계: 12000원

type="number" 여도 e.target.value 는 문자열입니다. 곱하기는 Number 를 안 씌워도 우연히 맞습니다. 자바스크립트가 알아서 숫자로 보거든요. 그런데 더하기는 "4000" + "2" 가 "40002" 가 됩니다. 그래서 계산 직전에 Number() 를 씌우는 것을 습관으로 하세요.

문제 7

```
function Q7() {
  const [agree, setAgree] = useState(false);

  return (
    <div className="demo"> <h3>문제 7 - 체크박스
  </h3> <label> <input type="checkbox" checked={agree} onChange={(e) =>
    setAgree(e.target.checked)}
    />
    약관에 동의합니다
    </label> <div className="output">동의: {agree ? "함" : "안 함"}</div> </div>
  );
}
```

화면 동의: 안 함

화면(누르면): 체크하면 → 동의: 함 / 다시 풀면 → 동의: 안 함

체크박스는 value 가 아니라 checked 입니다. 두 자리 모두 그렇습니다.

붙일 때 checked={agree}

읽을 때 e.target.checked e.target.value 를 읽으면 켜도 꺼도 "on" 이 들어옵니다. "on" 은 truthy 라서 한 번 켜면 영원히 "함" 이 됩니다.

문제 8

```
function Q8() {
  const [size, setSize] = useState("M");

  return (
    <div className="demo"> <h3>문제 8 - select</h3> <select value={size} onChange=
    {(e) => setSize(e.target.value)}>
      <option value="S">작은 것</option> <option value="M">보통
    </option> <option value="L">큰 것</option> </select> <div className="output">고른
    사이즈: {size}</div> </div>
  );
}
```

화면 목록에 '보통' 이 골라져 있고 → 고른 사이즈: M

화면(누르면): '큰 것' 을 고르면 → 고른 사이즈: L

value 는 option 이 아니라 select 에 줍니다. option 에 selected 를 붙이면 React 가 콘솔로 그러지 말라고 알려 줍니다. 고른 값은 option 의 value 입니다. 화면에 보이는 "큰 것" 이 아닙니다.

문제 9

```
function Q9() {
  const [pay, setPay] = useState("카드");

  return (
    <div className="demo"> <h3>문제 9 - 라디오
  </h3> <label> <input type="radio" name="pay" value="카드" checked={pay === "카드"}
    onChange={(e) => setPay(e.target.value)}
  />
    카드
    </label> <label> <input type="radio" name="pay" value="현금" checked={pay ===
"현금"}
    onChange={(e) => setPay(e.target.value)}
  />
    현금
    </label> <div className="output">결제 방법: {pay}</div> </div>
  );
}
```

화면 '카드' 가 켜져 있고 → 결제 방법: 카드

화면(누르면): '현금' 을 누르면 → 결제 방법: 현금

checked 자리에 비교식을 그대로 넣었습니다. pay === "카드" 는 true 아니면 false 입니다. 라디오가 두 개여도 state 는 하나면 됩니다. 골라진 것이 하나뿐이니까요. 읽을 때는 e.target.value 입니다. 체크박스과 다릅니다. "켜졌나" 가 아니라 "무엇을 골랐나" 가 알고 싶은 것이니까요.

문제 10

```
const Q10_TOPPINGs = ["시럽", "휘핑크림"];

function Q10() {
  const [picked, setPicked] = useState([]);

  function handleChange(e) {
    const value = e.target.value;

    if (e.target.checked) {
      setPicked([...picked, value]); // 켜짐 → 더한 새 배열
    }
  }
}
```

```

    } else {
      setPicked(picked.filter((item) => item !== value)); // 꺼짐 → 뺀 새 배열
    }
  }

  return (
    <div className="demo"> <h3>문제 10 - 여러 개 체크</h3>
      {Q10_TOPPING.map((topping) => (
        <label key={topping} style={{ marginRight: "12px"
      }}> <input type="checkbox" value={topping} checked=
      {picked.includes(topping)} onChange={handleChange}
        />
        {topping}
      </label>
      )))}
      <div className="output">고른 것: {picked.join(", ")}
    </div> <div className="output">모두 {picked.length}개</div> </div>
  );
}

```

화면 고른 것: / 모두 0개

화면(누르면): '시럽' 을 켜면 → 고른 것: 시럽 / 모두 1개 화면(누르면): '휘핑크림' 도 켜면 → 고른 것: 시럽, 휘핑크림 / 모두 2개 화면(누르면): '시럽' 을 끄면 → 고른 것: 휘핑크림 / 모두 1개

여기서는 e.target.checked 와 e.target.value 를 둘 다 씁니다. checked → 켜나 껴나 value → 어느 토핑인가 체크박스에 value 를 안 적으면 전부 "on" 이라 구분이 안 됩니다. 끝 때 filter 를 빼먹으면 같은 토핑이 계속 쌓입니다. 에러는 안 납니다.

문제 11

```

function Q11() {
  const [name, setName] = useState("김민준");
  const [message, setMessage] = useState("아직 제출 전입니다");

  function handleSubmit(e) {
    e.preventDefault();
    setMessage(name + "님 환영합니다");
  }

  return (
    <div className="demo"> <h3>문제 11 - onSubmit</h3> <form onSubmit=
    {handleSubmit}> <input value={name} onChange={(e) => setName(e.target.value)} />
      <button type="submit">확인</button> </form> <div className="output">{message}
    </div> </div>
  );
}

```



```
);
}
```

화면 아직 제출 전입니다

화면(누르면): '확인' 을 누르면 → 김민준님 환영합니다 입력칸에서 Enter 만 쳐도 결과가 같습니다.

버튼의 onClick 이 아니라 form 의 onSubmit 에 붙였습니다. 그래야 버튼을 누른 사람과 Enter 를 친 사람이 같은 곳으로 옵니다. e.preventDefault() 를 빼면 페이지가 새로고침되어 state 가 처음 값으로 돌아갑니다.

문제 12

```
function Q12() {
  const [text, setText] = useState("우유 사기");
  const [todos, setTodos] = useState(["장보기"]);

  function handleSubmit(e) {
    e.preventDefault();
    setTodos([...todos, text]);
  }

  return (
    <div className="demo"> <h3>문제 12 - 목록에 더하기</h3> <form onSubmit=
    {handleSubmit}> <input value={text} onChange={(e) => setText(e.target.value)} />
    <button type="submit">추가</button> </form> <ul>
      {todos.map((todo, index) => (
        <li key={index}>{todo}</li>
      ))}
    </ul> </div>
  );
}
```

화면 목록에 장보기 한 줄

화면(누르면): '추가' 를 누르면 → 장보기 / 우유 사기 두 줄

...todos, text · 는 "todos 를 그대로 펼쳐 담고 뒤에 하나 더" 라는 뜻입니다.

대괄호를 빠뜨려 setTodos(text) 라고 쓰면 목록이 글자 하나로 덮여 버립니다. push 를 쓰면 화면이 아예 안 바뀝니다. 이유는 07단원 개념01 에서 다룹니다.

여기서는 입력칸을 안 비웠습니다. 그래서 한 번 더 누르면 같은 것이 또 들어갑니다. 비우는 것은 문제 13·14 에서 합니다.

문제 13 [응용]

```
function Q13() {
  const [text, setText] = useState("");
  const [error, setError] = useState("");
  const [saved, setSaved] = useState("(아직 없음)");

  function handleSubmit(e) {
    e.preventDefault();

    const value = text.trim(); // 앞뒤 공백 제거 if (!value) {
      setError("후기를 입력해 주세요");
      return; // * return 이 없으면 아래 줄이 그대로 실행됩니다
    }

    if (value.length > 10) {
      setError("10자 이내로 써 주세요");
      return;
    }

    setSaved(value);
    setError("");
    setText("");
  }

  return (
    <div className="demo"> <h3>문제 13 - [응용] 검사하고 에러 보여 주기
    </h3> <form onSubmit={handleSubmit}> <input value={text} onChange={(e) =>
    setText(e.target.value)} />
      <button type="submit">저장</button> <button type="button" onClick={() =>
    setText("")}>
        지우기
      </button> </form> <div className="error">{error}</div>

      <div className="output">저장된 후기: {saved}</div>
    </div>
  );
}
```

화면 저장된 후기: (아직 없음)

화면(누르면): 빈 칸에서 '저장' → 후기를 입력해 주세요 스페이스 세 칸만 치고 '저장' → 화면: 후기를 입력해 주세요 "맛있어요" 를 치고 '저장' → 화면: 저장된 후기: 맛있어요 (입력칸이 비워집니다)
"아메리카노가 정말 맛있어요" 를 치고 '저장' → 화면: 10자 이내로 써 주세요

검사를 위에서 하나씩 하고 안 맞으면 바로 `return` 합니다. 조기 반환입니다. `else` 가 하나도 없어서 읽기 쉽습니다.

지우기 버튼에는 `type="button"` 을 붙였습니다. 안 붙이면 `form` 안이라 기본이 `submit` 이 되어, 지우면서 제출까지 합니다. 그러면 방금 비운 값 때문에 “후기를 입력해 주세요” 가 함께 뜹니다.

`trim()` 한 `value` 로 검사하고 `value` 로 저장했습니다. 검사한 값과 저장하는 값이 다르면 검사가 헛됩니다.

문제 14 [도전]

```
function Q14() {
  const [text, setText] = useState("");
  const [orders, setOrders] = useState([
    { id: 1, text: "아메리카노" }
  ]);
  const [nextId, setNextId] = useState(2);
  const [error, setError] = useState("");

  function handleSubmit(e) {
    e.preventDefault();

    const value = text.trim(); // (1) if (!value) {
      setError("메뉴를 입력해 주세요"); // (2) return;
    }

    setOrders([
      ...orders,
      { id: nextId, text: value }
    ]); // (3) setNextId(nextId + 1); // (4) setError(""); // (5) setText(""); // (5)
  }

  return (
    <div className="demo">
      <h3>문제 14 - [도전] id 를 붙여 담기</h3>
      <form onSubmit={handleSubmit}>
        <input value={text} onChange={(e) => setText(e.target.value)} />
        <button type="submit">담기</button>
      </form>
      <div className="error">{error}</div>
      <div className="output">모두 {orders.length}잔</div>
      <ul>
        {orders.map((order) => (
          <li key={order.id}>
            {order.id}번 - {order.text}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```

    )))}
  </ul>
</div>
);
}

```

화면 모두 1잔 / 목록에 "1번 - 아메리카노"

화면(누르면): 빈 칸에서 '담기' → 메뉴를 입력해 주세요 "라떼" 를 치고 '담기' → 화면: 모두 2잔 / "2번 — 라떼" 가 붙고 입력칸이 비워집니다 "라떼" 를 한 번 더 담으면 → 화면: 모두 3잔 / "3번 — 라떼" 가 또 붙습니다

set 함수를 네 번이나 불렀지만 화면은 한 번만 다시 그려집니다. React 가 모아서 한 번에 처리하기 때문입니다.

nextId 를 안 올리면 두 번째 항목도 2번이 되고, 그 다음도 2번이 됩니다. key 가 겹치면 콘솔에 "same key" 경고가 나옵니다.

id 덕분에 같은 글자를 여러 번 담아도 됩니다. 글자 자체를 key 로 썼다면 "라떼" 를 두 번 담는 순간 경고가 났을 것입니다.

문제 15 (에러 확인)

```

function Q15() {
  const [text, setText] = useState("김민준");

  function handleChange(e) {
    setText(e.target.value);
  }

  return (
    <div className="demo"> <h3>문제 15 - 에러 확인</h3> <input value={text} onChange=
{handleChange} /> <div className="output">지금 값: {text}</div> </div>
  );
}

```

onChange={handleChange} 를 지우면 이렇게 됩니다.

왜 안 써지나:

value={text} 는 "이 칸에 보이는 글자는 언제나 text 다" 라는 약속입니다.

키를 눌러도 setText 를 부르는 코드가 없으니 text 는 그대로입니다. React 는 이 약속을 지키려고, 키를 누를 때마다 입력칸의 DOM 값을 곧바로 text 값으로 되돌려 놓습니다. 컴포넌트를 다시 그리는 것과는 다른 일입니다. ★ 화면은 state 를 그대로 비춥니다. state 가 안 바뀌면 화면도 안 바뀝니다.

경고 첫 문장: You provided a `value` prop to a form field without an `onChange` handler.

이어지는 문장까지 옮기면 이렇습니다.

This will render a read-only field. If the field should be mutable use `defaultValue`. Otherwise, set either `onChange` or `readOnly`.

우리말로: "value 는 줬는데 onChange 를 안 줬습니다. 읽기 전용 칸이 됩니다.

고칠 수 있어야 하면 `defaultValue` 를, 아니면 `onChange` 나 `readOnly` 를 붙이세요."

이 경고는 에러가 아닙니다. 화면은 계속 돌아갑니다. 그래서 더 잘 놓칩니다. "입력칸에 글자가 안 써져요" 를 만나면 제일 먼저 `onChange` 를 찾으세요.

화면 조립

```
export default function ExerciseAnswers() {
  return (
    <div> <h1>06단원 연습문제 정답 - 폼과 입력</h1> <p className="guide">
      정답 파일입니다. <strong>먼저 직접 풀어 보고</strong> 나서 여세요.
    <br /> <br />
      코드만 보지 말고 <strong>왜 그런지</strong> 적어 둔 줄을 함께 읽으세요. 답이
      맞았어도 이유가 다르면 다음 문제에서 또 틀립니다.
    </p> <div> <Q1 /> <Q2 /> <Q3 /> <Q4 /> <Q5 /> <Q6 /> <Q7 /> <Q8 /> <Q9 /> <Q10
    /> <Q11 /> <Q12 /> <Q13 /> <Q14 /> <Q15 /> </div> </div>
  );
}
```

07단원 연습문제 정답.jsx 정답 state 설계와 불변성

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

직접 눌러 보세요. F12 → Console 도 함께.

```
import { useState } from "react";

let nextId = 100;
```

문제 1

```
function Q1() {
  const [items, setItems] = useState(["아메리카노"]);

  function handleAdd() {
    setItems([...items, "케이크"]);
  }

  return (
    <div className="demo"> <h3>문제 1 – 담기</h3> <button onClick={handleAdd}>케이크
    담기</button> <ul>
      {items.map((name, index) => (
        <li key={index}>{name}</li>
      ))}
    </ul> </div>
  );
}
```

화면(누르면): 아메리카노 · 케이크 · 케이크 ... 대괄호를 새로 열었으니 결과는 반드시 새 배열입니다.

`items.push("케이크")` 뒤에 `setItems(items)` 를 하면 화면이 안 바뀝니다. 배열 안만 바뀌고 배열 자체는
전과 같은 것이라 React 가 넘어가기 때문입니다.

문제 2

```
function Q2() {
  const [items, setItems] = useState([
    { id: 1, name: "아메리카노" },
```

```

{ id: 2, name: "케이크" },
  { id: 3, name: "라떼" },
]);

function handleRemove(targetId) {
  setItems(items.filter((item) => item.id !== targetId));
}

return (
  <div className="demo"> <h3>문제 2 - 빼기</h3> <ul>
    {items.map((item) => (
      <li key={item.id}>
        {item.name}
        <button onClick={() => handleRemove(item.id)}>빼기</button> </li>
    ))}
  </ul> </div>
);
}

```

화면(누르면): '케이크'의 빼기 → 아메리카노 · 라떼 filter는 '남길 것'을 고릅니다. 그래서 !== 입니다.
 === 로 쓰면 누른 것만 남습니다. splice 는 원본을 바꾸므로 쓰지 않습니다.

문제 3

```

function Q3() {
  const [items, setItems] = useState([
    { id: 1, name: "아메리카노", qty: 1 },
    { id: 2, name: "케이크", qty: 1 },
  ]);

  function handlePlus(targetId) {
    setItems(
      items.map((item) =>
        item.id === targetId ? { ...item, qty: item.qty + 1 } : item
      )
    );
  }

  return (
    <div className="demo"> <h3>문제 3 - 수량 늘리기</h3> <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} {item.qty}개
          <button onClick={() => handlePlus(item.id)}>+1</button> </li>
      ))}
    </ul>
  );
}

```

```

    )))
  </ul> </div>
);
}

```

화면(누르면): 아메리카노 2개 / 케이크 1개 삼항의 오른쪽에 item 을 그대로 돌려주는 것이 중요합니다. 그 자리를 빼먹으면 안 고칠 줄이 `undefined` 가 되어 화면이 에러로 멈춥니다.

문제 4

```

function Q4() {
  const [items, setItems] = useState([
    { id: 1, name: "케이크", price: 6000 },
    { id: 2, name: "아메리카노", price: 4000 },
    { id: 3, name: "라떼", price: 4500 },
  ]);

  function handleSort() {
    setItems([...items].sort((a, b) => a.price - b.price));
  }

  return (
    <div className="demo"> <h3>문제 4 - 정렬</h3> <button onClick={handleSort}>싼
    것부터</button> <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} - {item.price}원
        </li>
      ))}
    </ul> </div>
  );
}

```

화면(누르면): 아메리카노 4000 → 라떼 4500 → 케이크 6000

`...items` 를 빼면 items 자신을 정렬하고 그것을 다시 넣게 됩니다.

정렬은 됐는데 화면만 안 바뀌는, 가장 헷갈리는 상태가 됩니다.

문제 5

```

function Q5() {
  const [user, setUser] = useState({ name: "김민준", age: 20, city: "서울" });

```


08단원 연습문제 정답.jsx 정답 파일 나누기와 스타일링

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 파일을 고르세요.

- ★ 먼저 스스로 풀어 보고 여세요. 막힌 문제만 보는 편이 훨씬 낫습니다.
- ★ 답이 하나뿐인 것은 아닙니다. 화면이 기대 결과대로 나오면 맞은 것입니다.
- ★ 개발 중에는 StrictMode 때문에 콘솔 줄이 두 번씩 찍힙니다. 정상입니다.

```
import { useState } from "react";
import Summary from "../ui/Summary.jsx";

import "../연습문제.css";
```

[import 연습 구역] 정답

문제를 풀면 이만큼이 쌓입니다. 어느 줄이 어느 문제인지 옆에 적어 두었습니다. 순서는 상관없습니다. 다만 보통은 ① 남이 만든 도구(react) → ② 자료가 준 부품(_ui) → ③ 내 파일 순서로 적습니다. 읽기 편하기 때문입니다.

```
import Welcome from "../부품/연습_Welcome.jsx"; // 문제 1
import { cakePrice, applyDiscount, formatPrice } from "../부품/연습_가격.js"; // 문제 2, 11
import { americanoPrice as coffeePrice } from "../부품/연습_가격.js"; // 문제 3
import Badge, { hotLabel } from "../부품/연습_배지.jsx"; // 문제 4
import CoffeeCard from "../부품/연습_카드.jsx"; // 문제 5
import CoffeeList from "../부품/연습_카드목록.jsx"; // 문제 6
import styles from "../연습문제.module.css"; // 문제 10
import OrderTable from "../부품/연습_주문표_정답.jsx"; // 문제 12
```

- ★ 문제 2와 문제 3은 같은 파일에서 가져오는데 줄이 두 개입니다. `as` 를 쓰는 것만 따로 떼어 놓으면 읽기 쉬워서 이렇게 나눴습니다. 한 줄로 합쳐도 똑같이 됩니다.

```
import { cakePrice, applyDiscount, formatPrice, americanoPrice as coffeePrice }
  from "../부품/연습_가격.js";
```

여기까지

```
const menuItems = [
  { id: 1, name: "아메리카노", price: 4000, soldOut: false },
  { id: 2, name: "라떼", price: 4500, soldOut: false },
  { id: 3, name: "케이크", price: 6000, soldOut: true },
  { id: 4, name: "삼각김밥", price: 1200, soldOut: false },
];
```

문제 1 정답 (개념02)

```
function Q1Welcome() {
  return <Welcome name="김민준" />;
}
```

export default 로 나온 것은 중괄호 없이 가져오고, 이름은 마음대로 지어도 됩니다. 다만 파일 이름과 같게 짓는 편이 나중에 찾기 좋습니다(개념02 섹션 2). <Welcome /> 은 컴포넌트이므로 대문자로 시작해야 합니다.

문제 2 정답 (개념02)

```
function Q2Discount() {
  const salePrice = applyDiscount(cakePrice);

  return (
    <div className="output">
      케이크 {cakePrice}원 → 할인가 {salePrice}원
    </div>
  );
}
```

이름 있는 export 는 중괄호 안에 이름을 정확히 적어야 합니다. 중괄호를 빼면 "그 파일의 대표를 주세요"가 되는데, 연습_가격.js 에는 대표가 없어서 값이 undefined 가 됩니다. ★ 그런데 에러가 안 납니다. 화면의 숫자 자리만 비어 버립니다. 이것이 개념02 섹션 6의 [실수 2] 입니다. 문제 13의 (다) 에서 직접 봅니다.

문제 3 정답 (개념02)

이 줄이 이미 있어서 이름이 부딪힙니다.

```
const americanoPrice = 9999;

function Q3Rename() {
  return (
    <div className="output">
      가져온 값 {coffeePrice} / 이 파일의 값 {americanoPrice}
    </div>
  );
}
```

as 를 안 쓰고 그냥 import { americanoPrice } 라고 쓰면

SyntaxError · Identifier 'americanoPrice' has already been declared 가 납니다.

“이미 있는 이름이다” 라는 뜻입니다. 화면이 통째로 빡니다.

as 는 이럴 때 씁니다. 겹치지 않는데 굳이 이름을 바꾸지는 마세요. 찾기만 어려워집니다.

문제 4 정답 (개념02)

```
function Q4Badge() {
  return (
    <div className="output">
      아메리카노 <Badge text={hotLabel} /> </div>
    );
}
```

대표(Badge)가 앞, 중괄호(hotLabel)가 뒤입니다. 순서가 정해져 있습니다. 중괄호가 없는 쪽이 대표라고 기억하면 됩니다.

```
import { Badge, hotLabel } 처럼 둘 다 중괄호에 넣으면 이런 에러가 납니다.
```

The requested module ... does not provide an export named 'Badge'

문제 5 정답 (개념03)

```
function Q5UseCard() {
  return (
    <div> <CoffeeCard name="아메리카노" price={4000} note="가장 많이 팔립니다"
    /> <CoffeeCard name="라떼" price={4500} note="우유가 들어갑니다"
    /> <CoffeeCard name="케이크" price={6000} note="달아요" /> </div>
  );
}
```

화면은 하나도 안 바뀌었습니다. 바뀐 것은 코드뿐입니다. 15줄이 3줄이 됐고, <small> 문구의 모양을 고치려면 이제 한 곳만 고치면 됩니다. 03단원 개념05에서 한 일과 같은데, 이번에는 파일이 나뉘어 있습니다.

문제 6 정답 (개념03)

```
function Q6UseList() {  
  return <CoffeeList />;  
}
```

한 줄입니다. 그리고 이 파일은 CoffeeCard 를 import 하지 않았습니다. CoffeeList 가 자기 안에서 알아서 가져다 씁니다.

(이 정답 파일은 문제 5 때문에 CoffeeCard 도 import 하고 있습니다. 문제 6만 놓고 보면 필요 없는 줄입니다)

이것이 파일을 나눠 얻는 것입니다. 쓰는 쪽은 안을 몰라도 됩니다. CoffeeList 안의 카드가 세 개든 열 개든 이 파일은 그대로입니다.

문제 7 정답 (개념03)

```
function Q7NoReact() {  
  console.log(typeof React);  
}
```

F12 콘솔 undefined

```
const element = <strong>라떼</strong>;  
  
console.log(element.type);
```

F12 콘솔 strong

```
return <div className="output">문제 7: 콘솔을 보세요 (F12 → Console)</div>;  
}
```

React 라는 이름이 이 파일에 없는데도 JSX 가 됩니다. Vite 가 JSX 를 react/jsx-runtime 의 함수로 바꾸고, 그 import 를 자동으로 넣어 주기 때문입니다. 개념03 섹션 4에서 본 그것입니다.

typeof 를 빼고 console.log(React) 라고 쓰면 ReferenceError 가 나면서 예제가 빨간 상자로 바뀝니다. typeof 는 없는 이름에 써도 되는 몇 안 되는 문법입니다.

element 는 화면이 아니라 값입니다. 평범한 객체이고 type 에 태그 이름이 들어 있습니다.

문제 8 정답 (개념04)

```
function Q8ClassName() {
  return (
    <div className="q08Card"> <strong>아메리카노</strong> -
    <span className="q08Price">4000원</span> </div>
  );
}
```

class 라고 쓰면 React 가 경고를 냅니다. Invalid DOM property class. Did you mean className? 경고만 나고 화면은 안 멈춥니다. 대신 스타일이 하나도 안 걸립니다. 문제 13의 (라) 에서 직접 봅니다.

문제 9 정답 (개념04)

```
function Q9Conditional() {
  const [isSale, setIsSale] = useState(false);
```

className 은 결국 글자입니다. 그러니 삼항으로 골라 넣으면 됩니다.

```
const cardClass = isSale ? "q08Card q08Sale" : "q08Card";

return (
  <div> <div className={cardClass}> <strong>케이크</strong> -
  <span className="q08Price">6000원</span> </div> <button onClick={() =>
    setIsSale(!isSale)}>
    {isSale ? "할인 끄기" : "할인 켜기"}
  </button> </div>
);
}
```

템플릿 리터럴로 써도 똑같습니다.

```
const cardClass = `q08Card ${isSale ? "q08Sale" : ""}`;
```

할인일 때 "q08Sale" 만 넣으면 카드 모양(테두리·여백)이 통째로 사라집니다. .q08Sale 은 색만 바꾸는 규칙이라 혼자서는 카드가 되지 않습니다. '기본 모양 + 상태' 로 나눠 두고 둘 다 붙이는 것이 요령입니다.

문제 10 정답 (개념04)

```
function Q10Modules() {
  return (
    <div className={styles.box}> <span className={styles.label}>삼각김밥</span> -
    1200원
    </div>
  );
}
```

전역 CSS 는 이름을 안 받고(import "./x.css"), CSS Modules 는 이름을 받습니다. styles 는 평범한 객체입니다. 키가 CSS 에 적은 이름, 값이 Vite 가 새로 지은 이름입니다.

className="box" 라고 글자로 적으면 아무 일도 안 일어납니다. 에러도 경고도 없습니다. Vite 가 이름을 바꿔 냈기 때문입니다.

문제 11 정답 [응용] (개념02 + 개념04)

```
function Q11List() {
  return (
    <ul>
      {menuItems.map((item) => (
        <li key={item.id}> <span className={item.soldOut ? "q08SoldOut" : ""}>
        {item.name}</span> -{" "}
        <span className="q08Price">{formatPrice(item.price)}</span> </li>
      ))}
    </ul>
  );
}
```

세 가지가 한 줄에 모여 있습니다. map + key 05단원 formatPrice 08단원에서 다른 파일에서 가져온 함수 조건부 className 08단원 개념04

className={item.soldOut ? "q08SoldOut" : ""} 에서 빈 문자열을 쓴 것에 주의하세요.

className 은 글자라서 "아무것도 안 붙임" 은 빈 글자입니다. null 을 넣어도 React 가 무시해 주지만, 빈 글자가 더 헷갈리지 않습니다.

문제 12 정답 [도전] (개념02 + 개념03)

```
function Q12OrderTable() {
  return <OrderTable items={menuItems} />;
}
```

★ 여러분이 만들 파일은 _부품/연습_주문표.jsx 입니다. 이 정답 파일은 _부품/연습_주문표_정답.jsx 라는 이름으로 미리 만들어 둔 것을 씁니다. 같은 이름으로 두면 여러분이 만든 파일과 부딪히기 때문에 이름만 다르게 했습니다. 그 파일을 열어 보세요. 내용은 여러분이 만들 것과 똑같습니다.

그 파일 안에서 중요한 것 두 가지입니다.

[1] 경로

그 파일도 _부품 폴더 안에 있으므로 "./연습_카드.jsx" 입니다.
 "./_부품/연습_카드.jsx" 라고 쓰면 _부품 안에 또 _부품 을 찾다가 실패합니다.
 경로는 늘 '그 줄이 적힌 파일' 이 기준입니다(개념02 섹션 4).

[2] 합계

```
items.reduce((sum, item) => sum + item.price, 0)
```

JS자료 08단원의 reduce 입니다. 시작값 0 을 빼면
 첫 번째 항목(객체)이 시작값이 되어 합계가 이상해집니다.

reduce 가 낫설면 이렇게 해도 됩니다. 결과는 같습니다.

```
let total = 0;
for (const item of items) { total = total + item.price; }
```

합계는 $4000 + 4500 + 6000 + 1200 = 15700$ 입니다.

문제 13 정답 (에러 확인)

정답은 이 파일 맨 아래 '문제 13 — 에러 확인 정답' 블록에 있습니다.

정리

- 1 export default 는 중괄호 없이, 이름 있는 export 는 중괄호 안에 정확한 이름으로 가져옵니다. 둘을 바꿔 쓰면 결과가 완전히 다릅니다.
- 2 이름이 겹치면 as 로 바꿉니다. 안 바꾸면 Identifier ... has already been declared 로 파일이 통째로 멈춥니다.
- 3 부품을 쓰는 쪽은 그 부품 안이 어떻게 생겼는지 몰라도 됩니다. CoffeeList 를 쓰면서 CoffeeCard 를 import 할 필요가 없습니다.
- 4 경로는 늘 그 줄이 적힌 파일이 기준입니다. _부품 안의 파일끼리는 ./ 만으로 서로를 부릅니다.
- 5 className 은 글자입니다. 조건부는 삼항이나 템플릿 리터럴로 글자를 만들면 됩니다. '기본 모양 + 상태' 로 나눠 두고 둘 다 붙이세요.
- 6 CSS Modules 는 이름을 Vite 가 새로 지어 주므로 className={styles.box} 처럼 값을 넣어야 합니다. 글자로 적으면 조용히 안 걸립니다.
- 7 import 실수 중 '이름 있는 export 를 중괄호 없이 가져오기' 와 'class 라고 쓰기' 는 에러가 안 납니다. 가장 찾기 어려운 종류입니다.


```

export default function Answer08Vite() {
  return (
    <div>
      <h1>08단원 연습문제 정답 - Vite 로 옮기기</h1>

      <p className="guide">
        <strong>먼저 스스로 풀어 보고 여세요.</strong> 막힌 문제만 보는 편이 훨씬
        남습니다.
        <br />
        <br />
        답이 하나뿐인 것은 아닙니다. <strong>기대 결과대로 나오면 맞은 것</strong>
        입니다.
        각 정답 아래에 왜 그런지와, 자주 하는 실수를 적어 두었습니다.
        <br />
        <br />
        <strong>F12 → Console</strong> 도 함께 열어 두세요. 문제 7은 콘솔로
        확인합니다.
      </p>

      <div className="demo">
        <h3>문제 1 - export default 가져오기</h3>
        <Q1Welcome />
      </div>
      <div className="demo">
        <h3>문제 2 - 이름 있는 export 가져오기</h3>
        <Q2Discount />
      </div>
      <div className="demo">
        <h3>문제 3 - as 로 이름 바꾸기</h3>
        <Q3Rename />
      </div>
      <div className="demo">
        <h3>문제 4 - default 와 이름 있는 export 를 한 줄에</h3>
        <Q4Badge />
      </div>
      <div className="demo">
        <h3>문제 5 - 반복을 부품으로 줄이기</h3>
        <Q5UseCard />

```

```

    </div>
    <div className="demo"> <h3>문제 6 – 부품이 부품을 부른다</h3> <Q6UseList
/> </div> <div className="demo"> <h3>문제 7 – React 없이 도는 JSX</h3> <Q7NoReact
/> </div> <div className="demo"> <h3>문제 8 – className 붙이기</h3> <Q8ClassName
/> </div> <div className="demo"> <h3>문제 9 – 조건에 따라 className 바꾸기
</h3> <Q9Conditional /> </div> <div className="demo"> <h3>문제 10 – CSS
Modules</h3> <Q10Modules /> </div> <div className="demo"> <h3>문제 11 [응용] – 목록 +
조건부 className</h3> <Q11List /> </div> <div className="demo"> <h3>문제 12 [도전] –
부품 파일 직접 만들기</h3> <Q12OrderTable /> </div> <div className="demo"> <h3>문제
13 – 에러 확인</h3> <p>이 문제는 화면이 아니라 코드와 콘솔로 확인합니다. 파일 맨
아래를 보세요.</p> </div>

</div>

);
}

```

문제 13 — 에러 확인 정답

(가) import 경로에서 ./ 를 지웠을 때

```

"./_부품/연습_Welcome.jsx" → "_부품/연습_Welcome.jsx"

```

화면 예제가 통째로 안 나옵니다.

터미널과 화면에 이 줄이 뜹니다.

```

Failed to resolve import "_부품/연습_Welcome.jsx" from "src/08_.../연습문제.jsx".
Does the file exist?

```

→ ./ 가 없으면 경로가 아니라 '도구 이름' 으로 읽힙니다.

node_modules 에서 _부품 이라는 도구를 찾다가 실패한 것입니다.

Vite 가 파일을 읽는 단계에서 막히므로 화면이 아예 안 그려집니다.

(나) 중괄호로 바꿨을 때

```

import Welcome from ... → import { Welcome } from ...

```

화면 빨간 상자로 바뀝니다.

F12 콘솔

```
SyntaxError: The requested module '/src/08.../_부품/연습_Welcome.jsx'
does not provide an export named 'Welcome'
```

- 파일은 잘 찾았습니다. 그 안에 'Welcome'이라는 이름으로 내보낸 것'이 없을 뿐입니다.
연습_Welcome.jsx 는 export default 로 내보냈습니다.
 - (가)와 (나)의 차이가 여기 있습니다.
 - (가) 파일 자체를 못 찾았다 → Failed to resolve import
 - (나) 파일은 찾았는데 그 이름이 없다 → does not provide an export named
- 메시지만 읽어도 어느 쪽인지 구분할 수 있습니다.

(다) 중괄호를 지웠을 때

```
import { cakePrice, applyDiscount } from ... → import cakePrice from ...
```

- ★ 에러가 안 납니다. 경고도 없습니다.

화면 "케이크 원 → 할인가 원" 처럼 숫자 자리가 비어 버립니다.

(그리고 applyDiscount 를 안 가져왔으므로 그 줄에서 다른 에러가 따로 납니다.
cakePrice 하나만 놓고 보면 조용히 undefined 가 됩니다)

- "그 파일의 대표를 주세요" 라고 말했는데 연습_가격.js 에는 대표가 없습니다.
없는 것을 달라고 했으니 undefined 가 옵니다.
{undefined} 는 화면에 아무것도 안 그림니다(05단원 개념05).
그래서 에러 없이 자리만 빕니다.

(라) className 을 class 로 바꿨을 때

- ★ 에러가 안 납니다. 화면도 안 멈춥니다.

화면 카드 모양이 사라지고 밋밋한 글자만 남습니다.

콘솔에 노란 경고가 뜹니다.

```
Invalid DOM property `class`. Did you mean `className`?
```

- 02단원 개념03에서 배운 것입니다. class 는 자바스크립트가 쓰는 낱말이라
React 는 className 을 씁니다. 경고를 안 읽으면 못 찾습니다.

에러가 안 나는데 틀린 것 — (다) 와 (라) 입니다.

(가)·(나)는 화면이 멈추니 바로 알아챱니다. 오히려 고치기 쉬운 쪽입니다. (다)·(라)는 화면이 계속 돌아갑니다. 값만 비고 스타일만 안 걸립니다. 그래서 며칠 뒤에야 발견되기도 합니다.

★ 값이 `undefined` 로 보이면 → 중괄호를 확인하세요 ★ CSS 가 안 걸리면 → `className` 철자와 `import` 를 확인하세요 ★ 그리고 콘솔의 노란 경고를 그냥 넘기지 마세요. 대부분 이런 것을 알려 줍니다.

09단원 ·

09단원 연습문제 정답.jsx 정답 useEffect와 API

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요. 7번부터는 인터넷 연결이 필요합니다.

먼저 스스로 풀어 본 다음에 보세요. 답이 달라도 결과가 같으면 맞은 것입니다. 여기 적힌 것은 한 가지 예입니다.

★ 콘솔에 같은 줄이 두 번씩 찍히는 것은 정상입니다(개념02 StrictMode). 이 파일의 // 콘솔: 에는 한 번만 적어 두었습니다.

★ 10 · 11번은 일부러 없는 글을 요청합니다. 콘솔의 빨간 Failed to load resource ... 404 줄은 정상입니다. 우리 코드가 낸 에러가 아니라 브라우저가 알려 주는 것입니다.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";

const BASE_URL = "https://jsonplaceholder.typicode.com";
```

문제 1 정답 (개념01)

처음 한 번만 실행하려면 의존성 배열을 [] 로 둡니다. 빈 배열은 “지켜볼 값이 없다” 는 뜻이라 달라질 것도 없어서 다시 실행되지 않습니다.

```
function Answer01() {
  useEffect(() => {
    console.log("09단원 시작합니다");
  }, []);
```

F12 콘솔 09단원 시작합니다

```
    }, []);

    return (
      <div className="demo"> <h3>문제 1 – 처음 나타날 때 한 번만
</h3> <p className="output">콘솔(F12)을 보세요</p> </div>
    );
  }
}
```

문제 2 정답 (개념01)

의존성 배열 자리를 아예 비워 두면 화면을 다시 그릴 때마다 실행됩니다. 쉼표와 대괄호를 통째로 빼는 것이지, 빈 배열([])을 넣는 것이 아닙니다. 둘은 정반대입니다. 이 차이가 09단원에서 가장 많이 틀리는 곳입니다.

```
function Answer02() {
  const [tick, setTick] = useState(0);

  useEffect(() => {
    console.log("다시 그려졌습니다");
  });
}
```

F12 콘솔 다시 그려졌습니다

```
});

return (
  <div className="demo"> <h3>문제 2 - 매번 실행되게 하기
</h3> <p className="output">다시 그린 횟수: {tick}</p>
  {/* 화면: 다시 그린 횟수: 0 */}
  <button onClick={() => setTick(tick + 1)}>다시 그리기</button>
  {/* 화면(누르면): 다시 그린 횟수: 1 */}
</div>
);
}
```

문제 3 정답 (개념01)

의존성 배열에 count 만 넣습니다. menu 는 넣지 않습니다.

메뉴 바꾸기 버튼을 누르면 화면은 다시 그려지지만 effect 는 가만히 있습니다.

```
function Answer03() {
  const [count, setCount] = useState(0);
  const [menu, setMenu] = useState("아메리카노");

  useEffect(() => {
    console.log("지금 잔 수: " + count);
  });
}
```

F12 콘솔
지금 잔 수: 0
지금 잔 수: 1

```

    }, [count]);

    return (
      <div className="demo"> <h3>문제 3 - 특정 값이 바뀔 때만
</h3> <p className="output">
        {menu} {count}잔
      </p>
      { /* 화면: 아메리카노 0잔 */ }
      <button onClick={() => setCount(count + 1)}>잔 수 +1</button> <button onClick=
{() => setMenu(menu === "아메리카노" ? "라떼" : "아메리카노")}>
        메뉴 바꾸기
      </button>
      { /* 화면(누르면): 아메리카노 1잔 → 라떼 1잔 */ }
    </div>
  );
}

```

문제 4 정답 (개념01)

`document.title` 은 React 가 그리는 `#root` 바깥에 있습니다. JSX 로는 손댈 수 없으니 `useEffect` 에서 바꿉니다. 전형적인 부작용입니다.

```

function Answer04() {
  const [todoCount, setTodoCount] = useState(0);

  useEffect(() => {
    document.title = `할일 ${todoCount}개`;
    console.log("탭 제목:", document.title);
  });
}

```

F12 콘솔 탭 제목: 할일 0개
 탭 제목: 할일 1개

```

    }, [todoCount]);

    return (
      <div className="demo"> <h3>문제 4 - 탭 제목 바꾸기</h3> <p className="output">할
일 {todoCount}개</p>
      { /* 화면: 할 일 0개 - 브라우저 탭 글자도 "할일 0개" 가 됩니다 */ }
      <button onClick={() => setTodoCount(todoCount + 1)}>할 일 추가</button>
      { /* 화면(누르면): 할 일 1개 - 탭 글자도 "할일 1개" 로 바뀝니다 */ }
    </div>
  );
}

```

문제 5·6 정답 (개념02)

setInterval 을 켜고, 정리 함수에서 clearInterval 로 끕니다.

setSeconds((prev) => prev + 1) 로 쓴 것이 중요합니다. setSeconds(seconds + 1) 로 쓰면 effect 가 처음 실행될 때의 seconds(0)를 계속 붙잡고 있어서 화면이 1에서 멈춥니다. 개념06 섹션 2의 오래된 값 문제입니다. 함수형 갱신을 쓰면 seconds 를 읽지 않으니 의존성 배열이 [] 여도 맞습니다.

문제 6 정답 · (다) 둘 다 입니다.

다른 예제를 고르면 이 컴포넌트가 화면에서 사라지므로 정리 함수가 불립니다. 돌아오면 새로 마운트되는데, StrictMode 가 붙었다 뗐다 다시 붙이므로 그때도 정리 함수가 한 번 불립니다. 그래서 콘솔에는 "켰습니다 → 켜졌습니다 → 켜졌습니다" 순서로 세 줄이 나옵니다.

```
function Answer05() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    console.log("타이머를 켜졌습니다");
  });
}
```

F12 콘솔 타이머를 켜졌습니다

```
const id = setInterval(() => {
  setSeconds((prev) => prev + 1);
}, 1000);

return () => {
  clearInterval(id);
  console.log("타이머를 켜졌습니다");
};
```

F12 콘솔 타이머를 켜졌습니다

```
};
}, []);

return (
  <div className="demo"> <h3>문제 5 · 6 – 타이머와 정리 함수
</h3> <p className="output">{seconds}초</p>
  { /* 화면: 0초 → 1초 → 2초 ... 1초마다 올라갑니다 */ }
</div>
);
}
```


문제 7 정답 (개념03)

useEffect 에 넘기는 바깥 함수에는 async 를 붙이지 않습니다. 안에 async 함수를 만들고, 만든 뒤에 부르는 줄을 잊지 마세요. 만들기만 하고 안 부르면 아무 일도 일어나지 않습니다. 에러도 안 납니다.

```
function Answer07() {
  const [title, setTitle] = useState("아직 못 받았습니다");

  useEffect(() => {
    async function loadPost() {
      const res = await fetch(`${BASE_URL}/posts/3`);
      const data = await res.json();

      console.log("문제7 제목:", data.title);
    }
  }, []);

  return (
    <div className="demo"> <h3>문제 7 - 글 하나 받아오기</h3> <p className="output">
      {title}</p>
      { /* 화면: ea molestias quasi exercitationem repellat qui ipsa sit aut */ }
    </div>
  );
}
```

F12 콘솔 문제7 제목: ea molestias quasi exercitationem repellat qui ipsa sit aut

```
setTitle(data.title);
}

loadPost();
}, []);

return (
  <div className="demo"> <h3>문제 7 - 글 하나 받아오기</h3> <p className="output">
    {title}</p>
    { /* 화면: ea molestias quasi exercitationem repellat qui ipsa sit aut */ }
  </div>
);
}
```

문제 8 정답 (개념03)

초기값을 빈 배열로 둔 것이 중요합니다. null 로 두면 첫 화면에서 null.map(...) 이 되어 그 자리에서 터집니다. 서버가 준 id 를 key 로 쓰면 딱 좋습니다(05단원 개념03).

```
function Answer08() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    async function loadUsers() {
      const res = await fetch(`${BASE_URL}/users?_limit=3`);
      const data = await res.json();

      console.log(
        "문제8 이름들:",
        data.map((user) => user.name)
      );
    }
  });
}
```

F12 콘솔 문제8 이름들: ['Leanne Graham', 'Ervin Howell', 'Clementine Bauch']

```
setUsers(data);
}

loadUsers();
}, []);

return (
  <div className="demo"> <h3>문제 8 - 목록 받아서 그리기</h3> <ul>
    {users.map((user) => (
      <li key={user.id}>{user.name}</li>
    ))}
  </ul>
  </* 화면: Leanne Graham / Ervin Howell / Clementine Bauch */>
</div>
);
}
```

문제 9 정답 (개념04)

loading의 초기값은 `true`입니다. 화면이 나타나는 순간 이미 요청이 나가 있으니까요. 다 받은 뒤에 `false`로 바꿉니다.

```
function Answer09() {
  const [title, setTitle] = useState("");
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function loadPost() {
      const res = await fetch(`${BASE_URL}/posts/3`);
      const data = await res.json();
    }
  });
}
```

10단원 · 연습문제 정답

10단원 연습문제 정답.jsx 정답 useRef와 커스텀 훅

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 파일을 고르세요.

F12 → Console 도 함께 보세요.

★ 먼저 스스로 풀어 보고 나서 여세요.

★ 정답이 하나뿐인 문제는 거의 없습니다. 화면이 기대 결과대로 나오면 맞은 것입니다.

여기 적힌 것과 코드 모양이 조금 달라도 괜찮습니다.

★ 개발 중에는 StrictMode 때문에 콘솔 줄이 두 번씩 찍힙니다. 정상입니다.

```
import { useState, useRef, useEffect, memo } from "react";
import Summary from "../_ui/Summary.jsx";

const menuNames = [
  "아메리카노",
  "카페라떼",
  "바닐라라떼",
  "카페모카",
  "녹차라떼",
  "케이크",
  "쿠키",
  "삼각김밥",
];

const menuItems = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
  { id: 4, name: "삼각김밥", price: 1200 },
];
```

문제 1 정답 (개념01)

```
function Q1Focus() {
```

1) 상자를 만듭니다. 아직 요소가 없으니 `null` 로 시작합니다.

```
const inputRef = useRef(null);
```

```
function handleFocus() {
```

3) 상자 안(current)에 든 요소에게 시킵니다.

```
inputRef.current.focus();
```

```
console.log("문제 1: 입력칸에 커서를 놓았습니다");
```

F12 콘솔 문제 1: 입력칸에 커서를 놓았습니다

```
}
```

```
return (
```

```
  <div className="output">
```

```
    { /* 2) 요소에 붙입니다. React 가 이 상자에 요소를 넣어 줍니다. */ }
```

```
    <input ref={inputRef} placeholder="여기에 커서가 와야 합니다"
```

```
  /> <button onClick={handleFocus}>커서 놓기</button> </div>
```

```
);
```

```
}
```

세 걸음이 모두 있어야 동작합니다. 하나만 빠져도 안 됩니다. 상자 만들기 → 요소에 붙이기 → current 로 쓰기

문제 2 정답 (개념01)

```
function Q2ClearAndFocus() {
```

```
  const [text, setText] = useState("아메리카노");
```

```
  const inputRef = useRef(null);
```

```
  function handleClear() {
```

비우는 일은 '화면에 보이는 값' 이므로 state 입니다.

```
    setText("");
```

커서를 놓는 일은 '브라우저에게 시키는 동작' 이므로 ref 입니다.

```
    inputRef.current.focus();
```

```
    console.log("문제 2: 지우고 커서를 놓았습니다");
```

F12 콘솔 문제 2: 지우고 커서를 놓았습니다

```

    }

    return (
      <div className="output"> <input ref={inputRef} value={text} onChange={(e) =>
setText(e.target.value)}
        />
      <button onClick={handleClear}>지우고 커서 놓기</button> <div>지금 값: {text} ||
"(비어 있음)"</div> </div> </div>
    );
  }
}

```

한 함수 안에서 state 와 ref 를 나눠 쓴 예입니다. inputRef.current.value = "" 로 비우면 안 됩니다. 개념01 섹션 1에서 본 것처럼 React 가 다음 렌더에서 되돌려 놓습니다.

문제 3 정답 (개념01)

```

function Q3Scroll() {
  const cakeRef = useRef(null);

  function handleGo() {
    cakeRef.current.scrollIntoView({ behavior: "smooth", block: "center" });

    console.log("문제 3: 케이크 줄로 스크롤했습니다");
  }
}

```

F12 콘솔 문제 3: 케이크 줄로 스크롤했습니다

```

    }

    return (
      <div className="output"> <button onClick={handleGo}>케이크로 가기
</button> <div style={{ height: 110,
      overflowY: "auto",
      border: "1px solid #ccc",
      background: "#fff",
      marginTop: 6,
    }}
    > <ul>
      {menuNames.map((name) => (
        <li key={name} ref={name === "케이크" ? cakeRef : null}>
          {name}
        </li>
      ))}
    </ul> </div> </div>
    );
  }
}

```

```
);
}
```

behavior 를 빼면 순간이동하고, block 을 빼면 케이크가 상자 맨 위에 붙습니다. 둘 다 '보기 좋으라고' 주는 값이라 없어도 동작은 합니다.

문제 4 정답 (개념02)

```
function Q4RefNoRender() {
  const countRef = useRef(0);
  const [tick, setTick] = useState(0);

  function handleUp() {
```

set 함수가 없습니다. 상자 안에 직접 넣습니다.

```
    countRef.current = countRef.current + 1;

    console.log(`문제 4: 지금 진짜 값: ${countRef.current}`);
```

```
F12 콘솔    문제 4: 지금 진짜 값: 1
```

(두 번째로 누르면 2, 세 번째로 누르면 3 이 찍힙니다)

```
  }

  return (
    <div className="output"> <div>화면에 보이는 값: {countRef.current}</div> <div>
    다시 그린 횟수: {tick}</div> <div style={{ marginTop: 6 }}> <button onClick=
    {handleUp}>ref 올리기</button> <button onClick={() => setTick(tick + 1)}>다시 그리기
    </button> </div> </div>
  );
}
```

ref 를 세 번 올려도 화면은 0 입니다. 콘솔에는 1, 2, 3 이 찍힙니다.

다시 그리기 · 를 누르는 순간 화면에 3 이 나타납니다.

값은 이미 3 이었고, 화면이 그것을 몰랐을 뿐입니다.

문제 5 정답 (개념02)

```
function Q5FixToState() {
```

ref 를 state 로 바꿨습니다.

```
const [count, setCount] = useState(0);

function handleAdd() {
  setCount(count + 1);

  console.log("문제 5: 답았습니다. state 라서 화면도 바뀝니다");
}
```

F12 콘솔 문제 5: 답았습니다. state 라서 화면도 바뀝니다

```
}

return (
  <div className="output">
    { /* .current 도 함께 지워야 합니다. state 는 그냥 count 입니다. */ }
    <div>아메리카노 {count} 잔</div> <button onClick={handleAdd}>담기
  </button> </div>
);
}
```

판단 기준은 하나였습니다. 이 값이 바뀌면 화면도 바뀌어야 하나? → 예. 그러니 state 입니다.

문제 6 정답 (개념02)

```
function Q6Timer() {
  const [sec, setSec] = useState(0);
  const timerRef = useRef(null);

  function handleStart() {
```

이미 돌고 있으면 아무것도 안 합니다. 이 줄이 없으면 타이머가 겹칩니다.

```
if (timerRef.current !== null) return;

timerRef.current = setInterval(() => {
```

함수형 갱신입니다(04단원 개념05). 안에서 옛날 sec 를 보지 않게 합니다.

```
setSec((prev) => prev + 1);
}, 500);

console.log("문제 6: 타이머 시작");
```

F12 콘솔 문제 6: 타이머 시작

```

    }

    function handleStop() {
      clearInterval(timerRef.current);
    }
  }

```

상자도 비웁니다. 안 비우면 다음 [시작] 이 위의 검사에 걸려 안 먹습니다.

```

    timerRef.current = null;

    console.log("문제 6: 타이머 멈춤");
  }

```

F12 콘솔 문제 6: 타이머 멈춤

```

    }

    useEffect(() => {
      return () => {
        clearInterval(timerRef.current);
      };
    }, []);

    return (
      <div className="output"> <div>{sec} 칸</div> <button onClick={handleStart}>시작</button> <button onClick={handleStop}>멈춤</button> </div>
    );
  }

```

sec 는 화면에 보이니 state, 타이머 id 는 화면에 안 보이니 ref 입니다. id 를 state 로 두어도 동작은 하지만, 시작할 때마다 쓸데없이 다시 그립니다.

문제 7 정답 (개념03)

```

function useToggle(initial = false) {
  const [on, setOn] = useState(initial);

  function toggle() {
    setOn((prev) => !prev);
  }
}

```

{ on: on, toggle: toggle } 을 짧게 쓴 것입니다(속성 축약).


```

return { on, toggle };
}

function Q7Toggle() {
  const open = useToggle(false);

  function handleToggle() {
    open.toggle();
    console.log("문제 7: 영업 상태를 뒤집었습니다");
  }
}

```

F12 콘솔 문제 7: 영업 상태를 뒤집었습니다

```

}

return (
  <div className="output"> <div>지금 상태: {open.on ? "영업 중" : "준비 중"}
</div> <button onClick={handleToggle}>바꾸기</button> </div>
);
}

```

setOn(!on) 으로 써도 동작합니다. 다만 한 번에 여러 번 부를 일이 생기면 어긋나므로 prev 를 쓰는 편이 안전합니다.

문제 8 정답 (개념03)

```

function useInput(initialValue = "") {
  const [value, setValue] = useState(initialValue);

  function onChange(e) {
    setValue(e.target.value);
  }

  function clear() {
    setValue("");
  }

  return { value, onChange, clear };
}

function Q8Form() {

```

훅을 두 번 부르면 state 도 두 개 생깁니다. 서로 따로 놓입니다.

```
const name = useInput("김민준");
const memo = useInput("아메리카노");

function handleClearName() {
  name.clear();
  console.log("문제 8: 이름 칸을 비웠습니다");
}
```

F12 콘솔 문제 8: 이름 칸을 비웠습니다

```
}

function handleClearMemo() {
  memo.clear();
  console.log("문제 8: 메모 칸을 비웠습니다");
}
```

F12 콘솔 문제 8: 메모 칸을 비웠습니다

```
}

return (
  <div className="output"> <div>
    이름 <input value={name.value} onChange={name.onChange} /> </div> <div style=
    {{ marginTop: 4 }}>
      메모 <input value={memo.value} onChange={memo.onChange} /> </div> <div style=
      {{ marginTop: 6 }}> <button onClick={handleClearName}>이름 비우기
    </button> <button onClick={handleClearMemo}>메모 비우기</button> </div> </div>
  );
}
```

문제 9 정답 (개념04)

```
function MenuPicker({ items }) {
```

혹을 '먼저' 부릅니다. 조건은 그 아래에서 겁니다. items 가 빈 배열이면 items[0] 이 없으므로 빈 문자열을 초기값으로 씁니다.

```
const [picked, setPicked] = useState(items.length > 0 ? items[0].name : "");

function handlePick(name) {
  setPicked(name);
  console.log(`문제 9: 고른 메뉴 - ${name}`);
}
```

F12 콘솔 문제 9: 고른 메뉴 - 아메리카노

```
}

```

return 은 훅을 다 부른 뒤에 합니다. 이 순서가 핵심입니다.

```
if (items.length === 0) {
  return <p>메뉴가 없습니다</p>;
}

return (
  <div> <div>
    {items.map((item) => (
      <button key={item.id} onClick={() => handlePick(item.name)}>
        {item.name}
      </button>
    ))}
    </div> <div style={{ marginTop: 6 }}>고른 메뉴: {picked}</div> </div>
);
}

function Q9Rules() {
  return (
    <div> <div className="output"> <MenuPicker items={menuItems}>
  /> </div> <div className="output" style={{ marginTop: 6 }}> <MenuPicker items={[]}>
  /> </div> </div>
  );
}
```

훅보다 위에서 return 하면 그 렌더에서는 훅이 안 불립니다. 그러면 훅 개수가 렌더마다 달라져 이 에러가 납니다. Rendered fewer hooks than expected. 09단원의 로딩 처리와 붙여 쓰다가 아주 자주 나옵니다.

문제 10 정답 (개념05)

memo 로 감쌉니다. 파일 맨 바깥에서 한 번만 만듭니다.

```
const PriceCard = memo(function PriceCard({ name, price }) {
  console.log(`문제 10: PriceCard 를 그렸습니다 - ${name}`);

```

F12 콘솔 문제 10: PriceCard 를 그렸습니다 - 아메리카노

```

return (
  <div>
    {name} - {price} 원
  </div>
);
});

function Q10Memo() {
  const [tick, setTick] = useState(0);

  console.log("문제 10: 부모를 그렸습니다");

```

F12 콘솔 문제 10: 부모를 그렸습니다

```

return (
  <div className="output"> <PriceCard name="아메리카노" price={4000} /> <div>부모의
숫자: {tick}</div> <button onClick={() => setTick(tick + 1)}>부모만 다시 그리기
</button> </div>
);
}

```

부모만 다시 그리기 · 를 누르면 부모 줄만 콘솔에 찍힙니다.

PriceCard 는 name 과 price 가 그대로라서 다시 안 그립니다.

주의 — memo(...) 를 컴포넌트 안에서 만들면 안 됩니다. 다시 그릴 때마다 새 컴포넌트가 되어 아무 효과가 없습니다.

문제 11 정답 [응용] (개념02 + 개념03)

```

function usePrevious(value) {

```

상자는 다시 그려도 그대로 남습니다. 그래서 '직전 값' 을 담아 둘 수 있습니다.

```

const ref = useRef("(없음)");

useEffect(() => {

```

화면을 다 그린 뒤에 넣습니다. 그리는 도중에 넣으면 '지금 값' 이 되어 버려서 늘 같은 값이 나옵니다.

```

  ref.current = value;
}, [value]);

return ref.current;

```

```

}

function Q11Previous() {
  const [menu, setMenu] = useState("아메리카노");
  const prevMenu = usePrevious(menu);

  function pick(name) {
    setMenu(name);
    console.log(`문제 11: 고른 메뉴 - ${name}`);
  }
}

```

F12 콘솔 문제 11: 고른 메뉴 - 라떼

```

}

return (
  <div className="output"> <div>지금: {menu}</div> <div>직전: {prevMenu}</div>
  <div style={{ marginTop: 6 }}> <button onClick={() => pick("라떼")}>라떼</button>
  <button onClick={() => pick("케이크")}>케이크</button> <button onClick={() => pick("삼각김밥")}>삼각김밥</button> </div> </div>
);
}

```

이 훅을 state 로 만들면 안 되는 이유가 있습니다. useEffect 안에서 setPrev 를 부르면 또 다시 그려지고, 그러면 useEffect 가 또 돌아서 무한 루프가 됩니다(09단원 개념06). ref 는 바뀌어도 다시 안 그리므로 안전합니다.

문제 12 정답 [도전] (개념01 + 개념03)

```

function useTodoList(initialTodos = []) {
  const [todos, setTodos] = useState(initialTodos);

  function add(text) {

```

06단원 trim. 공백만 있는 글자는 넣지 않습니다.

```

const trimmed = text.trim();
if (trimmed === "") return;

```

07단원 불변 갱신. push 가 아니라 새 배열을 만듭니다.

```

setTodos((prev) => [...prev, trimmed]);
}

function remove(index) {

```

filter 로 그 자리만 빼고 새 배열을 만듭니다.

```
setTodos((prev) => prev.filter((todo, i) => i !== index));
}

return { todos, add, remove };
}

function Q12Todo() {
  const list = useTodoList(["아메리카노 사기", "케이크 사기"]);
  const [text, setText] = useState("삼각김밥 사기");
  const inputRef = useRef(null);

  function handleAdd() {
    console.log(`문제 12: 추가 - ${text}`);
  }
}
```

F12 콘솔 문제 12: 추가 - 삼각김밥 사기

```
list.add(text); // 커스텀 훅에게 시킵니다 setText(""); // 입력칸을 비웁니다
(state)
inputRef.current.focus(); // 커서를 다시 놓습니다 (ref)
}

function handleRemove(index) {
  list.remove(index);
  console.log("문제 12: 한 줄 지웠습니다");
}
```

F12 콘솔 문제 12: 한 줄 지웠습니다

```
}

return (
  <div className="output"> <input ref={inputRef} value={text} onChange={(e) =>
setText(e.target.value)}
  placeholder="할 일"
  />
  <button onClick={handleAdd}>추가</button> <ul>
    {list.todos.map((todo, index) => (
      <li key={index}>
        {todo} <button onClick={() => handleRemove(index)}>지우기</button> </li>
      )))}
  </ul> </div>
);
}
```

세 가지가 한 함수 안에 모여 있습니다. 각각 어느 도구인지 보세요.

| | | |
|---------------------------------------|--------------------|------------------|
| <code>list.add(text)</code> | 커스텀 훅 | (목록 다루기를 묶어 둔 것) |
| <code>setText("")</code> | <code>state</code> | (화면에 보이는 값) |
| <code>inputRef.current.focus()</code> | <code>ref</code> | (브라우저에게 시키는 동작) |

key 에 index 를 쓴 것이 마음에 걸릴 수 있습니다(05단원 개념03). 지우기가 있는 목록이라 원래는 고유한 id 를 붙이는 편이 낫습니다. 여기서 이 단원의 주제에 집중하려고 index 로 두었습니다.

정리

- 1 ref 는 세 걸음입니다. useRef 로 상자 만들기 → 요소에 ref 붙이기 → current 로 쓰기.
- 2 화면에 보이는 값은 state, 브라우저에게 시키는 동작은 ref 입니다. 한 함수 안에서 같이 씁니다.
- 3 타이머는 시작할 때 겹치지 않게 검사하고, 멈출 때 상자를 null 로 비웁니다.
- 4 커스텀 훅은 그냥 함수입니다. 부를 때마다 새 state 가 생겨 서로 따로 놓입니다.
- 5 훅을 먼저 다 부르고, early return 은 그 아래에 둡니다.
- 6 memo 는 파일 맨 바깥에서 감쌉니다. 컴포넌트 안에서 감싸면 효과가 없습니다.

```

export default function Answer10RefAndHooks() {
  return (
    <div>
      <h1>10단원 연습문제 정답 - useRef 와 커스텀 훅</h1>

      <p className="guide">
        <strong>먼저 스스로 풀어 보고 나서 여세요.</strong>
        <br />
        코드 모양이 조금 달라도 기대 결과대로 나오면 맞은 것입니다.
        <br />
        F12 → Console 도 함께 보세요.
      </p>

      <div className="demo">
        <h3>문제 1 - 커서 놓기</h3>
        <Q1Focus />
      </div>
      <div className="demo">
        <h3>문제 2 - 지우고 커서 놓기</h3>
        <Q2ClearAndFocus />
      </div>
      <div className="demo">
        <h3>문제 3 - 스크롤 이동</h3>
        <Q3Scroll />
      </div>
      <div className="demo">
        <h3>문제 4 - ref 는 다시 안 그린다</h3>
        <Q4RefNoRender />
      </div>
      <div className="demo">
        <h3>문제 5 - ref 를 state 로 고치기</h3>
        <Q5FixToState />
      </div>
      <div className="demo">
        <h3>문제 6 - 타이머 id 보관</h3>
        <Q6Timer />
      </div>
    </div>
  )
}

```



```

    </div>
    <div className="demo"> <h3>문제 7 – useToggle</h3> <Q7Toggle
  /> </div> <div className="demo"> <h3>문제 8 – useInput</h3> <Q8Form
  /> </div> <div className="demo"> <h3>문제 9 – 훅의 규칙</h3> <Q9Rules
  /> </div> <div className="demo"> <h3>문제 10 – memo</h3> <Q10Memo
  /> </div> <div className="demo"> <h3>문제 11 [응용] – usePrevious</h3> <Q11Previous
  /> </div> <div className="demo"> <h3>문제 12 [도전] – 할일 목록</h3> <Q12Todo
  /> </div> <div className="demo"> <h3>문제 13 – 에러 확인</h3> <p>이 문제의 정답은 이
파일 맨 아래 주석에 있습니다.</p> </div>

  </div>
);
}

```

문제 13 정답 — 에러 확인

(가) `inputRef.current.focus()` → `inputRef.focus()`

TypeError: inputRef.focus is not a function

useRef 가 돌려주는 것은 요소가 아니라 { current: 요소 } 상자입니다.
상자에는 focus 라는 메소드가 없습니다. 요소는 상자 안에 있습니다.

(나) input 에서 `ref={inputRef}` 를 지움

TypeError: Cannot read properties of null (reading 'focus')

상자는 만들었는데 아무 요소에도 안 붙였습니다.
그래서 current 가 처음 값인 null 그대로입니다.
null 에서 focus 를 꺼내려 해서 나는 에러입니다.
JS자료 10단원에서 `querySelector` 가 null 을 돌려줬을 때 본 그 에러와 같습니다.

(가)와 (나)의 차이:

(가) 는 상자를 요소인 줄 알고 쓴 것 - `.current` 를 빠뜨렸습니다

(나) 는 상자가 아직 비어 있는 것 - `ref={...}` 를 안 붙였습니다

메시지에서 `'inputRef.focus'` 인지 `'reading focus of null'` 인지를 보면 구분됩니다.

(다) 화면 줄에서 `{countRef.current}` → `{countRef}`

Objects are not valid as a React child (found: object with keys {current})

상자는 객체입니다. 객체는 화면에 그릴 수 없습니다.
화면에 그릴 수 있는 것은 글자, 숫자, JSX 입니다.
메시지에 `keys {current}` 라고 친절히 알려 줍니다. ref 를 통째로 쓴 것입니다.

(라) `if (count > 0) { const [bonus] = useState("보너스"); }` 를 넣고 [담기] 클릭

누르기 전에는 멀쩡합니다. count 가 0 이라 혹은 하나였습니다.
누르면 count 가 1 이 되어 혹은 두 개가 됩니다. 그 순간 이렇게 됩니다.

— 콘솔 경고

React has detected a change **in** the order **of** Hooks called by Q5FixToState.
This will lead to bugs and errors **if** not fixed.

— 에러

Rendered more hooks than during the previous render.

개념04에서 본 그대로입니다. 혹은 개수가 렌더마다 달라졌습니다.

네 가지를 한 줄로 구분하면 이렇습니다. (가) 상자와 요소를 헛갈림 → is not a **function** (나) 상자가 비어 있음 → reading ... of **null** (다) 상자를 화면에 그리려 함 → Objects are not valid as a React child (라) 혹은 개수가 달라짐 → Rendered more hooks than ...

에러 메시지를 끝까지 읽는 습관이 붙으면 이 넷을 보자마자 구분할 수 있습니다.

부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.